Constraint-driven modernization

Incremental legacy modernization

A 12-year-old .NET monolith with a shared SQL Server database, changed one measured constraint at a time while it keeps serving customers.

The system and its rules

Part	Today
Application	.NET Framework monolith on IIS, server-rendered UI
Data	One shared SQL Server database
Integrations	ERP by nightly CSV, legacy SMTP mail, one payment integration
Modules	Catalog, Pricing, Orders, Inventory, Fulfillment, Billing, ERP Integration
Hard rule	No big-bang rewrite; the old application keeps serving customers
Hard rule	No loss of acknowledged orders; old clients and batch jobs work until migrated
Hard rule	One authority per writable domain, aggregate or partition
Hard rule	Each change is independently verifiable and preferably reversible

Four things migrate separately

Dimension	What moves	Typical safety
Code	Implementation is isolated and replaced	Characterization tests, seams
Traffic	Clients or internal callers reach the new path	Routing flag, canary, fallback route
Data	Data is copied, synchronized, validated, archived	Backfill, reconciliation, lag threshold
Write ownership	One writer is fenced, drained and switched	Write fencing, cutoff, reconciliation

Traffic can be switched back at any time. Data written since the cutover cannot.

Six phases, 50 steps

Phase	Steps	Focus
1 Stabilize and characterize	01–07	Architecture map, behavior capture, test data, deployment, drift, evidence, reversibility
2 Introduce replacement seams	08–14	Routing and logic seams, dependency swap, data access, vendor model, schema compatibility, modules
3 Read-only strangler extraction	15–21	First read slice: populate, compare, shadow, canary, migrate callers, retire the old branch
4 Data synchronization and writer handover	22–33	Backfill, sync, translation, reconciliation, single writer, failback, sync debt
5 Business capability extraction	34–42	Notifications, ERP, billing, inventory, checkout, UI routes, contracts
6 Infrastructure change and retirement	43–50	Rehost, runtime, database hosting, identity, audit, latency, decommission, stop

Phase 1

Stabilize and characterize

Steps in this phase

Step	Forcing constraint
M01 Recover architecture map	Pricing change unexpectedly modifies invoices
M02 Characterize behavior	Discount rule undocumented and broken by small release
M03 Create representative test data	Production defects cannot be reproduced safely
M04 Automate deployment	Changes inconsistent and rollback hard
M05 Reduce environment drift	Production behaves differently from tested stage
M06 Introduce production evidence	Release increases errors but cause unknown
M07 Create reversibility	Bad discount logic must be disabled without undoing other changes

M01 · Stabilize and characterize

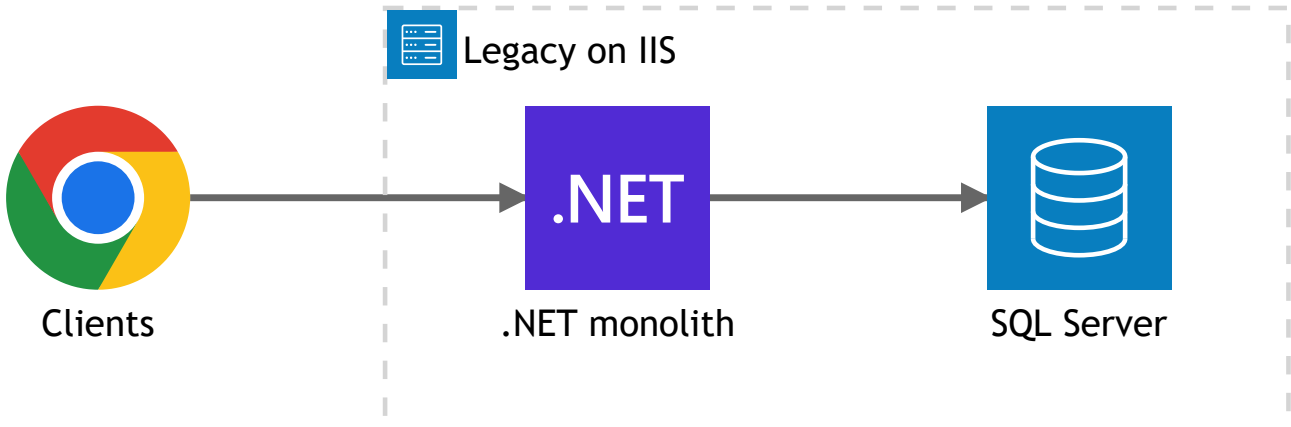
Recover the architecture map

A pricing change unexpectedly modifies invoices, and no document shows why.

Nothing documents how the modules depend on each other

Component	What it is	What it does
Legacy monolith	12-year-old .NET Framework application	Runs Catalog, Pricing, Orders, Inventory, Fulfillment, Billing and ERP Integration
IIS	Microsoft web server that hosts the application	Serves the server-rendered UI and runs the monolith
SQL Server	One shared relational database	Stores the data of every module, readable by all of them
Documentation	Almost none; rules live in code and production behavior	Cannot say which module reads or changes what
Long-serving developers	The people who remember how it works	Answer questions from memory

Current architecture



A pricing change that touched invoices

A developer changes one pricing rule that looks local. The next invoices carry different amounts, because another module depends on pricing data in a way no document shows.

Invoices change

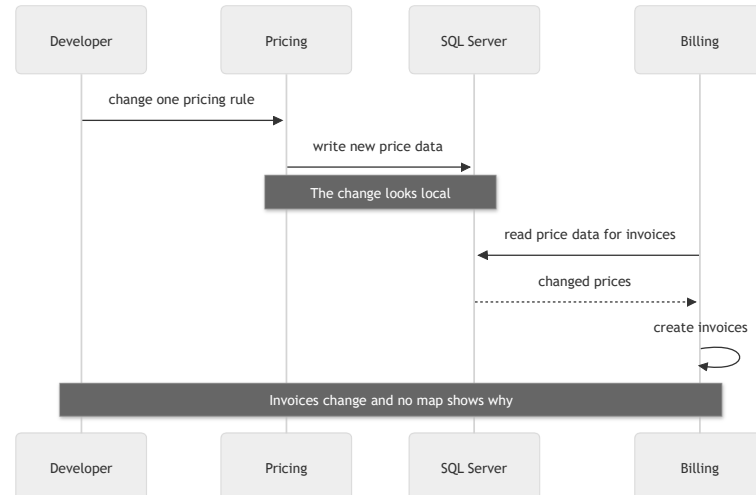
after a pricing change, and the Pricing to Billing link is not documented

7 modules, 1 database

with no map of who reads or writes what

Every path confirmed

from a pricing change to the invoices, by code and by a runtime sample



Each option replaces guessing with a different kind of evidence

Solution	What it is	What it does
Architecture Recovery + dependency maps	Extracting system structure from code, data and runtime behavior and drawing it as dependency maps	Finds which modules, tables and jobs depend on each other, including links nobody documented
C4 Model + traces	C4 diagrams of context, containers, components and code, filled in with recorded request traces	Shows the system at the right level for each reader and the calls seen in real requests
Interviews with long-serving developers	Asking the people who know the system to describe how the modules depend on each other	Collects remembered dependencies and business intent into notes

Evidence costs time, and memory costs accuracy

Solution	Gains	Costs
Architecture Recovery + dependency maps	Reveals coupling Finds hidden dependencies Evidence from code, data and runtime	Takes time Evidence may be incomplete Map needs an owner to stay current
C4 Model + traces	Right level for each stakeholder Traces show real calls	Incomplete picture May miss dynamic and runtime behavior Needs tracing the monolith lacks
Interviews with long-serving developers	Cheapest and quickest Explains business intent	Memory gaps hide dependencies Nothing confirms the answers Knowledge leaves with the people

Deciding signal: does runtime evidence find dependencies that code reading and memory missed? One confirmed hidden link means interviews alone fail the constraint. If the question is only who owns which module, C4 views are enough; traces alone suffice only when every path of interest can be exercised on demand.

Smallest safe move

Architecture Recovery + dependency maps for one slice

- **Scope:** evidence only; no code, traffic, data or write ownership changes
- **Change:** scan code and SQL for references from Pricing to invoice data, map the paths, then confirm each with a runtime sample from one pricing change in a test environment
- **Safety net:** read-only work; every edge is marked confirmed or suspected, so an interview answer is never taken as proof
- **Escape route:** nothing to undo; correct or drop an edge when a runtime sample contradicts it
- **Deferred:** C4 Model + traces when new teams need the map; Interviews with long-serving developers only to name suspected edges

```
-- database objects only; SQL text inside the .NET code needs the code scan
SELECT OBJECT_NAME(referencing_id), referenced_entity_name
FROM sys.sql_expression_dependencies
WHERE referenced_entity_name IN ('Invoice', 'InvoiceLine');
```

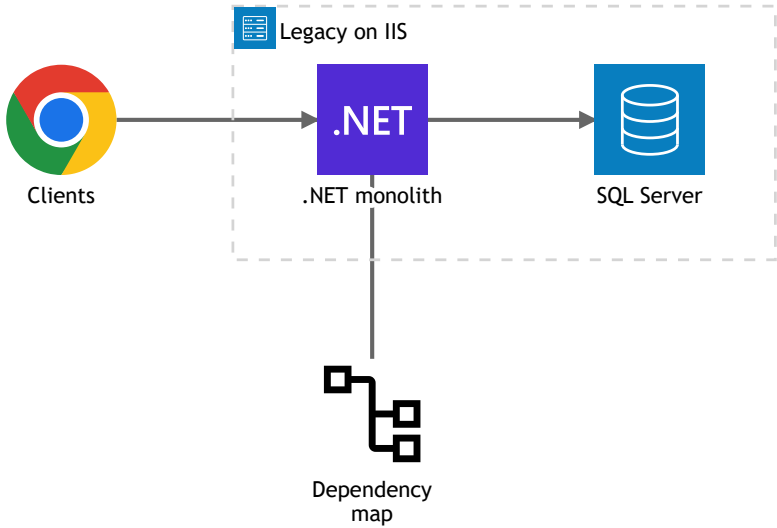
Verification: a map that predicts the invoices

Check	Pass condition
Static scan of Pricing references to invoice data	Every reference classified as call, shared table, view, stored procedure or job, each with an owning module
Runtime sample: one pricing change run in a test environment	Every table, view and procedure touched appears on the map; 0 touched items missing
Prediction: a second pricing change	Invoice fields predicted from the map equal the fields that changed
Edge status review	Every edge tagged confirmed or suspected; 0 untagged edges
Fitness function	A scheduled scan job reads Pricing code and SQL for references to invoice tables, views and procedures and fails when one is not on the map

Rollback: routing rollback is not needed because no traffic moves; state rollback is not needed because nothing is written. A wrong map is corrected by the next runtime sample.

Stop: when every path from Pricing to the invoices is confirmed and the prediction matches, no further mapping is added; other modules are mapped only when a constraint reaches them.

Solution architecture



Nothing in the running system changes. The dependency map is new and describes the monolith from its code, data and runtime behavior.

Glossary

Used in this step

Term	Definition
Monolith	Entire application deployed as a single unit.
Shared database	One SQL Server database that every module reads and writes
Hidden dependency	A link between modules that no document or interface declares
Architecture Recovery	Extract system structure from code, data and runtime behavior
Dependency map	Diagram of which module, table or job depends on which, with evidence per link
C4 Model	Architecture visualization through context, containers, components and code
Trace	Record of the calls one request makes across components
Runtime sample	Observation of what a real or test action touches while it runs
Confirmed edge	A map link verified by both code reading and a runtime sample
Interviews with long-serving developers	Asking experienced people how the system works; fast but unverified
Fitness function	Automated check that keeps a stated property true over time

Related concepts

Term	Definition
Characterization Testing	Tests documenting actual legacy behavior before refactoring
Golden Master	Capture existing observable outputs as compatibility reference
Refactor in Place	Improve internal structure without changing external behavior
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations
Seam	A place where behavior can be switched between old and new without editing callers

M02 · Stabilize and characterize

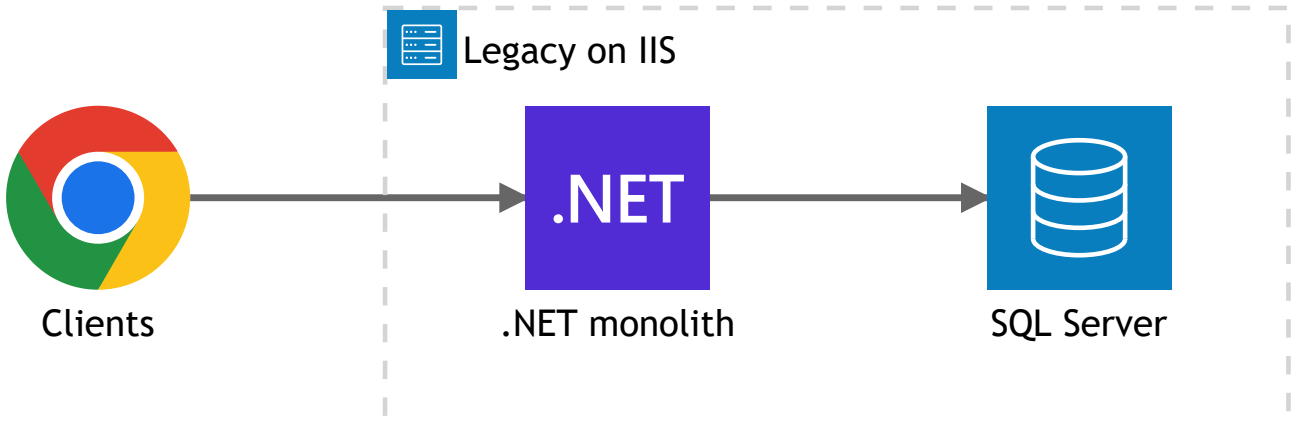
Characterize behavior

An undocumented discount rule was broken by a small release, and no test noticed.

The legacy system has no tests that fail when behavior changes

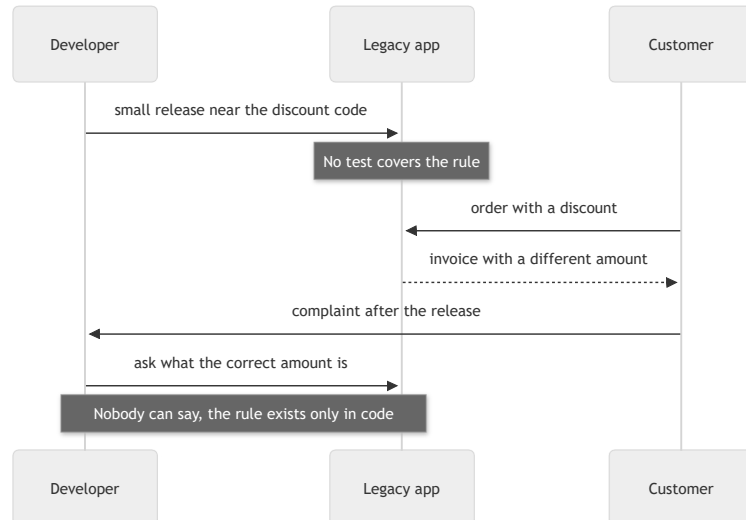
Component	What it is	What it does
Legacy monolith	.NET Framework application on IIS	Applies discounts and builds invoices; behavior is known only from code
Discount rule	Business logic inside the Pricing module	Reduces the price of an order; no specification exists
Tests	Sparse, outdated or missing checks	Stay green when a release changes a discount
Releases	Small changes shipped to the monolith	Can alter behavior without any check failing
Dependency map	Confirmed paths from Pricing to Billing	Shows which outputs a discount change can reach

Current architecture



A small release that broke a discount

A small release changes how a discount applies, and the amounts customers see change with it. The rule is written nowhere, so nobody can say what the correct result was.



Broken by a small release

and no test failed

Only in code

the discount rule has no specification and no reliable tests

Baseline recorded

for the old release, known defects reviewed, any later difference shows as a failed test

Each option pins behavior in a different place

Solution	What it is	What it does
Characterization Testing + Golden Master	Tests that record what the current release actually returns for chosen inputs and keep those outputs as the reference	Fails when a later release returns a different discount for the same input
Contract Testing	Tests that check the interface consumers rely on against the provider	Verifies requests and responses between callers and the application, not what happens inside
Manual regression checklist	A written list of discount scenarios that testers replay by hand before each release	Compares each result with what the tester expects

Capturing behavior protects it and also preserves its bugs

Solution	Gains	Costs
Characterization Testing + Golden Master	Protects unknown contracts Characterizes behavior from real outputs Repeatable on every run	Preserves existing bugs Intended changes need baseline review Volatile values must be masked
Contract Testing	Verifies interfaces Protects independent releases	Does not verify internals Wrong values pass if the shape is right Test lifecycle and contract churn
Manual regression checklist	Cheapest to start No tooling needed	Checks expectations, not actual behavior Slow and not repeatable on every change Misses rules nobody listed

Deciding signal: does the failure change an interface shape or only values? A release that changes values while every response keeps its shape leaves contract tests green, so Contract Testing alone cannot meet the constraint. A manual checklist suffices only while the input combinations are few enough for a person to replay; contract tests join once a consumer is released on its own schedule.

Smallest safe move

Characterization Testing + Golden Master

- **Scope:** evidence only: the discount rule's inputs and outputs are read; no code, traffic, data or write ownership changes
- **Change:** record the old release's outputs for inputs that cover the rule (customer type, quantity, price, date) as the golden master, then fail on any later difference
- **Safety net:** label each known defect keep or fix, so capturing behavior does not decide that bugs persist; mask timestamps and ids
- **Escape route:** delete the golden master files; production is untouched
- **Deferred:** Contract Testing when a consumer is released on its own schedule; Manual regression checklist only for rules not yet captured

```
// illustrative: replay every recorded input and compare with the recorded output
foreach (var input in DiscountInputs.All)
    Assert.Equal(GoldenMaster.Read(input), Pricing.ApplyDiscount(input));
```

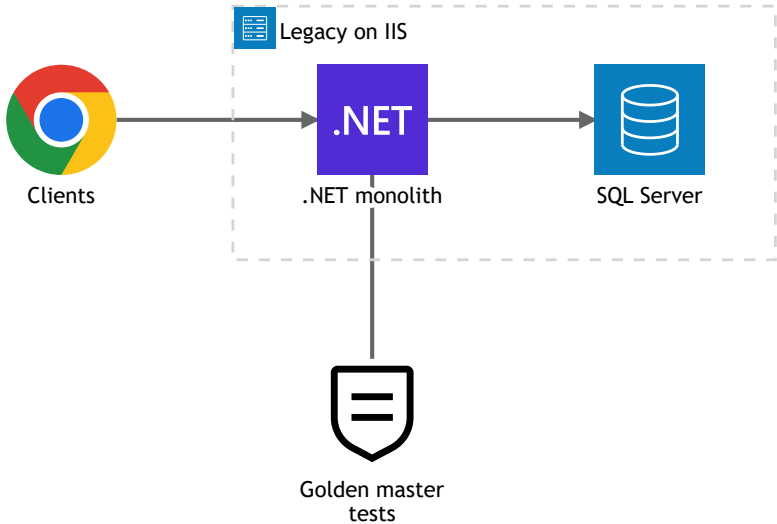
Verification: the old release as the reference

Check	Pass condition
Baseline run twice over the old release	Identical outputs both times; 0 unexplained differences
Known defects reviewed	Each known defect labelled keep or fix with an owner; 0 unlabelled defects
Replay of the small release that broke the discount	The golden master test fails and shows the changed discounts
A deliberate, approved rule change	The diff is reviewed and the baseline is updated in the same change
Fitness function	The golden master comparison runs on every change to Pricing code; any unapproved difference fails the test run

Rollback: routing rollback is not needed because no traffic moves; state rollback is not needed because the tests only read outputs. Baseline files are versioned, so reverting the commit restores the previous baseline.

Stop: when the golden master fails on the broken release and passes on the old one, no further test layer is added: no contract tests and no full-system suite until a constraint needs them.

Solution architecture



The monolith, its database and its clients stay as they were. A golden master test harness replays recorded inputs against the monolith and compares the outputs.

Glossary

Used in this step

Term	Definition
Characterization Testing	Tests documenting actual legacy behavior before refactoring
Golden Master	Capture existing observable outputs as compatibility reference
Contract Testing	Consumer expectations verified against a provider
Manual regression checklist	Written scenarios that testers replay by hand and judge against expectations
Baseline	Recorded outputs of the current release for chosen inputs
Known defect	A wrong behavior the team has already observed in the legacy system
Test harness	Code that runs the system with chosen inputs and compares the outputs
Discount rule	Business rule that sets the price reduction for an order; exists only in code
Masking	Replacing volatile values so that comparisons stay stable
Fitness function	Automated check that keeps a stated property true over time

Related concepts

Term	Definition
Refactor in Place	Improve internal structure without changing external behavior
Differential Testing	Compare outputs of old and new implementations for same input
Branch by Abstraction	Introduce stable interface then change the implementation behind it gradually
Feature Flag	Runtime control enabling behavior without deploying new code
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations

M03 · Stabilize and characterize

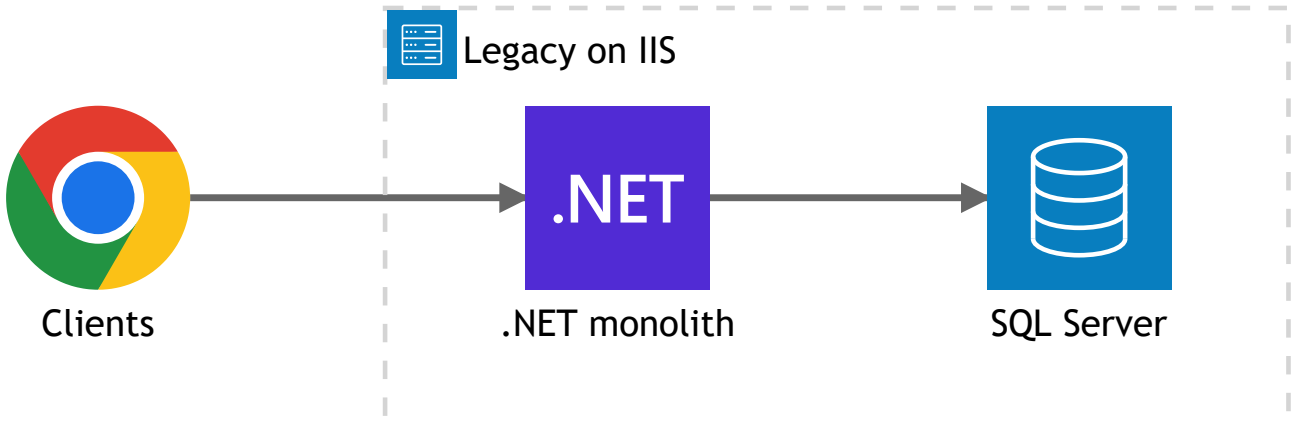
Create representative test data

Production defects cannot be reproduced safely, because no realistic isolated dataset exists.

Test environments hold only hand-made rows

Component	What it is	What it does
Legacy monolith	.NET Framework application on IIS	Handles the orders, discounts and invoices in which defects appear
Production database	One shared SQL Server database	Holds the real data, the unusual order shapes and the sensitive records
Staging	Test environment with a small hand-made dataset	Runs without the unusual shapes, so defects do not reproduce
Golden master tests	Recorded outputs of the old release	Need realistic inputs to be worth running
Dependency map	Confirmed paths from Pricing to Billing	Shows which data a reproduction must include

Current architecture



A defect that staging cannot reproduce

A defect shows up in production only for an unusual order shape, and the team cannot recreate it. Testing on live data would put customers at risk.

Not reproducible

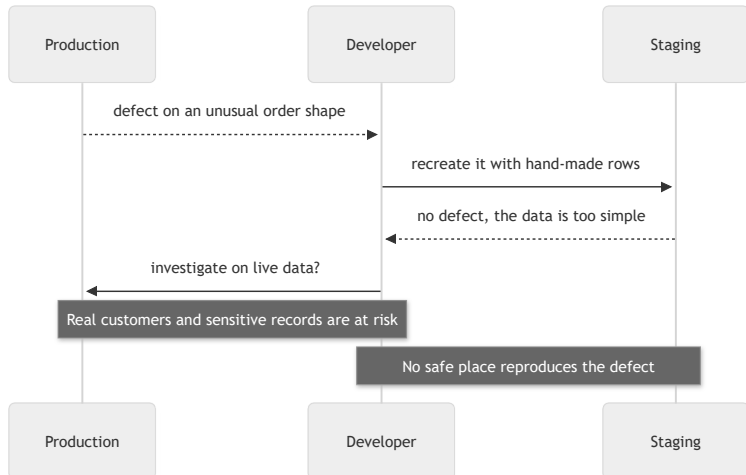
a production defect cannot be reproduced in a safe environment

No realistic dataset

test environments hold only hand-made rows

0 sensitive records

in staging while unusual data shapes are exercised



Each option trades realism against safety and size

Solution	What it is	What it does
Sanitized snapshot	A copy of selected production data with personal and sensitive fields masked or removed	Gives staging the real data shapes and distributions without real identities
Synthetic fixtures	Hand-written or generated records built to cover chosen cases	Provides repeatable data with no link to real people
Restored DB clone	A production backup restored into a separate environment	Reproduces the full database state, volume and indexes as production holds them

Realistic data brings privacy or resource costs

Solution	Gains	Costs
Sanitized snapshot	Realistic data shapes Contains the unusual cases production produced Small when cut to one slice	Privacy: masking must be complete Masking rules need upkeep Masking can alter the shapes that matter
Synthetic fixtures	Safe: no real data Repeatable and small Easy to share and version	Limited edge cases Only shapes someone thought of Fixtures need upkeep
Restored DB clone	Representative of production Includes volume and indexes	Resources: storage and restore time Copies sensitive records unless masked Stale until restored again

Deciding signal: can the defect be reproduced from a written description of the data shape? If a fixture reproduces it, Synthetic fixtures suffice. If the shape appears only in real data, a Sanitized snapshot is needed and ships only when the scan finds 0 sensitive records. A Restored DB clone is justified only when the defect depends on volume, indexes or full database state and storage and restore time fit.

Smallest safe move

Sanitized snapshot of one slice

- **Scope:** data only: a masked slice of production data is copied to staging; production, code, traffic and write ownership do not change
- **Change:** export read-only with personal and payment fields masked before leaving production, load it into staging, keep the unusual shapes found in defects
- **Safety net:** a scan must find no sensitive field before staging loads the data; an unmasked copy never reaches staging
- **Escape route:** drop and rebuild the staging dataset; production is never written to
- **Deferred:** Synthetic fixtures when masking cannot be trusted; Restored DB clone when a defect depends on volume or indexes

```
-- illustrative: keep the shape, drop the identity
SELECT OrderId, Total, DiscountCode, 'customer-' + CAST(CustomerId AS varchar(12)) AS CustomerRef
FROM Orders WHERE DiscountCode IS NOT NULL;
```

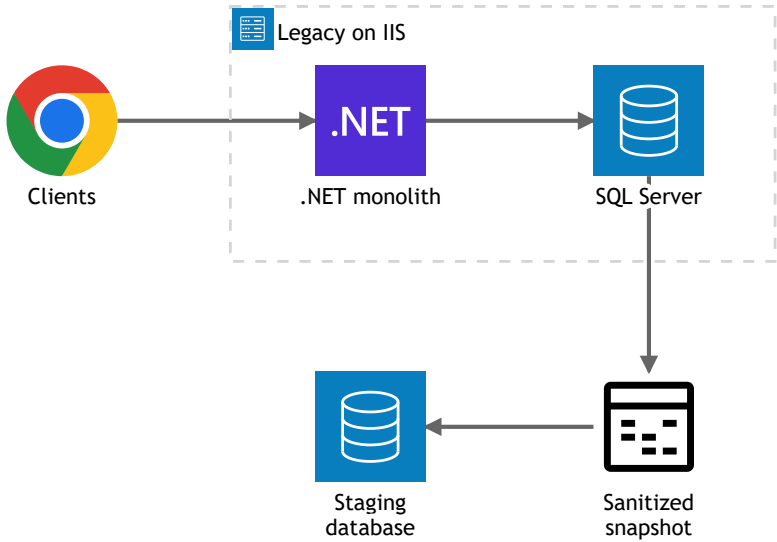
Verification: unusual shapes without sensitive records

Check	Pass condition
Sensitive-field scan of the staging dataset	0 personal or payment fields in clear text
Unusual shapes listed from production defects run in staging	Every listed shape runs through Pricing, Orders and Billing and the result is recorded; production is not touched
One past production defect replayed in staging	The defect is reproduced with the same wrong result
Dataset rebuilt twice	Same row counts and checksums both times, so golden master runs are comparable
Fitness function	The dataset build runs as a scheduled job and fails if the scan finds any hit or a listed shape has no rows

Rollback: routing rollback is not needed because no traffic moves. State rollback is limited to staging: production is never written, so the staging dataset is dropped and rebuilt. If a sensitive field slipped through, purge the staging copy and its backups, fix the masking rule and rebuild (forward repair).

Stop: when staging reproduces the known production defects from the masked slice, no larger dataset and no full restored clone is added.

Solution architecture



Production is unchanged. A masked slice leaves it read-only and loads a staging database, where unusual data shapes can be exercised safely.

Glossary

Used in this step

Term	Definition
Sanitized snapshot	Copy of selected production data with personal and sensitive fields masked or removed
Synthetic fixtures	Hand-written or generated records that cover chosen cases
Restored DB clone	A production backup restored into a separate environment
Data shape	The structure and combination of values in a record, such as an order with stacked discounts
Data masking	Replacing sensitive values with realistic but meaningless ones
Sensitive field	Personal or payment data that must not leave production unprotected
Staging	The environment where changes are tested before production
Edge case	A rare input combination at the border of expected behavior
Data slice	A narrow part of the data that covers one path, such as orders with discounts
Fitness function	Automated check that keeps a stated property true over time

Related concepts

Term	Definition
Golden Master	Capture existing observable outputs as compatibility reference
Characterization Testing	Tests documenting actual legacy behavior before refactoring
Shadow Traffic	Send production requests to a candidate implementation without using candidate responses
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Differential Testing	Compare outputs of old and new implementations for same input

M04 · Stabilize and characterize

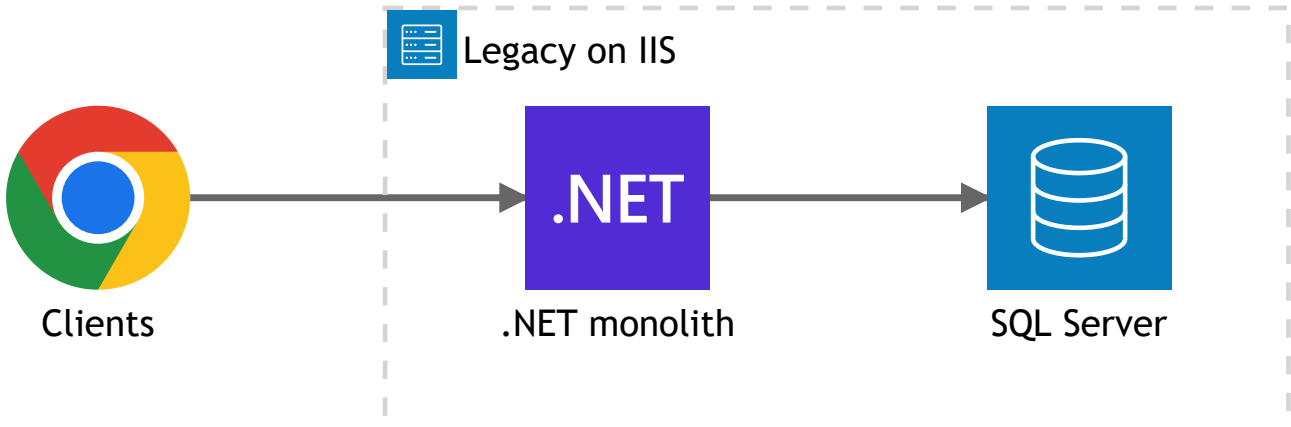
Automate deployment

A manual two-hour release makes changes inconsistent and rollback hard.

A two-hour manual procedure ships every release

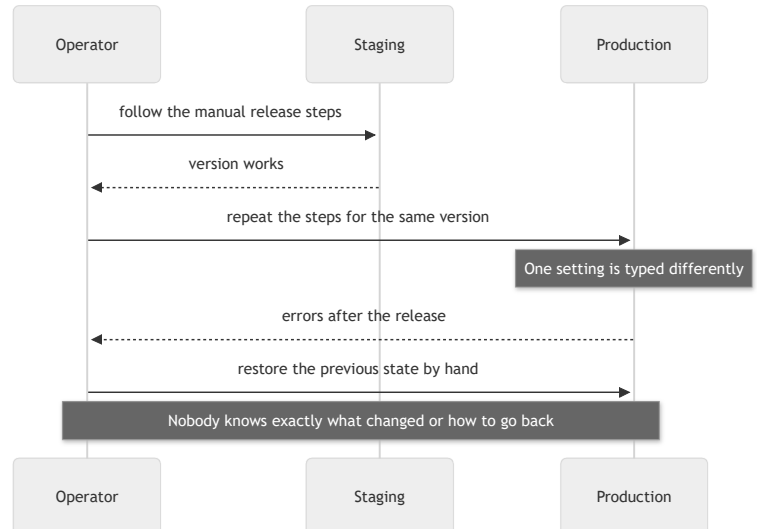
Component	What it is	What it does
Release procedure	Manual two-hour sequence of build, copy and configure steps	Puts each release on staging and production by hand
Legacy monolith	.NET Framework application on IIS	Is built and copied to the servers during the release
SQL Server	One shared relational database	Receives any schema change as one more manual step
Golden master tests	Recorded outputs of the old release	Exist, but run only when someone remembers
Staging dataset	Masked slice of production data	Lets a release be tried before production

Current architecture



The same version, released differently

Every release is a manual procedure, so two deployments of one version end up different. When a release goes wrong, nobody can say exactly what changed or how to return to the previous state.



Inconsistent changes

between releases, and rollback is hard

Two-hour manual release

repeated for every release

Identical hashes

when one version is deployed twice to a clean environment

Each option automates a different part of the release

Solution	What it is	What it does
Scripted deployment	The manual release steps written down as scripts that an operator runs	Repeats the same file copies and settings changes without hand typing
CI/CD	A pipeline that builds, tests and deploys every change the same way	Runs the same validated steps on every change and every release
Immutable artifacts	A versioned package built once and never changed after the build	Deploys the identical package to every environment; rollback installs the previous package

Automation costs scripts, setup or packaging work

Solution	Gains	Costs
Scripted deployment	Low effort Removes typing errors Steps become readable	Scripts to maintain Build and tests stay manual Rollback is still a rerun by hand
CI/CD	Validated repeatability Tests gate every release History of what was deployed	Setup effort Pipeline needs an owner Legacy build may need cleanup first
Immutable artifacts	Consistent deployments Rollback installs the previous package	Packaging work for a legacy app Configuration must live outside the package Package storage and retention

Deciding signal: where do two clean deployments of one version differ? Only in manual steps: Scripted deployment is enough. In the built files: Immutable artifacts. If every change must also pass the tests before release: CI/CD. The choice changes at any hash difference between two clean deployments of one commit, or any release that reaches production without the tests having run.

Smallest safe move

CI/CD for one clean environment

- **Scope:** delivery only: one pipeline, one clean environment; production stays manual until hashes match; no traffic, data or write ownership change
- **Change:** build once per commit into a versioned package, run the golden master tests, deploy that package by script
- **Safety net:** one package for every environment, never rebuilt, configuration kept outside it; every script in version control so scripts do not become the new undocumented code
- **Escape route:** redeploy the previous package; the manual procedure remains
- **Deferred:** Scripted deployment alone if no build agent exists; Immutable artifacts with a repository when builds of one commit differ

```
# illustrative pipeline outline
steps: [build-once, golden-master-tests, deploy-to-clean-env, compare-hashes]
```

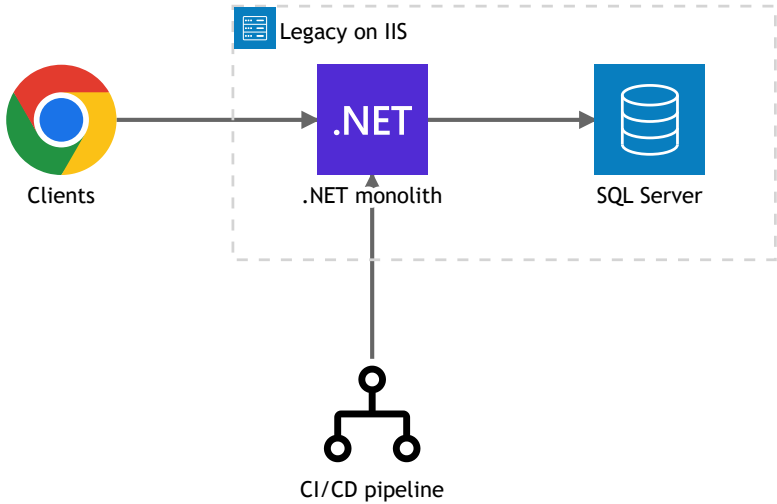
Verification: the same hashes twice

Check	Pass condition
One version deployed twice to a clean environment	Hashes of the deployed files are identical both times
Golden master tests run in the pipeline	They run on every change and a changed discount output fails the run
Previous package redeployed	Hashes match the earlier deployment of that version
Pipeline duration against the manual baseline	Duration recorded and compared with the two-hour manual release
Fitness function	The pipeline runs the double deployment and the hash comparison on every change and fails on any difference

Rollback: routing rollback is redeploying the previous package, which brings the previous version back; the pipeline itself moves no traffic. State rollback is different: redeploying files does not undo database changes, so schema or data changes stay out of this move and are reversed by a restore or a forward repair.

Stop: when two clean deployments hash-match and the previous version can be redeployed, no further delivery layer is added: no blue-green environments, container platform or orchestration.

Solution architecture



Clients, monolith and database are as before. A pipeline now builds one versioned package, tests it and deploys that same package to the monolith.

Glossary

Used in this step

Term	Definition
Scripted deployment	Release steps written as scripts that an operator runs
CI/CD	Pipeline that builds, tests and deploys every change the same way
Immutable artifacts	A versioned package built once and never changed after the build
Rollback	Return to a previously running version or state
Manual release	A release done by an operator following a remembered or written procedure
Versioned package	A build output with a version, kept so that it can be redeployed
Clean environment	An environment rebuilt from nothing before a deployment test
Hash	A fingerprint of a file; equal hashes mean identical content
Golden Master	Capture existing observable outputs as compatibility reference
Fitness function	Automated check that keeps a stated property true over time

Related concepts

Term	Definition
Blue-Green Deployment	Maintain two release environments and switch traffic
Feature Flag	Runtime control enabling behavior without deploying new code
Canary Routing	Shift a small controlled segment of live traffic to a new implementation
Infrastructure as Code	Version-controlled machine-readable infrastructure configuration
Container Image	Packaged filesystem and metadata from which containers are created

M05 · Stabilize and characterize

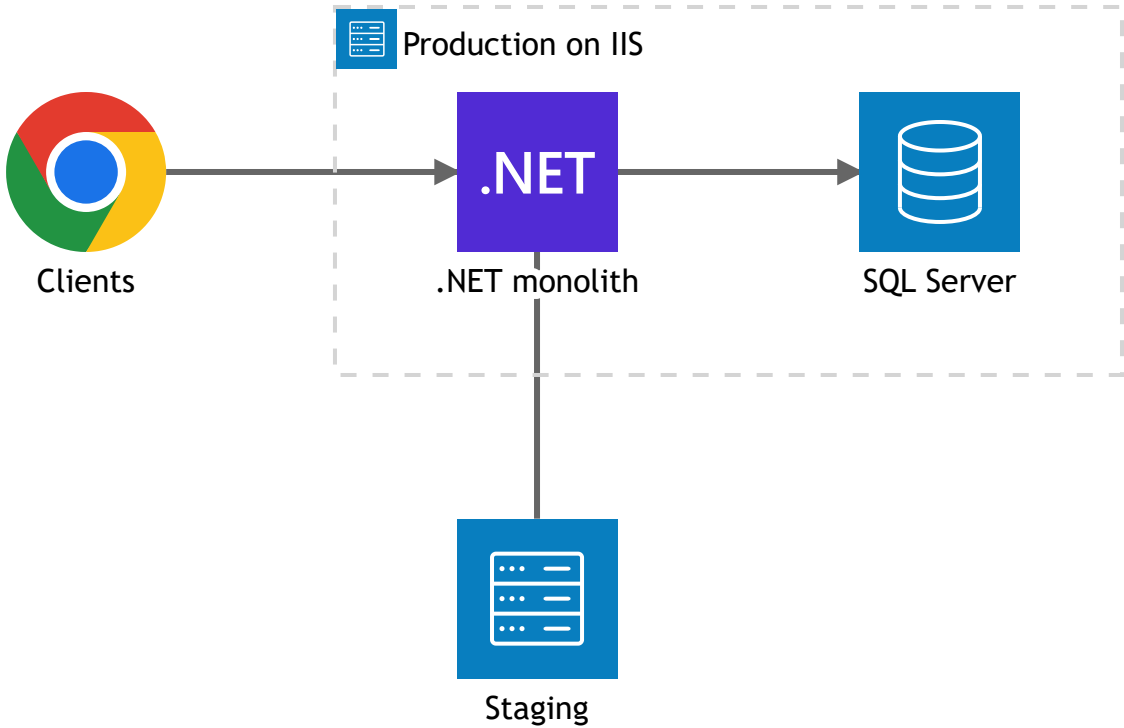
Reduce environment drift

Production behaves differently from the tested stage, because both were configured by hand.

Two hand-configured environments are assumed to match

Component	What it is	What it does
Production	Hand-configured IIS and SQL Server hosts	Serves customers with settings accumulated over years
Staging	Hand-configured test environment meant to mirror production	Is where every release is tested first
Release pipeline	Builds one versioned package and deploys it by script	Delivers the same files, not the same settings
Golden master tests	Recorded outputs of the old release	Pass on staging and say nothing about production settings
Configuration	Web server, application and database settings	Lives on each server, with no list of the differences

Current architecture



Passes in staging, behaves differently in production

A release passes every check in staging and then behaves differently in production. Both environments were configured by hand, so nobody can list how they differ.

Different behavior

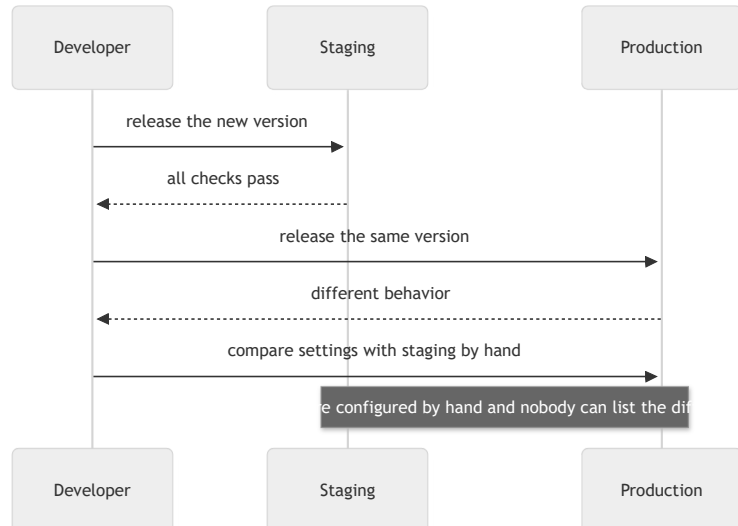
a release that passes in staging behaves differently in production

Configured by hand

staging and production, with no list of their differences

Only intended differences

in the configuration inventory diff between production and staging



Each option makes environments reproducible at a different level

Solution	What it is	What it does
Configuration as Code	Environment settings kept as version-controlled files that the pipeline applies	Makes the settings of each environment reproducible and comparable by diff
IaC	Version-controlled, machine-readable definitions of servers and infrastructure	Builds every environment from the same definitions
Containers	The application and its dependencies packaged in an isolated runtime image	Runs the same package with the same runtime in every environment

Reproducibility costs migration, providers or runtime compatibility

Solution	Gains	Costs
Configuration as Code	Reproducibility Changes reviewed and versioned Differences visible by diff	Migration of hand-set values into files Secrets need separate handling Hand edits on servers bring drift back
IaC	Consistent infrastructure Repeatable environments A lost environment can be rebuilt	Provider dependence Provider and module maintenance Heavy when only settings differ
Containers	Packaged: same runtime everywhere Reproducible deployment artifacts	Runtime compatibility: legacy runtime may not run unchanged Image lifecycle and security patching New platform to operate

Deciding signal: how do the environments differ, and in how many independent ways? A small group of settings points to Configuration as Code. Differences in servers, operating system or network that settings files cannot express bring IaC. Differences in the application's own runtime dependencies bring Containers, only if the legacy runtime runs in one.

Smallest safe move

Configuration as Code, staging first

- **Scope:** infrastructure configuration only: both environments' settings are exported, compared and versioned; no code, traffic, data or write ownership change
- **Change:** diff the inventories, mark each difference intended or drift, version per-environment files, apply the staging file first
- **Safety net:** secrets stay in a secret store; a scheduled drift check catches hand edits on servers, the way drift returns
- **Escape route:** production changes one setting group at a time; re-apply the previous version from version control
- **Deferred:** IaC when servers, operating system or network differ; Containers when runtime dependencies differ and the monolith runs in one

```
# illustrative inventory diff
setting                 production   staging
executionTimeout        110          90
.NET Framework version  4.8          4.7.2
```

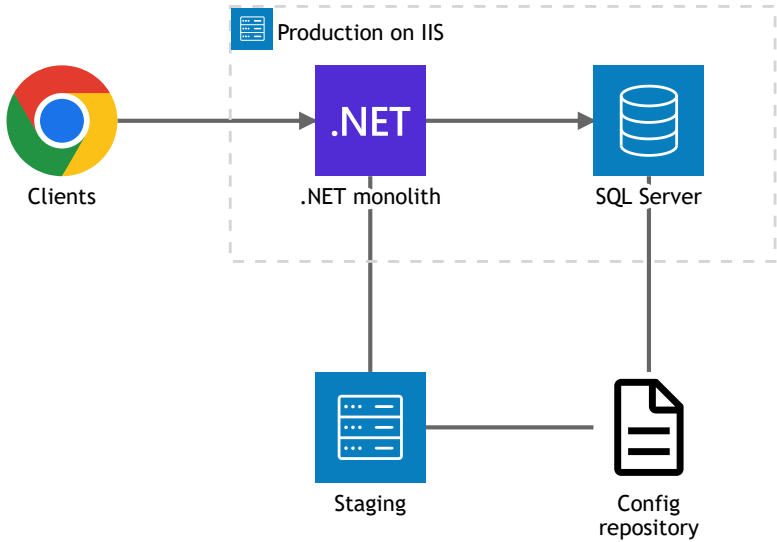
Verification: a diff with no surprises

Check	Pass condition
Inventory diff of production and staging	Every difference listed and marked intended with a reason or drift; 0 unexplained differences
Repository configuration applied to staging, then diffed again	The second diff shows only intended differences
Golden master tests run on staging after the change	No output differences from the baseline
A hand edit made on purpose in staging	The next drift check reports that setting
Fitness function	A scheduled job exports both inventories, diffs them and fails when a difference is not on the list of intended differences

Rollback: routing rollback is not needed because no traffic moves. State rollback: configuration is the state of an environment, so re-apply the previous version from version control one setting group at a time; data is not touched, so no data restore is needed.

Stop: when the inventory diff shows only intended differences and staging results match production behavior, no IaC and no containers are added.

Solution architecture



Both environments keep running as before. One configuration repository now describes the settings of staging and production, so their differences can be diffed and reviewed.

Glossary

Used in this step

Term	Definition
Configuration as Code	Settings kept as version-controlled files that the pipeline applies instead of being typed by hand
IaC	Version-controlled machine-readable infrastructure configuration
Containers	Lightweight isolated environment packaging an application and its dependencies.
Environment drift	Untracked divergence between environments that should match
Configuration inventory	A list of the settings and installed components of an environment
Intended difference	A documented difference between environments, with a reason
Drift check	Scheduled comparison that reports differences between environments
Secret store	A protected place for passwords and keys, kept outside version control
Staging	The environment where changes are tested before production
Fitness function	Automated check that keeps a stated property true over time

Related concepts

Term	Definition
Container Image	Packaged filesystem and metadata from which containers are created
Feature Flag	Runtime control enabling behavior without deploying new code
Rollback	Return to a previously running version or state
Rehost	Move workload to new infrastructure with minimal application changes
Blue-Green Deployment	Maintain two release environments and switch traffic

M06 · Stabilize and characterize

Introduce production evidence

A release increases errors, but the cause stays unknown because plain text server logs cannot follow one checkout through the database and the payment call.

Plain text logs record errors but not their cause

Component	What it is	What it does
Clients	Browsers that place orders	Call the monolith over HTTP
Legacy monolith	12-year-old .NET Framework application on IIS	Runs checkout and every other module
SQL Server	One shared relational database	Stores the data of every module
Payment integration	The one integration that takes card payments	Is called by the checkout code during an order
Plain text logs	Free-text log files on each server	Record errors as lines that only people read

Current architecture

Errors rise after a release and nobody can say why

A release goes live and checkout errors rise. The logs are plain text files on each server and nothing ties the lines of one request together, so the cause stays unknown.

Errors rise

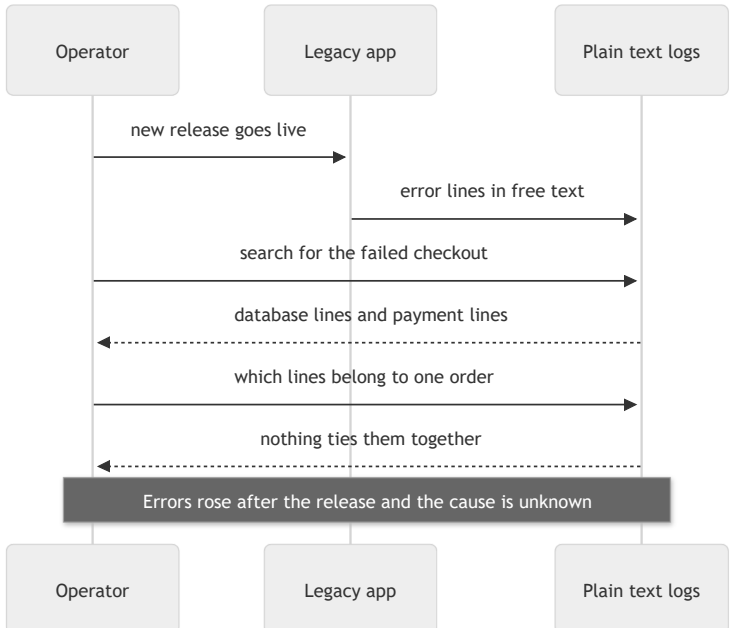
after a release, and the cause stays unknown

Plain text local logs

no shared key between the request, the database call and the payment call

1 failed checkout

traced through the database and payment calls by one id, with an error baseline per release



Each option gathers evidence from a different angle

Solution	What it is	What it does
Structured Logging	Log lines written as named fields such as time, level, release and order id instead of free text	Makes logs searchable and countable by field, for example errors per release
Metrics	Numerical measurements such as request count, error count and latency collected over time	Shows the live health of checkout and the error rate of each release
Correlation ID	One identifier created per request and carried across every related operation	Tags every log line of one checkout, from the request through the database call to the payment call

Evidence costs storage, operations or discipline

Solution	Gains	Costs
Structured Logging	Searchable by field Errors countable per release Readable by tools, not only by people	Log volume and storage cost Field names must be agreed and kept stable Does not join the lines of one checkout by itself
Metrics	Live health view Error rate per release at a glance Easy to alert on	Shows that errors rose, not why Collector and dashboards to operate Needs an agreed definition of an error
Correlation ID	Connects log and trace evidence Follows one failed checkout end to end Works with the logs that already exist	Must propagate consistently Every hop has to pass it on Does not count or rate errors by itself

Deciding signal: can one failed checkout be followed through the request, the database call and the payment call, and can errors be counted per release? Lines without a shared key need a Correlation ID; lines that cannot be searched or counted by field need Structured Logging; errors that customers report before any measurement shows them need Metrics.

Smallest safe move

A correlation id on the checkout path

- **Scope:** evidence and code on the checkout path; traffic, data and write ownership unchanged
- **Change:** add one id and the release version to each checkout log line, including the database and payment calls; count errors per release as the baseline SLI
- **Safety net:** no card or personal data in lines; a failed log write never fails a checkout; id propagation tested at every hop; rising errors alone are not diagnosis
- **Escape route:** redeploy the previous package; appended fields keep old readers working
- **Deferred:** Structured Logging when finding lines by field is too slow; Metrics when errors are first noticed from complaints

```
# illustrative log lines of one failed checkout
corr=7f3a9c release=2.14 step=request   POST /checkout
corr=7f3a9c release=2.14 step=database  insert order
corr=7f3a9c release=2.14 step=payment   ERROR declined
```

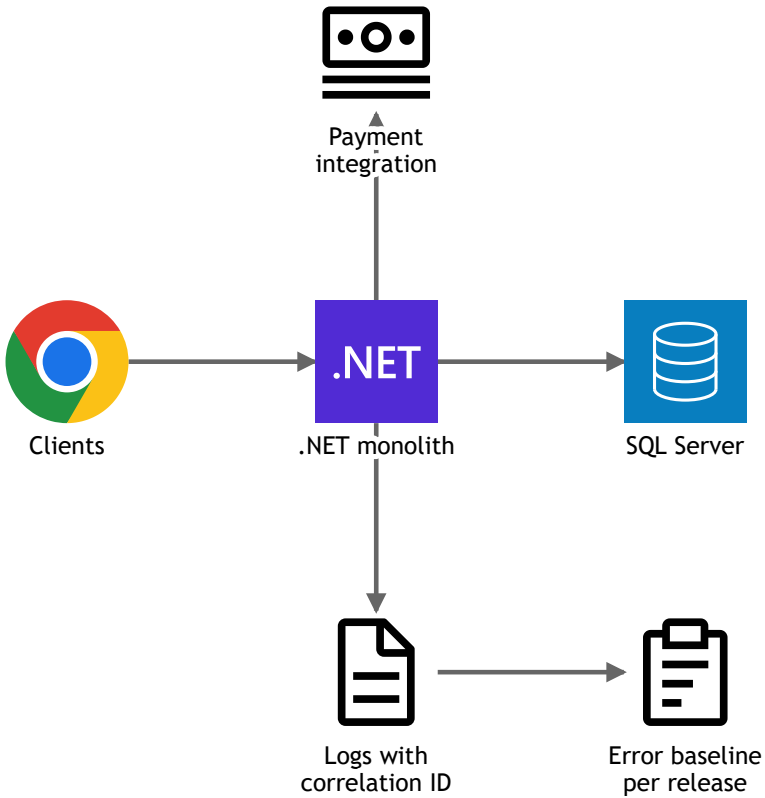
Verification: one failed checkout, traced end to end

Check	Pass condition
A failed checkout forced in staging (payment declined)	One search for its correlation id returns the request, the database call and the payment call in order
Checkout log lines sampled in production	100% of the sampled lines carry the id and the release version
Checkout errors counted per release	A baseline error count for the current release is recorded for later comparison
Log content scanned and a log write failure forced	No card number or personal data in any line, and the checkout still completes
Fitness function	A pipeline test forces a payment failure and fails when an id search misses a call; a log scan fails on card number patterns

Rollback: routing rollback is not needed because no traffic moves. State rollback is not needed either: only log lines gain fields and no business data changes; redeploying the previous package removes the fields.

Stop: when one id traces a failed checkout through the database and the payment call and the error count per release is recorded, no log platform, metrics stack or distributed tracing is added.

Solution architecture



The call path is unchanged. Every checkout log line now carries one correlation id and the release version, and the error count per release is the first production evidence that later releases are compared with.

Glossary

Used in this step

Term	Definition
Structured Logging	Log lines written as named fields instead of free text
Metrics	Captures numerical measurements of system performance and behavior.
Correlation ID	Identifier carried across related operations
Production evidence	What the running system records about a release: logs, ids and measurements
Plain text log	Free-text lines in a file on one server, searchable only by text
Propagation	Passing the id on to every call made for the same request
Trace	All log lines of one request, joined by its correlation id
Baseline	Measured state of the current release that later releases are compared with
SLI	Measured numerical indicator of service behavior
Fitness function	Automated check that keeps a stated property true over time

Related concepts

Term	Definition
Distributed Tracing	Tracks requests across multiple services and dependencies.
Centralized Logging	Aggregates application logs from multiple services.
SLO	Target for SLI over a defined time window
Alerting System	Notifies operators when monitored conditions exceed thresholds.
Feature Flag	Runtime control enabling behavior without deploying new code

M07 · Stabilize and characterize

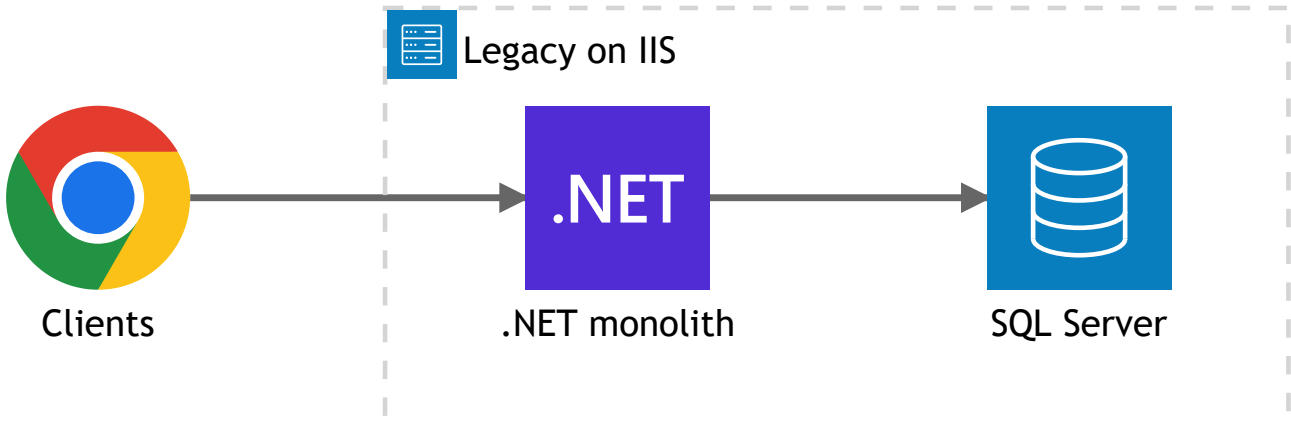
Create reversibility

A bad discount change must be switched off alone within the recovery time objective, while every other change of the release stays active.

One bundled release carries every changed feature

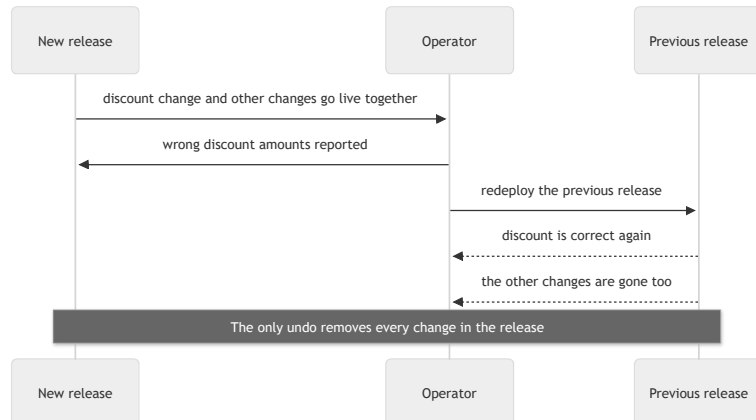
Component	What it is	What it does
Clients	Browsers and other callers	Use the application over HTTP
Legacy monolith	12-year-old .NET Framework application on IIS	Ships all changed features in one bundled release
SQL Server	One shared relational database	Stores the data of every module
Release pipeline	Builds one versioned package and deploys it by script	Can redeploy the previous package and nothing smaller
Production evidence	Correlation id, release version and error count per release in the logs	Shows that a release went wrong and which orders it touched

Current architecture



One bad discount change should not cost the whole release

A release ships several changed features together. One discount change is wrong, and the only undo redeploys the previous release, which also removes the other changes.



Bundled release

one bad discount change cannot be undone alone

All changes bundled

the other changes must keep working

Within the RTO

for example 5 minutes (illustrative), the bad change is off and the rest stays active

Each option undoes a release at a different size

Solution	What it is	What it does
Feature Flag	A runtime switch that turns one changed behavior on or off without deploying code	Wraps the new discount logic so that it can be switched off alone while the other changes stay active
Canary/Blue-Green	Canary sends a small share of live traffic to the new release; blue-green keeps two environments and switches traffic between them	Limits who sees a bad release, or switches the whole release back at once
Rollback	Redeploy the previous package so that the previous version runs again	Replaces the whole release with the earlier one, undoing every change in it

A smaller undo costs discipline, a larger one costs data safety

Solution	Gains	Costs
Feature Flag	Targeted undo Reversible activation Switch-off needs no deploy	Flag debt: cleanup after use Test combinations multiply Flag setting is a new dependency
Canary/Blue-Green	Limited rollout Rapid cutover back Limits exposure	Capacity: double temporary capacity Needs good metrics and routing Undoes the whole release, not one change
Rollback	Version return Already available from the pipeline Shortens failed release recovery	Data risk: reversal may be unsafe Also undoes the other changes Database state is not rolled back

Deciding signal: must one change be undone alone, how long may recovery take (the RTO) and did the change write state? A bad change inside a bundled release that must go alone picks the Feature Flag; a whole-release failure that wrote no state picks Rollback; exposure to a small share of users, with routing and metrics in place, picks Canary/Blue-Green. The choice changes when the change cannot be wrapped in a flag or when written state makes a version return unsafe.

Smallest safe move

A Feature Flag around the discount change

- **Scope:** code only: one switch around the changed discount logic; traffic, data and write ownership unchanged
- **Change:** read the flag where the logic is used, new when on and old when off; log its state with the release version
- **Safety net:** golden master in both positions; fail-safe default to the old logic; owner and removal date per flag; no Rollback once data was written
- **Escape route:** switch the flag off; if that fails, redeploy the previous package
- **Deferred:** Canary/Blue-Green when a change cannot be wrapped in a flag or exposure must be limited; Rollback only when no state was written

```
# illustrative flag setting, read at runtime
discount.newLogic = on     # off returns the old discount logic
```

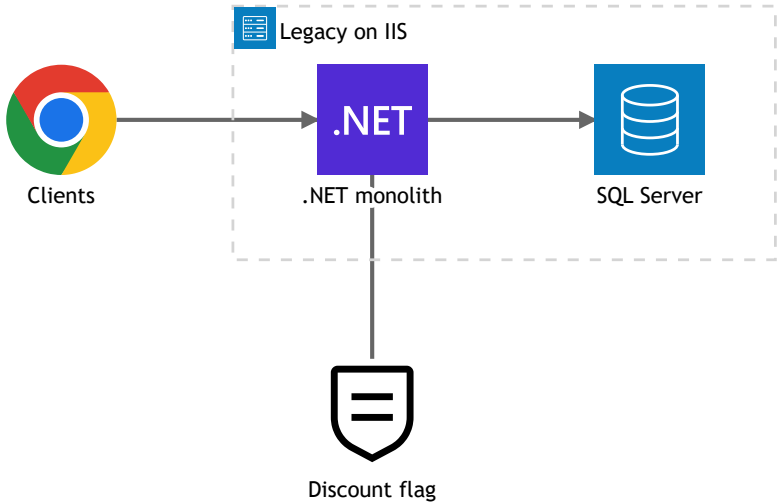
Verification: a failed release switched off in time

Check	Pass condition
Failed release injected in staging: the changed discount logic returns wrong amounts	Golden master tests fail on the discount outputs and on nothing else
Flag switched off in staging	Outputs match the golden master baseline again without a deploy, within the RTO (for example 5 minutes)
Other changes of the same release after the switch-off	Still active, and their tests pass
Orders priced while the flag was on	Listed from the flag state and release version in the logs; none lost, each one marked for review
Fitness function	A pipeline test runs the golden master in both positions; a scheduled drill fails when the switch-off exceeds the RTO; a stale flag report fails past the removal date

Rollback: routing rollback: switching the flag off returns the old discount logic to new requests at once and the other changes stay. State rollback: orders already priced with the bad discount keep their amounts, so they are found from the logs and corrected by a forward fix or reconciliation; redeploying the previous package would not restore them either.

Stop: when the bad discount change can be disabled alone within the RTO and the other changes keep working, no canary, blue-green or routing layer is added for this need.

Solution architecture



The monolith reads one discount flag at runtime to choose between the old and the new discount logic; every other change of the release stays untouched.

Glossary

Used in this step

Term	Definition
Feature Flag	Runtime control enabling behavior without deploying new code
Canary/Blue-Green	Shift a small share of traffic to the new release, or switch between two environments
Rollback	Return to a previously running version or state
Bundled release	One release that ships several changed features together
RTO	Maximum targeted recovery duration after disruption
Fail-safe default	Value a flag takes when its setting cannot be read: the old behavior
Flag debt	Flags left in the code after they are no longer needed
Golden Master	Capture existing observable outputs as compatibility reference
Routing rollback	Switching traffic or a flag back to the previous path
State rollback	Restoring data to its earlier content, which a version return does not do

Related concepts

Term	Definition
Canary Routing	Shift a small controlled segment of live traffic to a new implementation
Blue-Green Deployment	Maintain two release environments and switch traffic
Forward Fix	Repair the new implementation without restoring the old system
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations

Phase 2

Introduce replacement seams

Steps in this phase

Step	Forcing constraint
M08 Intercept clients at boundary	First replacement API must coexist without client updates
M09 Extract business logic seam	New endpoint must reuse pricing behavior
M10 Replace internal dependency gradually	Cannot change all callers at once
M11 Centralize database access	Schema change touches hundreds of files
M12 Isolate vendor model	ERP provider will be replaced
M13 Preserve old schema consumers	Schema rename required but consumers cannot move together
M14 Extract domain modules first	Changes cross order/inventory/pricing files daily

M08 · Introduce replacement seams

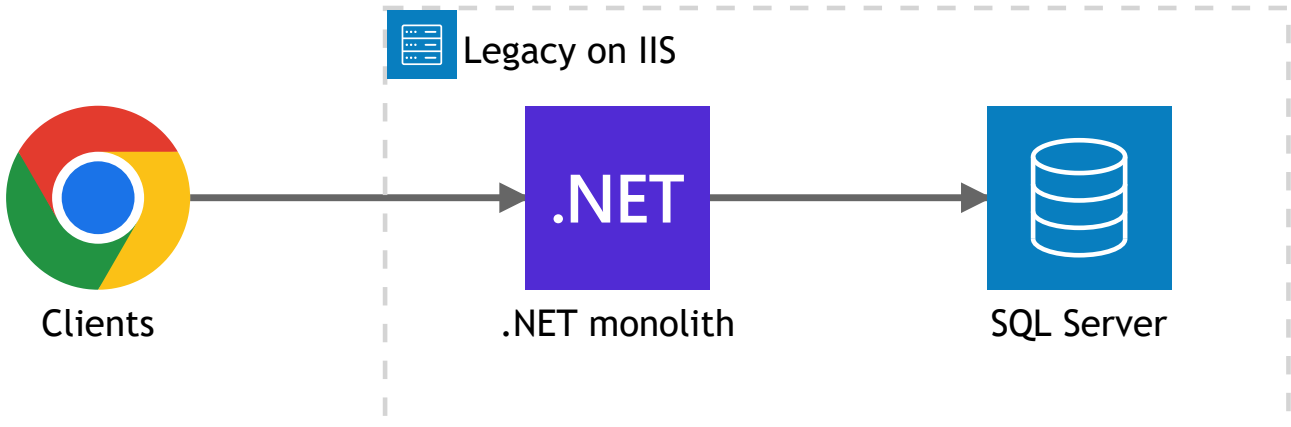
Intercept clients at the boundary

The first replacement API must coexist with the legacy application without a single client update.

Clients call the legacy monolith directly

Component	What it is	What it does
Clients	Anything that calls the application over HTTP	Call legacy URLs directly
Legacy monolith	12-year-old .NET Framework application	Serves the UI and APIs; runs Catalog, Pricing, Orders, Inventory, Fulfillment, Billing and ERP Integration
IIS	Microsoft web server that hosts the application	Terminates HTTP and runs the monolith
SQL Server	One shared relational database	Stores the data of every module
Feature flags	Runtime switches for single behaviors	Turn one change off without undoing the others

Current architecture



A replacement API with nobody to call it

The first replacement catalog read API is ready, yet every client calls legacy URLs directly and cannot be released with it. One endpoint has to move without any client noticing.

No seam

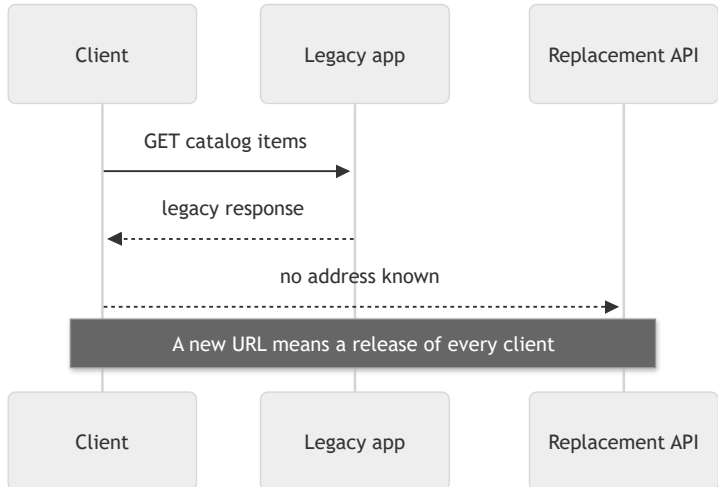
clients call legacy URLs directly, so one endpoint cannot move alone

0 client updates

allowed in the same release as the replacement

1 safe read endpoint

served by the replacement, then returned to legacy



Each option decides where old and new meet

Solution	What it is	What it does
Routing Façade + Reverse Proxy	A stable public entry point that forwards each route to the old or the new system	Keeps every client URL unchanged and moves one route at a time by flag or rule
API Gateway	A central entry point for routing, authentication, rate limiting and API policies	Routes like a proxy and also enforces cross-cutting policies for all clients
Client-side switch	Each client is changed to call the replacement URL directly	Sends requests to the replacement without an intermediary

Control over routing costs a new dependency

Solution	Gains	Costs
Routing Façade + Reverse Proxy	Stable URL for every client One switch per route Cheap: forwarding rules only	New critical dependency in front of everything Routing rules accumulate and need an owner One extra network hop
API Gateway	Governance: one place for authentication, rate limits and policies Routing and policy together	More complexity than the first move needs Legacy policies must be re-expressed first Gateway ownership and release process to set up
Client-side switch	No extra hop No new component to operate	Every client needs a release, which the constraint forbids No single switch back Old clients stay on legacy

Deciding signal: client-facing policies the replacement must enforce, east-west traffic needs and operational capacity. A routing-only need picks the proxy; policy that legacy lacks picks the gateway; a client release fails the constraint.

Smallest safe move

A routing proxy in front of legacy

- **Scope:** traffic only; legacy code, database and clients stay unchanged
- **Change:** route 100% of traffic to legacy through the proxy, then move one catalog GET route to the replacement by flag or rule
- **Safety net:** health checks and a fallback route to legacy; every façade rule has an owner and a removal trigger, so the compatibility layer does not become migration debt
- **Escape route:** switch the route flag back to legacy; clients never saw another URL
- **Deferred:** API Gateway until clients need authentication or rate limiting that proxy and legacy lack; Client-side switch only for a client that is released anyway

```
location /catalog/items { proxy_pass http://replacement; }   # moved by flag or rule
location /              { proxy_pass http://legacy; }        # everything else stays
```

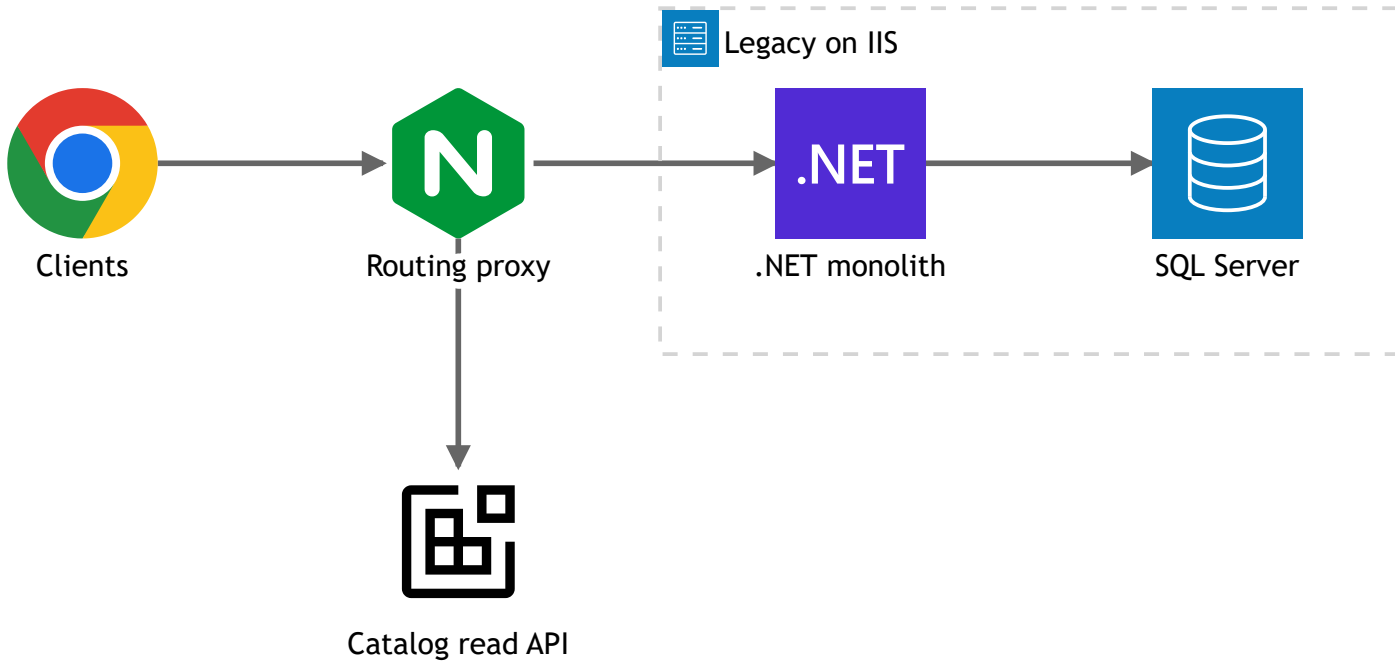
Verification: one route out and back

Check	Pass condition
Proxy with 100% to legacy	Sampled responses identical to direct calls; added latency recorded against the baseline
One read route moved to the replacement	Same URL, same contract, 0 client changes
Replacement made unhealthy on purpose	Health check fails; proxy serves legacy with no client-visible errors
Flag switched back	All traffic on legacy after one configuration change, no failed requests
Fitness function	A pipeline smoke test calls each routed public URL, asserts the target and that the flag restores legacy; proxy error rate is alerted

Rollback: routing rollback only: the flag returns the route to legacy at once. No state rollback is needed, because only a read endpoint moved and nothing was written.

Stop: when clients keep their URLs and the proxy meets the routing need, no gateway or further layer is added.

Solution architecture



Clients keep one URL. The routing proxy decides per route whether legacy or the catalog read API answers.

Glossary

Used in this step

Term	Definition
Legacy monolith	The 12-year-old .NET application on IIS that every client calls today
Seam	A place where behavior can be switched between old and new without editing callers
Routing Façade	Stable public entrypoint routing selected capabilities to old or new system
Reverse Proxy	Intermediary that forwards client requests to backend services
API Gateway	Central entry point for routing, authentication, rate limiting, and API policies
Replacement API	The new implementation of one capability that runs beside legacy
Routing rule	Configuration that sends one route to legacy or to the replacement
Fallback route	Path that sends traffic back to legacy when the replacement is unhealthy
Health check	Probe that tells the proxy whether a target can take traffic
Compatibility layer	Code or configuration that keeps old clients working while the system changes

Related concepts

Term	Definition
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations
Feature Flag	Runtime control enabling behavior without deploying new code
Canary Routing	Shift a small controlled segment of live traffic to a new implementation
Branch by Abstraction	Introduce stable interface then change the implementation behind it gradually
Expand-Contract	Add compatible representation, migrate consumers, then remove old representation

M09 · Introduce replacement seams

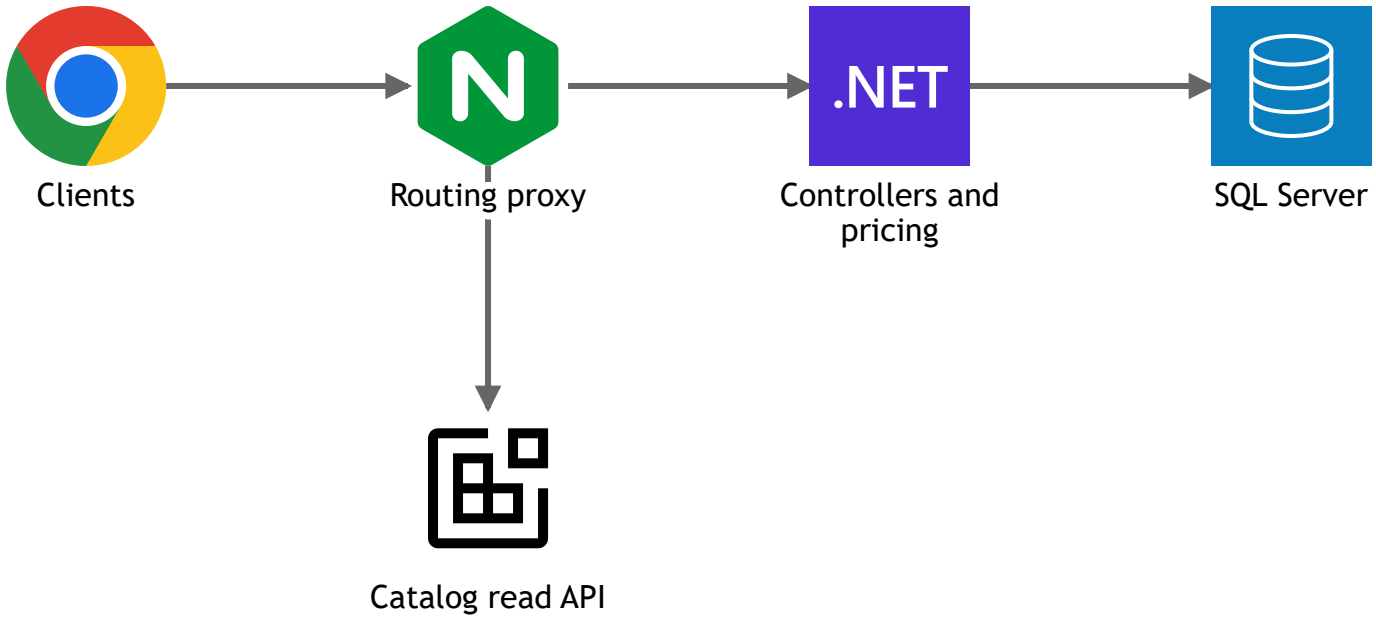
Extract business logic seam

A new endpoint must reuse the pricing behavior that lives inside the controllers, without a second copy of the rules.

Controllers own the pricing rules, so nothing else can call them

Component	What it is	What it does
Clients	Browsers and other callers	Reach the application through the routing proxy
Routing proxy	Public entry point in front of legacy	Sends each route to legacy or to the replacement
Controllers	Web layer classes of the legacy monolith	Handle requests and apply the pricing rules themselves
SQL Server	One shared relational database	Holds prices, discounts and orders
New endpoint	An endpoint of the replacement catalog read API	Must show prices and discounts but cannot call controller code

Current architecture



A new endpoint can only copy undocumented rules

The new endpoint has to show the same prices and discounts as the legacy pages, but the rules live inside controller code. Copying them leaves two versions that drift apart.

Rules inside controllers

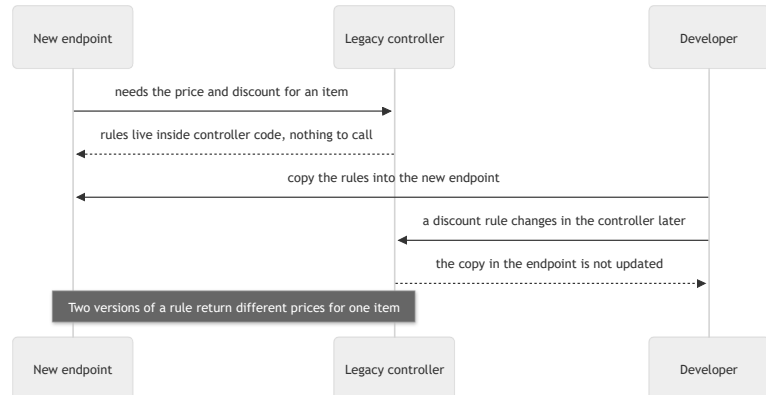
the new endpoint cannot call them

Two callers

the new endpoint and the legacy controllers need the same price and discount

0 unexplained differences

between the facade and the old controller path, on the same pricing tests



Each option decides where the pricing behavior lives

Solution	What it is	What it does
Facade/service extraction	The pricing rules moved out of the controllers into one pricing component with a narrow interface	Every caller, legacy controller or new endpoint, asks the same component for price and discount
Duplicate temporarily	The pricing rules copied into the code of the new endpoint	Lets the new endpoint ship without touching the controllers
Hexagonal Architecture	Business logic isolated from infrastructure through ports and adapters	Puts the pricing core behind ports, with controllers, the new endpoint and the database as adapters

A shared facade costs a refactor, a copy costs drift

Solution	Gains	Costs
Facade/service extraction	One source of pricing behavior New endpoint reuses the rules unchanged One target for the characterization tests	Refactor of controller code Regression risk while rules move Interface must stay stable for callers
Duplicate temporarily	Fast to start Legacy controllers untouched No refactor risk now	Drift: two versions of each rule Every rule fix needed twice Differences show up as wrong prices
Hexagonal Architecture	Ports give clean boundaries Pricing testable without web or database Prepares later extraction	Upfront work Large restructuring for one seam Delays the first reuse

Deciding signal: how many callers need the pricing behavior, how often the rules change and whether a seam preserves behavior on one narrow slice, shown by identical golden master outputs. A second caller of changing rules picks the facade, because a copy drifts; several modules needing isolation from infrastructure pick Hexagonal Architecture. The choice changes at the first difference between a copy and the controller output.

Smallest safe move

Facade/service extraction for one pricing path

- **Scope:** code only: one pricing path moves into a pricing facade; traffic, data and write ownership unchanged
- **Change:** move the calculation unchanged behind one narrow interface; the legacy controller calls it behind a feature flag, then the new endpoint
- **Safety net:** golden master on facade and old path; rules moved, not rewritten; no Duplicate temporarily, as copies drift
- **Escape route:** switch the flag off; the old controller code runs again
- **Deferred:** Hexagonal Architecture when a second capability needs the same isolation or the facade leaks infrastructure types

```
// both callers use the same method; the flag selects the old inline code or the facade
var quote = flags.PricingFacade ? pricing.Quote(order) : LegacyQuote(order);
```

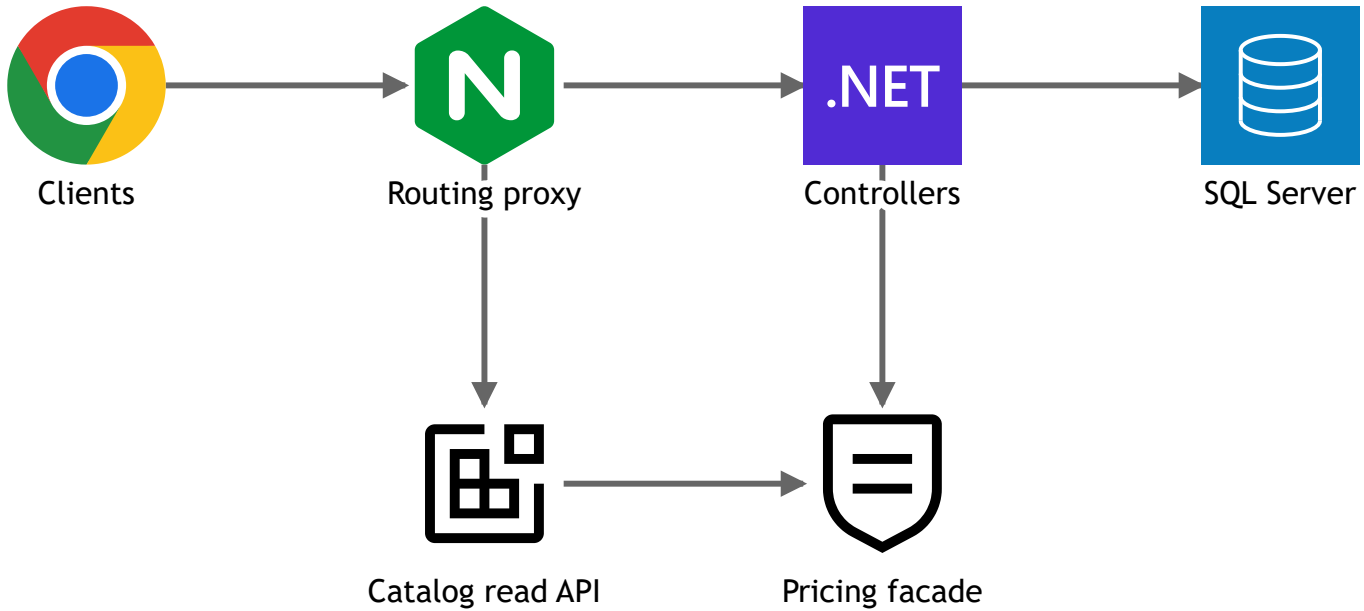
Verification: same tests, same answers

Check	Pass condition
Golden master pricing tests run against the facade	Same outputs as the old controller path for every recorded input; 0 unexplained differences
New endpoint and legacy screen called with the same inputs	Identical price and discount for each input
Flag switched off for the legacy controller	Old inline code serves it again and the outputs match the baseline
Facade path in production compared with the error baseline	Error rate not above the baseline for the release
Fitness function	The golden master pricing suite runs in the pipeline on every change; an architecture test fails when a controller or the new endpoint calculates prices itself

Rollback: routing rollback: switching the flag off returns the legacy controller to its old inline code at once, and the new endpoint stays unreleased or is routed back to legacy by the proxy. State rollback is not needed: the facade only calculates and writes nothing.

Stop: when the legacy controllers and the new endpoint both call one pricing facade and the golden master shows identical outputs, no hexagonal restructuring and no separate pricing service is added.

Solution architecture



The controllers and the new endpoint now call one pricing facade, so price and discount have one source; the controllers keep their old code behind the flag until the golden master matches.

Glossary

Used in this step

Term	Definition
Facade/service extraction	Pricing rules moved behind one narrow interface that every caller uses
Duplicate temporarily	Copying the rules into the new code so that it ships without touching the old
Hexagonal Architecture	Business logic isolated from infrastructure through ports and adapters.
Pricing facade	The one component that calculates price and discount for every caller
Seam	A place where behavior can be switched between old and new without editing callers
Drift	Two copies of one rule that no longer give the same result
Golden Master	Capture existing observable outputs as compatibility reference
Architecture test	Automated check that code follows a structural rule, such as no pricing in controllers
Feature Flag	Runtime control enabling behavior without deploying new code
Routing rollback	Switching traffic or a flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Branch by Abstraction	Introduce stable interface then change the implementation behind it gradually
Characterization Testing	Tests documenting actual legacy behavior before refactoring
Refactor in Place	Improve internal structure without changing external behavior
Modular Monolith	Single deployment with clearly separated internal modules.
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations

M10 · Introduce replacement seams

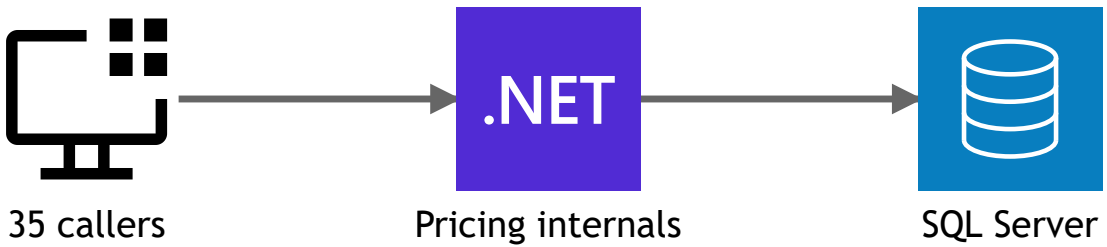
Replace internal dependency gradually

The pricing internals must be replaced, but 35 callers invoke them directly and cannot all be changed at once.

Callers reach straight into the pricing internals

Component	What it is	What it does
35 callers	Code in the monolith that is not behind the facade yet, for example jobs, reports and other modules	Invoke the pricing internals directly
Pricing internals	The classes and methods that calculate prices and discounts	Are called by every caller with no stable interface in between
Pricing facade	The narrow interface for price and discount	Serves the controllers and the new endpoint, but not these callers
SQL Server	One shared relational database	Holds prices, discounts and orders
Feature flags	Runtime switches for single behaviors	Turn one behavior off without undoing the others

Current architecture



A new implementation would break every caller at once

The pricing internals are to be replaced, but 35 callers invoke them directly and cannot all be changed and released together. The new implementation can only arrive if old and new callers work side by side.

Cannot change all callers at once

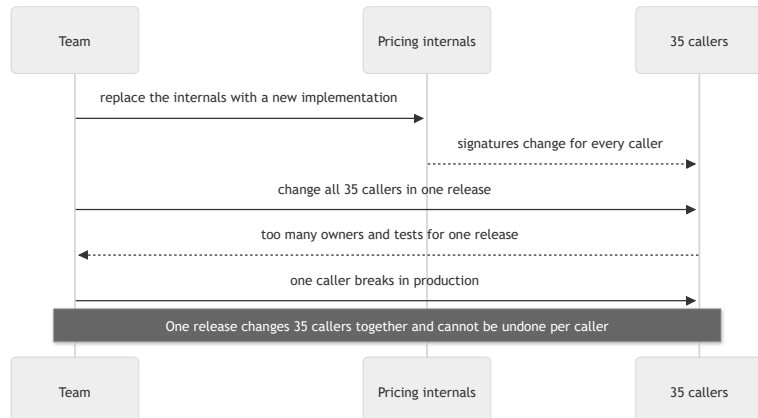
a new implementation would break every caller that invokes the internals

35 callers

invoke pricing internals directly

1 caller

switched to the new implementation and back, with identical outputs and no side effects



Each option decides how callers reach the new implementation

Solution	What it is	What it does
Branch by Abstraction	A stable interface that callers use, with the old and the new implementation behind it	Moves callers to the interface one at a time and switches the implementation behind it by flag
Adapter	A wrapper that offers the old internal calls and forwards them to the new implementation	Lets callers keep their code while the adapter translates each call
Change all callers at once	Every caller is edited to use the new implementation in one release	Removes the old internals in a single cutover

Coexistence costs a transitional layer, a big switch costs control

Solution	Gains	Costs
Branch by Abstraction	Switch implementation under stable API Callers migrate in small batches Each caller can switch back alone	Transitional layer Every caller is still edited once Layer must be retired after the last caller
Adapter	Isolates differences Callers need no edit at first One place for the translation	More code to write and test Mapping needs retirement Callers stay bound to the old internals
Change all callers at once	No transitional layer Fastest end state One release to test	35 callers change and release together One bad caller blocks the release No switch back per caller

Deciding signal: how many callers exist and can they be released together, how many implementations must coexist and how stable is the common behavior? 35 callers on different releases rule out changing them all at once; callers that can be edited pick Branch by Abstraction; callers that cannot be edited at all pick Adapter. The choice changes when so few callers remain that one release can carry them.

Smallest safe move

Branch by Abstraction, one caller first

- **Scope:** code only: one of the 35 callers moves to a stable interface; traffic, data and write ownership unchanged
- **Change:** widen the pricing facade into the interface, add the new implementation beside the old, switch that caller by flag
- **Safety net:** golden master on both implementations; reference count only goes down; no Change all callers at once
- **Escape route:** switch the flag back; each caller reverts on its own
- **Deferred:** Adapter for a caller that cannot be edited; Change all callers at once only when one release can carry the last ones

```
// every migrated caller asks the interface; a per-caller flag picks the implementation behind it
IPricing pricing = flags.UseNew("InvoiceJob") ? new NewPricing() : new LegacyPricing();
```

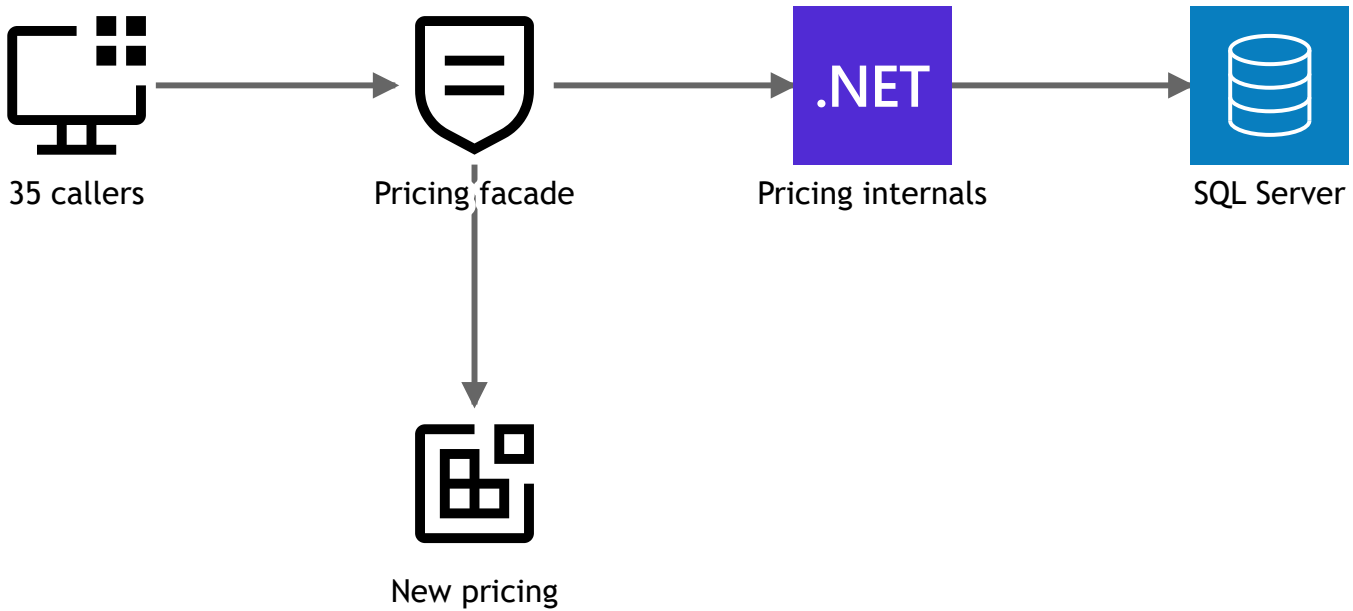
Verification: one caller out and back

Check	Pass condition
Caller inventory of the pricing internals	All 35 callers listed with an owner and the way each one calls the internals
One caller switched to the new implementation through the interface	Golden master outputs for that caller identical to the baseline; 0 unexplained differences
The same caller switched back	The old implementation serves it again; outputs identical and no side effects such as changed rows or audit lines
Switched caller in production compared with the error baseline	Error rate not above the baseline for the release
Fitness function	A ratchet check counts direct references to the internals and fails when the count rises above 35 callers; the golden master suite runs against both implementations in the pipeline

Rollback: routing rollback: switching the flag back returns the caller to the old implementation at once and the interface stays. State rollback is not needed while pricing only calculates; rows written by a switched caller are found from the logs, compared with the golden master and repaired forward.

Stop: when callers move in small separately released batches and each can be switched back, no adapter and no all-at-once change is added; after the last caller has moved, the transitional layer is retired.

Solution architecture



Callers move to the pricing facade one at a time, and a flag picks the old internals or the new pricing behind it; until the last caller has moved, some still call the internals directly.

Glossary

Used in this step

Term	Definition
Branch by Abstraction	Introduce stable interface then change the implementation behind it gradually
Adapter	A wrapper that translates between the interface callers use and the implementation behind it
Change all callers at once	Editing every caller to use the new implementation in one release
Pricing internals	The classes and methods of the pricing module that callers invoke directly today
Stable interface	The one interface callers use while the implementation behind it changes
Transitional layer	Temporary indirection between callers and implementation, retired after the last caller moves
Caller inventory	List of every caller with its owner and the way it calls the internals
Ratchet check	A check that lets a count only go down
Feature Flag	Runtime control enabling behavior without deploying new code
Golden Master	Capture existing observable outputs as compatibility reference
Routing rollback	Switching traffic or a flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Anti-Corruption Layer (ACL)	Protects a domain model from incompatible external system models.
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations
Refactor in Place	Improve internal structure without changing external behavior
Hexagonal Architecture	Business logic isolated from infrastructure through ports and adapters.
Canary Routing	Shift a small controlled segment of live traffic to a new implementation

M11 · Introduce replacement seams

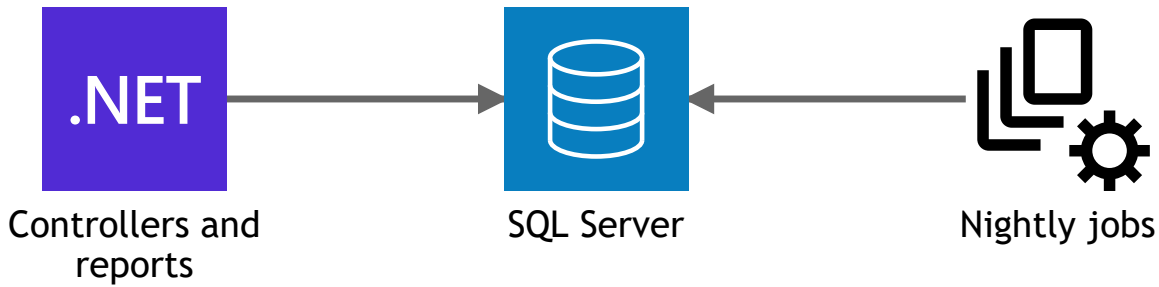
Centralize database access

A schema change would touch hundreds of files, because SQL statements are scattered throughout the code.

SQL statements are scattered through controllers, jobs and reports

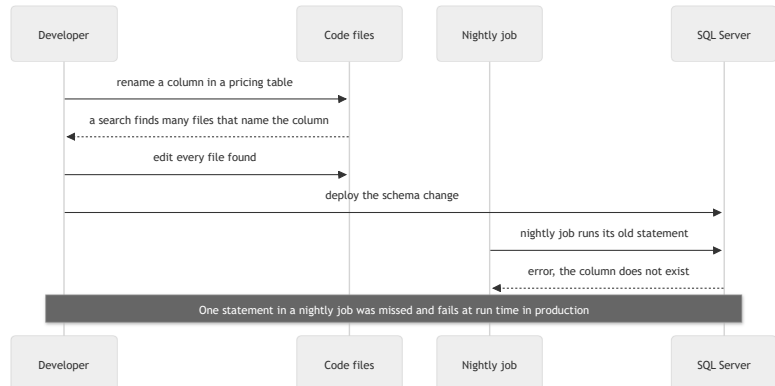
Component	What it is	What it does
SQL statements	Inline queries written wherever the code needs data	Name tables and columns in every file that uses them
Controllers	Web layer classes of the legacy monolith	Issue SQL of their own against the tables
Reports	Report and export code of the monolith	Query the tables with SQL of their own
Jobs	Scheduled background work, such as the nightly runs	Read and write the same tables with SQL of their own
SQL Server	One shared relational database	Holds the tables of every module, among them the pricing tables
Feature flags	Runtime switches for single behaviors	Turn one change off without undoing the others

Current architecture



A missed file fails only in production

A schema change would have to be applied in hundreds of files, because SQL is scattered through controllers, jobs and reports. Nobody can list every statement that touches the tables, so a missed file fails only at run time.



Hundreds of files

change for one schema change, because SQL statements are scattered throughout code

Scattered SQL

controllers, jobs and reports reach the tables directly

Through one repository

a query trace shows every migrated call path reaching the tables there, so a schema change touches only the repository

Each option decides who knows the tables and columns

Solution	What it is	What it does
Repository	A data-access layer with named operations that hides tables and SQL from callers	Callers ask the repository, and only the repository knows tables and columns
Database View	A named query that presents an abstraction over underlying tables	Keeps the old names and shapes readable after the tables change, so existing SQL keeps working
Keep direct SQL	SQL statements stay where they are and each file is edited when the schema changes	Spreads every schema change over all the files that touch the tables

Hiding the tables costs a layer, exposing them costs every schema change

Solution	Gains	Costs
Repository	Contained query layer A schema change touches one layer Call paths can be traced and moved one at a time	Abstraction Moving hundreds of statements takes time Leaky abstraction if tables are exposed
Database View	Preserves the old schema Schema compatibility boundary No application code change at first	Database coupling Can obscure cost and dependencies Hides the scattered SQL, does not remove it
Keep direct SQL	Zero migration No new layer No refactor risk now	Later risk Schema change touches hundreds of files Missed statements fail at run time

Deciding signal: how many files hold SQL for the tables that will change, how often schema and code change together, and who else reads the tables directly. SQL only in the application picks the Repository; readers outside the application, such as reports or exports, that must survive a rename pick the Database View; no planned schema change picks Keep direct SQL. The choice changes at the first schema change that has to ship while direct readers or unmigrated paths still use the old shape.

Smallest safe move

A Repository for one call path

- **Scope:** code only: one call path on the pricing tables; schema, data, traffic and write ownership unchanged
- **Change:** list every statement, copy each unchanged into a repository method, move one call path behind a feature flag, tag each with the repository name
- **Safety net:** verbatim statements checked by golden master and query trace; named operations only; no Keep direct SQL for planned schema changes
- **Escape route:** switch the flag back to the inline SQL
- **Deferred:** Database View when a schema change must ship before every path moves or outside readers need the old shape; Keep direct SQL only without planned schema changes

```
// one method per statement: SQL copied unchanged and tagged for the query trace
decimal PriceOf(int id) ⇒ db.QueryOne<decimal>(
    "/* PriceRepository */ SELECT Amount FROM Prices WHERE ItemId = @id", id);
// a flag picks the repository or the old inline SQL
var amount = flags.PriceRepository ? repo.PriceOf(id) : LegacyPriceSql(id);
```

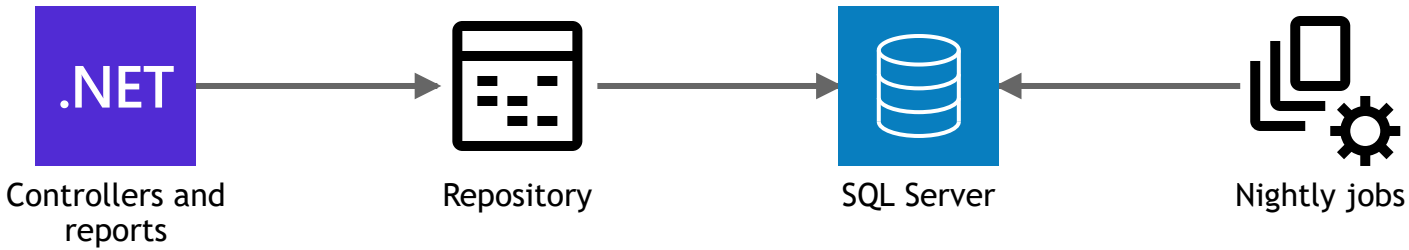
Verification: a query trace shows the migrated path using the repository

Check	Pass condition
Inventory of SQL statements against the chosen tables	Every statement listed with its file; the file count is recorded as the baseline
Query trace of one migrated call path	Every statement for that path is issued by the repository and none by a controller, job or report
Same inputs against the old inline SQL and the repository	Same rows and golden master outputs; query count and duration per request not worse than the measured baseline
Column rename rehearsed in staging for the migrated paths	Only the repository changes for those paths, and their golden master tests pass
Flag switched back for that call path	Inline SQL serves it again and the outputs match
Fitness function	A ratchet check fails when the count of files outside the repository that hold SQL for the chosen tables rises; a staging query trace fails when a migrated path issues SQL from outside the repository; golden master tests run on every change

Rollback: routing rollback: switching the flag back returns the call path to its inline SQL at once. State rollback is not needed: the repository runs the same statements against the same schema, so no schema or data changes in this move.

Stop: when every call path to the chosen tables goes through the repository and a schema change on them touches only the repository, no view layer, separate data service or database split is added.

Solution architecture



A migrated call path now asks the repository, which alone issues its SQL; jobs and other paths that have not moved still reach SQL Server directly.

Glossary

Used in this step

Term	Definition
Repository	A data-access layer with named operations that hides tables and SQL from callers
Database View	Named query presenting an abstraction over underlying tables
Keep direct SQL	Leaving SQL in the code where it is and editing each file when the schema changes
Query trace	Record of the SQL statements a request runs and the code that issued them
Call path	The chain of code from a request to the SQL it runs
Schema change	A rename, split or retyping of tables or columns
Leaky abstraction	An interface that exposes details, such as table names, that it should hide
Ratchet check	A check that lets a count only go down
Feature Flag	Runtime control enabling behavior without deploying new code
Golden Master	Capture existing observable outputs as compatibility reference
Routing rollback	Switching traffic or a flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Expand-Contract	Add compatible representation, migrate consumers, then remove old representation
Branch by Abstraction	Introduce stable interface then change the implementation behind it gradually
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations
Anti-Corruption Layer (ACL)	Protects a domain model from incompatible external system models.
Refactor in Place	Improve internal structure without changing external behavior

M12 · Introduce replacement seams

Isolate vendor model

The ERP provider will be replaced, so a fake ERP client must swap in for the real one with 0 domain code changes.

Order code speaks the ERP provider’s language

Component	What it is	What it does
Order code	The Orders module of the legacy monolith	Builds orders and submits them to the ERP using the provider’s own calls
ERP provider	The external system that receives orders	Accepts submissions and answers with its own status codes
Vendor model	The provider’s own types, field names and status codes	Appears in many order files, not in one place
Golden master tests	Recorded outputs of the legacy application	Show when order output changes
Feature flags	Runtime switches for single behaviors	Turn one change off without undoing the others

Current architecture



A provider swap would reopen every order file

Order code calls the ERP provider directly and uses its types, field names and status codes everywhere. When the provider is replaced, each of those files has to change while orders keep flowing.

Vendor model everywhere

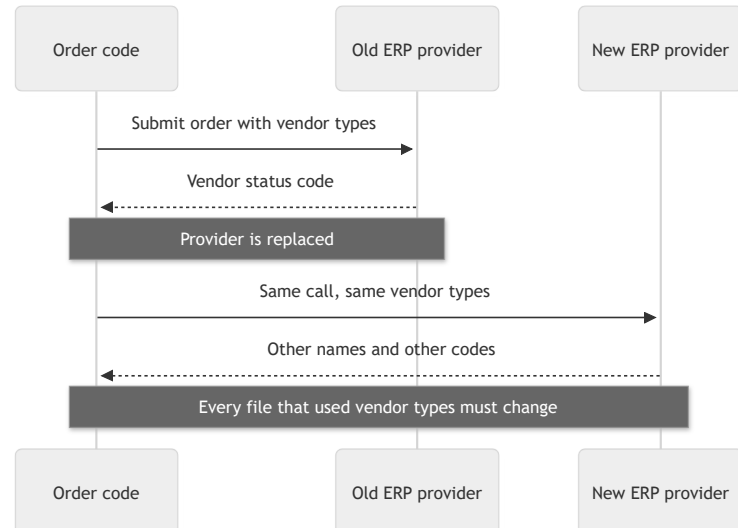
types, field names and status codes throughout order code

Provider replaced

while orders keep flowing to the ERP

0 domain code changes

when a fake ERP client replaces the real one in a test



Each option decides where the vendor model stops

Solution	What it is	What it does
Anti-Corruption Layer	A translation layer with its own model between the order domain and the ERP provider	Maps every vendor concept to a domain concept so the order model never sees the vendor’s
Ports & Adapters	An interface owned by the order domain with one swappable adapter per provider	Order code calls the port; an adapter talks to the current provider and a fake adapter serves tests
Direct adapter	A thin wrapper class around the vendor client that hands vendor types to callers	Puts every ERP call in one class without changing the types the callers see

A cheap wrapper keeps the leak, a layer costs mapping

Solution	Gains	Costs
Anti-Corruption Layer	Protects the order model One place for all vendor mapping	Mapping to build and maintain Heavier than a plain interface Mapping needs retirement later
Ports & Adapters	Swappable implementation Fake adapter for fast tests Domain owns the contract	Interface design effort Temporary indirection One adapter per provider
Direct adapter	Cheapest to write No interface to design	Vendor types still leak into order code Swap still changes domain files Fails when the provider changes

Deciding signal: order files that reference vendor types, and whether the new provider differs in meaning or only in names. Names and formats only: a port with one adapter is enough. The same concept with another lifecycle, for example a status: an Anti-Corruption Layer. Any vendor type left outside the adapter means a swap still changes domain code, so Direct adapter fails.

Smallest safe move

Ports & Adapters on one ERP call

- **Scope:** code only; one port, one adapter, one fake; traffic, data and write ownership unchanged
- **Change:** define the port in order terms, map vendor types in the adapter, move call sites behind a flag one by one
- **Safety net:** direct call kept as fallback until payloads match; Direct adapter is never used, since vendor types would leak; port and adapter get an owner and a retirement trigger
- **Escape route:** flip the flag back to the direct call
- **Deferred:** Anti-Corruption Layer when the new provider gives a concept another meaning or several modules need the translation; Direct adapter is not brought back

```
public interface IErpOrders { void Submit(OrderRequest order); }  // port: domain types only
public class VendorErpOrders : IErpOrders { /* maps vendor types, calls the provider */ }
public class FakeErpOrders : IErpOrders { /* keeps submitted orders in memory */ }
```

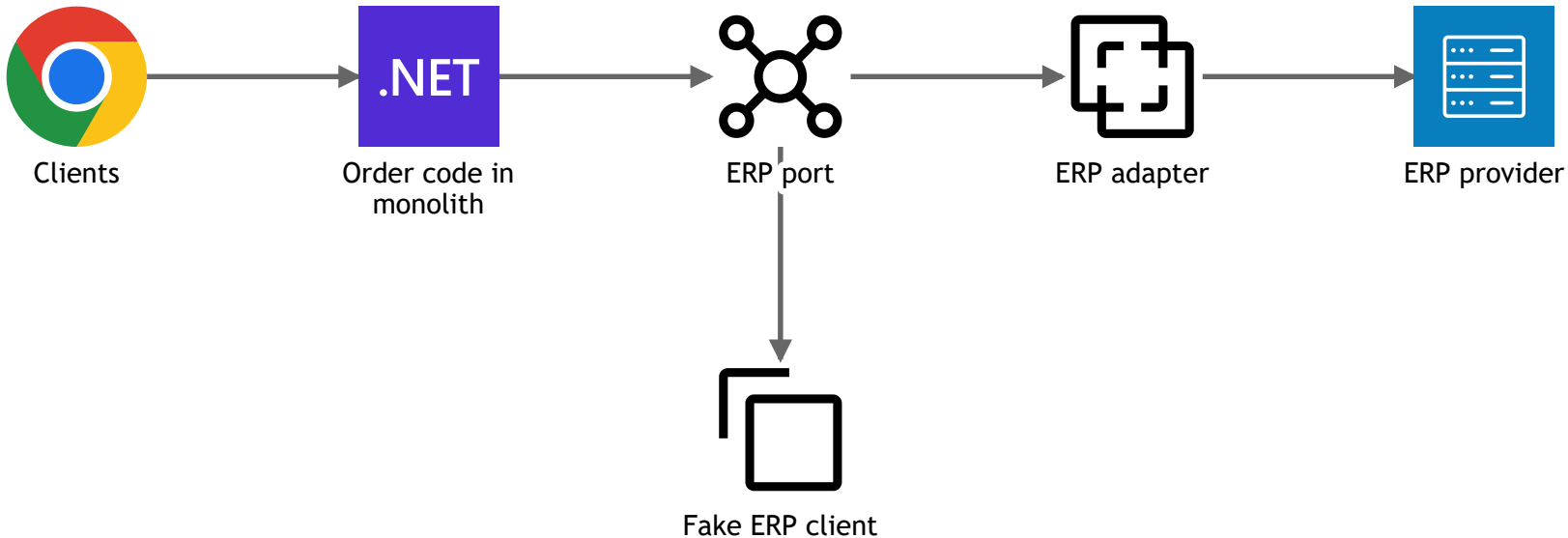
Verification: a fake client swaps in

Check	Pass condition
Dependency scan of order code	0 references to the vendor namespace outside the adapter
Order tests run against the fake ERP client	The same tests pass with 0 domain code changes
Sampled orders through the adapter versus the old direct call	ERP payload identical for every sampled order; golden master diff empty
Flag switched back to the direct call	Orders reach the ERP as before, no failed submissions
Fitness function	A dependency rule fails the build when order code references the vendor namespace; the fake-adapter tests run on every build

Rollback: routing rollback: the flag returns each call site to the direct vendor call at once. State rollback is not needed for the seam itself, because no data moved; if a wrong mapping already sent bad payloads to the ERP, repair forward by re-sending those orders and reconciling them with the ERP.

Stop: when order code references only the port and the fake client swaps in without domain changes, no Anti-Corruption Layer, second adapter or integration platform is added until the new provider is chosen.

Solution architecture



Order code now calls the ERP port in domain terms. The adapter holds the vendor model for the current provider, and the fake client proves that a provider swap leaves the domain untouched.

Glossary

Used in this step

Term	Definition
Anti-Corruption Layer	Protects a domain model from incompatible external system models.
Ports & Adapters	A domain-owned interface with one swappable implementation per outside system
Direct adapter	A thin wrapper that passes the vendor's own types through to callers
Vendor model	The provider's own types, field names and status codes
Port	The interface the order domain defines for what it needs from the ERP
Adapter	Code behind a port that talks to one specific provider
Fake ERP client	A test double that stands in for the ERP provider
Seam	A place where behavior can be switched between old and new without editing callers
Dependency rule	A build check that fails when code references a forbidden namespace
Golden Master	Capture existing observable outputs as compatibility reference
Fitness function	Measurable or executable architectural quality rule

Related concepts

Term	Definition
Branch by Abstraction	Introduce stable interface then change the implementation behind it gradually
Domain Translation Adapter	Translate one system's terms and records into another's semantics
Hexagonal Architecture	Business logic isolated from infrastructure through ports and adapters.
Bounded Context	Boundary inside which a domain model and vocabulary have specific meaning
Feature Flag	Runtime control enabling behavior without deploying new code

M13 · Introduce replacement seams

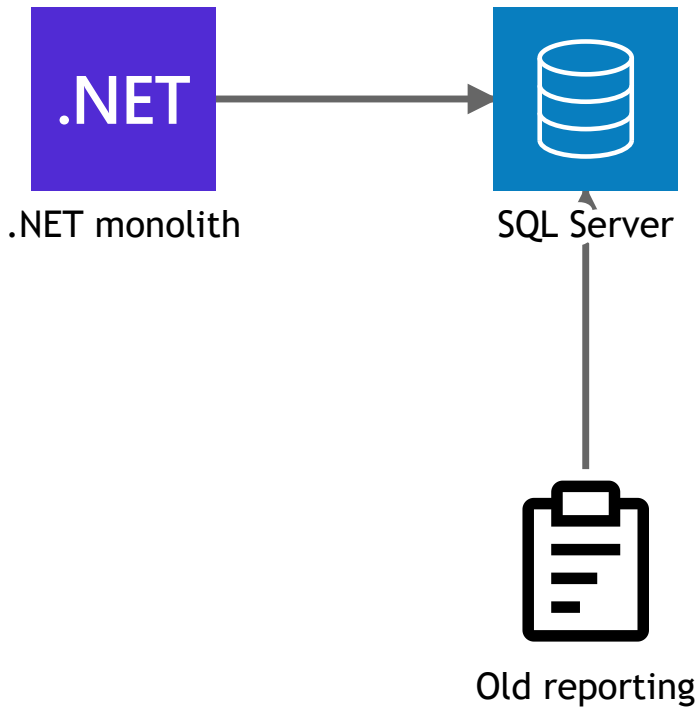
Preserve old schema consumers

A schema rename is required, but the consumers cannot move together, so old and new reads must work in the same release.

Old reports read the Customer fields by name

Component	What it is	What it does
Legacy application	The .NET monolith with one centralized data access layer	Reads and writes the Customer fields through that layer
SQL Server	One shared relational database	Holds the Customer table that every module and report reads
Customer fields	The columns that carry customer data under today’s names	Are referenced by name in application code and in reports
Old reporting	Reports and queries written against those field names	Read the table directly and are released on their own schedule
Golden master tests	Recorded outputs of the legacy application	Show when a result changes

Current architecture



A rename in one release stops the reports

The Customer fields have to be renamed, but the old reports read them by name and are released separately from the application. A rename in one release would stop the reports while the application already uses the new names.

Rename breaks reports

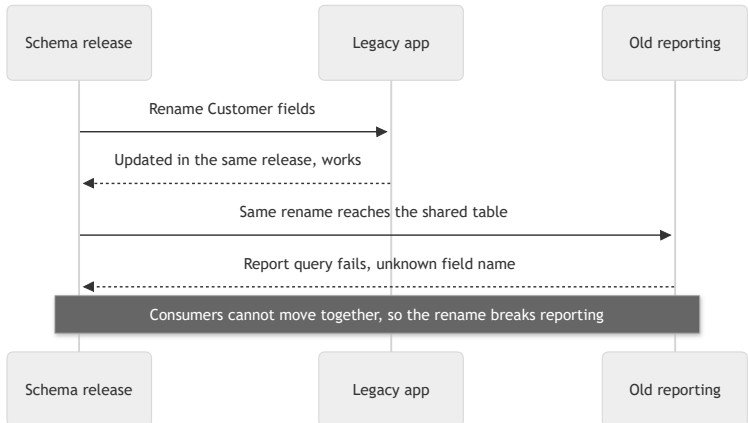
old reporting queries fail on the renamed Customer fields

Consumers apart

the application and old reporting read the same fields and cannot release together

Same release

old and new reads both work



Each option keeps old readers alive in a different way

Solution	What it is	What it does
Expand-Contract	Add the new field names beside the old ones, migrate consumers, then remove the old names	Keeps both names readable until the last old consumer has moved, then drops the old columns
Compatibility View	A database view that presents the old table and field names over the renamed schema	Old SQL keeps running against the view while the tables take their new names
Immediate rename	Rename the fields in a single deployment	Gives the schema its final names at once

Compatibility costs temporary fields, breaking costs every report

Solution	Gains	Costs
Expand-Contract	Compatibility for every consumer Consumers move one at a time Each phase can be undone	Temporary duplicated fields Both columns written until the sunset Old columns need an explicit sunset
Compatibility View	Old SQL survives unchanged Tables can be renamed freely	View needs upkeep with every schema change Can hide query cost and dependencies Old writers need extra work
Immediate rename	No duplicated fields Final names from day one	Breaks old reporting at once All consumers must release together No way back without restoring data

Deciding signal: unmigrated consumers and how each reads the fields: by name, by SELECT-star, by saved report, by export. Readers of renamed columns on a table that keeps its name: Expand-Contract. Consumers that need the old table name or shape, or cannot be changed: Compatibility View. Only when every consumer can release together is an Immediate rename safe; the choice changes when the usage audit finds a reader nobody owns.

Smallest safe move

Expand-Contract on the Customer fields

- **Scope:** data and code; expand phase only; old reporting stays on the old names; traffic and write ownership unchanged
- **Change:** add the new Customer columns, run a Data Backfill, write both in one transaction, move the application to the new names
- **Safety net:** old columns stay authoritative until a sunset with an owner and a date; Immediate rename is not used
- **Escape route:** deploy the previous application version; the new columns sit unused
- **Deferred:** Compatibility View when a table itself must change or a reader cannot; Immediate rename only if every consumer releases together; contract phase only after the usage audit shows no reader

```
ALTER TABLE Customer ADD FullName nvarchar(200) NULL;           -- expand: new name beside the old
UPDATE Customer SET FullName = CustName WHERE FullName IS NULL;  -- backfill from the old column
-- contract, only after the usage audit: ALTER TABLE Customer DROP COLUMN CustName;
```

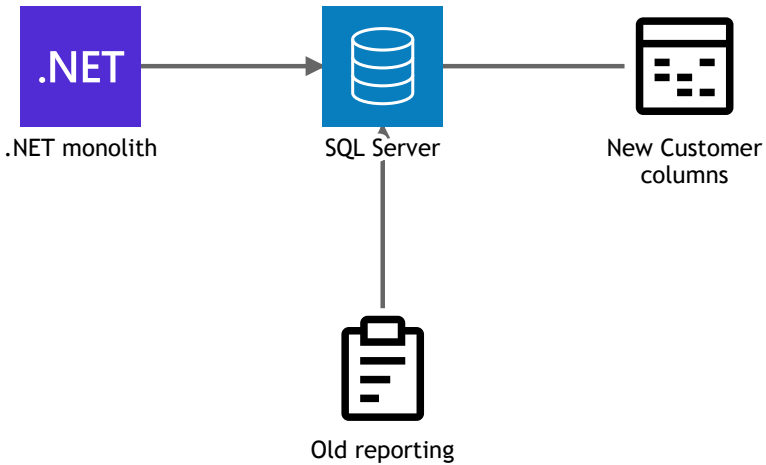
Verification: old and new reads in one release

Check	Pass condition
Old reporting queries on the expanded schema	Result sets identical to those before the change
Previous application version on the expanded schema	Golden master tests pass unchanged; old queries and inserts still succeed
New application version writes the new names	Old and new columns hold the same value for every sampled row after writes
Usage audit before any old column is dropped	Zero reads of the old names over one full reporting cycle
Fitness function	Every schema change runs the old and new queries on a staging copy and fails on any difference; an alert fires when an old column passes its sunset date or still shows reads

Rollback: routing rollback: deploy the previous application version, which still uses the old names. State rollback: the expand phase only adds columns and drops nothing, so there is nothing to restore; if a bad backfill leaves the columns different, repair forward by re-running the backfill from the old authoritative column, never by dropping it.

Stop: when old reporting and the new application both read the expanded schema in the same release, no view, table rename or column removal is added until the usage audit shows the old names are unused.

Solution architecture



Both name sets now exist in the Customer table. The application moves to the new names, old reporting keeps reading the old ones, and the old columns go only after the usage audit.

Glossary

Used in this step

Term	Definition
Expand-Contract	Add compatible representation, migrate consumers, then remove old representation
Compatibility View	Database view presenting old shape over new data
Immediate rename	Rename a field in one deployment, so every reader must change at once
Consumer	An application, report or export that reads a table by field name
Expand phase	The step that adds the new names while the old ones keep working
Contract phase	The step that removes the old names after the last reader has moved
Sunset	Agreed date and owner for removing an old column or view
Data Backfill	Populate a new representation using already-stored historical data
Usage audit	A check of which queries and reports still read the old names
Golden Master	Capture existing observable outputs as compatibility reference
Fitness function	Measurable or executable architectural quality rule

Related concepts

Term	Definition
Consumer-Driven Contract Test	Consumer expectations verified against a provider
Branch by Abstraction	Introduce stable interface then change the implementation behind it gradually
Feature Flag	Runtime control enabling behavior without deploying new code
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence

M14 · Introduce replacement seams

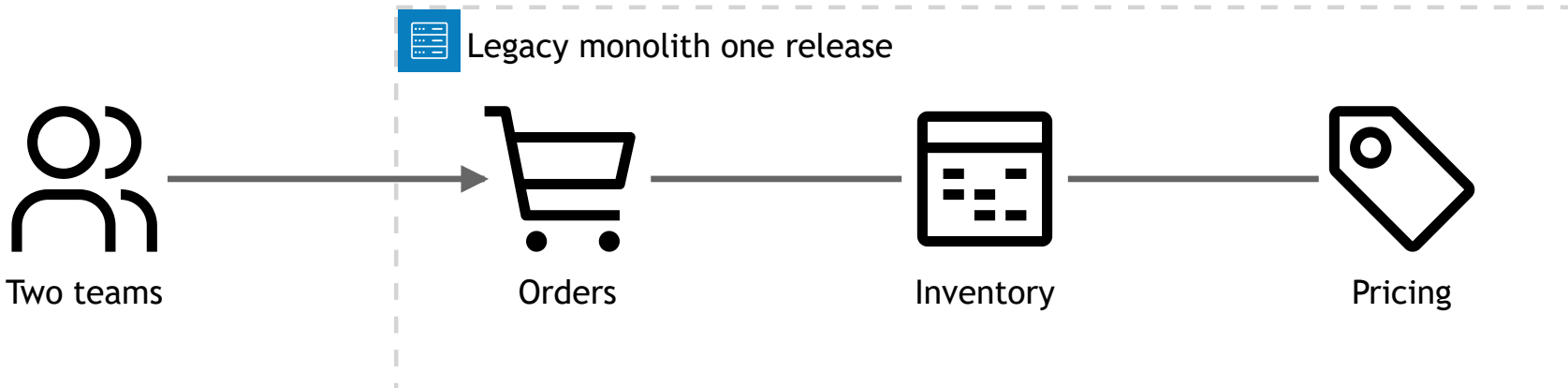
Extract domain modules first

Changes cross order, inventory and pricing files daily, so import boundary checks must run on every build and fail any new cross-module import.

One large codebase, two teams, three tangled modules

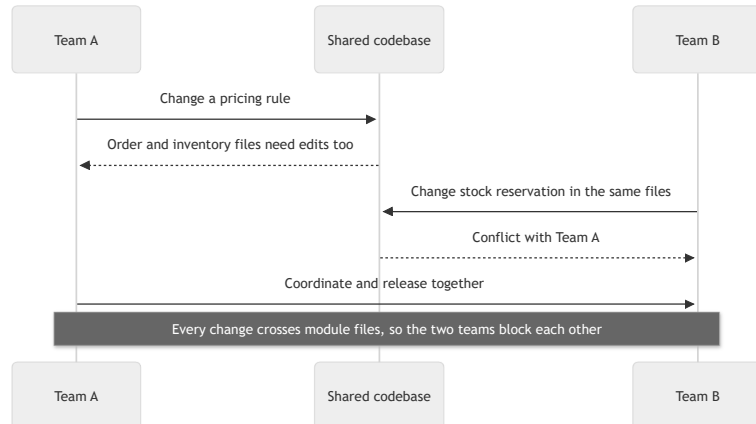
Component	What it is	What it does
Legacy monolith	One large .NET codebase deployed as a single unit on IIS	Runs all seven modules in one release
Two teams	The people who own the codebase	Change Orders, Inventory and Pricing files in the same release
Orders, Inventory, Pricing	Three modules that import each other’s code freely	Change together: one change edits files in all three
Pricing entry	The stable entry to pricing from the earlier seam	Is the one module boundary that exists today
Golden master tests	Recorded outputs of the legacy application	Show when behavior changes

Current architecture



Every change drags three modules along

Two teams share one large codebase in which Orders, Inventory and Pricing code import each other freely, so almost every change edits files of all three. The teams block each other in reviews and releases.



Daily cross-module changes

changes cross order, inventory and pricing files

Two teams, one release

one large codebase shipped as one unit

Boundary checks

run on every build and fail any new cross-module import

Each option draws the module boundary in a different place

Solution	What it is	What it does
Modular Monolith + DDD	One deployable with explicit domain modules, named owners and public interfaces	Keeps in-process calls while each team changes its own module without editing the others
Package extraction	Move the code of a module into a separately versioned package	Other modules use the package version instead of the source files
Immediate Microservices	Split each module into its own deployable service now	Gives each module its own deployment and runtime

Boundaries stay cheap in one release, autonomy costs distribution

Solution	Gains	Costs
Modular Monolith + DDD	Clear module boundaries In-process calls stay cheap No distributed failure modes	Shared release stays Boundary design needs domain collaboration The check and its baseline need an owner
Package extraction	Small code change Explicit versioned interface	Dependencies and version bumps to manage A cross-module change becomes two releases Import rules still unchecked
Immediate Microservices	Independent deployment Team autonomy	Distributed failure and latency risk Operations burden for several services Cross-module rules need new mechanisms

Deciding signal: coordinated change frequency, independent deployment need, operations skill and cross-domain invariants; measure changes touching more than one module and imports across boundaries. Daily cross-module changes on one release cadence: Modular Monolith + DDD. A module a second application needs: Package extraction. Immediate Microservices only for a boundary that needs independent deployment, keeps its invariants inside and can be run by operations.

Smallest safe move

Modular Monolith + DDD, enforced by the build

- **Scope:** code only; one deployment, no traffic or data change
- **Change:** name Orders, Inventory and Pricing as bounded contexts with owners; record the import baseline; fail the build on new imports outside a public interface
- **Safety net:** only new imports are blocked, so behavior is unchanged; Immediate Microservices is not used, since calls would become network calls; baseline entries get an owner and a removal trigger
- **Escape route:** set the check to warning only or revert
- **Deferred:** Package extraction when a second application needs a module; Immediate Microservices when a boundary needs independent deployment and keeps its invariants inside

```
rule: Orders may call Pricing only through the Pricing public interface
baseline: today's cross-module imports are listed and may only shrink
```

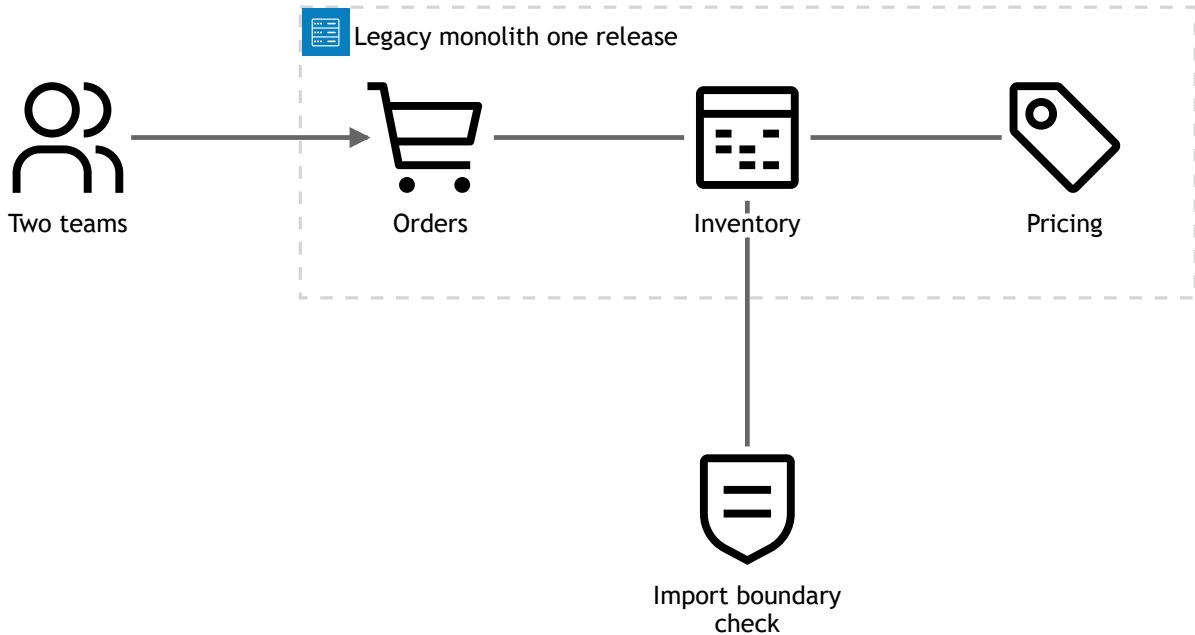
Verification: boundaries enforced by the build

Check	Pass condition
Baseline of cross-module imports among Orders, Inventory and Pricing	Count and list recorded; the count never rises on any build
Forbidden import added on purpose in a test branch	Build fails and names the importing file and the module
Change confined to one module, for example a pricing rule	Needs edits only inside that module’s files and its tests pass
Golden master tests and the deployment	Outputs identical to before; the build still deploys as one unit
Fitness function	Boundary rules run on every build, new imports fail it and the baseline may only fall; the share of changes touching more than one module is tracked per release

Rollback: routing rollback is not needed because no traffic path changed: revert the commit or set the check to warning only, and the same deployment runs as before. State rollback is not needed because no data or write ownership moved.

Stop: when the check blocks new cross-module imports and changes stay inside one module, no package extraction or service split is added until a boundary needs independent deployment.

Solution architecture



Still one deployment with in-process calls. The build now blocks any new import across module boundaries, so each team changes its own module without editing the others.

Glossary

Used in this step

Term	Definition
Modular Monolith + DDD	Single deployment with clearly separated internal modules aligned to business concepts
Package extraction	Moving a module's code into a separately versioned package
Immediate Microservices	Splitting modules into independently deployable services right away
Monolith	Entire application deployed as a single unit.
Bounded Context	Boundary inside which a domain model and vocabulary have specific meaning
Domain module	A part of the codebase with one owner and a public interface
Public interface	The few types a module lets other modules call
Import boundary check	A build rule that fails when code imports another module's internals
Import baseline	Cross-module imports allowed today; the list may only shrink
Coordinated change	A change that must edit files of more than one module
Golden Master	Capture existing observable outputs as compatibility reference
Fitness function	Measurable or executable architectural quality rule

Related concepts

Term	Definition
Microservices	Independently deployable services organized around business capabilities.
Hexagonal Architecture	Business logic isolated from infrastructure through ports and adapters.
Branch by Abstraction	Introduce stable interface then change the implementation behind it gradually
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations
Database Per Service	Each service owns its private persistence and schema

Phase 3

Read-only strangler extraction

Steps in this phase

Step	Forcing constraint
M15 Add first new read endpoint	New search API required without replacing current UI
M16 Populate read-only replacement	New API must query production products
M17 Compare outputs	Old behavior has undocumented filters and rounding
M18 Shadow production inputs safely	Real data exposes irregularities
M19 Canary read traffic	New catalog must be tested with a small user cohort
M20 Migrate internal catalog consumers	All catalog callers must converge on replacement
M21 Retire old read branch	Duplicate code imposes maintenance

M15 · Read-only strangler extraction

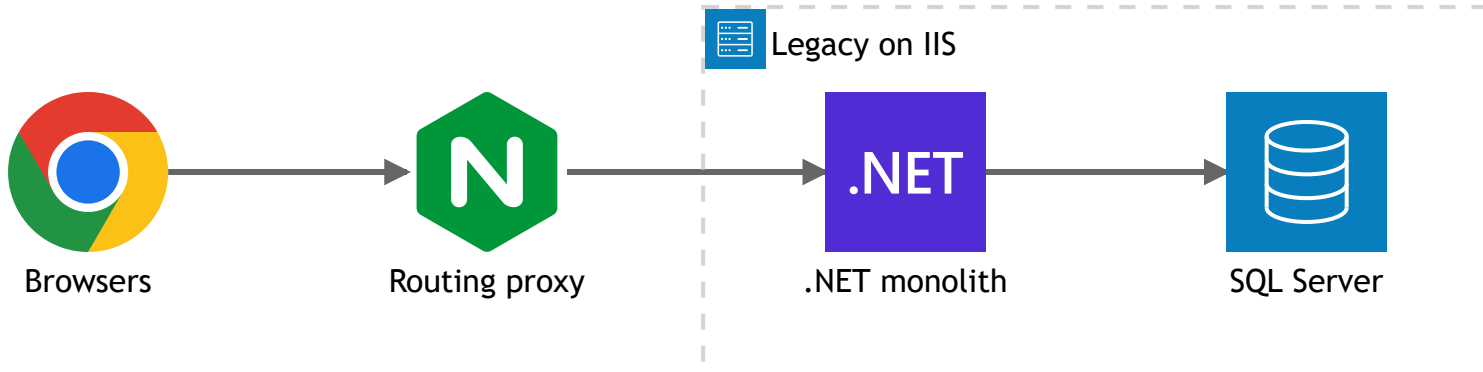
Add first new read endpoint

A new search API is required without replacing the current UI, so 1 new read endpoint must match the old search fixtures.

Every catalog read runs inside legacy

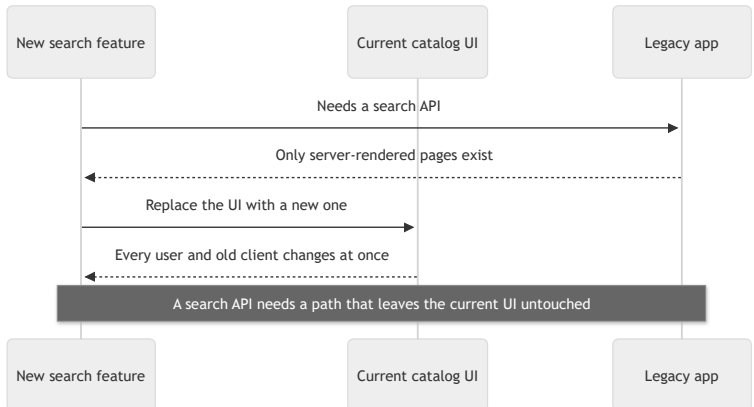
Component	What it is	What it does
Browsers	The users' browsers and the old clients	Call the server-rendered catalog UI at the public URL
Routing proxy	The Routing Façade created earlier, a reverse proxy at the public URL	Forwards every route to legacy; a route can move by flag
Legacy monolith	The .NET application with the Catalog module	Serves every catalog read: search, browse and product pages
SQL Server	One shared relational database	Holds all catalog data
Search fixtures	Recorded search requests with the old results, a golden master	Define what the old search returns

Current architecture



A search API needs a home beside the current UI

A new catalog search API is required, but every catalog read runs inside legacy and its server-rendered UI, and the UI must stay as it is. There is no separate place to add an API that can be released and switched off on its own.



All reads in legacy

no search API exists beside the current UI

Current UI unchanged

it keeps serving users while the API is added

1 new read endpoint

matches the old search fixtures

Each option decides where the search API lives

Solution	What it is	What it does
New Catalog API	A separate read-only service with a search endpoint for the catalog	Answers search requests beside legacy and is reached through the routing proxy on its own route
Monolith new endpoint	A new search action added to the legacy application	Serves search from the same process, code and database as the current UI
Replace the current UI	A new catalog front end built on the new search API that retires the current UI	Moves every search user to the new screens at once

Isolation costs a runtime, simplicity costs a shared release

Solution	Gains	Costs
New Catalog API	Isolated from legacy code Deploys on its own schedule Base for the later catalog extraction	New runtime to operate Proxy route needs an owner No catalog data until it is populated
Monolith new endpoint	Simpler, no new runtime Reuses existing queries and data	Shared release with legacy More code in the monolith Search load and failures shared with legacy
Replace the current UI	Clean new front end One search path	Violates the constraint: the current UI must stay Every user changes at once No fallback to the current UI

Deciding signal: independent deployment need and operations skill: how often the search API must change without a legacy release, how much search load legacy can absorb (p95 against the production evidence baseline) and whether the team can run another runtime. Its own schedule or load isolation: New Catalog API. Neither need and no capacity for a new runtime: Monolith new endpoint. Anything that replaces the current UI fails the constraint.

Smallest safe move

New Catalog API behind a flagged route

- **Scope:** traffic and code; one read-only service, one route; UI, legacy and database unchanged; test data only
- **Change:** deploy the service, add a flagged proxy route for internal testers, run the old search fixtures on it
- **Safety net:** read-only endpoint with no write path, no production data, route off for users; Replace the current UI is never used; owner and removal trigger for the route
- **Escape route:** turn the route flag off; the UI never left legacy
- **Deferred:** Monolith new endpoint when no independent release or runtime capacity exists; Replace the current UI only once results match and users move by route

```
location /api/catalog/search { proxy_pass http://catalog-api; }   # new route, flag-controlled
location /                   { proxy_pass http://legacy; }        # everything else stays
```

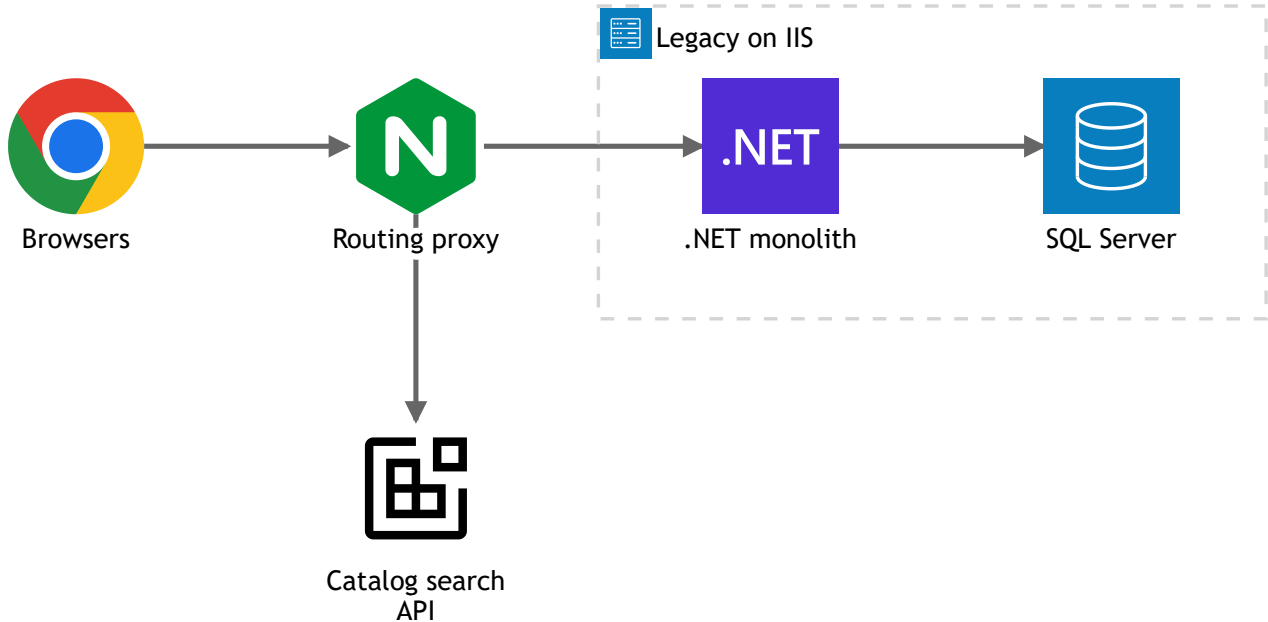
Verification: the old search fixtures on the new endpoint

Check	Pass condition
Old search fixtures run against the new endpoint	Every fixture returns the same items as the recorded old search results
Current UI before and after the proxy route is added	Sampled responses identical, no client changes
New route called through the public URL	Answered by the new API only when the flag is on, never when it is off
Search API deployed twice with no legacy release	Both deployments succeed and the current UI is unaffected
Fitness function	A pipeline contract test runs the old fixtures on every build and fails on any difference; a smoke test asserts which target answers; proxy and API error rates are alerted

Rollback: routing rollback: switch the route flag off or remove the proxy rule at once; users never saw the new API and the current UI kept using legacy. State rollback is not needed: the API is read-only and holds only test data, so nothing was written.

Stop: when the new endpoint matches the old search fixtures and the current UI is unchanged, no second endpoint, gateway or UI replacement is added.

Solution architecture



The current UI path through legacy is unchanged. The routing proxy can also send one new route to the read-only catalog search API, which has no database connection yet.

Glossary

Used in this step

Term	Definition
New Catalog API	A separate read-only service that answers catalog search beside legacy
Monolith new endpoint	A new search action added to the legacy application
Replace the current UI	Retiring the current screens for a new front end in one move
Monolith	Entire application deployed as a single unit.
Routing Façade	Stable public entrypoint routing selected capabilities to old or new system
Route flag	Switch that sends one route to legacy or to the replacement
Read-only endpoint	An endpoint that only reads and exposes no write path
Search fixtures	Recorded search requests with the results the old search returned
Golden Master	Capture existing observable outputs as compatibility reference
Contract test	A test that checks a service against an agreed response shape
Fitness function	Measurable or executable architectural quality rule

Related concepts

Term	Definition
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations
API Gateway	Central entry point for routing, authentication, rate limiting, and API policies
Feature Flag	Runtime control enabling behavior without deploying new code
Differential Testing	Compare outputs of old and new implementations for same input
Canary Routing	Shift a small controlled segment of live traffic to a new implementation

M16 · Read-only strangler extraction

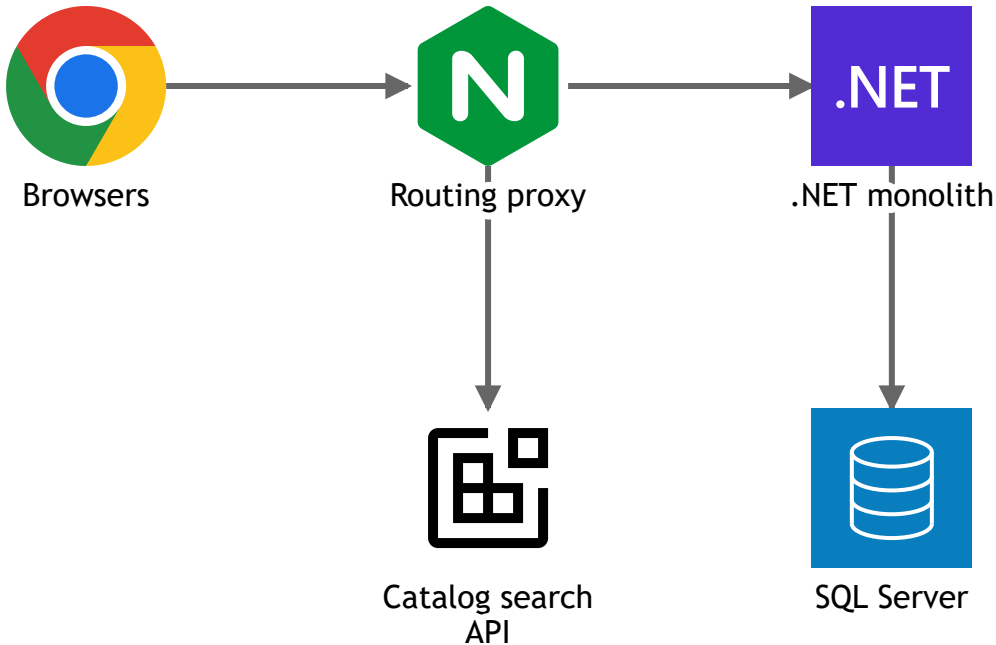
Populate read-only replacement

The new API must query production products, and its read endpoint must expose no write path.

The new endpoint answers, but has nothing to read

Component	What it is	What it does
Catalog search API	The read-only service behind a flagged proxy route	Answers search requests from test data only
Routing proxy	The Routing Façade at the public URL	Sends one new route to the API and everything else to legacy
Legacy monolith	The .NET application with the Catalog module	Stays the write owner of every product
SQL Server	One shared relational database	Holds the only production copy of the products
Search fixtures	Recorded search requests with the old results	Pass on the test data, not yet on production products

Current architecture



Production products live behind legacy

The new search endpoint answers with an empty result because production products live only in the legacy SQL Server database. A way to read them must not become a way to change them or to slow legacy.

No catalog data

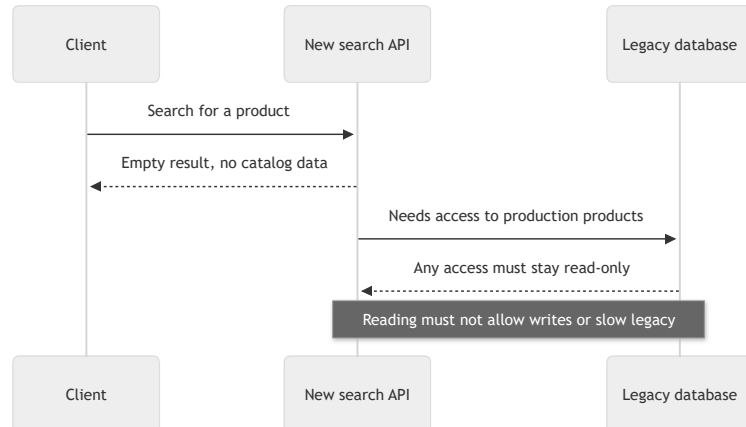
the new endpoint has nothing to query

Legacy still writes

production products live only in the legacy database

No write path

the read endpoint exposes none



Each option gives the replacement data in a different way

Solution	What it is	What it does
Read-only SQL access	A SELECT-only database login on the catalog tables, used by the replacement	The replacement queries production products directly and reads them live
Read Replica	A copy of the legacy database that serves read queries	The replacement queries the copy, so its reads never touch the production primary
ETL copy	A scheduled extract, transform and load of products into the replacement's own store	The replacement queries its own copy, independent of legacy at query time
Shared read-write login	The replacement reuses the legacy application's own database login	Reads production tables and could also write to them

Live reads cost coupling, copies cost lag, shared logins cost control

Solution	Gains	Costs
Read-only SQL access	Fast: nothing to copy or build Always current data	Tightly coupled to the legacy schema Search load hits the production database Login must stay SELECT only
Read Replica	Read isolation from production Same schema, no transformation	Replication lag Replica to run and pay for Still coupled to the legacy schema
ETL copy	Independent of legacy at query time Data can be reshaped for search	Stale between runs Pipeline and mapping to maintain An unowned copy drifts unnoticed
Shared read-write login	Fastest to wire No new login to create	A write path exists Writes bypass legacy rules Two writers for one product

Deciding signal: whether read load threatens legacy (replacement query rate and legacy p95 against the production evidence baseline), how stale the search may be and how tightly it depends on the legacy schema. Within the baseline and live data: Read-only SQL access. Above it, for example 10%: Read Replica, if lag is tolerable. Own schema or no dependence on legacy at query time: ETL copy. A write path fails the invariant.

Smallest safe move

Read-only SQL access with a SELECT-only login

- **Scope:** data and code; one SELECT-only login; legacy code, schema and writers unchanged
- **Change:** create the login on the catalog tables, deny all other grants, set a query timeout and connection limit
- **Safety net:** Least privilege: no write grant, no write route; Shared read-write login is never used; tables read are listed to limit schema coupling
- **Escape route:** flag the route off or disable the login
- **Deferred:** Read Replica when legacy p95 rises above baseline, for example 10%; ETL copy when the API needs its own schema or must not depend on legacy at query time

```
GRANT SELECT ON Catalog.Product TO catalog_search;          -- read only, catalog tables only
DENY INSERT, UPDATE, DELETE, EXECUTE TO catalog_search;     -- no write path
```

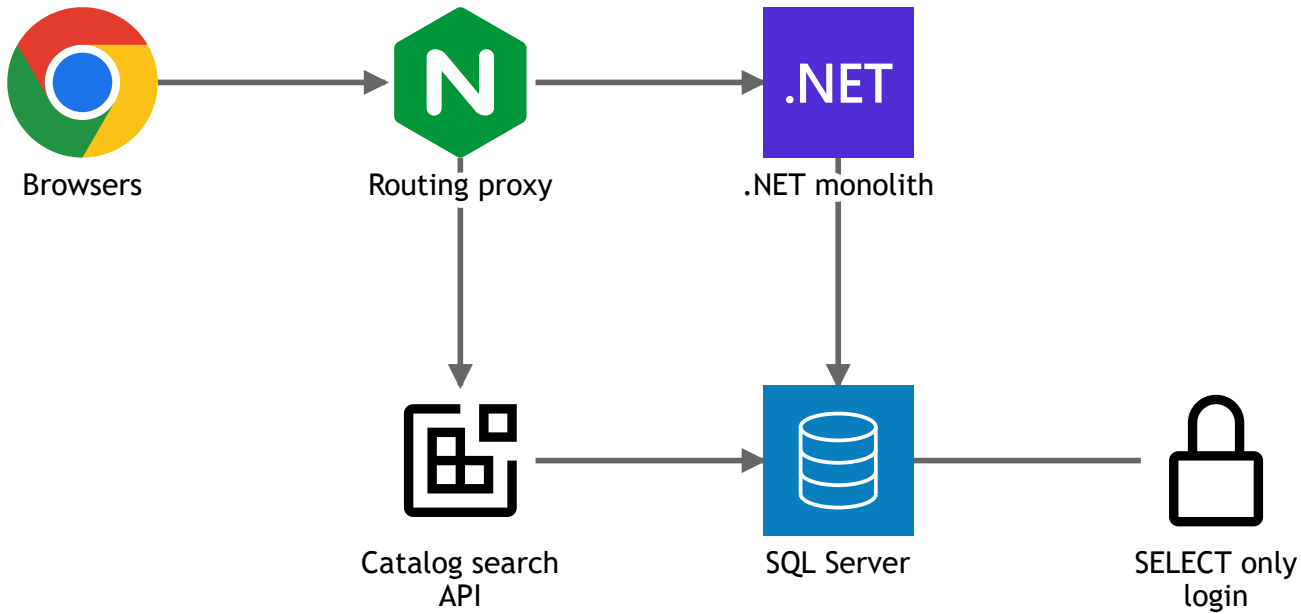
Verification: the endpoint exposes no write path

Check	Pass condition
Write attempts with the replacement's login: insert, update, delete, execute	Every attempt rejected; legacy product tables unchanged
Write methods called on the search endpoint: POST, PUT, PATCH, DELETE	Rejected; no write route exists
Product counts and sampled rows read by the replacement compared with legacy	Equal for the same keys, because the data is read live
Legacy p95 while the replacement is queried	Stays within the production evidence baseline, for example no more than 10% above it
Fitness function	A pipeline test tries each write with the replacement's login and fails if any succeeds; an alert fires when the login gains a write grant or legacy p95 leaves its baseline

Rollback: routing rollback: switch the route flag off or disable the login; legacy answers as before. State rollback is not needed because read-only access changes no data; if a write grant is ever found, revoke it at once and reconcile the affected rows against legacy audit data and backups, repairing forward.

Stop: when the replacement reads production products through a SELECT-only login, exposes no write path and legacy p95 stays within its baseline, no replica, copy or write access is added.

Solution architecture



The catalog search API now reads production products from SQL Server through a login that can only SELECT. Legacy remains the only writer, and the replacement has no write path.

Glossary

Used in this step

Term	Definition
Read-only SQL access	A SELECT-only database login on production tables, used by the replacement
Read Replica	Database copy serving read operations to distribute query load.
ETL copy	A scheduled extract, transform and load of products into the replacement's own store
Shared read-write login	The legacy application's own database login, reused by the replacement
Write path	Any route by which a request or a login can change stored data
Write owner	The one system whose writes are authoritative for a piece of data
Replication lag	Delay before a change on the primary shows up on the copy
Schema coupling	Dependence of one system on another's table and column names
Least privilege	A login gets only the permissions its job needs
Baseline	Measured normal legacy behavior that later changes are compared with
Fitness function	Measurable or executable architectural quality rule

Related concepts

Term	Definition
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations
Data Backfill	Populate a new representation using already-stored historical data
Change Data Capture (CDC)	Detects and propagates changes made to database records.
Database View	Named query presenting an abstraction over underlying tables
Feature Flag	Runtime control enabling behavior without deploying new code

M17 · Read-only strangler extraction

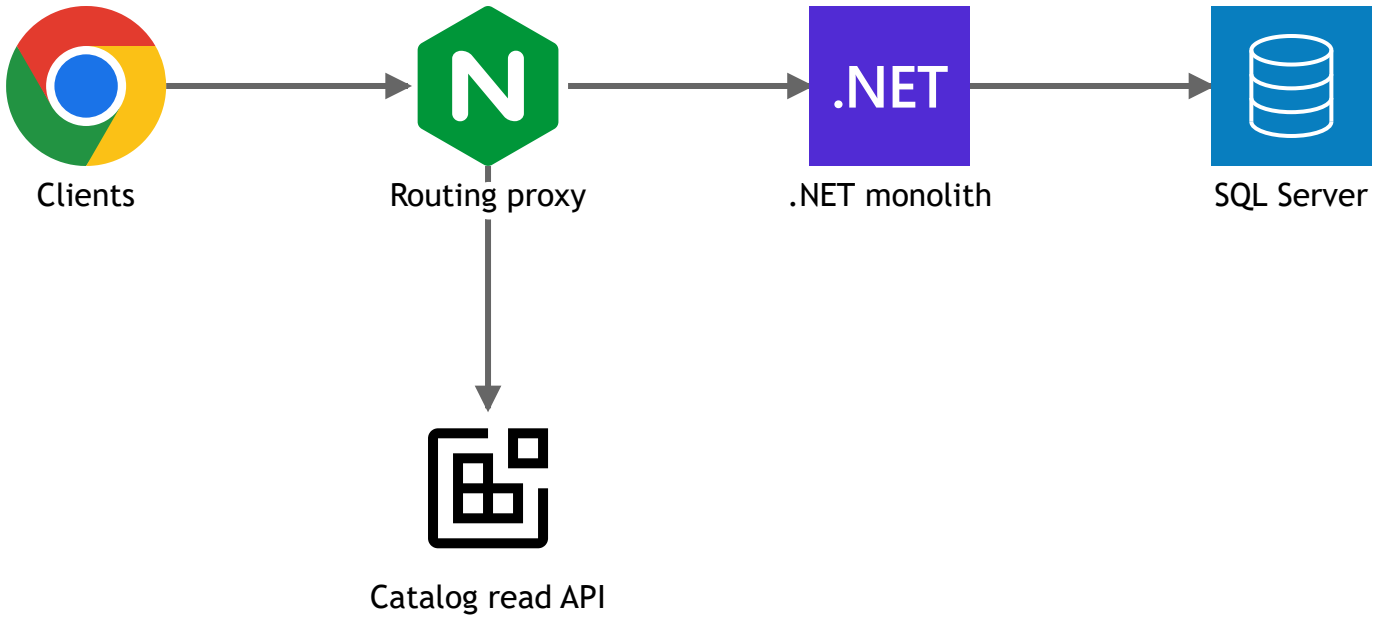
Compare outputs

Old behavior has undocumented filters and rounding, so every difference between old and new search must be classified as intended or defect: 0 unclassified mismatches.

Two search implementations answer differently

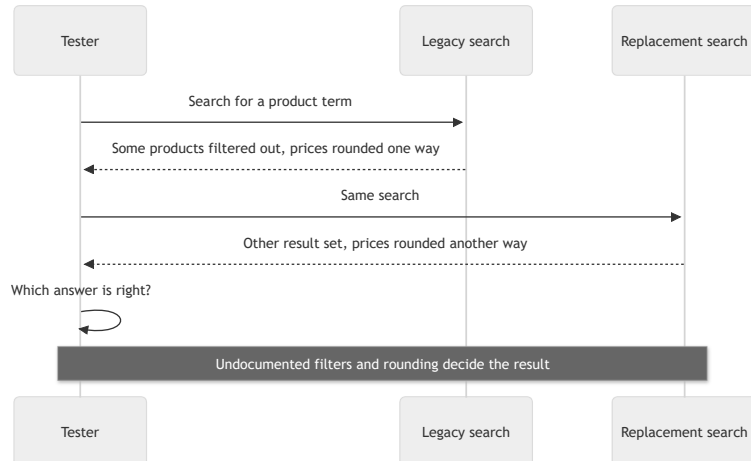
Component	What it is	What it does
Clients	Anything that calls the application over HTTP	Search the catalog through the routing proxy
Routing proxy	Stable public entry point	Sends every request to legacy; can route a catalog read to the replacement by flag
Legacy monolith	12-year-old .NET Framework application	Answers every search with its own filters and rounding
SQL Server	One shared relational database	Holds the production product data that the replacement reads without writing
Catalog read API	The replacement search endpoint beside legacy	Passes the search fixtures; no customer calls it yet
Feature flags	Runtime switches for single behaviors	Turn one route back to legacy without undoing other changes

Current architecture



Nobody knows which difference is a bug

The replacement passes the fixtures, yet for some queries it returns other results than legacy. Legacy hides some products by filters nobody wrote down and rounds prices in its own way.



Different results

old and new search for the same query

Fixed query set

run read-only against both implementations

0 unclassified mismatches

every one marked intended or defect

Each option decides what counts as the reference

Solution	What it is	What it does
Differential Testing + Golden Master	Capture legacy outputs as the reference and run old and new on the same input	Reveals every mismatch between legacy and the replacement and lists it for classification
Approved change	A recorded decision that one difference is intended behavior	Fixes the legacy behavior or declares the new result the contract, and both sides follow it
Manual spot checks	A tester compares a handful of searches by hand	Judges the replacement by sampled queries, with no recorded reference

Finding every mismatch costs review time

Solution	Gains	Costs
Differential Testing + Golden Master	Reveals mismatches Characterizes unknown legacy behavior Repeatable on every build	Every mismatch needs a human review Legitimate differences require rules May preserve historical defects
Approved change	Fixes legacy defects Rule documented once decided	Changes the contract consumers rely on Each difference needs an owner decision Touches legacy code that is hard to release
Manual spot checks	No tooling to build Quick first look	Misses rare filters and rounding Not repeatable; no regression gate Undocumented rules stay unknown

Deciding signal: mismatch rate across a fixed query set that covers every filter and price band, and the share of mismatches that are defects versus intended rules. Any unclassified mismatch keeps the comparison running; a legacy defect that the owner confirms moves to Approved change for that difference; one differing result outside the sampled queries disqualifies Manual spot checks.

Smallest safe move

Differential Testing + Golden Master on a fixed query set

- **Scope:** evidence and code; traffic stays 100% on legacy, nothing is written, no contract changes
- **Change:** capture legacy answers for a fixed query set as the golden master, run the same queries on the replacement, classify every mismatch as intended or defect
- **Safety net:** the golden master is a reference, not a verdict: each mismatch gets an owner decision; Manual spot checks are never evidence
- **Escape route:** nothing reaches customers; the routing proxy still sends every request to legacy
- **Deferred:** Approved change for a defect that the owner and consumers accept; Manual spot checks only as a first look

```
for each query in the query set:
    golden = answer(legacy, query)          # captured once
    actual = answer(replacement, query)
    if golden ≠ actual: list the mismatch for classification
```

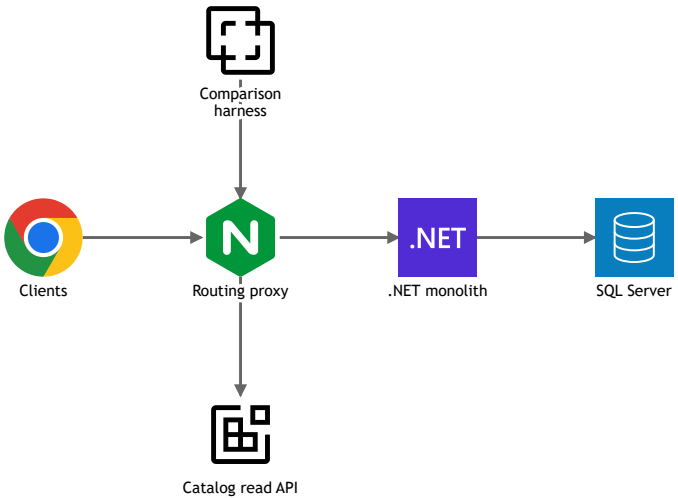
Verification: classify every mismatch

Check	Pass condition
Same query set run on legacy and on the replacement	Every answer pair diffed by result set, applied filters, order and price rounding; the mismatch list is complete
Each mismatch reviewed	0 unclassified mismatches: every one marked intended or defect, with an owner
Defects resolved	Each defect fixed in the replacement or recorded as an Approved change; none accepted silently
Rerun after the fixes	Only the listed intended differences remain
Fitness function	The comparison runs in the pipeline on every build of the replacement and fails when a mismatch is not on the list of intended differences

Rollback: routing rollback is not needed, because customers never see replacement output and all traffic stays on legacy through the proxy. State rollback is not needed either, because the comparison only reads. An Approved change that edits legacy behavior would ship later as its own flagged release.

Stop: when every remaining difference is intended and recorded, or the replacement already matches legacy on the whole query set, no further comparison tooling is added.

Solution architecture



The harness sends each query through the routing proxy twice, once pinned to legacy and once to the catalog read API, and diffs the answers. Customers still receive only legacy answers.

Glossary

Used in this step

Term	Definition
Differential Testing + Golden Master	Run old and new on the same input and diff both against the captured legacy output
Approved change	A recorded decision that one difference is intended behavior
Manual spot checks	A tester compares a handful of searches by hand
Differential Testing	Compare outputs of old and new implementations for same input
Golden Master	Capture existing observable outputs as compatibility reference
Mismatch	A difference between the old and the new answer for the same input
Query set	A fixed list of searches run against both implementations
Intended difference	A mismatch accepted on purpose and recorded as a rule
Defect	A mismatch that is a bug in the replacement or in legacy
Catalog read API	The replacement search endpoint that runs beside legacy and only reads
Routing proxy	Intermediary that sends each route to legacy or to the replacement

Related concepts

Term	Definition
Characterization Testing	Tests documenting actual legacy behavior before refactoring
Shadow Traffic	Send production requests to a candidate implementation without using candidate responses
Canary Routing	Shift a small controlled segment of live traffic to a new implementation
Feature Flag	Runtime control enabling behavior without deploying new code
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations

M18 · Read-only strangler extraction

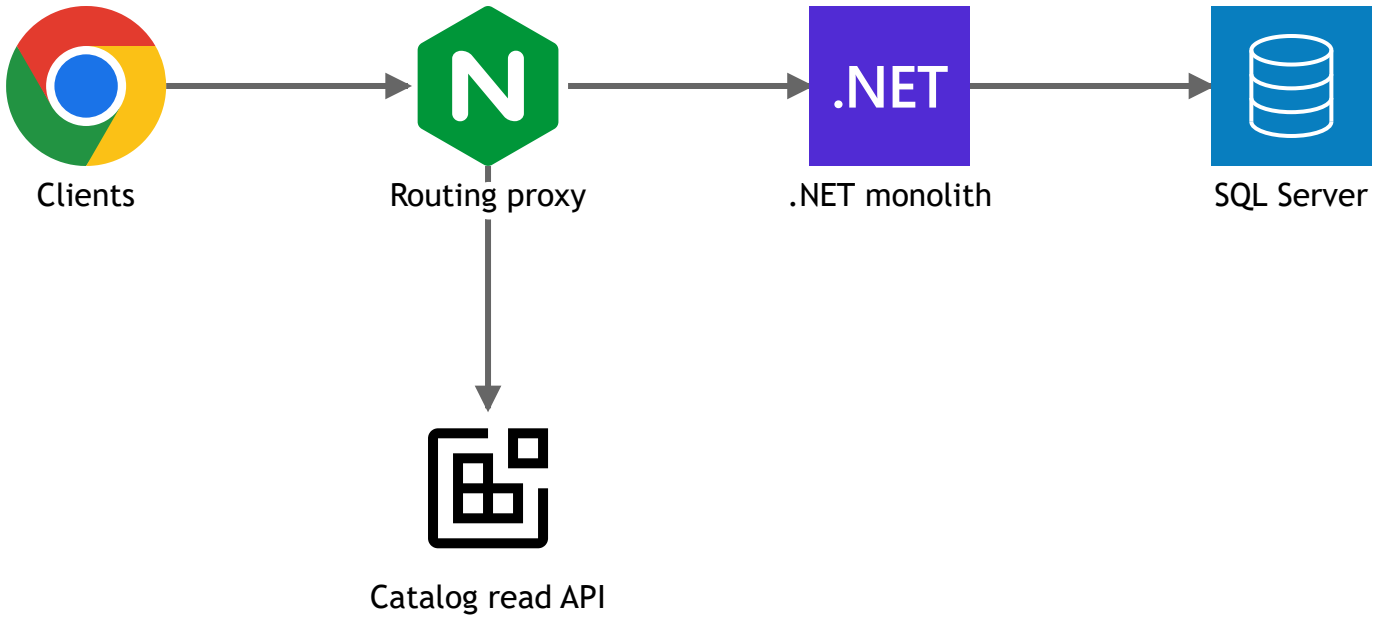
Shadow production inputs safely

Real data exposes irregularities that fixtures hide, and mirroring live requests must cause 0 candidate side effects: no second write, payment or mail.

The candidate has only seen fixtures

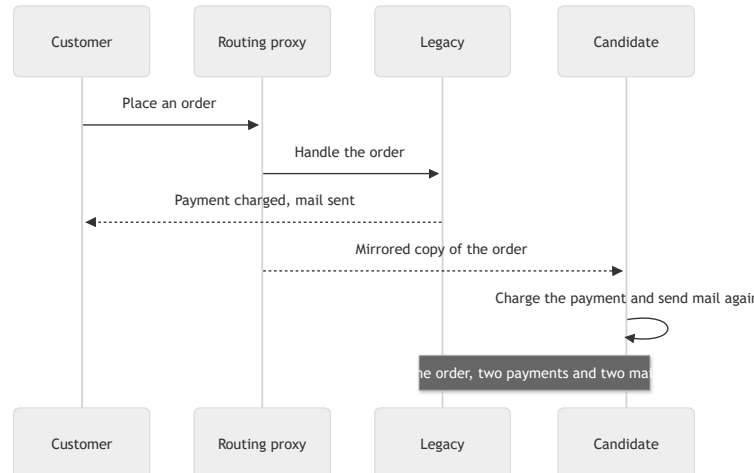
Component	What it is	What it does
Clients	Anything that calls the application over HTTP	Send every search to legacy through the routing proxy
Routing proxy	Stable public entry point	Keeps all traffic on legacy; no live request reaches the candidate
Legacy monolith	12-year-old .NET Framework application	Answers every customer request with its own behavior
SQL Server	One shared relational database	Holds the production product data
Catalog read API	The candidate search endpoint beside legacy	Reads production products without writing; tested only with fixtures
Comparison harness	Test tool with a fixed query set	Diffs candidate and legacy answers and classifies each mismatch

Current architecture



A copied order would be paid twice

Fixtures hold only what someone thought of, so irregularities in real data stay hidden. Copying live requests to the candidate would show them, but a copied request that writes, pays or sends mail would do it a second time.



Irregularities

that real data exposes and fixtures never contained

Mirrored request

could trigger a payment, a mail or a write twice

0 candidate side effects

writes, payments and mail disabled

Each option trades realism against safety

Solution	What it is	What it does
Shadow Traffic	Send production requests to a candidate implementation without using candidate responses	Copies live search reads to the replacement and compares its answers with legacy offline
Offline replay	Record sanitized production requests and replay them against the candidate later	Runs the recorded set in a controlled environment, away from production
Unfiltered mirror	Copy every production request, writes, payments and mail included, to the candidate	Runs the candidate on the full live input with nothing suppressed

Realistic input brings load and side effects

Solution	Gains	Costs
Shadow Traffic	Tests realistic behavior Real queries and data expose irregularities Customers see only legacy answers	Double load on the shared database Dangerous side effects if not suppressed Personal data copied unless stripped
Offline replay	Controlled: no live side effects Personal data removed once, at capture Repeatable on every build	Incomplete: misses rare and new requests Recording and sanitizing pipeline to build Recording goes stale as data changes
Unfiltered mirror	Most complete live input No filter to build	Payments and mail would fire twice Writes duplicate or corrupt data Personal data reaches the candidate and its logs

Deciding signal: side effects, personal data and writes in the mirrored request, the production realism needed, and the load the shared database can absorb. Search reads pick Shadow Traffic on an allow-list; personal data that cannot be stripped, or load beyond the measured headroom, moves a request class to Offline replay; a mirror that cannot guarantee writes and payments disabled fails the invariant.

Smallest safe move

Shadow Traffic on allow-listed reads

- **Scope:** traffic and evidence; the candidate stays read-only, legacy answers every customer
- **Change:** mirror a sample of search reads (for example 10% of search reads) through an allow-list; discard responses, strip personal data, give the candidate a read-only role, disable writes, payments and mail
- **Safety net:** requests off the allow-list are dropped; a guard probe must be rejected; Unfiltered mirror is never used
- **Escape route:** switch the mirror flag off; customers never saw candidate answers
- **Deferred:** Offline replay when personal data cannot be stripped or mirrored load exceeds the database headroom

```
proxy: if route in allow-list and sample() → copy to candidate, discard its response
candidate: read-only role; payment and mail clients replaced by no-ops
```

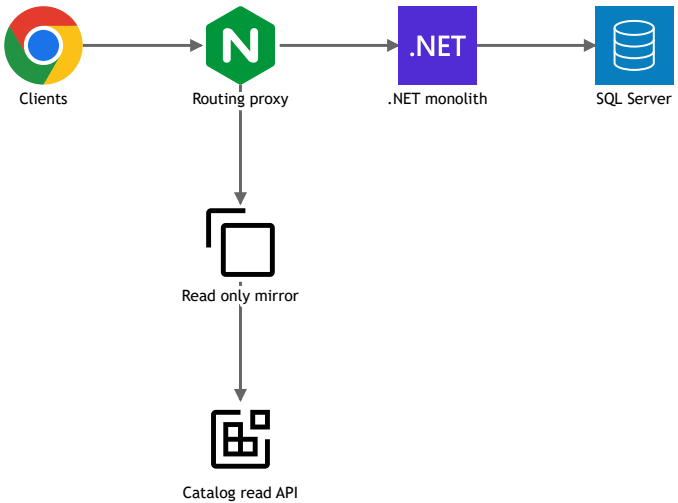
Verification: prove the candidate cannot act

Check	Pass condition
Write, payment and mail attempts made against the replacement while the mirror runs	All blocked; legacy and catalog data unchanged
Request classes and fields in the mirrored copies and replacement logs	Only allow-listed search reads; 0 mirrored calls to payment and mail routes; no personal data fields
Legacy search p95 and error rate with the mirror on	Within the baseline noise measured from production evidence before the mirror
Shadow answers diffed against legacy	Each irregularity from real data classified as intended or defect; nothing reaches customers
Fitness function	A scheduled probe sends a write, a payment and a mail request along the mirror path and must be rejected; an alert fires on any write accepted by the replacement or any request mirrored off the allow-list

Rollback: routing rollback: switch the mirror flag off; no customer was served from the replacement. State rollback is not needed, because the replacement is read-only and its writes are disabled. Exception: a leaked side effect, such as a duplicated mail or payment, is not undone by the flag; it needs a forward fix and a compensating action by its owner, so the guard probe must be rejected before the mirror starts.

Stop: when every irregularity from live reads is classified and the guards hold, no Offline replay pipeline and no wider mirror is added.

Solution architecture



The proxy copies only allow-listed search reads through the read-only mirror to the catalog read API and drops everything else. Customers still receive only legacy answers.

Glossary

Used in this step

Term	Definition
Shadow Traffic	Send production requests to a candidate implementation without using candidate responses
Offline replay	Record sanitized production requests and replay them against the candidate later
Unfiltered mirror	Copy every production request, writes, payments and mail included, to the candidate
Candidate	The replacement under test, which never answers a customer in this step
Mirror flag	Switch that turns the copying of live requests on or off
Allow-list	The read routes that may be mirrored; any other request is dropped
Side effect	A change outside the answer itself, such as a write, a payment or a mail
Personal data	Data that identifies a customer and must not reach the candidate or its logs
Read-only role	Database role that can read but not change data
Guard probe	A scheduled request that tries a write, a payment and a mail and must be rejected
Fixtures	Hand-made test inputs with their expected outputs

Related concepts

Term	Definition
Canary Routing	Shift a small controlled segment of live traffic to a new implementation
Differential Testing	Compare outputs of old and new implementations for same input
Golden Master	Capture existing observable outputs as compatibility reference
Feature Flag	Runtime control enabling behavior without deploying new code
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations

M19 · Read-only strangler extraction

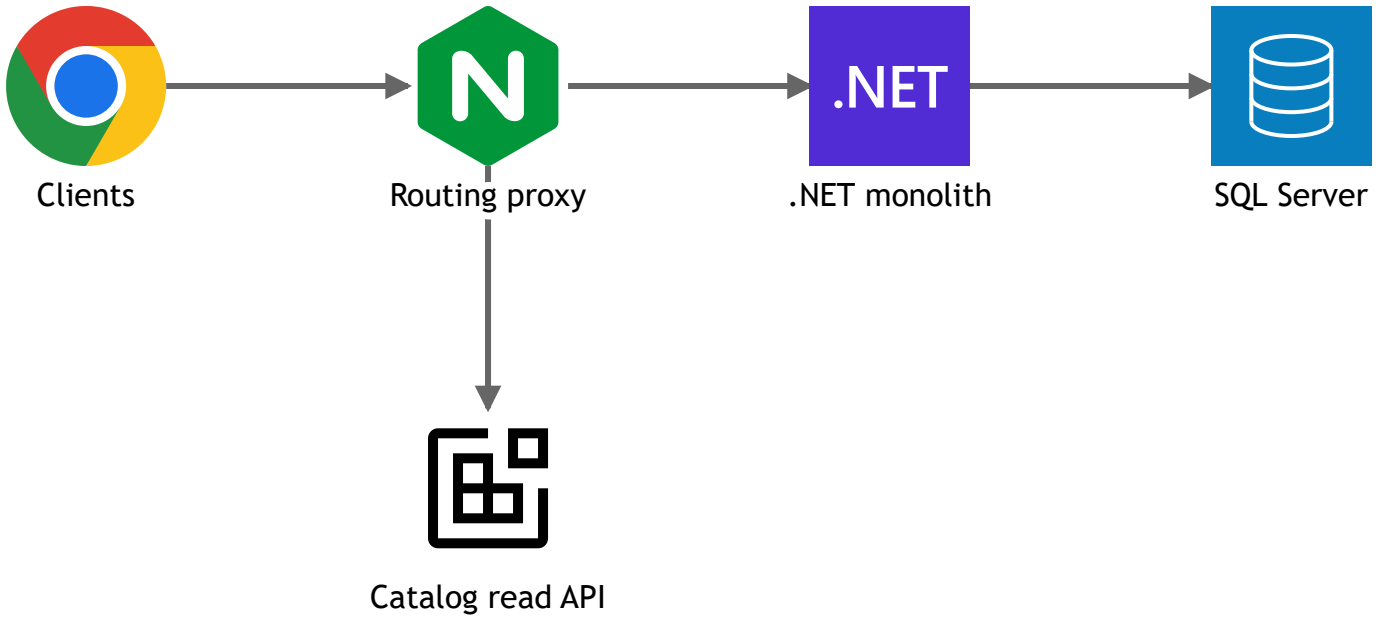
Canary read traffic

The new catalog must be tested with a small user cohort, in stages of 1%, 10%, 50% and 100% of read traffic, with a route switch back to legacy.

Both paths work, but only legacy serves users

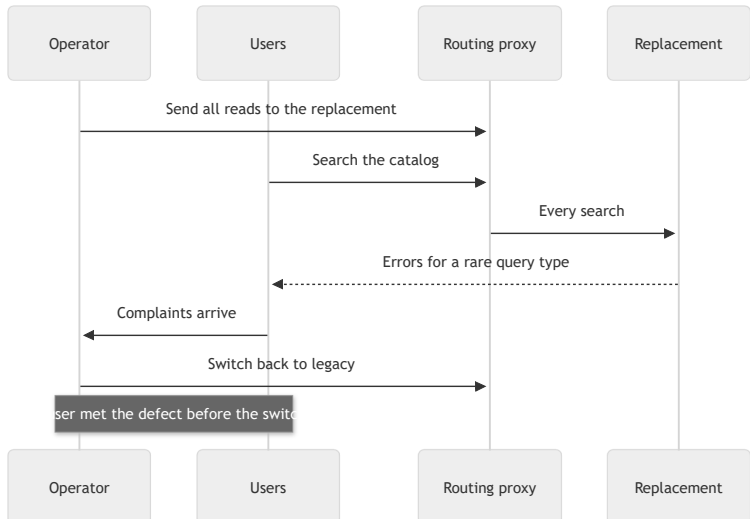
Component	What it is	What it does
Clients	Anything that calls the application over HTTP	Send every read through the routing proxy to legacy
Routing proxy	Stable public entry point	Can send one catalog route to legacy or to the replacement by flag or rule
Legacy monolith	12-year-old .NET Framework application	Answers every read and every write
SQL Server	One shared relational database	Holds the production product data
Catalog read API	The candidate search endpoint beside legacy	Matches legacy on the classified query set and on shadow reads; serves no user yet
Production evidence	Metrics and logs per route	Show the error rate and p95 of each route

Current architecture



The first live users need a small cohort

Comparison and shadow traffic show that the replacement answers like legacy, yet no real user has ever been served by it. Switching every user at once would expose all of them to any remaining defect.



No real user
has been served by the candidate; all reads are on legacy

1%, 10%, 50% and 100%
of live read traffic, shifted in stages

Error and p95 gates
pass at every stage; one route switch returns the cohort to legacy

Each option decides who meets a defect first

Solution	What it is	What it does
Canary Routing 1%-10%-50%-100%	Shift a small controlled segment of live traffic to the new implementation, in four stages	Routes 1% of reads to the replacement, then 10%, 50% and 100%, gated by error and p95 metrics
Tenant-based routing	Route all reads of chosen tenants or accounts to the replacement	Serves one fixed group of accounts entirely from the replacement
All-at-once switch	Move all read traffic to the replacement in one flag change	Replaces the legacy read path for every user at once, with a flag to switch back

Small exposure needs metrics and routing

Solution	Gains	Costs
Canary Routing 1%-10%-50%-100%	Limits exposure Low blast radius Each stage returns by route switch	Requires good metrics and routing Routing rules and sticky routing to keep Four observation windows take time
Tenant-based routing	Isolation of the affected group Same users always see the same implementation Group chosen on purpose	Skew: the group may not represent all users Needs a stable tenant identity at the proxy Large tenants carry unequal load
All-at-once switch	Simplest rule: one flag Fast cutover No cohort logic to build	Every user meets every defect No small cohort to measure against Undo comes after users saw errors

Deciding signal: can error rate and p95 be split by cohort, and can the proxy route by percentage or by account? Cohort-split metrics and percentage routing pick Canary Routing; a stable tenant identity and a group to isolate first pick Tenant-based routing; without split metrics no canary starts; exposing every user at once fails the small-cohort constraint.

Smallest safe move

Canary Routing 1%-10%-50%-100% at the proxy

- **Scope:** traffic and evidence; the proxy sends a share of reads to the replacement, legacy keeps the rest
- **Change:** raise the share in stages of 1%, 10%, 50% and 100%, each held until the cohort's error rate and p95 are no worse than legacy's
- **Safety net:** gates at every stage, automatic switch back on failure, sticky routing, cohort-split metrics first; no All-at-once switch as a first move
- **Escape route:** set the share to 0% by flag or rule
- **Deferred:** Tenant-based routing when a stable account identity exists and a fixed group should be isolated first, or cohorts hide per-tenant defects

```
route /catalog/search: legacy 99%, replacement 1%      # stage 1 of 4
sticky by user; gates: error rate and p95 against legacy, same window
```

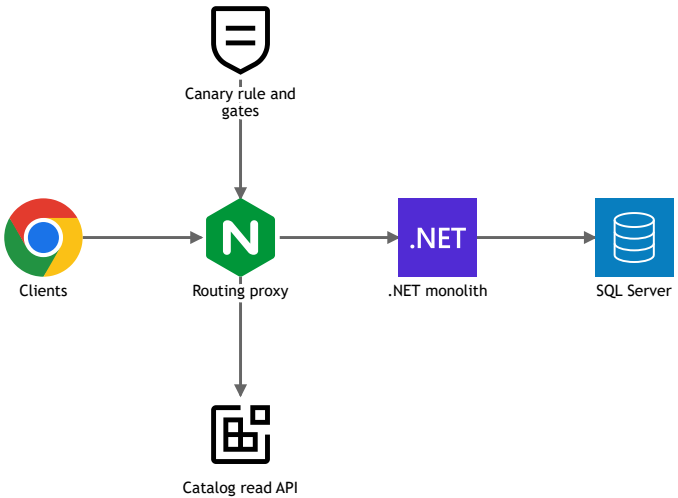
Verification: gates at every stage

Check	Pass condition
Metrics split by implementation before the first stage	Error rate and p95 of the cohort and of legacy shown side by side for the same window
Each stage 1%, 10%, 50% and 100%	Cohort error rate and p95 no worse than the legacy baseline; the stage advances only on green gates
Failure injected: the replacement made slow or failing for the cohort	A gate fails and the proxy returns the cohort to legacy without a manual step
Route switched back at any stage	All reads on legacy after one configuration change, no failed requests
Fitness function	A rollout job evaluates the gates for each stage and returns the route to legacy on a breach; an alert fires if the replacement's share is above the last stage that passed

Rollback: routing rollback: set the replacement's share to 0%, so one configuration change returns the whole cohort to legacy. State rollback is not needed, because only reads moved and nothing was written; the trap that rollback code does not roll back database state starts to apply when writes move.

Stop: when the 1%-10%-50%-100% stages pass their gates and the replacement serves all read traffic, no tenant routing and no blue-green environment is added.

Solution architecture



The canary rule sets what share of reads the proxy sends to the catalog read API. A failed gate stops the rollout and returns the route to legacy.

Glossary

Used in this step

Term	Definition
Canary Routing 1%-10%-50%-100%	Shift a small controlled segment of live traffic to a new implementation, in four stages
Tenant-based routing	Route all reads of chosen tenants or accounts to the replacement
All-at-once switch	Move all read traffic to the replacement in one flag change
Cohort	The group of users whose reads the replacement serves at a stage
Gate	A metric condition that must hold before a stage advances
p95	Latency under which 95% of requests complete
Cohort-split metrics	Error rate and p95 measured separately for legacy and for the replacement
Sticky routing	A user keeps being served by the same implementation
Blast radius	The share of users or data affected when something fails
Routing rollback	Switching traffic back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Blue-Green Deployment	Maintain two release environments and switch traffic
Shadow Traffic	Send production requests to a candidate implementation without using candidate responses
Feature Flag	Runtime control enabling behavior without deploying new code
Rollback	Return to a previously running version or state
Forward Fix	Repair the new implementation without restoring the old system

M20 · Read-only strangler extraction

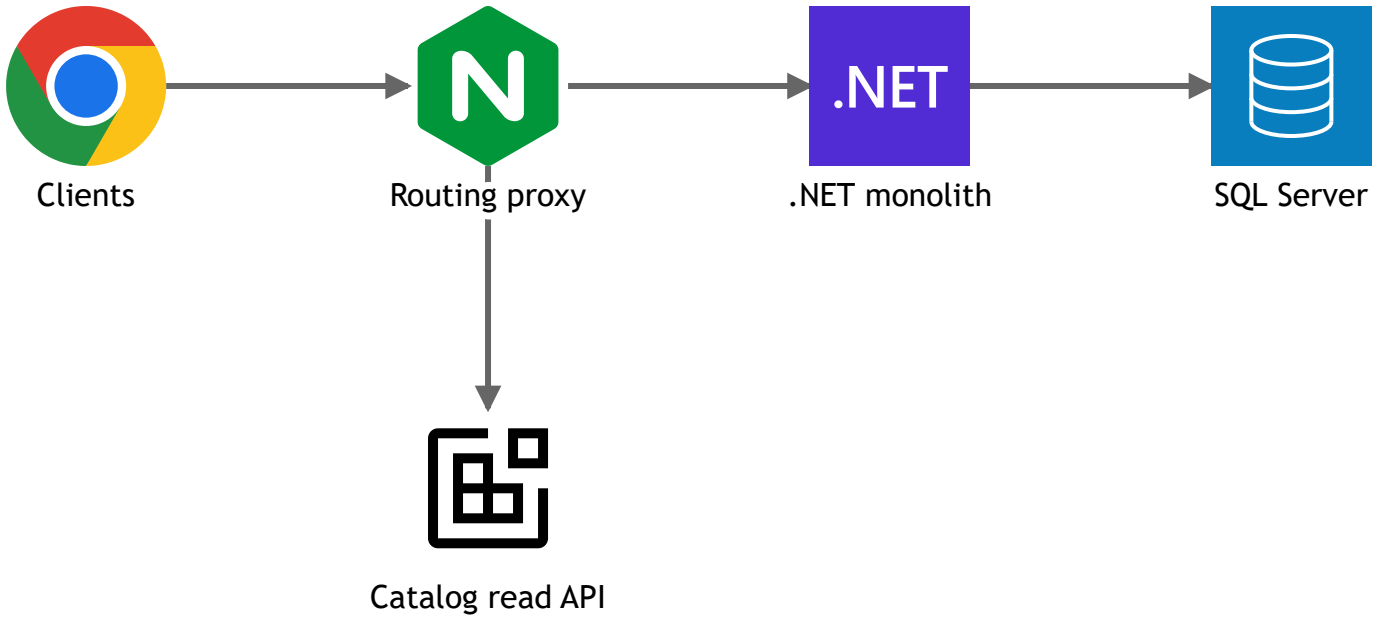
Migrate internal catalog consumers

All catalog callers must converge on the replacement, and internal calls bypass the proxy: the goal is 0 remaining old call-sites.

Old pages still call legacy catalog methods

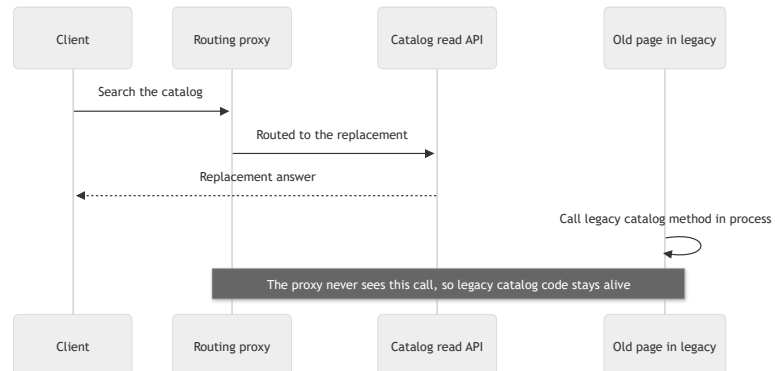
Component	What it is	What it does
Clients	Anything that calls the application over HTTP	Reach the catalog through the routing proxy
Routing proxy	Stable public entry point	Sends catalog reads to the replacement, other routes to legacy
Legacy monolith	12-year-old .NET Framework application	Its pages and modules call legacy catalog methods in process
SQL Server	One shared relational database	Holds the production product data that both implementations read
Catalog read API	The replacement search endpoint	Serves all external catalog reads; no internal caller uses it
Feature flags	Runtime switches for single behaviors	Turn one behavior back without undoing the others

Current architecture



The proxy never sees internal calls

The proxy moves external clients, but the monolith's own pages and modules, the internal consumers, call the legacy catalog methods in process. While one caller still uses them, the old read code cannot be retired and two catalog implementations must be kept.



Internal calls bypass the proxy

old pages still call legacy catalog methods

Every call-site

of the legacy catalog methods, counted and listed

0 remaining old call-sites

all catalog callers use the replacement

Each option decides how callers reach the replacement

Solution	What it is	What it does
Branch by Abstraction	Introduce a stable interface, then change the implementation behind it gradually	Moves callers to one catalog interface whose implementation switches between legacy and replacement by flag
Proxy route	Route catalog requests to the replacement at the routing proxy	Sends every external catalog call to the replacement by rule, without touching callers
Direct call-site edits	Edit every internal call-site to call the replacement directly	Replaces each legacy catalog call with a call to the replacement in one release

A phased move costs a dual path

Solution	Gains	Costs
Branch by Abstraction	Replace internals without rewriting all callers Phased: one caller group at a time Per-group flag returns to the old path	Dual path while callers move Temporary indirection needs an owner Interface must hide differences exactly
Proxy route	No change in callers One switch per route Already in place	External only: internal calls bypass the proxy Old pages keep calling legacy methods Old read code cannot be retired
Direct call-site edits	No dual path to maintain No extra indirection	Big-bang change across all callers No per-caller switch back A missed call-site goes unnoticed

Deciding signal: the number of call-sites that call the legacy catalog methods in process, counted by a reference search. One internal call-site that bypasses the proxy rules out Proxy route alone, and only external callers make it enough. Call-sites that cannot change safely in one release pick Branch by Abstraction; changing every caller at once fails the constraint.

Smallest safe move

Branch by Abstraction behind a catalog interface

- **Scope:** code and traffic inside the monolith; the interface is the seam; no data change
- **Change:** count every call-site, put a catalog interface in front of the old code, add an adapter to the replacement, move caller groups behind flags, recount
- **Safety net:** the dual path stays until the last caller moves; the proxy route alone never counts; no Direct call-site edits; the interface has an owner and a removal trigger
- **Escape route:** flip the caller group's flag back to the old implementation
- **Deferred:** Proxy route covers external callers only; Direct call-site edits only for a lone, tested call-site

```
interface ICatalog { Product[] Search(Query q); }       // the seam
class LegacyCatalog : ICatalog { /* old code */ }
class ReplacementCatalog : ICatalog { /* adapter to the catalog read API */ }
ICatalog Resolve(CallerGroup g) ⇒ flags.On(g) ? replacement : legacy;
```

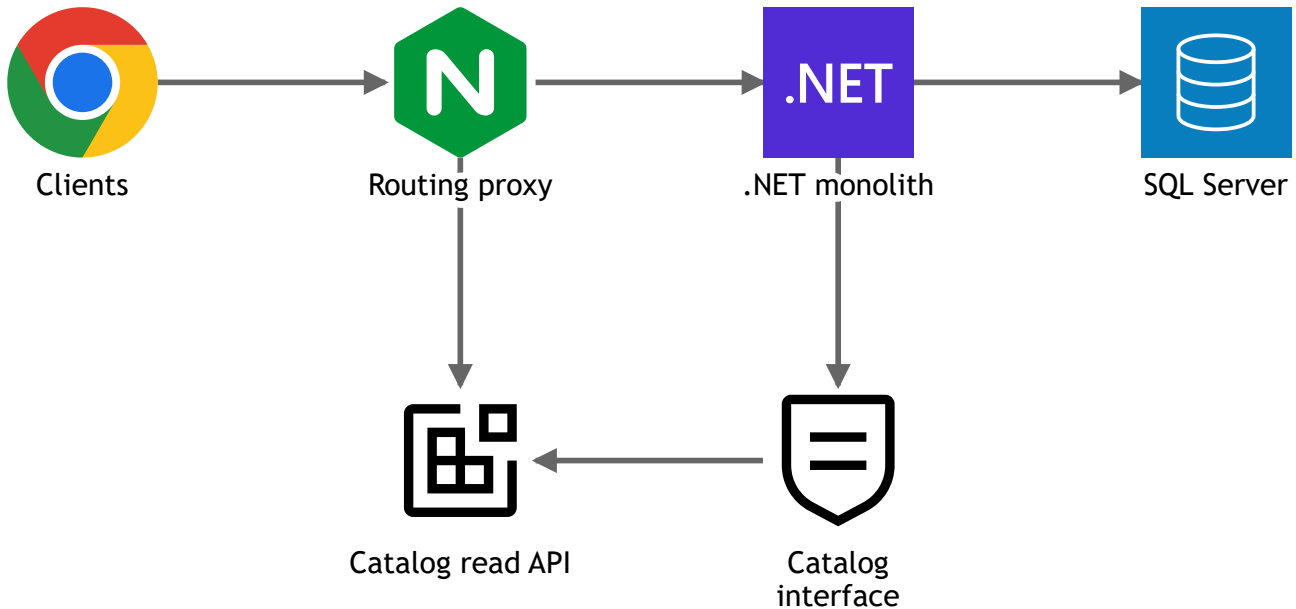
Verification: count the call-sites down to 0

Check	Pass condition
Reference search for calls to the legacy catalog methods	Every call-site listed with an owner; the count is the baseline
One caller group moved to the interface with its flag on	Same answers as the old implementation, differences classified, no failed calls
The flag of that group switched back	The group is served by the old implementation again, no failed requests
Count after each batch of callers	The count of old call-sites falls to 0; no direct call remains outside the interface
Fitness function	A build step fails when a direct call to a legacy catalog method appears outside the interface; a dashboard shows the remaining old call-sites and the share of calls answered by the replacement

Rollback: routing rollback: flip the flag of a caller group back to the old implementation behind the interface, per group and at once. State rollback is not needed, because every call only reads and nothing was written.

Stop: when the count of old call-sites is 0 and every caller reaches the catalog through the interface served by the replacement, no further indirection or routing layer is added.

Solution architecture



Old pages call the catalog interface, and a flag per caller group sends each call to the old implementation or to the catalog read API.

Glossary

Used in this step

Term	Definition
Branch by Abstraction	Introduce stable interface then change the implementation behind it gradually
Proxy route	Routing a catalog request to the replacement at the routing proxy
Direct call-site edits	Edit every internal call-site to call the replacement directly
Call-site	A place in the code that calls a legacy catalog method
Catalog interface	The stable interface all internal callers use, with the old code and an adapter to the replacement behind it
Caller group	A batch of call-sites that move together behind one flag
Dual path	Old and new implementation both live behind the interface until the last caller moves
Internal consumer	A page or module inside the monolith that calls catalog code in process
Seam	A place where behavior can be switched between old and new without editing callers
Routing proxy	Intermediary that sends each route to legacy or to the replacement
Routing rollback	Switching traffic back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations
Feature Flag	Runtime control enabling behavior without deploying new code
Differential Testing	Compare outputs of old and new implementations for same input
Expand-Contract	Add compatible representation, migrate consumers, then remove old representation
Refactor in Place	Improve internal structure without changing external behavior

M21 · Read-only strangler extraction

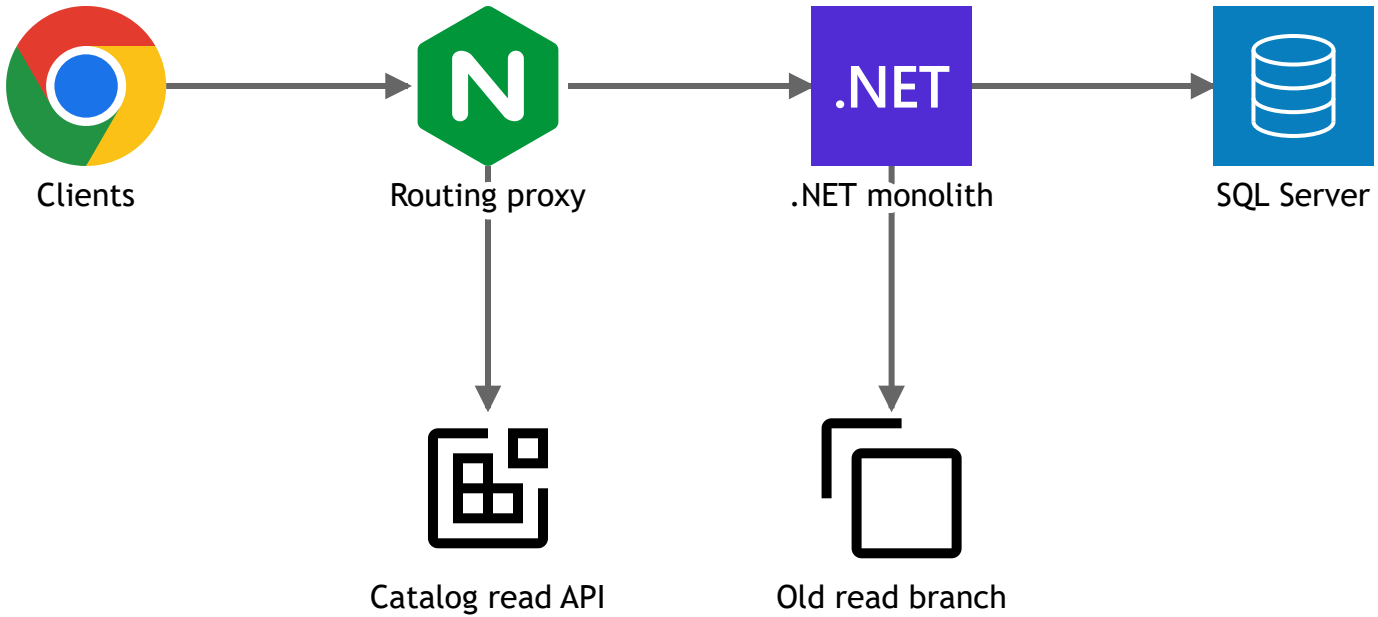
Retire old read branch

Duplicate code imposes maintenance, so the old read branch goes once the new read path handles 100% load, and it stays the escape route until the soak passes.

The new path serves all reads, the old branch remains

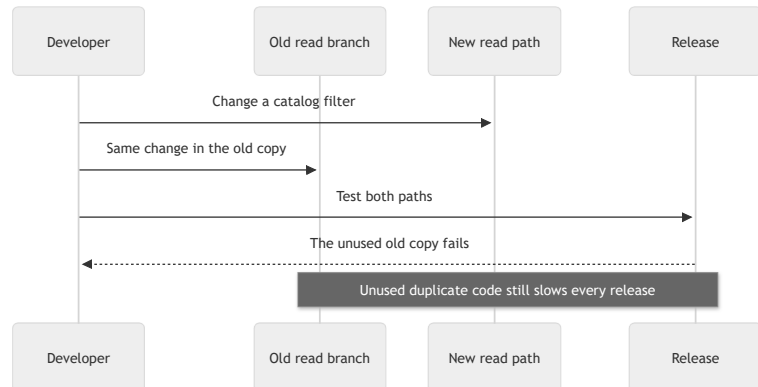
Component	What it is	What it does
Routing proxy	Stable public entry point	Sends 100% of catalog reads to the replacement; a fallback route to legacy remains
Catalog read API	The replacement search endpoint	Handles 100% load with its gates held
Legacy monolith	12-year-old .NET Framework application	Still contains the old read branch behind the catalog interface
Old read branch	The legacy implementation of catalog reads	Waits as a dormant fallback; every catalog change must be made and tested twice
SQL Server	One shared relational database	Holds the production product data
Feature flags	Runtime switches for single behaviors	Turn one caller group back to the old branch

Current architecture



Two read paths cost twice

The replacement answers all catalog reads, yet the old read branch is still in the monolith. Deleting it removes the debt and also the quickest way back, so it has to wait until the new path has proven itself.



Two read implementations
to maintain, test and release

100% load
handled by the new read path

1 read implementation
after a defined retention window, with no fallback used

Each option decides when the escape route goes

Solution	What it is	What it does
Delete old read code after soak	Remove the old read branch once the new path has carried all load through a soak and a retention window	Deletes the duplicate code, the fallback route and the flags, and keeps the last release containing it tagged
Dormant fallback	Keep the old read branch deployed but switched off behind a flag	Returns reads to legacy by flag if the replacement fails
Delete at 100% without soak	Delete the old read code in the release that reaches 100% load	Removes the duplicate at once, with no soak and no retention window

Less debt means a slower way back

Solution	Gains	Costs
Delete old read code after soak	Removes debt One read implementation to maintain Fewer routes and flags	Less immediate undo Undo means redeploying a tagged release A late-found rare case has no instant fallback
Dormant fallback	Instant revert by flag No redeploy to undo	Duplicate code keeps costing maintenance Untested fallback rots Dead paths hide in every release
Delete at 100% without soak	Fastest debt removal No soak to track	Escape route gone before success criteria pass Rare cases appear after the fallback is gone Undo is a redeploy under pressure

Deciding signal: how long the replacement has handled 100% load with gates held and no fallback use, how many references to the old code remain, and the retention window the longest business cycle needs. Zero references and a clean soak pick deletion; one reference or one fallback use keeps the Dormant fallback and restarts the soak; deleting at 100% load breaks the rule to keep the old path until success criteria pass.

Smallest safe move

Delete old read code after soak, behind an audit

- **Scope:** code and traffic; the old read code, its fallback route and flags go; replacement, data and writers stay
- **Change:** audit the soak, search all references, delete the branch in one reviewed change, tag the last release containing it
- **Safety net:** soak at 100% load with gates held and no fallback use; Delete at 100% without soak is never used; an undo drill restores the tag
- **Escape route:** flag back before deletion; afterwards redeploy the tag or fix forward
- **Deferred:** Dormant fallback while a reference remains or a fallback use restarts the soak

```
audit: soak window clean, references = 0   →  delete branch, fallback route, flags
tag:   last release containing the branch, kept for the retention window
```

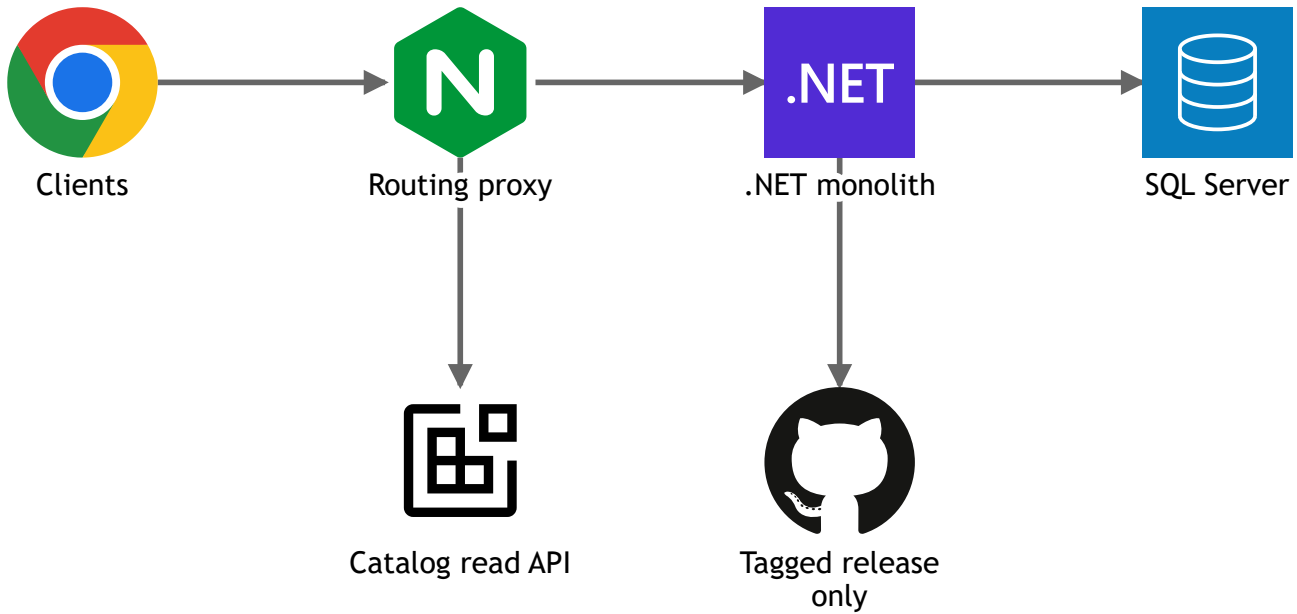
Verification: audit, search and an undo drill

Check	Pass condition
Reference search for the old read code, its routes and its flags	0 references outside the branch to delete; the old call-site count is 0
Release audit of the soak window	100% of read traffic on the replacement for the whole window, gates held, no fallback use
Release with the branch removed	Build and regression suite green; search answers equal the golden master except the listed intended differences
Undo drill: tagged release redeployed in a test environment	The old read branch works again from the tag, with the steps written down
Fitness function	A CI check fails when a reference to the retired read code, route or flag reappears; a flag audit lists every migration flag with an owner and a removal date

Rollback: routing rollback is possible only while the old branch exists: flip the flag back to legacy until the deletion. After it, undo means redeploying the tagged release or fixing forward on the replacement. State rollback is not needed, because nothing was written.

Stop: when one read implementation remains and the retention window ends without a revert, no fallback branch and no compatibility layer for reads is added.

Solution architecture



The old read branch no longer runs in the monolith; only a tagged release of it is kept for the retention window, and the replacement is the single read implementation.

Glossary

Used in this step

Term	Definition
Delete old read code after soak	Remove the old read code once the soak and retention window have passed
Dormant fallback	Keep the old read branch deployed but switched off behind a flag
Delete at 100% without soak	Delete the old read code in the release that reaches 100% load
Old read branch	The legacy implementation of catalog reads, kept as the escape route
Soak	The period in which the new path carries all load while gates are watched
Retention window	How long the tagged release is kept so that the branch can be restored
Release audit	Review of the soak window: load share, gates held, fallback use
Reference search	Search of the code for every use of the old read code, routes and flags
Tagged release	The last release that still contains the old branch
Undo drill	A rehearsal that restores the old branch from the tagged release
Routing rollback	Switching traffic back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations
Branch by Abstraction	Introduce stable interface then change the implementation behind it gradually
Feature Flag	Runtime control enabling behavior without deploying new code
Forward Fix	Repair the new implementation without restoring the old system
Expand-Contract	Add compatible representation, migrate consumers, then remove old representation

Phase 4

Data synchronization and writer handover

Steps in this phase

Step	Forcing constraint
M22 Backfill independent catalog DB	Catalog requires own schema and indices
M23 Keep database synchronized	Prices and availability change continuously
M24 Translate between domain models	New Catalog model cannot share old tables as-is
M25 Reconcile state	Cannot cut over while discrepancies exist
M26 Cut new reads to replica	Catalog p95 ≤150 ms without old DB queries
M27 Keep legacy as write owner initially	Merchandisers need changes while old processes still update product
M28 Expand compatible schema	Schema must evolve without breaking old consumers
M29 Transfer write authority	Legacy release controls product changes
M30 Feed legacy consumers from new source	New DB now owns writes but old apps must keep functioning
M31 Prevent old batch writers	Two authorities will cause lost updates
M32 Plan state-aware failback	Routing back would lose new state
M33 Retire sync debt	Replication and duplicate tables are expensive

M22 · Data synchronization and writer handover

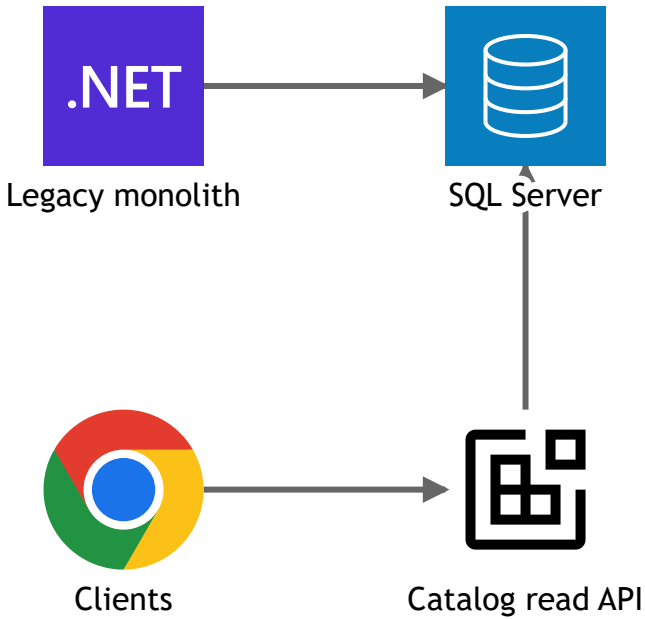
Backfill independent catalog DB

The catalog requires its own schema and indices, so the product data must be backfilled into a separate database that equals legacy at one stable snapshot marker.

The catalog already reads alone, but from legacy tables

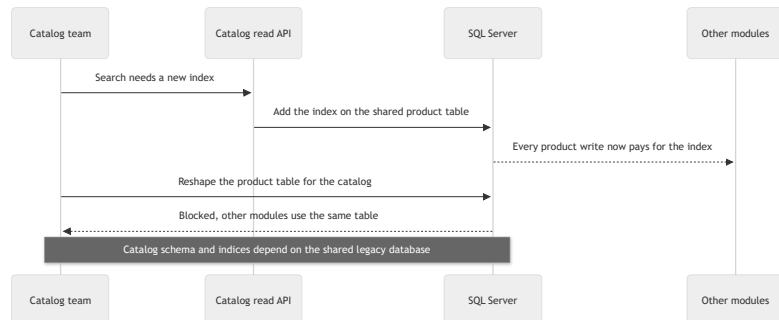
Component	What it is	What it does
Clients	Browsers and callers of the public URL	Reach catalog reads through the routing proxy
Routing proxy	The stable public entry point in front of legacy	Sends catalog reads to the replacement and everything else to legacy
Catalog read API	The replacement that serves 100% of catalog reads	Queries the legacy product tables, read-only
Legacy monolith	12-year-old .NET application on IIS	Stays the only writer of product data
SQL Server	One shared relational database	Holds the tables of every module, products included

Current architecture



The catalog depends on tables it does not own

The catalog still queries tables that the other legacy modules share. Every index it needs adds write cost for them, and every schema change waits for a legacy release.



Shared tables

catalog schema and indices depend on every legacy module

Legacy still writes

the copy is built while legacy is the only writer

Own schema and indices

counts and checksums equal legacy at one stable snapshot marker

Each option decides where the catalog’s own data lives

Solution	What it is	What it does
Data Backfill + Snapshot	Copy the existing products into a new database at a stable snapshot marker	Builds the catalog’s own schema and indices, loads the history in throttled batches, then compares counts and checksums at the marker
Shared DB	Add catalog tables and indexes inside the legacy database	Keeps catalog data on the same SQL Server and reads it from there, with no copy
Live bulk copy	A one-off script copies the live tables into a new database without a snapshot marker	Reads rows while legacy keeps writing and loads them as they are found

Independence costs a transfer, sharing costs coupling

Solution	Gains	Costs
Data Backfill + Snapshot	Independent schema and indices Complete state before any sync Marker shows where changes resume	Large one-time transfer Load on legacy while copying Copy is stale after the marker
Shared DB	Simple: no copy, no sync Always current	Still coupled to legacy schema and releases Extra database indexes slow legacy writes Shared instance load and locks
Live bulk copy	Quick to write No snapshot setup	Rows change while copying No marker where sync can resume Counts and checksums cannot prove equality

Deciding signal: how often catalog changes need a coordinated legacy release, and what its indexes cost legacy writes. If changes fit the legacy release cadence and write latency stays at its baseline, Shared DB is enough; otherwise Data Backfill + Snapshot. A copy that cannot be proven equal at one marker fails.

Smallest safe move

Data Backfill + Snapshot into catalogdb

- **Scope:** data only; legacy code, proxy and the catalog read path stay unchanged
- **Change:** create catalogdb with its own schema and indices, record a snapshot marker, copy the products in throttled batches, compare counts and checksums
- **Safety net:** copy one consistent snapshot, never live rows (Live bulk copy is the trap); throttling keeps legacy load at its baseline
- **Escape route:** drop or reload catalogdb; the catalog read API keeps reading the legacy tables
- **Deferred:** Shared DB when catalog changes fit legacy releases and index cost stays flat; Live bulk copy is not brought back

```
-- every batch reads the snapshot taken at the marker, never the live rows
SELECT * FROM snapshot_at_marker.product ORDER BY product_id;
```

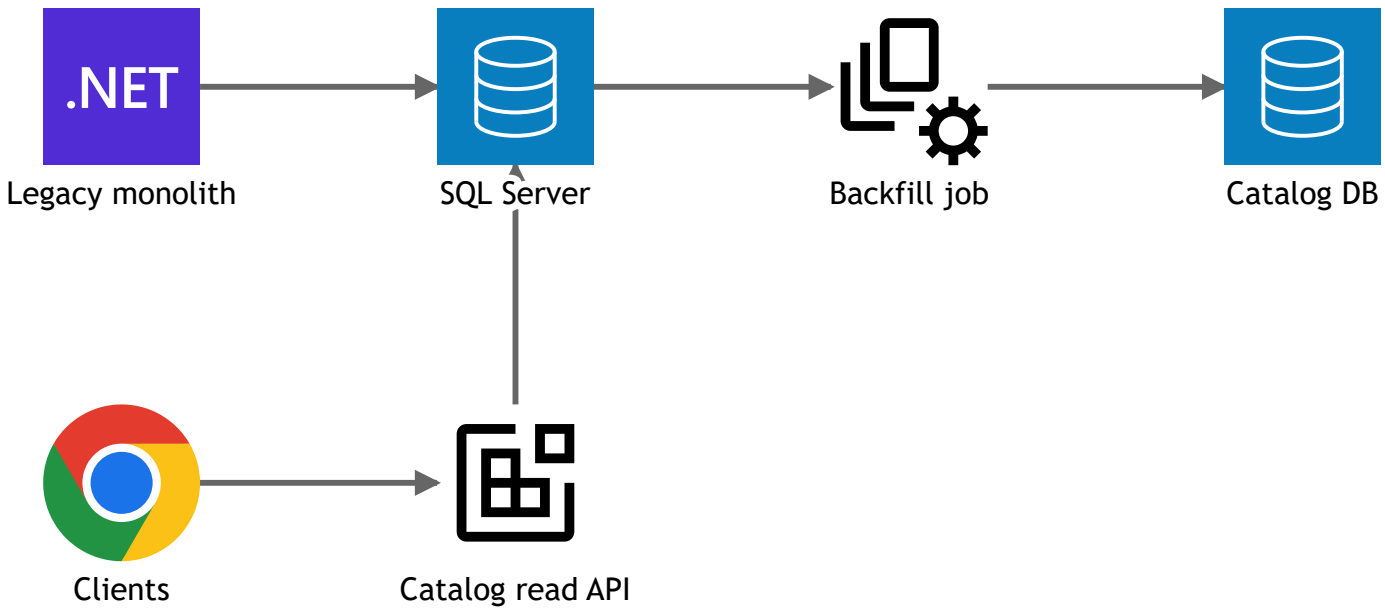
Verification: counts and checksums at one marker

Check	Pass condition
Row counts per product table at the marker	Equal in the legacy snapshot and in catalogdb for every table
Checksums per key range at the marker	Equal for every range; any mismatch is explained or fixed before the next step
Top catalog queries run against catalogdb as test traffic	Plans use the new database indexes, with no full-table scans
Legacy under normal load while the copy runs	Legacy query latency stays at its recorded baseline; the job slows itself above it
Fitness function	A scheduled job rebuilds a scratch copy from a fresh snapshot, compares counts and checksums per table and fails the pipeline on any difference; the marker is stored with the result

Rollback: routing rollback is not needed, because no traffic moves and the catalog read API still queries the legacy tables. State rollback concerns the copy only: drop or reload catalogdb. Legacy stays the only writer and is never changed, so no authority and no data are at risk.

Stop: when catalogdb matches the snapshot at the marker and carries its own schema and indices, nothing more is added: no synchronization, no read cutover and no writer change until the next constraint forces it.

Solution architecture



The catalog read API still queries the legacy tables. A backfill job has filled the new catalog database from a snapshot, and legacy remains the only writer.

Glossary

Used in this step

Term	Definition
Data Backfill + Snapshot	Copy the existing products into a new database at a stable snapshot marker
Shared DB	Several modules or services use same persistence system or schema
Live bulk copy	One-off script that copies live tables without a snapshot marker
Data Backfill	Populate a new representation using already-stored historical data
Snapshot marker	Stable position in legacy against which the copy is built and compared
Checksum	Short value computed from rows that changes when any row differs
Database Index	Data structure supporting efficient access paths
Catalog read API	The replacement that serves catalog reads, read-only
Routing proxy	Stable public entry point that sends each route to legacy or the replacement
Catalog database	The new store with its own schema and indices, backfilled from legacy
Routing rollback	Switching traffic or the writer flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Change Data Capture (CDC)	Detects and propagates changes made to database records.
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Database Per Service	Each service owns its private persistence and schema
Read Replica	Database copy serving read operations to distribute query load.
Anti-Corruption Layer (ACL)	Protects a domain model from incompatible external system models.

M23 · Data synchronization and writer handover

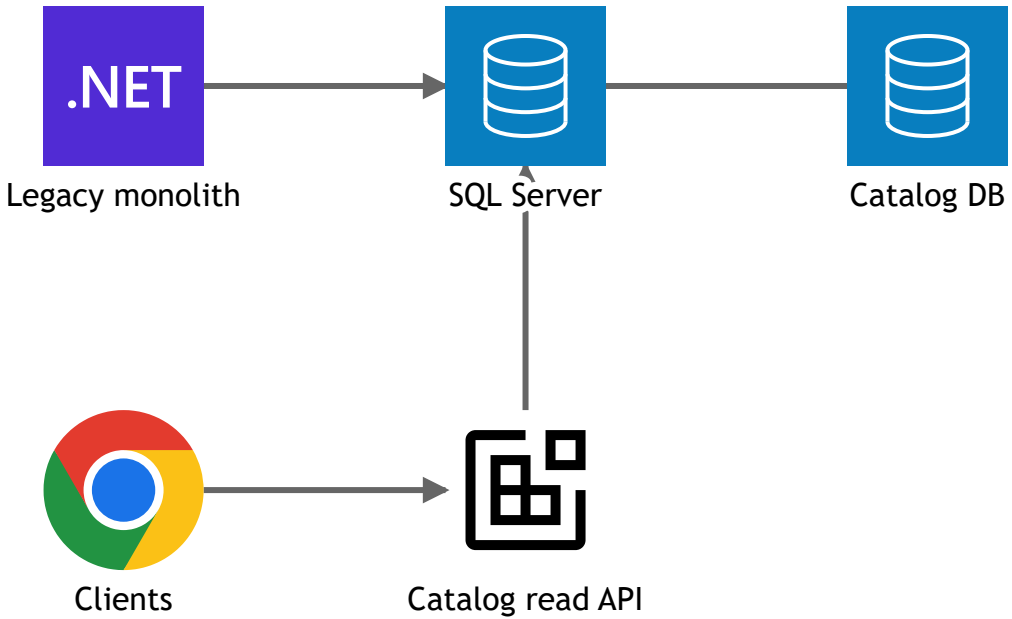
Keep database synchronized

Prices and availability change continuously, so every update and delete after the snapshot marker must reach the catalog database once and in order.

The catalog database holds a photograph of legacy

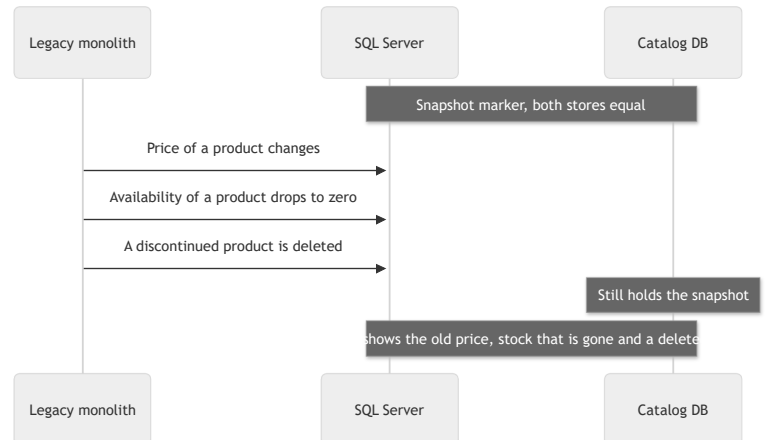
Component	What it is	What it does
Catalog database	The new store with its own schema and indices	Holds the snapshot copy, equal to legacy only at the marker
Snapshot marker	The recorded position of the backfill in legacy	Marks the point after which changes are missing
Catalog read API	The replacement that serves catalog reads	Still queries the legacy product tables
Legacy monolith	12-year-old .NET application on IIS	Stays the only writer and changes prices and availability all day
SQL Server	One shared relational database	Holds the live product tables, deletes included

Current architecture



The copy ages with every price change

The catalog database was equal to legacy only at the snapshot marker. Prices and availability change all day and products get deleted, so the copy drifts and cannot serve reads.



Snapshot only

the copy is correct at the marker and drifts with every change

Continuous changes

price and availability updates and deletes keep arriving in legacy

Every change applied

each update and delete after the marker once and in order, verified with offsets

Each option decides how changes leave the legacy database

Solution	What it is	What it does
CDC	Reads the legacy transaction log and emits every committed row change	Applies inserts, updates and deletes to the catalog database in commit order, resuming from an offset
Timestamp polling	A job asks the legacy tables for rows changed since the last timestamp	Copies the changed rows to the catalog database on a schedule
Outbox	Legacy writes a business event to an outbox table in the same transaction as the change	Publishes stable business events that the catalog database applies

Completeness costs maintenance, simplicity misses deletes

Solution	Gains	Costs
CDC	Captures updates and deletes No change to legacy code Commit order and resumable offsets	Log-based tooling to run and maintain Raw row changes, not business events Log retention and schema drift need care
Timestamp polling	Simple to build and troubleshoot No legacy code or log access needed	Deletes are missing Lag is at least the polling interval Writers that skip the timestamp are missed
Outbox	Business events, not raw rows Consumers independent of table layout Event recorded atomically with the change	Legacy code changes on every write path Old imports and admin writes are easily missed Event design and ownership to agree

Deciding signal: raw table changes or business events, and how many legacy write paths can change. Deletes that must propagate with legacy code left alone pick CDC; reliable timestamps, soft deletes and freshness longer than the polling interval allow Timestamp polling; business meaning with every write path changeable picks Outbox.

Smallest safe move

CDC from the snapshot marker into catalogdb

- **Scope:** data; legacy code, the catalog read path and routing stay unchanged, and legacy remains the only writer
- **Change:** enable capture on the product tables from the snapshot marker, apply inserts, updates and deletes to catalogdb in commit order
- **Safety net:** idempotent apply and a stored offset make restarts safe, a lag alert fires; Timestamp polling is no shortcut, it misses deletes
- **Escape route:** stop the capture job; replay from the last offset or reload from a fresh snapshot
- **Deferred:** Timestamp polling when timestamps are reliable, deletes are soft and freshness exceeds the interval; Outbox when every legacy write path can emit events

```
capture  product tables, starting at the snapshot marker
apply    insert, update, delete by key, in commit order
store    the last applied offset after every batch
```

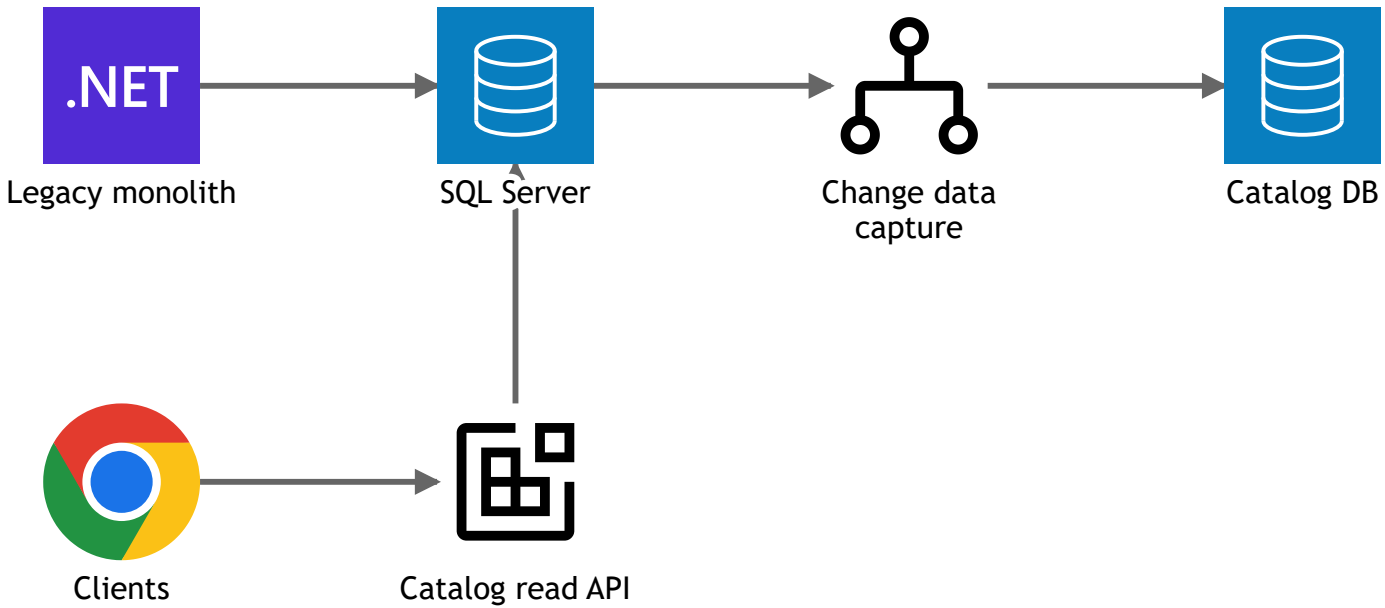
Verification: updates, deletes and order with offsets

Check	Pass condition
Burst of price and availability updates in legacy	Each change reaches catalogdb once; final values equal legacy
Products deleted in legacy	Rows are removed in catalogdb and no orphans are left
Several changes to one product in quick succession	Applied in commit order; catalogdb ends with the last value
Capture job stopped and restarted mid-stream	Resumes from the stored offset with no gap and no duplicate effect
Fitness function	A scheduled probe updates and deletes a marker product in legacy and fails if catalogdb does not show both within the agreed lag; offset lag is alerted; counts are compared per table after each run

Rollback: routing rollback is not needed, because no traffic moves and the catalog read API still queries the legacy tables. State rollback concerns the copy only: stop the capture, then replay from the last offset or reload from a fresh snapshot. The pipeline never writes to legacy, so no authority and no data are at risk.

Stop: when the feed keeps catalogdb within the agreed lag for updates and deletes, no outbox, no business event layer and no second synchronization mechanism is added.

Solution architecture



Product changes now flow from SQL Server into the catalog database through change data capture, while the catalog read API still queries the legacy tables and legacy remains the only writer.

Glossary

Used in this step

Term	Definition
Change Data Capture (CDC)	Detects and propagates changes made to database records.
Timestamp polling	Job that repeatedly asks for rows changed since the last timestamp
Outbox	Atomically records business changes and outgoing events in the same database transaction.
Polling	Repeated requests to determine whether data changed
Offset	Position in the change feed up to which changes have been applied
Lag	Delay between a change in legacy and its appearance in catalogdb
Idempotent apply	Applying the same change twice leaves the same result
Commit order	Order in which legacy committed changes, kept when they are applied
Snapshot marker	Stable position in legacy against which the copy is built and compared
Catalog database	The new store with its own schema, backfilled and kept in sync from legacy
Routing rollback	Switching traffic or the writer flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
SQL Server Change Tracking	SQL Server facility tracking changed row keys and versions
Asynchronous Replication	Acknowledge commit before all replicas confirm persistence
Materialized Query Projection	Read-optimized representation built from updates or events
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Anti-Corruption Layer (ACL)	Protects a domain model from incompatible external system models.

M24 · Data synchronization and writer handover

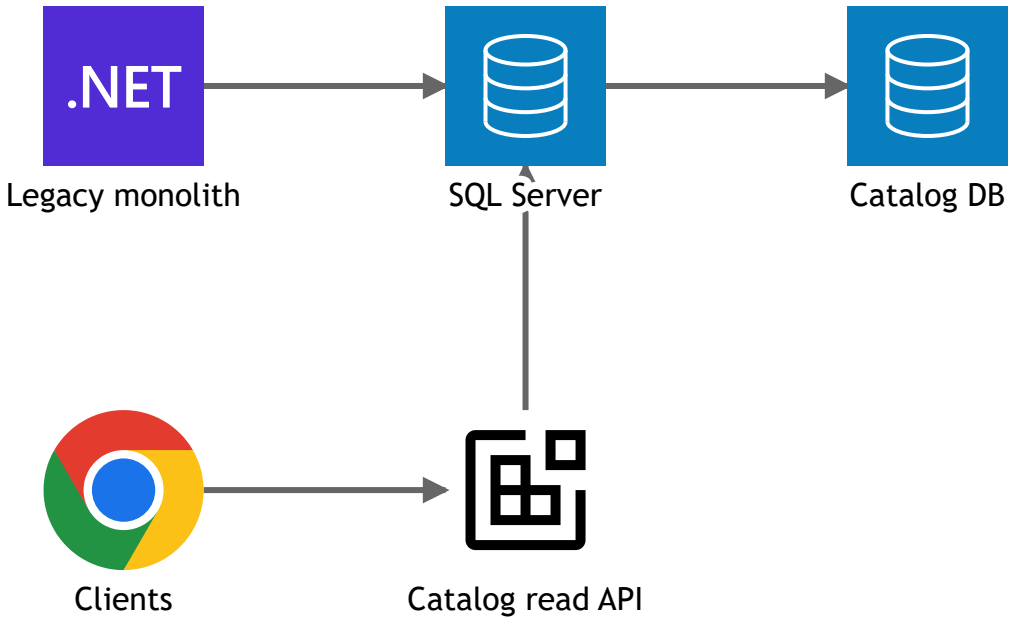
Translate between domain models

The new catalog model cannot share the old tables as they are, so legacy SKU and StockItem records need a translation into the catalog model.

Changes reach the catalog database in legacy shape

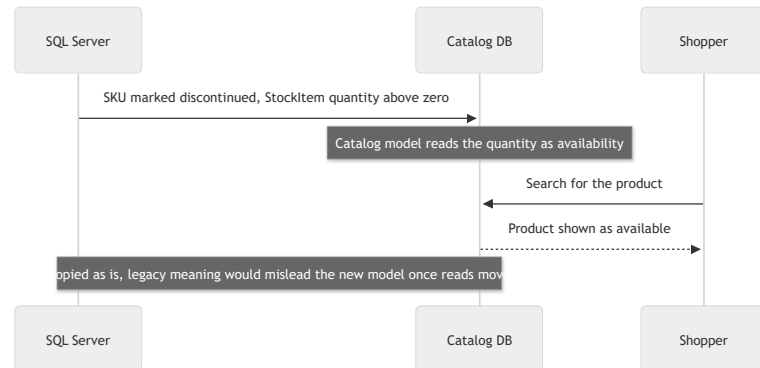
Component	What it is	What it does
SKU	Legacy record of a sellable item with its price and pack data	Uses words that differ in meaning from the catalog model
StockItem	Legacy record of the stock quantity of a sellable item	Describes what is in stock in legacy terms, not in catalog terms
Catalog database	The new store with its own schema and indices	Receives backfilled rows and feed changes in the legacy shape
Catalog read API	The replacement that serves catalog reads	Still queries the legacy product tables
Legacy monolith	12-year-old .NET application on IIS	Stays the only writer of SKU and StockItem data

Current architecture



Copied legacy rows mislead the catalog model

Legacy keeps a SKU and a separate StockItem, and the same word can mean different things in each. Copied as they are, the rows put legacy meaning into the catalog model.



Different semantics

legacy SKU and StockItem rows carry meanings the catalog model does not share

Legacy shape in

backfilled rows and every change from the feed enter catalogdb as they are

Lifecycle matches

transformed lifecycle equals the domain examples for every mapped field

Each option decides who owns the meaning

Solution	What it is	What it does
ACL + mappings	An anti-corruption layer translates legacy rows into the catalog model through explicit mappings	Maps SKU and StockItem records to catalog products, variants, prices and availability before they reach catalogdb
Identical schema	The catalog database copies the legacy table layout and column meanings unchanged	Carries SKU and StockItem tables across one to one, so nothing needs translating
Database View over raw copy	Raw legacy tables are copied unchanged and database views present them in the catalog shape	Catalog queries read the views, so the translation runs inside the database at query time

Isolating the model costs code, copying it costs debt

Solution	Gains	Costs
ACL + mappings	Domain model isolated from legacy Mappings explicit and testable Catalog schema free to differ	Mapping code to build and maintain Legacy semantics must be understood first One more component in the feed
Identical schema	Easy: no mapping code Feed is a straight copy	Legacy model leaks into the catalog Schema debt carried into the replacement Own schema and indices lose their point
Database View over raw copy	Cheap: SQL only, no new service Raw copy stays complete	Logic hidden in database views Translation cost paid on every read Legacy shape stays in storage

Deciding signal: how many served fields differ in meaning between legacy and the catalog model, and how often legacy changes. None means Identical schema is enough; only renames and joins that fit the read budget allow Database View over raw copy; any difference in lifecycle or identity needs ACL + mappings, because only code rejects unmapped values and passes domain examples.

Smallest safe move

ACL + mappings between the feed and catalogdb

- **Scope:** code and data; one translation layer for the served fields; routing, the read path and legacy stay unchanged
- **Change:** map SKU and StockItem to product, variant, price and availability; the backfill and the feed both go through it
- **Safety net:** unmapped values are rejected and logged, never guessed; Identical schema is no shortcut; both paths share one mapping version
- **Escape route:** disable the layer; reload catalogdb from the snapshot marker with the previous mapping version
- **Deferred:** Database View over raw copy when differences are only renames and joins; Identical schema where meaning already matches

```
# example rule, illustrative
SKU discontinued and StockItem quantity above zero
  → variant status discontinued, availability not sold
unmapped value → reject and log, never guess
```

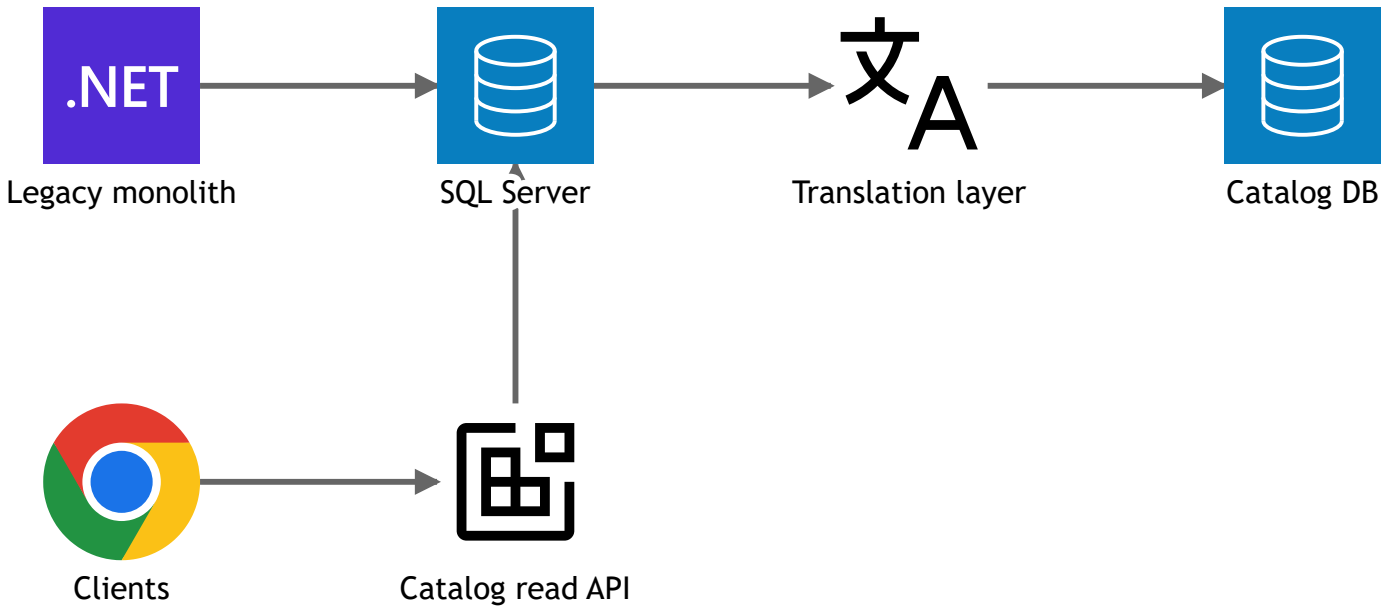
Verification: transformed lifecycle against domain examples

Check	Pass condition
Domain examples for the SKU lifecycle, agreed with merchandisers	Each legacy example maps to the expected catalog product and variant state
Domain examples for StockItem quantity cases, including zero	Availability equals the expected state for every case
Unmapped or invalid legacy value injected	Rejected and logged; no guess and no row dropped silently
Same legacy rows sent through the backfill and through the change feed	Both paths produce identical catalog rows
Fitness function	The domain examples run as contract tests in the pipeline and fail on any difference; a production counter of rejected records is alerted above zero until each one is explained

Rollback: routing rollback is not needed, because no traffic moves and the catalog read API still queries the legacy tables. State rollback concerns the copy only: disable the layer and reload catalogdb from the snapshot marker with the previous mapping version. Legacy stays the only writer, so a wrong mapping is repaired forward by correcting it and reloading.

Stop: when the mappings pass the domain examples for the fields the catalog serves, no canonical model, translation platform or extra schema layer is added; unmapped fields wait until the catalog needs them.

Solution architecture



Legacy records now reach the catalog database only through the translation layer, which maps SKU and StockItem to the catalog model; the read path and the single legacy writer are unchanged.

Glossary

Used in this step

Term	Definition
ACL + mappings	An anti-corruption layer that translates legacy records into the catalog model through explicit mappings
Identical schema	Catalog database with the legacy table layout and column meanings unchanged
Database View over raw copy	Raw legacy tables copied unchanged, with views presenting them in the catalog shape
Anti-Corruption Layer (ACL)	Protects a domain model from incompatible external system models.
Database View	Named query presenting an abstraction over underlying tables
SKU	Legacy record of a sellable item with its price and pack data
StockItem	Legacy record of the stock quantity of a sellable item
Catalog model	The new catalog's own meaning of product, variant, price and availability
Domain example	A legacy record paired with the catalog state it must produce
Mapping	Rule that turns one legacy value into one catalog value
Routing rollback	Switching traffic or the writer flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Domain Translation Adapter	Translate one system's terms and records into another's semantics
Bounded Context	Boundary inside which a domain model and vocabulary have specific meaning
Shared Database	Several modules or services use same persistence system or schema
Schema Compatibility View	Database view presenting old shape over new data
Expand-Contract	Add compatible representation, migrate consumers, then remove old representation

M25 · Data synchronization and writer handover

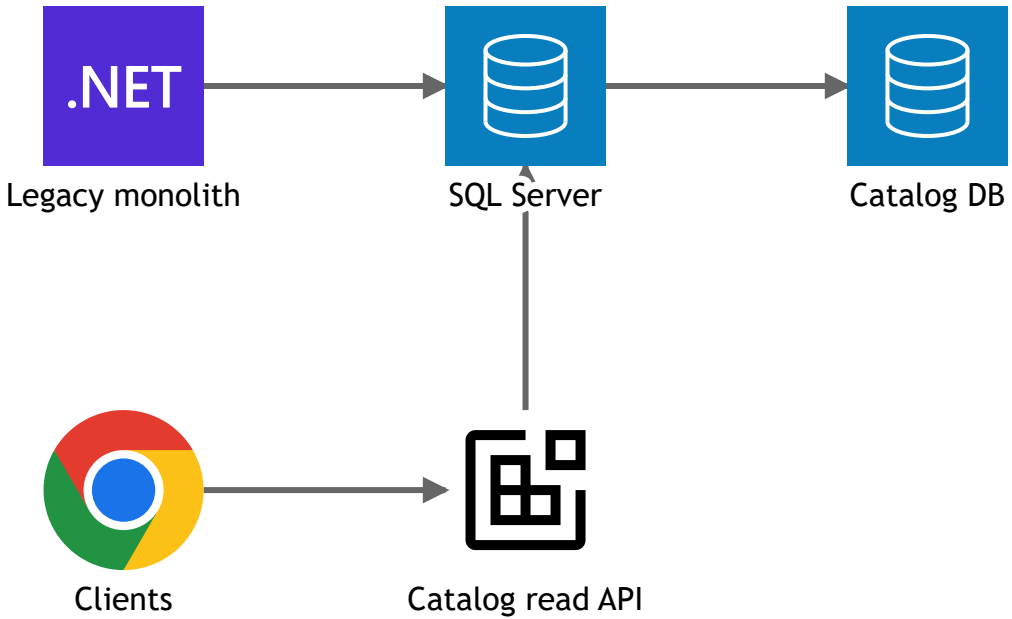
Reconcile state

A cutover cannot happen while discrepancies exist, so legacy and the catalog database must be compared until no drift is left unexplained.

Two databases hold products and nobody has compared them

Component	What it is	What it does
SQL Server	One shared relational database	Holds the authoritative product rows; legacy is the only writer
Catalog database	The new store with its own schema and indices	Filled by backfill, then kept current by the feed through the translation
Catalog read API	The replacement that serves catalog reads	Still queries the legacy product tables
Legacy monolith	12-year-old .NET application on IIS	Keeps changing prices and availability and deleting products

Current architecture



A cutover needs proof, not hope

Two databases now hold products. Nobody can say whether they agree: a missed update or a surviving deleted product would show only after reads move.

No comparison

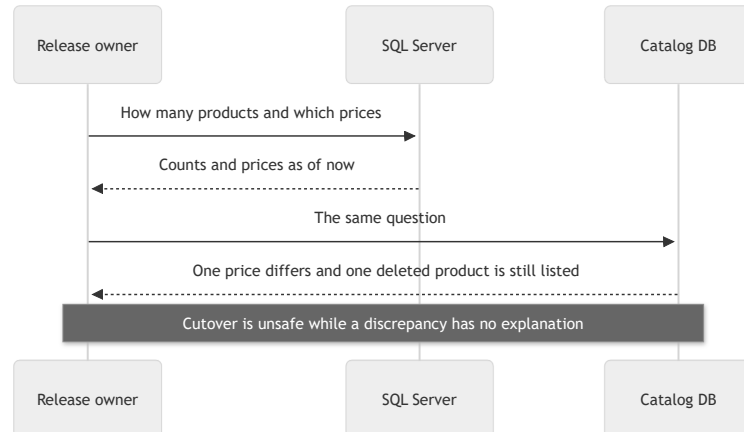
nobody can say whether legacy and catalogdb hold the same products

Everything has run

backfill, the feed and translation worked while legacy kept changing

Zero unexplained drift

for sampled and critical keys before any cutover



Each option decides what is compared

Solution	What it is	What it does
Data Reconciliation	Compare the two stores by key with counts, checksums and domain invariants	Runs a read-only job over key ranges and reports every difference as explained or unexplained
Differential Reads	Send the same catalog query to legacy and to catalogdb and compare the answers	Shows the result differences a user would see, such as filters and rounding
Manual spot checks	People compare a handful of products in the old and the new screens	Looks for visible differences in chosen examples

Data proof costs effort, result proof costs load

Solution	Gains	Costs
Data Reconciliation	Proof of migration correctness Finds missing, extra and changed rows Cheap to rerun on a schedule	Effort to build and tune Edge cases need rules, such as changes in flight Compares data, not query results
Differential Reads	Actual results users would see Catches query and mapping differences	Extra load on both stores Legitimate differences need rules Covers only the queries that are sent
Manual spot checks	No tooling needed Quick to start	Samples hide missing and changed rows Cannot prove zero unexplained drift Not repeatable

Deciding signal: whether the risk sits in the data or in the results. Counts, checksums and invariants that compare clean for sampled and critical keys pick Data Reconciliation; data that matches while query results differ in filters or rounding adds Differential Reads; Manual spot checks cannot show zero unexplained drift and approve nothing. Measure key count, change rate, changes in flight and the extra load on each store.

Smallest safe move

Data Reconciliation over critical keys and a sample

- **Scope:** data only; a read-only job over legacy and catalogdb; the feed, routing and legacy stay unchanged
- **Change:** compare counts and checksums of translated legacy rows per key range, plus domain invariants; classify each difference as explained or unexplained
- **Safety net:** the job never writes; changes in flight count as explained only within the lag; Manual spot checks prove nothing
- **Escape route:** stop the job; reload the affected keys from legacy through the mapping and compare again
- **Deferred:** Differential Reads when data matches but results differ, and before reads move; Manual spot checks only as a first look

```
for each key range, critical keys and a sample:
  compare count and checksum of translated legacy rows with catalogdb
  explained = in flight within the lag, or a known mapping rule
  unexplained must be zero before any cutover
```

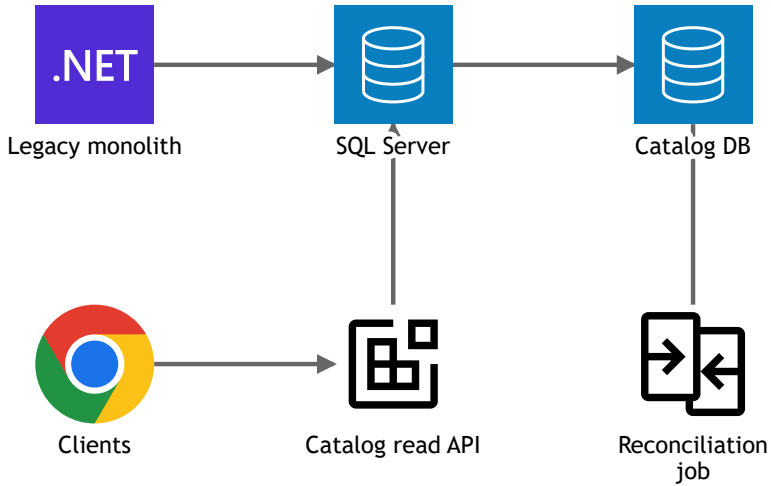
Verification: zero unexplained drift for sampled and critical keys

Check	Pass condition
Counts and checksums per key range, translated legacy rows against catalogdb	Equal, apart from changes in flight within the lag
Domain invariants on catalogdb for critical keys	No violation: every variant has a price and a valid availability
Drift injected into a scratch copy: one changed price and one deleted product	Both reported as unexplained, so the job can fail
Every reported difference	Classified as explained or repaired; unexplained count is zero before any cutover
Fitness function	A scheduled run publishes the unexplained count per table; a pipeline gate and the later cutover runbook refuse to proceed unless it is zero for sampled and critical keys

Rollback: routing rollback is not needed, because no traffic moves and the job only reads. State rollback concerns the copy only: a defect is repaired by a Forward Fix, reloading the affected keys from legacy through the mapping, never by editing catalogdb by hand. Legacy stays the only writer, so no authority is at risk.

Stop: when reconciliation reports zero unexplained drift for sampled and critical keys, no further comparison layer is added; Differential Reads wait for the read cutover.

Solution architecture



A read-only reconciliation job now compares the catalog database with SQL Server, translated the same way as the feed, and reports every difference as explained or unexplained.

Glossary

Used in this step

Term	Definition
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Differential Reads	The same read sent to both stores and the answers compared
Manual spot checks	People compare a few chosen products by eye
Unexplained drift	A difference between the stores with no known cause
Changes in flight	Changes committed in legacy that the feed has not applied yet
Checksum	Short value computed from rows that changes when any row differs
Domain invariant	Business rule that must hold in every state of the data
Sampled and critical keys	A random sample of keys plus the keys whose errors would cost most
Forward Fix	Repair the new implementation without restoring the old system
Catalog database	The new store with its own schema, backfilled and kept in sync from legacy
Routing rollback	Switching traffic or the writer flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Data Cutover	Point at which authority for selected reads or writes changes to new system
Differential Testing	Compare outputs of old and new implementations for same input
Shadow Traffic	Send production requests to a candidate implementation without using candidate responses
Canary Routing	Shift a small controlled segment of live traffic to a new implementation
Write Ownership Handover	Transfer authoritative writes for bounded data from legacy to replacement

M26 · Data synchronization and writer handover

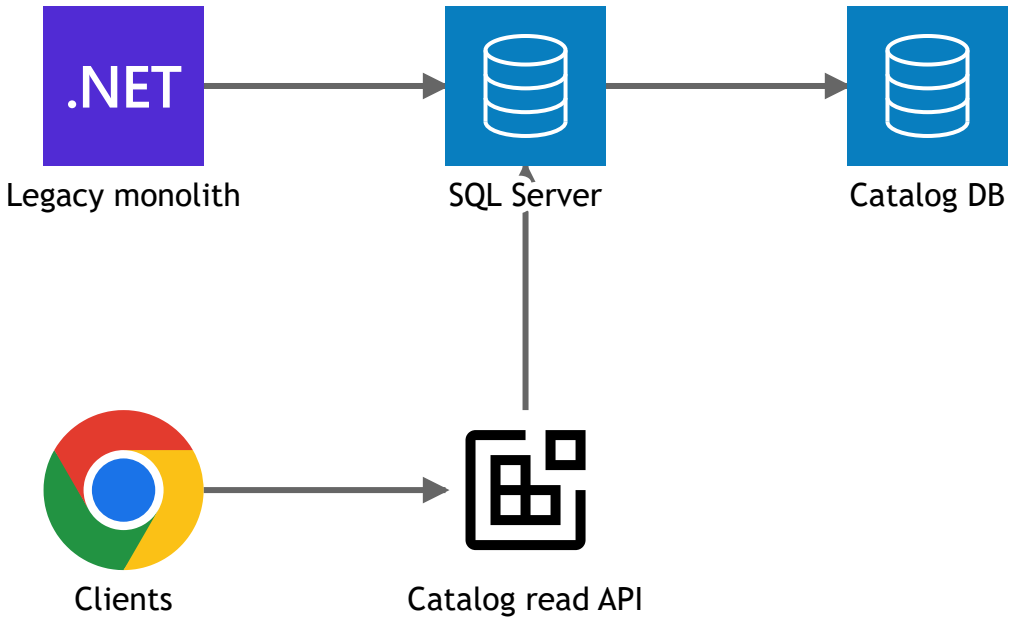
Cut new reads to replica

Catalog p95 ≤150 ms without old DB queries, so reads move to the synchronized database under a lag threshold, with no silent fallback.

The new database is current, but nothing reads it

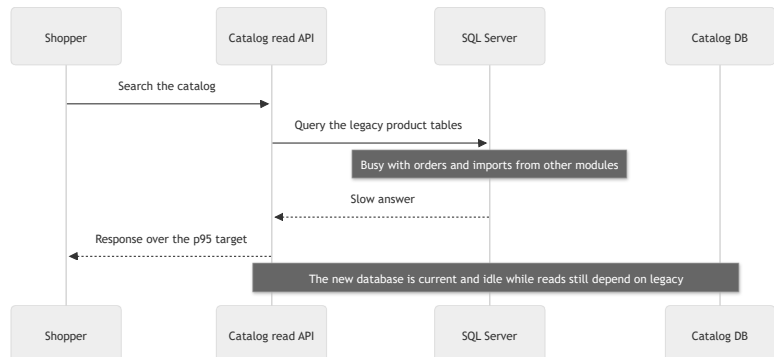
Component	What it is	What it does
Catalog database	The new store with its own schema and indices	Reconciled with legacy and kept current by the feed
Catalog read API	The replacement that serves catalog reads	Still queries the legacy product tables, the old database
SQL Server	One shared relational database	Answers catalog reads and the queries of every other module
Legacy monolith	12-year-old .NET application on IIS	Stays the only writer of product data
Feature flags	Runtime switches for single behaviors	Switch one route or source without undoing other changes

Current architecture



A reconciled database that nothing reads

The new database is reconciled, yet every catalog read still queries the legacy tables. Moving reads over exposes replication lag to shoppers, and a quiet fallback would hide a defect instead of showing it.



Old DB reads

every catalog read still queries the old database

Changes keep flowing

price and availability changes arrive through the feed while reads move

p95 ≤150 ms

with 0 old DB queries and lag within the max threshold

Each option decides what answers a catalog read

Solution	What it is	What it does
New DB direct	The catalog read API reads only from catalogdb	Serves every catalog read from the synchronized database and never queries the legacy tables
Hybrid fallback	The catalog read API tries catalogdb first and falls back to the legacy tables on a miss, an error or a stale answer	Keeps the legacy tables as a per-request safety net behind the new database
TTL cache on legacy reads	A cache with a time-to-live sits in front of the legacy tables	Answers repeated catalog reads from memory and queries legacy only on a miss

Independence costs lag, a safety net costs visibility

Solution	Gains	Costs
New DB direct	Independence from legacy load Own indices serve the p95 target Defects stay visible	CDC lag shows to shoppers Staleness limit needs enforcement Defects reach shoppers until switched back
Hybrid fallback	Resilience: legacy answers when the new DB cannot Gentle first cutover	Conceals defects Old DB queries continue Two read paths to run and test
TTL cache on legacy reads	Cheap: no data movement Fast for repeated reads	Misses still query the old DB TTL alone cannot bound staleness Cold and expired entries break p95

Deciding signal: the staleness the business accepts against observed lag, plus p95 and the old DB query count. Peak lag within the max lag threshold (for example 30 s, illustrative) with p95 ≤150 ms and 0 old DB queries picks New DB direct; lag that cannot be bounded is fixed in the feed, since Hybrid fallback only hides it; a TTL cache cannot bound staleness and still queries the old DB on every miss.

Smallest safe move

New DB direct behind a read-source flag

- **Scope:** traffic and data; one read source moves by canary routing cohort; the feed and the single legacy writer stay
- **Change:** a read-source flag points a cohort at catalogdb; the offset enforces the max lag threshold; Differential Reads compare outputs
- **Safety net:** no silent fallback, since Hybrid fallback conceals defects; a lag breach alerts and flips the flag visibly
- **Escape route:** set the read-source flag back to the legacy tables
- **Deferred:** Hybrid fallback when a stale answer beats an error and each fallback is alerted; TTL cache on legacy reads for staleness-tolerant reads

```
read-source flag = catalogdb for cohort N, widened until every read
lag above the max threshold → alert, visible flip back to the legacy tables
```

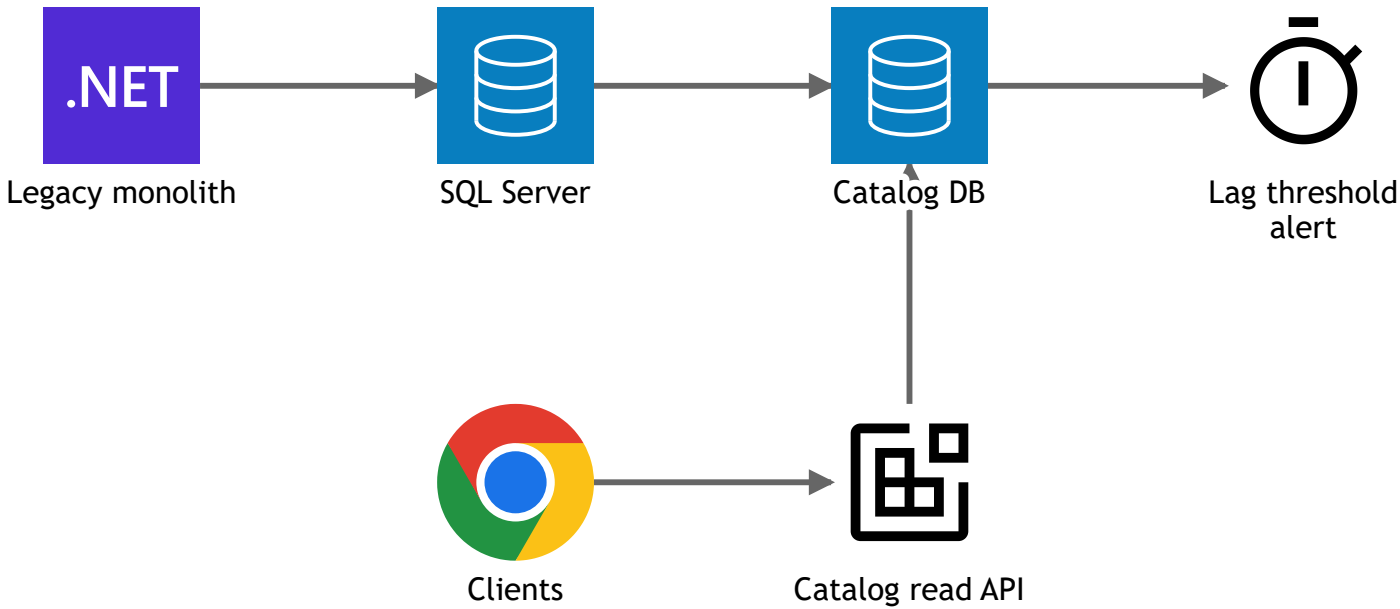
Verification: lag threshold and compared outputs

Check	Pass condition
Catalog reads from catalogdb under production-like load	p95 ≤150 ms, with 0 queries from the catalog read path on the legacy tables
Burst of price and availability changes through the feed	Lag stays within the max lag threshold (for example 30 s, illustrative); a breach raises an alert
The same sampled requests sent to the legacy tables and to catalogdb	Outputs equal; every difference is explained by a rule or fixed
Read-source flag switched back to the legacy tables	Reads served by legacy after one configuration change, with no failed requests
Fitness function	A pipeline load test asserts p95 ≤150 ms and zero queries on the legacy tables; an alert compares the feed offset with the max lag threshold; a scheduled output comparison fails on unexplained differences

Rollback: routing rollback: set the read-source flag back to the legacy tables. It is safe because legacy is still the only writer and holds every change, so authority has not moved. State rollback is not needed for reads; if catalogdb is wrong, repair it forward by reloading the affected keys from legacy and reconcile again.

Stop: when catalog reads meet p95 ≤150 ms from catalogdb with no old DB queries, lag stays within the threshold and outputs match, no hybrid fallback, cache or extra read layer is added.

Solution architecture



The catalog read API now reads the catalog database, so no catalog read touches SQL Server. A lag alert guards the feed, and legacy remains the only writer.

Glossary

Used in this step

Term	Definition
New DB direct	The catalog read API reads only from the catalog database
Hybrid fallback	Reads try the new database first and fall back to legacy on a miss, an error or a stale answer
TTL cache on legacy reads	A cache with a time-to-live in front of the legacy tables
Max lag threshold	Largest delay between legacy and the catalog database that the business accepts for served data
Lag	Delay between a change in legacy and its appearance in catalogdb
Offset	Position in the change feed up to which changes have been applied
Read-source flag	Switch that selects whether the catalog read API reads the legacy tables or the catalog database
Differential Reads	The same read sent to both stores and the answers compared
Canary Routing	Shift a small controlled segment of live traffic to a new implementation
Catalog database	The new store with its own schema, backfilled and kept in sync from legacy
Routing rollback	Switching traffic or the writer flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Feature Flag	Runtime control enabling behavior without deploying new code
Read Replica	Database copy serving read operations to distribute query load.
Asynchronous Replication	Acknowledge commit before all replicas confirm persistence
In-Memory Cache	Stores frequently accessed data in memory for faster retrieval.
Circuit Breaker	Stops requests to failing dependencies temporarily.
Data Cutover	Point at which authority for selected reads or writes changes to new system

M27 · Data synchronization and writer handover

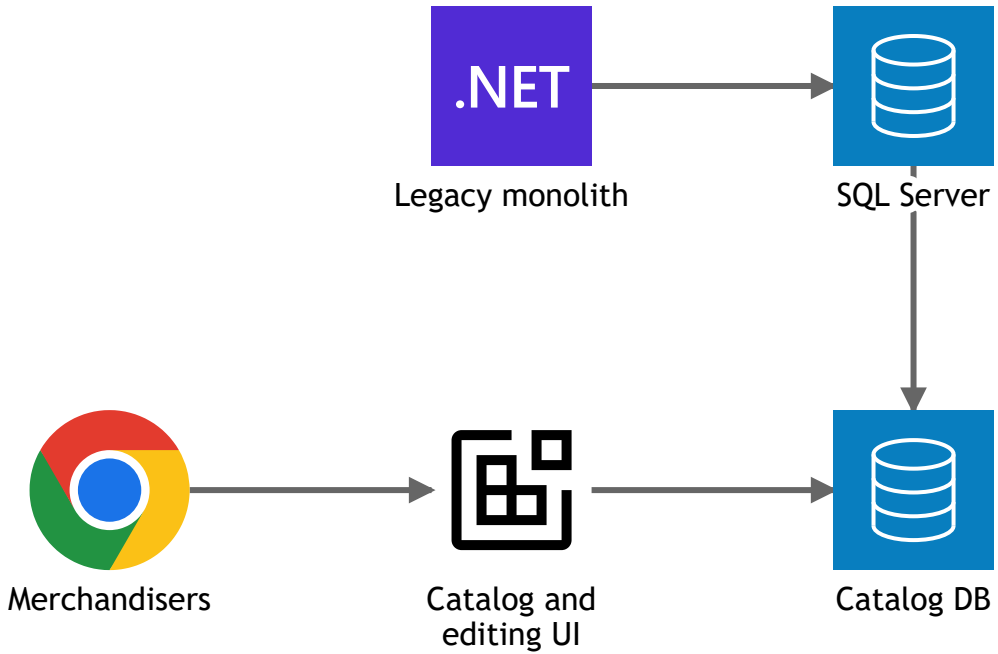
Keep legacy as write owner initially

Merchandisers need changes while old processes still update product, so exactly 1 write authority per product must accept them.

The editing UI exists, but legacy is still the write owner

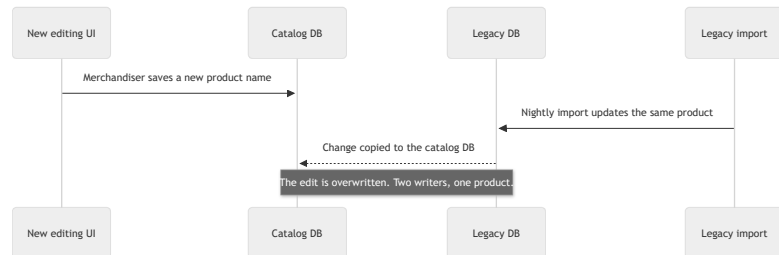
Component	What it is	What it does
New editing UI	The new screen where merchandisers change products	Shows products from the catalog database; has no write path yet
Catalog service	The replacement catalog application	Serves catalog reads from the catalog database
Catalog database	New store with its own schema, backfilled and kept in sync from legacy	Receives every legacy product change through change data capture
Legacy monolith	12-year-old .NET application with imports and admin screens	Acts as write owner: still updates the product tables
SQL Server	One shared relational database	Holds the authoritative product rows

Current architecture



Two writers, one product

Merchandisers have a new editing UI, yet old imports and admin processes keep updating the same products in legacy. A UI that writes to the new database gets overwritten by an old process.



Two writers per product
the new editing UI and the old processes

Old imports and admin processes
still update product tables in legacy

1 write authority
per product: only the old primary accepts writes

Every option answers who may write a product

Solution	What it is	What it does
New UI calls legacy writer	The new editing UI sends every product change to the existing legacy write path	Legacy stays the only writer; each change reaches the catalog database through the existing synchronization
Dual Write	Attempt to update two independent stores from one business operation	The new UI writes each edit to legacy and to the catalog database in the same request
Early partition handover	Make the catalog database the only writer for one bounded product partition now	Sends edits for that partition to the new store while the rest stays with legacy

Coupling is cheaper than two authorities

Solution	Gains	Costs
New UI calls legacy writer	One authority per product Legacy validation and concurrency rules still apply No new write logic to prove	New UI coupled to the legacy write path Saved edit visible only after sync lag Product changes still ship with legacy releases
Dual Write	Simple prototype sync Both stores current when both writes succeed	Partial failure leaves the stores different Split brain: two authorities per product No single order of writes
Early partition handover	Independent deployment sooner Smaller blast radius than all products	Needs fencing, draining and reconciliation first Routing by partition key Old processes touching the partition lose updates

Deciding signal: ownership change, partial write failure, conflict risk and the aggregate invariant. Count the old processes still writing product tables and check reconciliation for unexplained drift: while either remains, legacy stays the only writer. One partial failure that leaves the stores different disqualifies Dual Write; a partition moves only after its old writers are fenced and reconciliation is clean.

Smallest safe move

Legacy stays the only writer

- **Scope:** write ownership and code; routing and synchronization stay unchanged
- **Change:** a writer flag selects legacy; the new UI calls a new legacy edit endpoint; application roles lose write grants on the catalog database
- **Safety net:** Dual Write is blocked by construction, as only the synchronization account writes the catalog database; UI edits and old imports meet in one store under legacy concurrency control
- **Escape route:** turn the writer flag off; merchandisers edit in the legacy screens again
- **Deferred:** Early partition handover once its old writers are fenced and reconciliation shows zero unexplained drift; Dual Write only for data that tolerates divergence

```
REVOKE INSERT, UPDATE, DELETE ON catalog.product FROM catalog_app;
-- only the synchronization account keeps write access
```

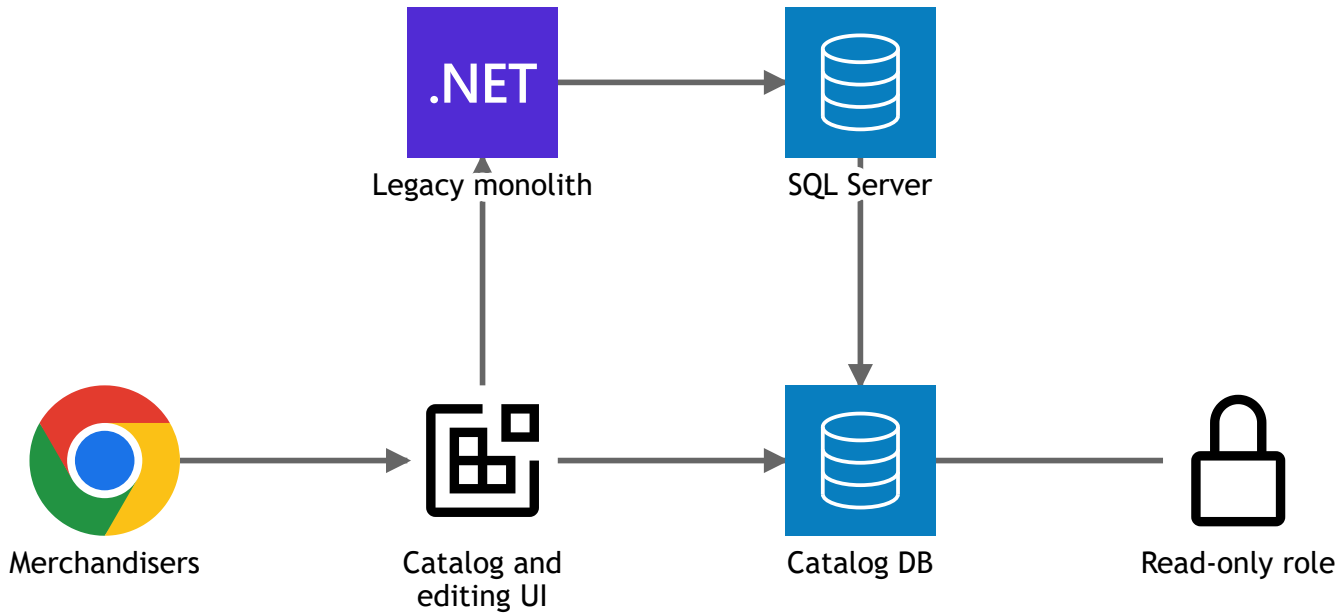
Verification: only the old primary accepts writes

Check	Pass condition
Write attempt on the catalog database with every application role	Rejected each time; only the synchronization account can write
Edit saved through the new UI	One updated row in legacy, same value in the catalog database after sync, zero unexplained drift in reconciliation
Import and UI edit on the same product at the same time	Both applied by legacy in one order, no edit lost, stores equal after sync
Legacy writer made unavailable during an edit	UI reports failure; neither store changed; no partial write
Fitness function	A pipeline test attempts a write on the catalog database with each application account and fails if any succeeds; scheduled reconciliation of counts and checksums alerts on unexplained drift

Rollback: routing rollback: switch the writer flag off and merchandisers edit in the legacy screens again. State rollback is not needed, because every edit already landed in legacy, the only authority, and synchronization carries it on; if the catalog database drifts, reconciliation repairs it from legacy.

Stop: when the new UI saves through the legacy writer and only the old primary accepts writes, no second writer and no dual write is added.

Solution architecture



The same stores and synchronization, plus an edit call from the catalog service to the legacy writer and a read-only role on the catalog database for application accounts.

Glossary

Used in this step

Term	Definition
New UI calls legacy writer	The new editing UI sends every product change to the existing legacy write path
Dual Write	Attempt to update two independent stores from one business operation
Early partition handover	Make the catalog database the only writer for one bounded product partition
Write owner	The one system whose writes are authoritative for a piece of data
New editing UI	The new screen where merchandisers change products
Catalog database	The new store with its own schema, backfilled and kept in sync from legacy
Change Data Capture (CDC)	Detects and propagates changes made to database records.
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Writer flag	Switch that selects which writer the new UI calls
Split brain	Two systems each act as the owner of the same data
Read-only role	Database role that can read but not change data

Related concepts

Term	Definition
Write Ownership Handover	Transfer authoritative writes for bounded data from legacy to replacement
Write Fencing	Prevent stale writers from modifying state after ownership changes
Transactional Outbox	Atomically records business changes and outgoing events in the same database transaction.
Data Cutover	Point at which authority for selected reads or writes changes to new system
Rollback	Return to a previously running version or state

M28 · Data synchronization and writer handover

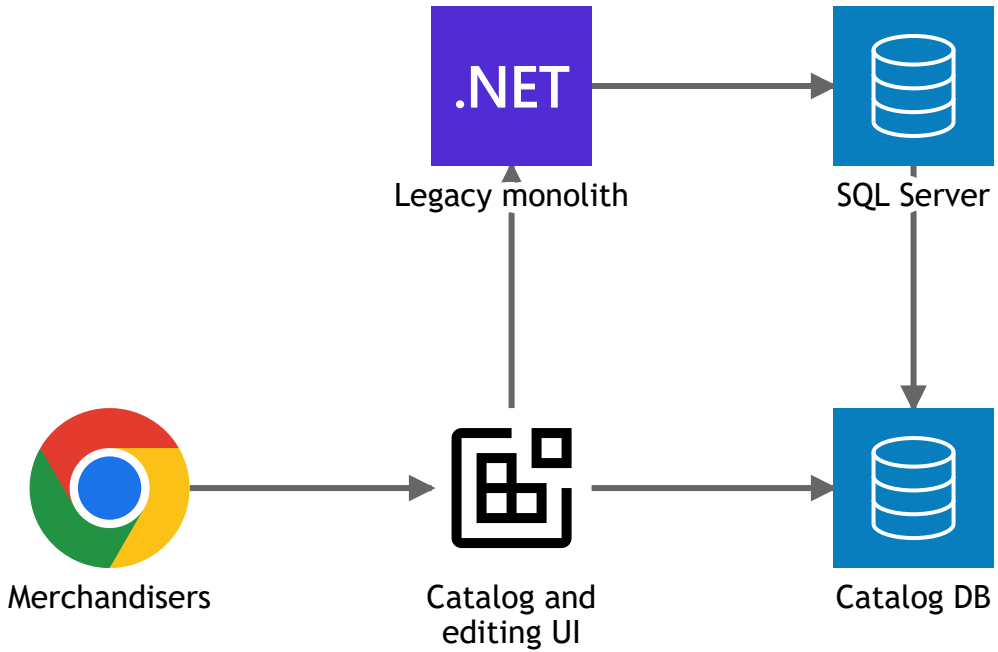
Expand compatible schema

The legacy schema must evolve for new catalog attributes without breaking old consumers.

New catalog attributes have no home in the legacy schema

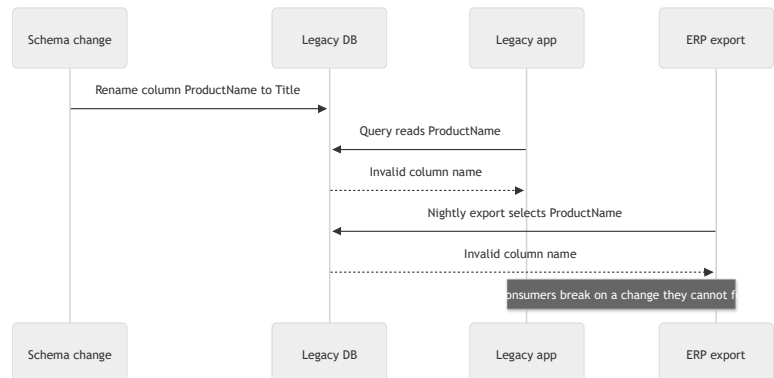
Component	What it is	What it does
New catalog attributes	Product attributes the catalog needs, for example display badges or search keywords	Have no column in the legacy tables
Legacy writer	The existing code path that saves products	Receives every edit from the new editing UI
SQL Server	One shared relational database	Holds the product tables that every old consumer reads
Old consumers	Legacy code, the nightly ERP export and reports	Read the same tables and cannot change in the same release
Catalog database	The new store, kept in sync from legacy	Receives product changes through change data capture

Current architecture



A renamed column stops every old reader

The new catalog needs attributes that legacy tables lack, so the legacy schema has to grow. The application, the nightly ERP export and the reports read the same tables and cannot all change in one release.



No column

for the new attributes, so edits cannot save them

Old consumers

read the same tables and cannot change together

Old and new versions

both run against the expanded schema

Each option changes the schema at a different pace

Solution	What it is	What it does
Expand-Contract	Add the compatible representation, migrate consumers, then remove the old representation	Adds new columns beside the old ones, backfills them, switches writers and readers, and removes old columns last
Immediate rename	Rename or replace the old column in a single deployment	Gives the schema its final shape at once
Database View	A named query that presents the old shape over the new tables	Lets the tables change underneath while old readers query the old name

Compatibility costs duplication, a breaking change costs consumers

Solution	Gains	Costs
Expand-Contract	Safe schema evolution Old and new versions run side by side Each phase can be undone	Temporary duplicated formats Backfill and two-column upkeep Old columns need an explicit sunset
Immediate rename	No duplicated formats Final schema from day one	Breaks every old consumer at once All consumers must release together No way back without restoring data
Database View	Keeps old readers operational Schema compatibility boundary	Legacy schema remains an obligation Can obscure cost and dependencies Old writers need extra work

Deciding signal: number of unmigrated consumers, retention requirements and how fast the interface evolves. Named-column queries ignore added columns, while SELECT-star readers and positional CSV exports need a check. A consumer that must see a restructured table by its old name moves the choice to a Database View; only when every consumer can release together is an immediate rename safe.

Smallest safe move

Expand-Contract: expand phase only

- **Scope:** data and code; write ownership and traffic stay unchanged
- **Change:** add nullable columns, backfill them, deploy the legacy writer that fills them and include them in the change data capture feed
- **Safety net:** Immediate rename is never applied; old columns and views get an owner, a removal trigger and a sunset; nullable columns keep old inserts working
- **Escape route:** deploy the previous application version; it ignores the new columns
- **Deferred:** Database View when a restructured table must keep its old name; column removal after a consumer audit shows no reader; Immediate rename only when every consumer releases together

```
ALTER TABLE product ADD badge_text NVARCHAR(MAX) NULL;  -- additive: old inserts still succeed
```

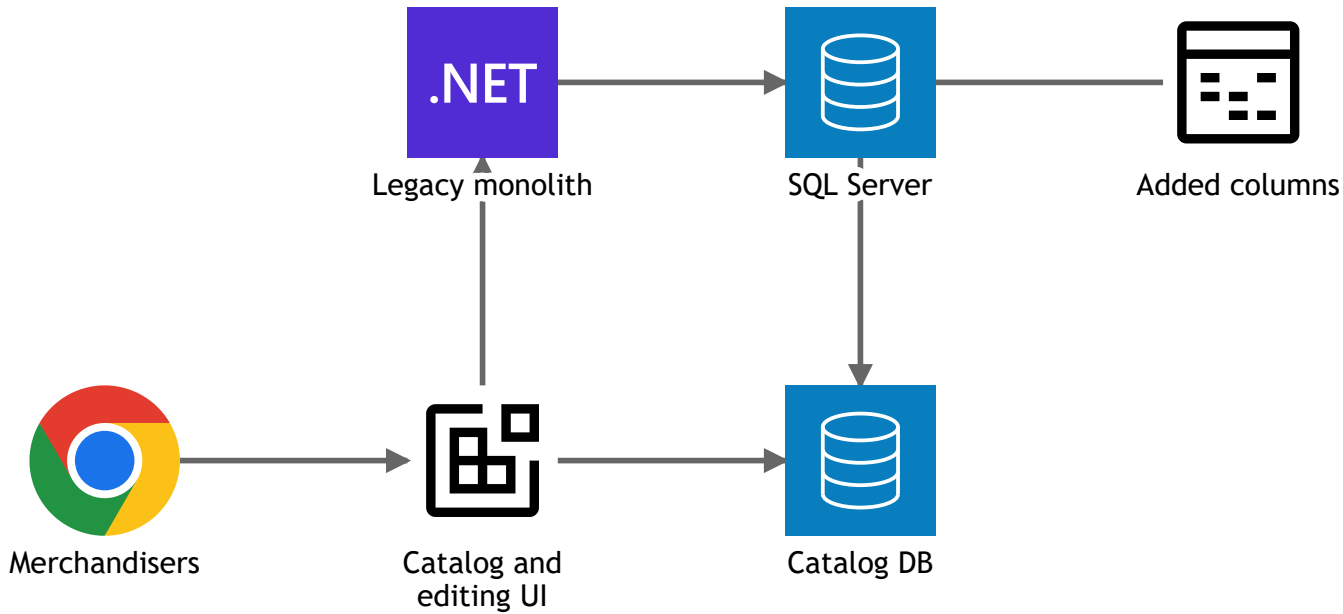
Verification: old and new versions on one schema

Check	Pass condition
Previous application version on the expanded schema	Characterization tests pass unchanged; all old queries and inserts still succeed
New application version on the expanded schema	Writes and reads the new attributes; old columns keep their values
Nightly ERP export before and after the expand	Same columns and same rows as the pre-expand export
Backfill and synchronization of the new columns	Counts and checksums match between legacy and the catalog database; zero unexplained drift
Fitness function	Each schema migration runs against the previous and current version and consumer-driven contract tests; a failing old consumer blocks the deploy; an alert fires when an old column or view passes its sunset

Rollback: routing rollback: deploy the previous application version, which ignores the new columns. State rollback: additive columns stay in place unused and nothing is dropped; a bad backfill is repaired forward by re-running it and reconciling legacy against the catalog database, never by dropping columns that a consumer may already read.

Stop: when old and new versions both pass on the expanded schema, no view, rename or column removal is added until a consumer audit shows the old columns are unused.

Solution architecture



The same stores and synchronization, plus added nullable columns beside the old ones; old consumers keep reading the old columns.

Glossary

Used in this step

Term	Definition
Expand-Contract	Add compatible representation, migrate consumers, then remove old representation
Immediate rename	Rename or replace a column in one deployment, so every reader must change at once
Database View	Named query presenting an abstraction over underlying tables
Old consumers	Legacy code, exports and reports that read the shared tables
Expanded schema	Schema that keeps the old columns and adds the new ones beside them
Nullable column	Column that may hold no value, so old inserts keep working
Data Backfill	Populate a new representation using already-stored historical data
Characterization test	Test that records current behavior as the reference
Consumer-Driven Contract Test	Consumer expectations verified against a provider
Sunset	Agreed date and owner for removing an old column or view
Change Data Capture (CDC)	Detects and propagates changes made to database records.

Related concepts

Term	Definition
Schema Compatibility View	Database view presenting old shape over new data
Branch by Abstraction	Introduce stable interface then change the implementation behind it gradually
Feature Flag	Runtime control enabling behavior without deploying new code
Database Per Service	Each service owns its private persistence and schema
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence

M29 · Data synchronization and writer handover

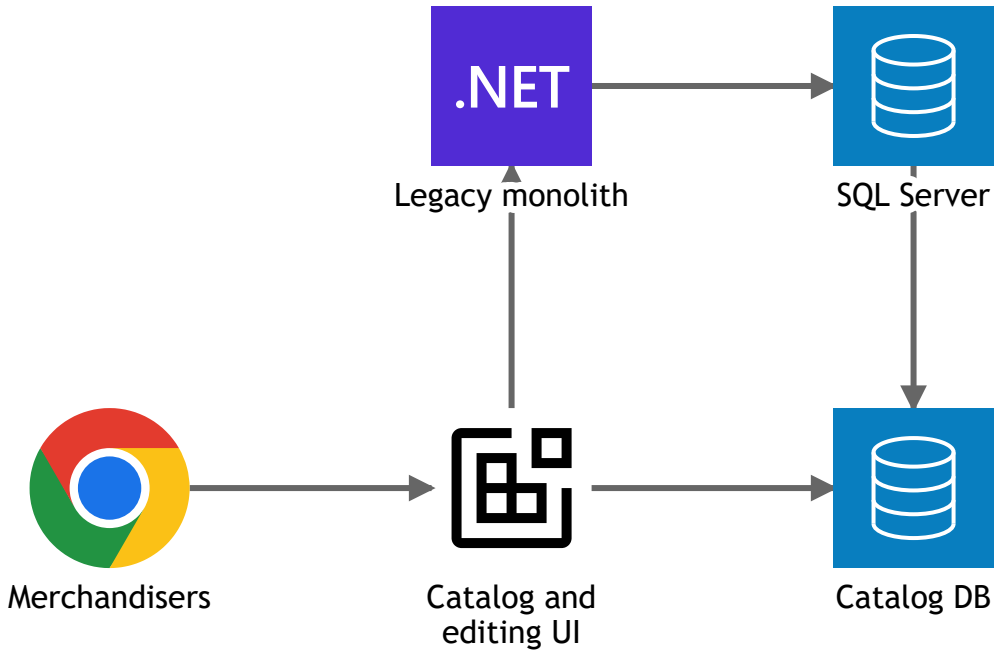
Transfer write authority

A legacy release controls product changes, so exactly 1 write authority per product must move to the new catalog with no overlap.

The catalog can deploy alone, but legacy still writes

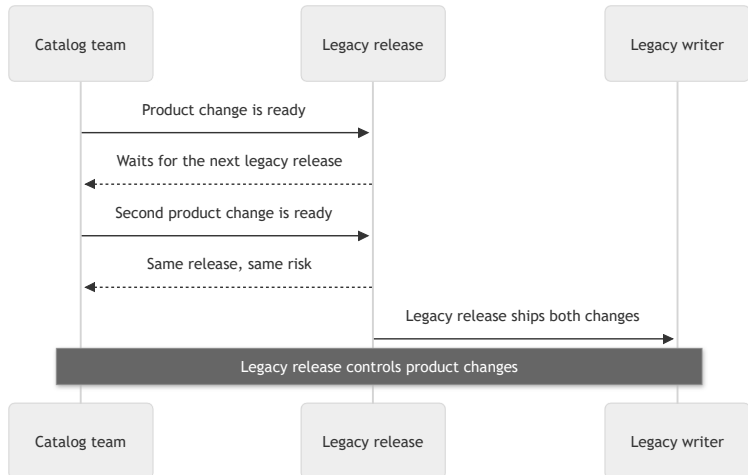
Component	What it is	What it does
Catalog service	The replacement catalog application with the new editing UI	Deploys on its own; saves every product edit through the legacy writer
Legacy writer	The existing code path that updates the product tables	Accepts every product write and ships with legacy releases
SQL Server	One shared relational database	Holds the authoritative product rows, with the expanded schema
Catalog database	The new store, reconciled with legacy	Serves reads; receives product changes through change data capture
Feature flags	Runtime switches for single behaviors	Turn one behavior off without undoing the others

Current architecture



Legacy releases gate every product change

The new catalog can deploy on its own schedule, yet every product change still ends in the legacy writer and waits for a legacy release. Authority has to move, and once the new side accepts a write, switching traffic back no longer restores the data.



Every product change
waits for a legacy release

Legacy writer
still holds the write authority for products

1 write authority
at every moment: reconcile up to cutoff, block legacy writes, activate the new writer

Each option decides how authority leaves legacy

Solution	What it is	What it does
Write Ownership Handover	Transfer authoritative writes for bounded data from legacy to replacement	Fences legacy writes, drains the change feed through a cutoff, reconciles, then activates the new writer
Partitioned keys	Split the product keys into partitions and hand over one partition at a time	Each partition has exactly one writer, legacy or catalog, chosen by key
Dual Write	Attempt to update two independent stores from one business operation	Writes every product change to legacy and to the catalog database until the new side is trusted

A clean switch costs a pause; two writers cost data

Solution	Gains	Costs
Write Ownership Handover	Safe autonomous extraction One authority at every moment Catalog releases without legacy releases	Requires fencing, draining and reconciliation Writes paused between fence and activation Routing rollback cannot restore new writes
Partitioned keys	Selective migration Smaller blast radius per handover Can pause between partitions	Routing by key on every write Partition map needs an owner Rules across partitions need care
Dual Write	Simple prototype sync No pause in accepting writes	Partial failures leave the stores different Split brain: two authorities per product Conflicts have no single arbiter

Deciding signal: ownership change, partial write failure, conflict risk and the aggregate invariant. Can product writes be fenced at a clean cutoff (legacy writers still active, write rate, drain time) with zero unexplained drift? Then the full handover. A long or noisy drain, or drift in part of the keys, picks Partitioned keys; accepting writes in both stores at once is Dual Write and fails the invariant.

Smallest safe move

Write Ownership Handover on the product write path

- **Scope:** write ownership and traffic; schema unchanged; legacy tables stay readable but frozen
- **Change:** run the sequence below, with Write Fencing on legacy product writes by feature flag, then set the writer flag to the catalog service
- **Safety net:** legacy is fenced before the new writer starts and activation needs zero unexplained drift; Dual Write is never a bridge
- **Escape route:** before the first new write, unfence legacy and flip the writer flag back; after it, repair forward or use reverse replication
- **Deferred:** Partitioned keys when the drain cannot be made clean or drift persists in part of the key space

```
reconcile to cutoff marker → fence legacy product writes → drain change feed to cutoff
→ reconcile again, zero unexplained drift required → writer flag = catalog
```

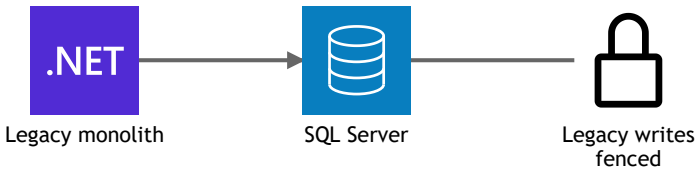
Verification: reconcile, block, activate

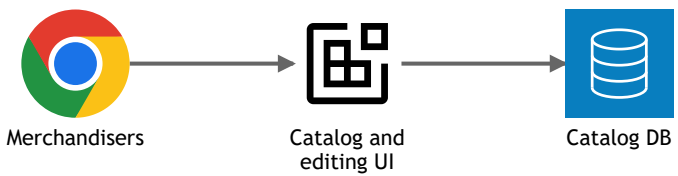
Check	Pass condition
Reconciliation at the cutoff marker, before and after the drain	Counts, checksums and invariants for sampled and critical keys show zero unexplained drift
Legacy product write attempted after the fence	Rejected; legacy product tables unchanged
First product edits through the new writer	Each accepted once, visible on read, stored only in the catalog database; none reaches legacy
Catalog change deployed in a window with no legacy release	Change takes effect and product writes keep working
Fitness function	A cutover runbook script refuses to activate the new writer without zero unexplained drift; a scheduled write probe on the fenced side must fail and alerts on any accepted write

Rollback: routing rollback is safe only before the new writer accepts its first write: unfence legacy and set the writer flag back. After that it is unsafe, because legacy lacks the new writes. State rollback then means reverse replication of the new writes into legacy plus reconciliation, or a forward fix on the new side, both planned before the cutover.

Stop: when the new catalog is the only writer, accepts all product changes and deploys without a legacy release, no partitioning or dual-write layer is added.

Solution architecture





Product edits now go from the catalog service to the catalog database. Legacy tables stay readable but frozen behind the fence, and the capture into the catalog database ended at the cutoff.

Glossary

Used in this step

Term	Definition
Write Ownership Handover	Transfer authoritative writes for bounded data from legacy to replacement
Partitioned keys	Product keys split into partitions, each with exactly one writer
Dual Write	Attempt to update two independent stores from one business operation
Write Fencing	Prevent stale writers from modifying state after ownership changes
Cutoff marker	Position in the change feed up to which the new store must be complete
Draining	Letting the change feed deliver every pending change before authority switches
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Writer flag	Switch that selects which system accepts product writes
Routing rollback	Switching traffic or the writer flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do
Reverse Replication	Propagate authoritative new-state changes back to old consumers temporarily

Related concepts

Term	Definition
Forward Fix	Repair the new implementation without restoring the old system
Data Cutover	Point at which authority for selected reads or writes changes to new system
Change Data Capture (CDC)	Detects and propagates changes made to database records.
Feature Flag	Runtime control enabling behavior without deploying new code
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations

M30 · Data synchronization and writer handover

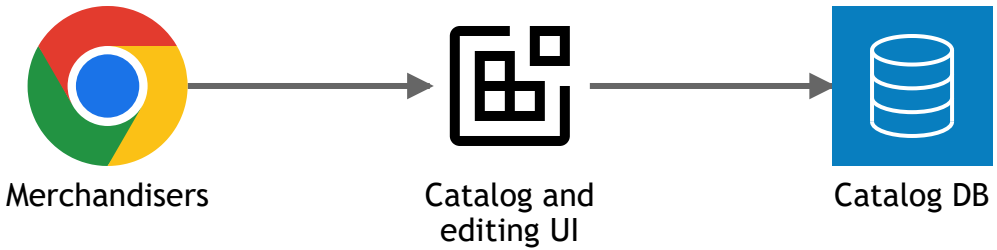
Feed legacy consumers from new source

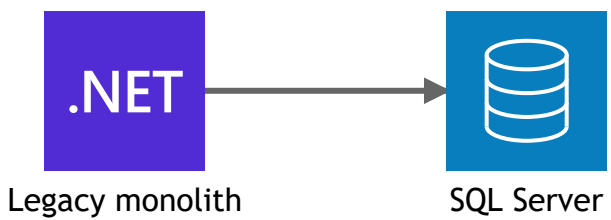
The new database now owns writes, but old apps must keep functioning without a change to their code.

Old ERP and order code still read legacy product tables

Component	What it is	What it does
Merchandisers	Staff who edit products in the new editing UI	Change products only in the catalog service
Catalog service	The replacement catalog application	Accepts product writes and serves reads
Catalog database	The new store, now the only write authority for products	Holds the current product state
Legacy tables	The product tables in the shared SQL Server database	Stopped changing when authority moved, but stay readable
Old ERP and order code	Legacy modules, including the nightly ERP export	Read product rows from the legacy tables

Current architecture





Old consumers keep running on frozen product data

The catalog database owns product writes, but the old ERP export and the order code read legacy tables that stopped changing at the handover. They see old prices and deleted products, and they cannot be rebuilt in the same release.

Frozen tables

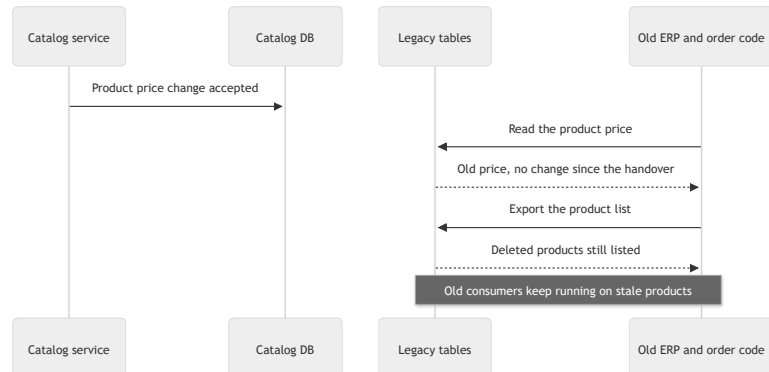
old ERP and order code read legacy product tables that no longer change

Every old consumer

still reads product rows from legacy tables, including deletes

Caught up, no missed delete

old consumer state follows the new source



Each option gets new data to old consumers differently

Solution	What it is	What it does
Outbox + Events	Record each product change as a business event in the same transaction as the change, and publish it	Delivers business events to old consumers, which update their own copy through a subscriber
Compatibility API	An API on the new catalog that returns products in the shape old code expects	Old code calls the API instead of reading the legacy tables
Reverse projection	A one-way copy of product data from the new source back into the old tables, in the old shape	Old consumers keep reading the legacy tables unchanged while a projection worker keeps them current

Every option trades freshness, change effort or duplicate state

Solution	Gains	Costs
Outbox + Events	Consumers decoupled from the new schema Business meaning, not raw rows No event lost between commit and publish	Lag behind the write Old code needs a subscriber Event versioning and delivery to manage
Compatibility API	Fresh data on every call No second copy of product state One place for the old contract	Network dependency in old code paths Old ERP and order code must change Availability and latency are shared
Reverse projection	Minimal client change: old code unchanged Old table shape preserved Updates and deletes carried by the feed	Duplicate state in two stores Lag behind the new source Mapping back to the old shape

Deciding signal: how many old consumers cannot change in this release, how fresh each needs its data against the projection lag, and whether raw row changes or business meaning (change data capture versus domain events) are needed. Unchangeable table readers pick Reverse projection; consumers that need business meaning and can add a subscriber pick Outbox + Events; consumers that need fresher data than the lag allows and can call out pick Compatibility API.

Smallest safe move

Reverse projection into the old product tables

- **Scope:** data; old code, routes and write ownership stay unchanged
- **Change:** a projection worker fed by change data capture on the catalog database writes inserts, updates and delete markers into the old tables in the old shape
- **Safety net:** one-way copy: the catalog database wins any conflict; every projected table gets an owner and a sunset, so the compatibility layer does not become migration debt
- **Escape route:** pause the projection worker; consumers keep the last projected state
- **Deferred:** Outbox + Events when consumers need business events or can add a subscriber; Compatibility API when a consumer needs fresher data than the lag allows

```
catalog DB change (insert, update, delete) → projection worker → old product table, old shape
a delete always arrives as a delete marker
```

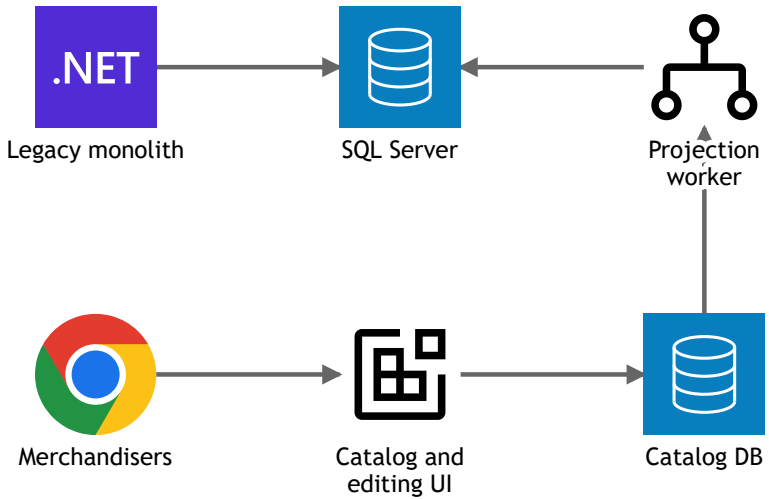
Verification: old consumers catch up with no missed delete

Check	Pass condition
Create, update and delete products in the catalog database	Old tables equal the catalog database once the projection catches up; every delete is present
Old ERP export and order code run unchanged on the projected tables	Output matches the characterization reference; no code change
Projection worker stopped and restarted mid-stream	Catches up from its offset with no gap and no duplicate effect
Projection lag against the freshness each old consumer needs	Lag stays below the freshness of the most demanding consumer
Fitness function	Scheduled reconciliation compares counts, checksums and delete markers after each batch and alerts on unexplained drift; a lag alert fires when the projection falls behind the agreed freshness

Rollback: routing rollback does not apply, because no client route changes; pausing the projection worker stops writes into the old tables. State rollback: the old tables are a copy, so a bad projection is repaired forward by rebuilding them from the catalog database and reconciling; copying them back would overwrite newer authoritative data.

Stop: when every old consumer runs unchanged on the projected tables and no delete is missed, no Compatibility API or event channel is added.

Solution architecture



The same stores, plus a projection worker that copies catalog database changes into the old product tables; old code and routes do not change.

Glossary

Used in this step

Term	Definition
Outbox + Events	Product changes recorded as business events in the same transaction and published to consumers
Compatibility API	An API on the new catalog that returns products in the shape old code expects
Reverse projection	One-way copy of new-source data back into the old tables in the old shape
Old consumers	Legacy code, exports and reports that read the shared tables
Projection worker	Process that applies new-source changes to the old tables
Domain Event	Statement that an important business fact occurred
Change Data Capture (CDC)	Detects and propagates changes made to database records.
Delete marker	Record that tells a consumer a row was deleted, so no delete is missed
Projection lag	Delay between a change in the new source and its appearance in the old tables
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Sunset	Agreed date and owner for removing an old column, table or view

Related concepts

Term	Definition
Transactional Outbox	Atomically records business changes and outgoing events in the same database transaction.
Reverse Replication	Propagate authoritative new-state changes back to old consumers temporarily
Schema Compatibility View	Database view presenting old shape over new data
Anti-Corruption Layer (ACL)	Protects a domain model from incompatible external system models.
Write Fencing	Prevent stale writers from modifying state after ownership changes

M31 · Data synchronization and writer handover

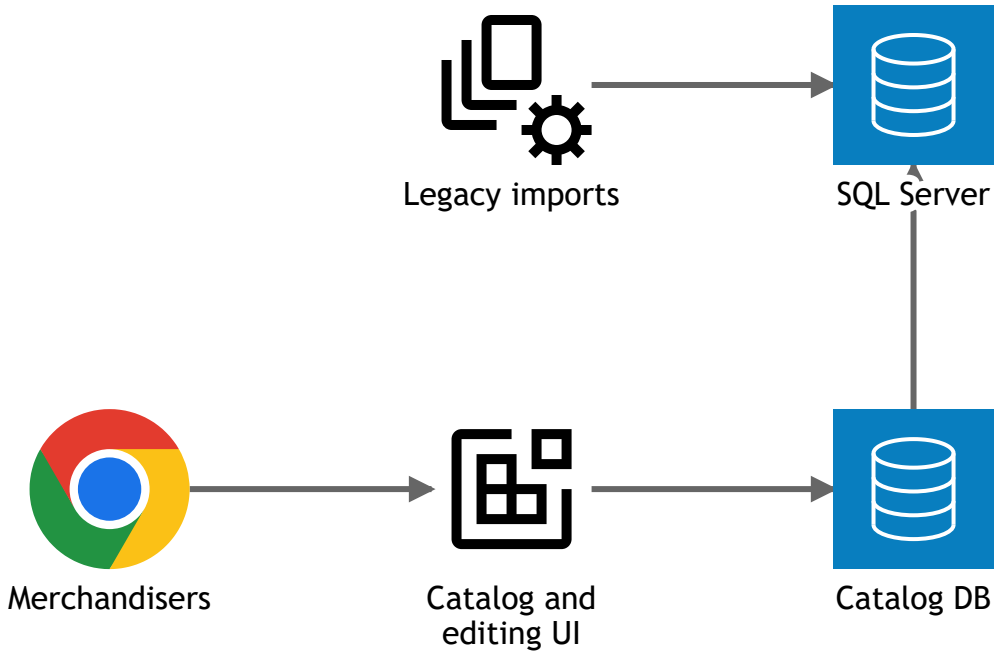
Prevent old batch writers

Two authorities will cause lost updates, so a stale legacy write must be rejected on every attempt.

Legacy imports still write the product tables

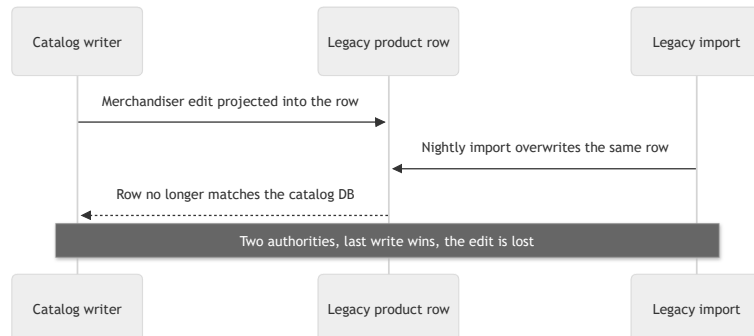
Component	What it is	What it does
Catalog service	The replacement catalog application	The only writer of product data in the catalog database
Catalog database	The new store, the write authority for products	Holds the current product state
Projection worker	One-way copy from the catalog database into the old tables	Keeps old consumers current; writes the legacy product tables
Legacy imports	Old batch jobs that load product data, for example supplier or ERP files	Still insert and update rows in the legacy product tables
SQL Server	One shared relational database	Holds the legacy product tables; write grants still allow the imports

Current architecture



Two authorities lose one of two changes

The catalog database is the authority, yet legacy import jobs still write the product tables that the projection also writes. When an import and a merchandiser edit touch one product, the last writer wins and the other change disappears without an error.



Last writer wins

an old import overwrites a catalog edit without an error

Legacy imports

still hold write access to the product tables

Stale write rejected

on every attempt by a legacy account

Each option stops a stale writer in a different place

Solution	What it is	What it does
Write Fencing / DB permissions	Prevent stale writers from modifying state after ownership changes, here by revoking database write permissions	The database rejects any write to the product tables from a legacy account, whatever the job does
Route all imports to new API	Every import job writes through the new catalog API instead of the legacy tables	Applies catalog validation and ownership to each import write
Import time window	A convention that imports run only outside merchandiser editing hours	Keeps the two writers apart by schedule instead of by enforcement

Enforcing the fence costs planning, a convention costs updates

Solution	Gains	Costs
Write Fencing / DB permissions	Stops split brain and lost updates Enforced by the database, not by agreement Stale writers fail loudly	Requires leases, tokens or privileges Needs an inventory of every writing account Needed imports break until re-pointed
Route all imports to new API	Controlled: imports follow catalog rules One audited write path Retries and idempotency handled once	An adapter for every import job Forgotten writers stay unchecked Bulk imports add API load
Import time window	Cheap: no code or permission change Quick to agree	Nothing stops a late or retried job Two authorities still exist Lost updates stay silent

Deciding signal: business invariant and conflict risk. Count the accounts and jobs with write grants on the product tables and how many are unknown or cannot be changed: any such writer requires a database fence, and routing imports to the API then adds validation on top. A convention such as a time window cannot pass, because one stale write breaks the invariant.

Smallest safe move

Write Fencing / DB permissions

- **Scope:** write ownership and data; old consumers keep read access
- **Change:** inventory every writing account, audit would-be rejections, then revoke write grants for legacy import accounts; only the projection account keeps write access
- **Safety net:** the database enforces the fence, so Import time window is not a control; audit mode shows which imports would break first
- **Escape route:** re-grant one account as a time-boxed break-glass grant, then reconcile its writes; the catalog database wins
- **Deferred:** Route all imports to new API one job at a time when a needed import is rejected; Import time window is rejected

```
REVOKE INSERT, UPDATE, DELETE ON legacy.product FROM import_job;  -- stale writes now fail
```

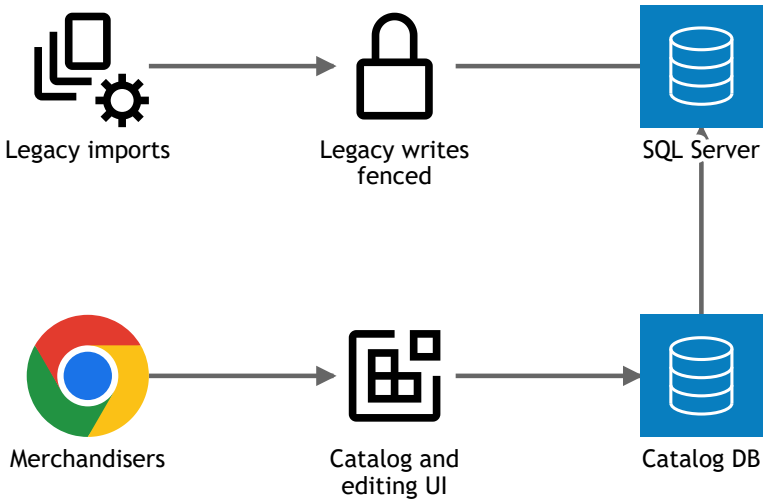
Verification: a stale write is rejected

Check	Pass condition
Stale write by each import account after the fence	Every attempt rejected with a permission error; product rows unchanged
Account inventory and audit log before the fence	Every writer of the product tables identified; no unknown account remains
A needed import re-pointed to the new API	Its change is accepted once by the catalog and projected to the old tables; no direct write to legacy
Merchandiser edit and import on the same product at the same time	Both ordered by the catalog, no change lost; old tables equal the catalog database after projection
Fitness function	A pipeline test attempts a write on the legacy product tables with every account except the projection account and fails if any succeeds; a scheduled permission audit alerts on a new write grant

Rollback: routing rollback does not apply, because the fence is a permission, not a route; re-granting write access restores the old writers and recreates the second authority, so it is only a time-boxed break-glass grant. State rollback: writes made in that window are reconciled against the catalog database, which wins, and a rejected legitimate import write is replayed through the new API as a forward repair.

Stop: when every stale write is rejected and every needed import writes through the new API, no further fencing layer such as lease tokens or a proxy is added.

Solution architecture



The same stores and projection, plus a database write fence between the legacy imports and SQL Server: their writes are rejected, and only the projection account writes the legacy tables.

Glossary

Used in this step

Term	Definition
Write Fencing / DB permissions	Prevent stale writers from modifying state after ownership changes, here with database permissions
Route all imports to new API	Every import job writes through the new catalog API instead of the legacy tables
Import time window	Convention that imports run only outside editing hours, with no enforcement
Legacy imports	Old batch jobs that load product data into the legacy tables
Lost update	A write that is silently overwritten by another writer
Split brain	Two systems each act as the owner of the same data
Write grant	Database permission to insert, update or delete rows
Audit mode	Fence that logs would-be rejections before it starts blocking
Break-glass grant	Time-boxed emergency write permission, reconciled afterwards
Projection account	The only database account allowed to write the legacy product tables
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence

Related concepts

Term	Definition
Write Ownership Handover	Transfer authoritative writes for bounded data from legacy to replacement
Dual Write	Attempt to update two independent stores from one business operation
Forward Fix	Repair the new implementation without restoring the old system
Reverse Replication	Propagate authoritative new-state changes back to old consumers temporarily
Data Cutover	Point at which authority for selected reads or writes changes to new system

M32 · Data synchronization and writer handover

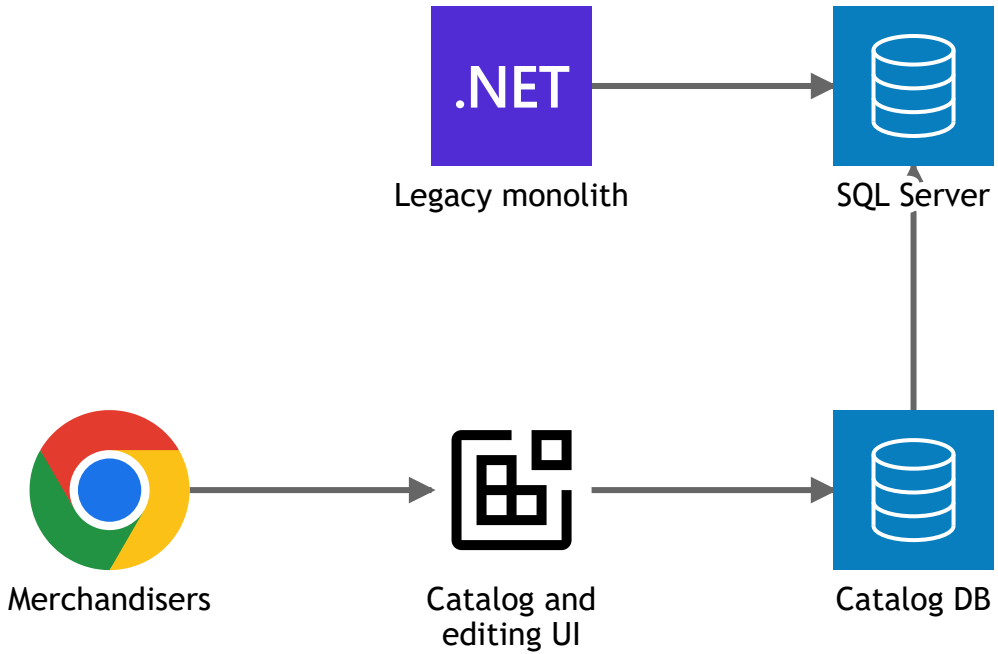
Plan state-aware failback

Routing back would lose new state, so a failback after post-cutover writes must lose 0 product edits.

The new writer is the authority; legacy holds a lagged copy

Component	What it is	What it does
Catalog service	The replacement catalog application with the editing UI	Accepts every product edit since the cutover
Catalog database	The new store and the write authority	Holds the current product state, including the newest edits
Projection worker	One-way copy from the catalog database into the old tables	Keeps old consumers current; the only writer of the legacy product tables
SQL Server	One shared relational database	Holds the legacy product tables as a lagged, old-shaped copy; the legacy write path is fenced
Writer flag	Switch that selects which system accepts product writes	Points at the catalog service

Current architecture



Flipping the flag back drops new edits

The new writer has accepted product edits since the cutover. If the team abandons it after an incident, the writer flag sends writes to legacy tables that hold only a lagged copy, so edits that are not in that copy disappear.

Edits missing

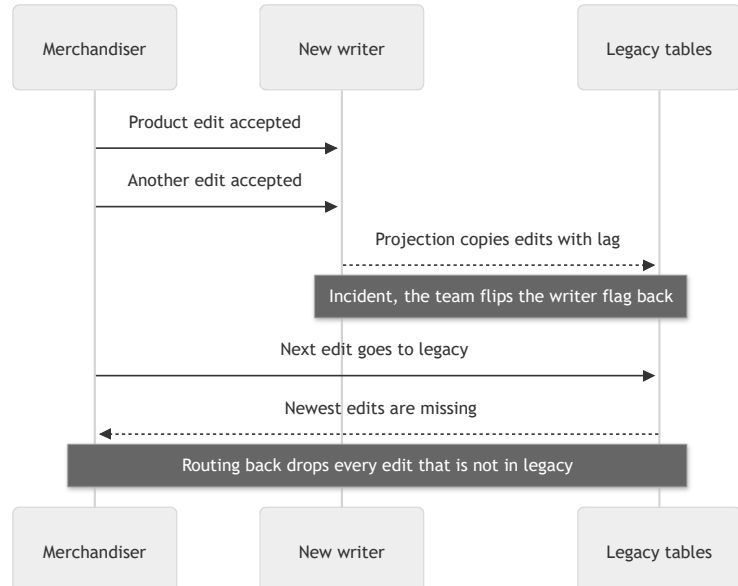
after the writer flag is flipped back to legacy

New writer keeps accepting

product edits; legacy holds only the lagged copy

0 lost product edits

in a failback rehearsed after post-cutover writes, one write authority at a time



Each option decides how the new state comes back

Solution	What it is	What it does
Reverse Replication + reconciliation	A feed that carries every new-side write back into the legacy tables, plus a comparison before authority returns	Keeps legacy current so authority can return after the new writer is fenced, the feed drained and the data reconciled
Forward Fix	Repair the new implementation without restoring the old system	Keeps authority on the new side and ships a code fix or a data repair
Routing rollback only	Flip the writer flag back to legacy and carry on	Sends writes to the legacy tables without moving the state the new writer accepted

A way back costs a feed; no way back costs data

Solution	Gains	Costs
Reverse Replication + reconciliation	Supports fallback Legacy tables stay current Failback lag is measurable	Complex: conflict and replay management Feed must carry every field and delete Return needs fence, drain and reconcile
Forward Fix	No migration reversal Avoids reverse data migration No reverse feed to run	Longer repair, incident may take longer Needs fast release and repair tooling No way back if the new store fails
Routing rollback only	Fastest switch No extra machinery	Drops edits made since the cutover Two truths: the catalog database diverges Rollback code does not roll back state

Deciding signal: post-cutover write amount, feed topology and failback RPO. If the feed lag stays within the failback RPO and legacy takes every field and delete, Reverse Replication + reconciliation passes. If a rehearsed forward repair fits the tolerated outage and the new store is repairable in place, Forward Fix wins. A plain routing switch fails once one write is accepted.

Smallest safe move

Reverse Replication + reconciliation on the existing projection

- **Scope:** write ownership and data; schema, routes and the catalog service unchanged
- **Change:** keep the projection as the reverse feed with every field and delete, measure its lag as the failback RPO, rehearse the runbook below
- **Safety net:** the writer flag moves only after zero unexplained drift; Routing rollback only is never the plan
- **Escape route:** abort before the flag moves; after it moved, repair forward
- **Deferred:** Forward Fix when the lag cannot meet the failback RPO or a repair beats a failback

```
Write Fencing on the new writer → drain the feed to the cutoff marker → Data Reconciliation, zero drift
→ unfence the legacy write path → writer flag = legacy
```

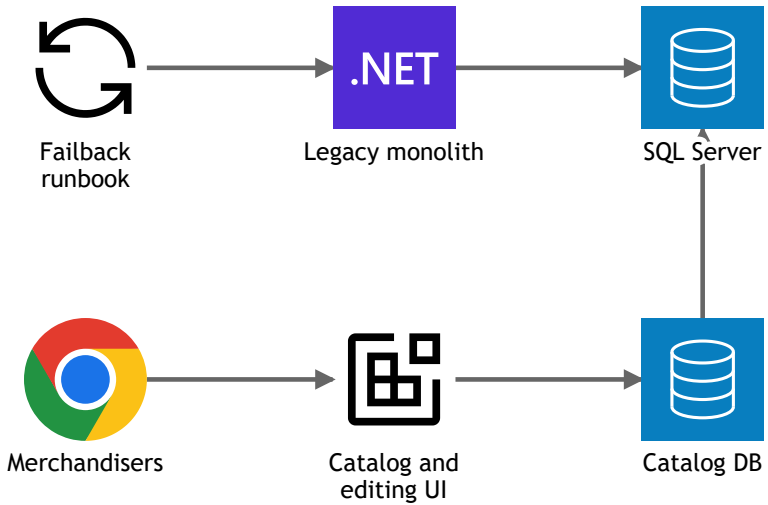
Verification: failback rehearsed after post-cutover writes

Check	Pass condition
Feed lag during normal writes, including deletes	Lag stays within the failback RPO; every delete reaches the legacy tables
Failback rehearsal after 100 post-cutover product edits (for example) with updates and deletes	Every edit is present in legacy after fence, drain and reconcile; 0 lost product edits
Write attempted on the new side after its fence, and on legacy before its unfence	Both rejected; only one side accepts writes at any moment
Failback aborted before the writer flag moves	New writer resumes with no lost edit; feed and flags unchanged
Fitness function	A scheduled drill runs the runbook after synthetic writes and fails the pipeline on drift; an alert fires when feed lag exceeds the failback RPO

Rollback: routing rollback of the writer flag is safe only while the new side has accepted no write. State rollback is the runbook: fence, drain, reconcile, then return authority; where it cannot complete, repair forward on the new side.

Stop: when the rehearsed failback loses no edit within the failback RPO, no further layer such as a second replication path is added.

Solution architecture



The same stores and projection, plus a failback runbook that fences the new writer, drains the reverse feed, reconciles and only then returns the writer flag to legacy.

Glossary

Used in this step

Term	Definition
Reverse Replication + reconciliation	Propagate new-state changes back to old consumers, then reconcile before authority returns
Forward Fix	Repair the new implementation without restoring the old system
Routing rollback only	Flip the writer flag back and carry on, without moving the state accepted since the cutover
Failback	Returning write authority from the new system to legacy after a cutover
Failback RPO	Largest window of recent writes that may be missing or re-created when authority returns
Routing rollback	Switching traffic or the writer flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Write Fencing	Prevent stale writers from modifying state after ownership changes
Cutoff marker	Position in the change feed up to which the new store must be complete
Projection worker	Process that applies new-source changes to the old tables
Writer flag	Switch that selects which system accepts product writes

Related concepts

Term	Definition
Rollback	Return to a previously running version or state
Data Cutover	Point at which authority for selected reads or writes changes to new system
Write Ownership Handover	Transfer authoritative writes for bounded data from legacy to replacement
Canary Routing	Shift a small controlled segment of live traffic to a new implementation
Feature Flag	Runtime control enabling behavior without deploying new code

M33 · Data synchronization and writer handover

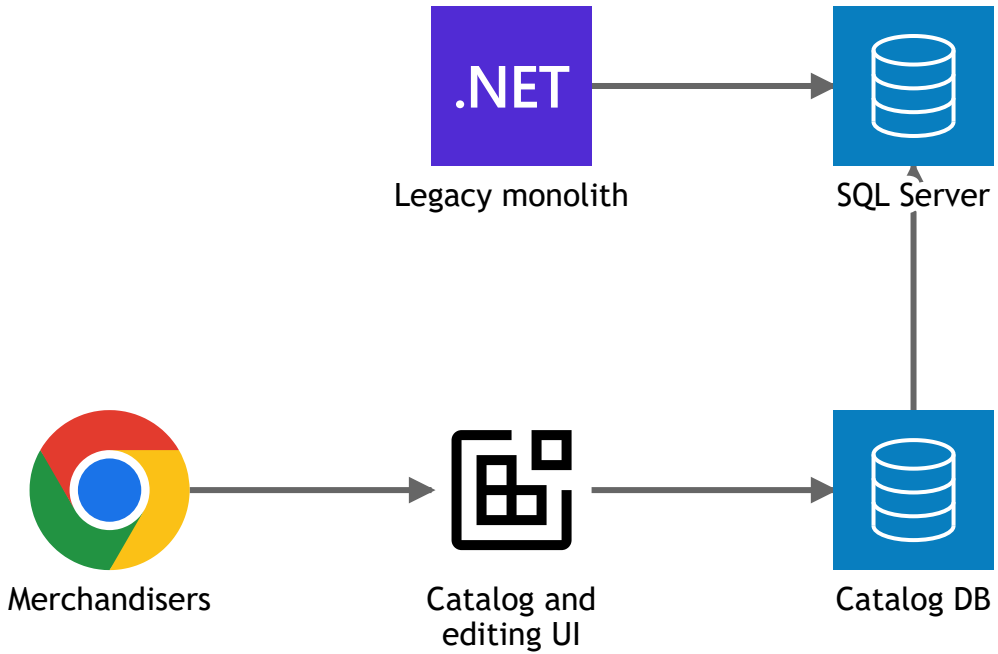
Retire sync debt

Replication and duplicate tables are expensive, so they go only after 0 required readers are proven and retention rules are met.

The sync still runs for a copy nobody is known to read

Component	What it is	What it does
Catalog database	The new store and the only write authority	Holds the one authoritative product state
Projection worker	Process that applies new-source changes to the old tables	Still copies every catalog change into SQL Server
Duplicate tables	The old product tables in SQL Server	Hold a second copy of every product row, including delete markers
Required consumers	The exports, reports and code that must keep working	None is known to read the old catalog tables
Way back to legacy	The projection used as the reverse feed	Depends on the duplicate tables staying current

Current architecture



A second copy costs money until it is proven unused

The catalog database serves every product read and write, and the team believes no required consumer reads the old tables. The projection and the duplicate tables still run, yet removing them is a one-way step.

Still running

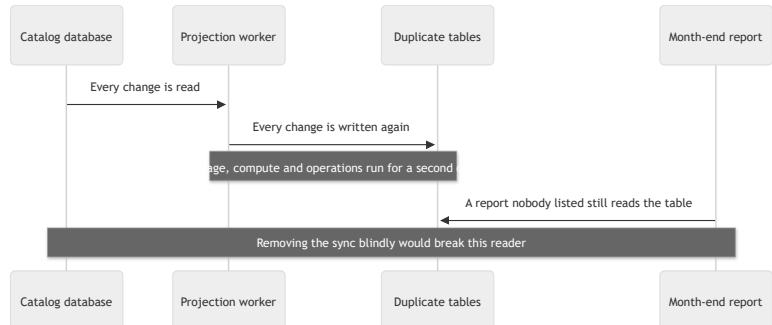
replication and duplicate tables cost storage, compute and operations

Readers unproven

whether a required consumer reads the old tables is a belief, not a measurement

0 required readers

over a 30 day audit window (for example) and retention rules met, then sync stopped and tables deleted



Each option decides when the second copy ends

Solution	What it is	What it does
Consumer audit + archive + stop sync + delete	A staged retirement: find every remaining reader, archive the data, stop the replication, then delete the duplicate tables	Removes the sync cost only after the audit and the retention rules say that nothing still needs it
Keep sync running	Leave the projection worker and the duplicate tables in place with no end date	Keeps every old reader and the way back to legacy fed while nobody decides to retire
Stop sync without audit	Switch off the projection and drop the duplicate tables on the assumption that nobody reads them	Ends the cost at once without searching for remaining readers

Removing debt costs the way back; keeping it costs money

Solution	Gains	Costs
Consumer audit + archive + stop sync + delete	Removes debt Cost stops: no replication, no copy Retention proven before delete	Limits rollback Way back to legacy ends with the sync Audit and archive effort first
Keep sync running	No reader can break Way back to legacy stays No effort now	Cost and operations continue Second copy can drift Debt grows with every schema change
Stop sync without audit	Fastest saving No audit effort	A hidden reader fails or reads stale data Dropped data can break retention rules No archive to restore from

Deciding signal: required readers and retention rules. Count the queries, jobs and reports that read the old tables over a full cycle (30 days for example, including a month end) and compare retention rules with the catalog database or an archive. Zero required readers and retention met picks the staged retirement; one reader that cannot move keeps the sync for that table; no audit rules out stopping.

Smallest safe move

Consumer audit + archive + stop sync + delete, one table at a time

- **Scope:** data; the projection and the duplicate tables, fewest readers first; write ownership and routes unchanged
- **Change:** audit readers for 30 days (for example), archive the table, stop its projection, keep it read-only for a hold period, then delete it and record the sunset
- **Safety net:** a reader found in the audit blocks its table; Stop sync without audit is never used
- **Escape route:** before the delete restart the projection and rebuild; after it restore from the archive
- **Deferred:** Keep sync running per table until its required reader migrates, with an owner and a sunset

```
audit readers → archive table → stop projection → read-only hold period → delete table, record sunset
```

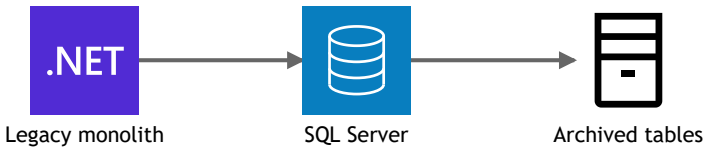
Verification: remaining queries and retention

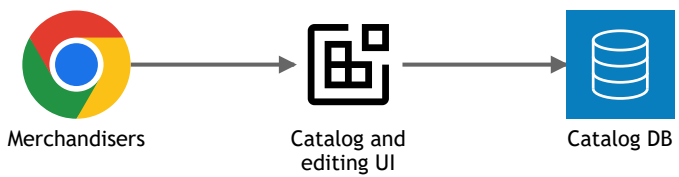
Check	Pass condition
Query log of the old tables over the audit window (30 days for example)	Every reader identified and classified; no required reader remains
Retention rules compared with the catalog database and the archive	Every record that must be kept is available after the tables are deleted
Archive restored into an empty database	Counts and checksums equal the table before archiving
Projection stopped and table read-only for the hold period	No access error, no support ticket and no read attempt in the log
Fitness function	A scheduled query-log check fails the pipeline when any login reads a retired table; a dependency scan rejects new references; cost and lag dashboards show the saving

Rollback: routing rollback does not apply, no route changes. State rollback works only before the delete: restart the projection, rebuild the table from the catalog database and reconcile. After the delete only the archive and a forward rebuild remain, and returning to legacy ends with the sync.

Stop: retirement is triggered only when the audit finds no required reader and retention is met; while one remains, retirement stops for that table and no new layer is added.

Solution architecture





The same stores, but the projection into SQL Server is gone: the duplicate tables are retired and only an archive keeps them for the retention period.

Glossary

Used in this step

Term	Definition
Consumer audit + archive + stop sync + delete	Find every remaining reader, archive the data, stop replication, then delete the duplicate tables
Keep sync running	Leave replication and duplicate tables in place with no end date
Stop sync without audit	Switch off replication and drop duplicate tables without searching for remaining readers
Consumer audit	Query-log and dependency search that finds every reader of the old tables
Sync debt	Replication and duplicate tables kept only to serve the migration
Retention rule	Requirement to keep certain records for a stated period
Archive	Read-only copy of data kept for retention, outside the live tables
Hold period	Time the table stays read-only after the sync stops, before it is deleted
Sunset	Agreed date and owner for removing an old column, table or view
Projection worker	Process that applies new-source changes to the old tables
Routing rollback	Switching traffic or the writer flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Rollback	Return to a previously running version or state
Expand-Contract	Add compatible representation, migrate consumers, then remove old representation
Reverse Replication	Propagate authoritative new-state changes back to old consumers temporarily
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations

Phase 5

Business capability extraction

Steps in this phase

Step	Forcing constraint
M34 Extract notification worker	Mail outages delay order acknowledgement
M35 Modernize ERP incremental sync	Stock sync freshness ≤5min
M36 Reconcile ERP vs fulfillment model	New fulfillment recognizes distinct lifecycle
M37 Switch billing provider	Old invoices remain available during transition
M38 Locate inventory transaction boundary	Inventory deployments needed independently
M39 Move inventory ownership	Concurrent requests may oversell
M40 Coordinate checkout across services	Partial failures leave paid-but-unreserved orders
M41 Migrate UI route by route	New account/catalog UI has different release schedule
M42 Guard independently deployed contracts	API schema change breaks old client

M34 · Business capability extraction

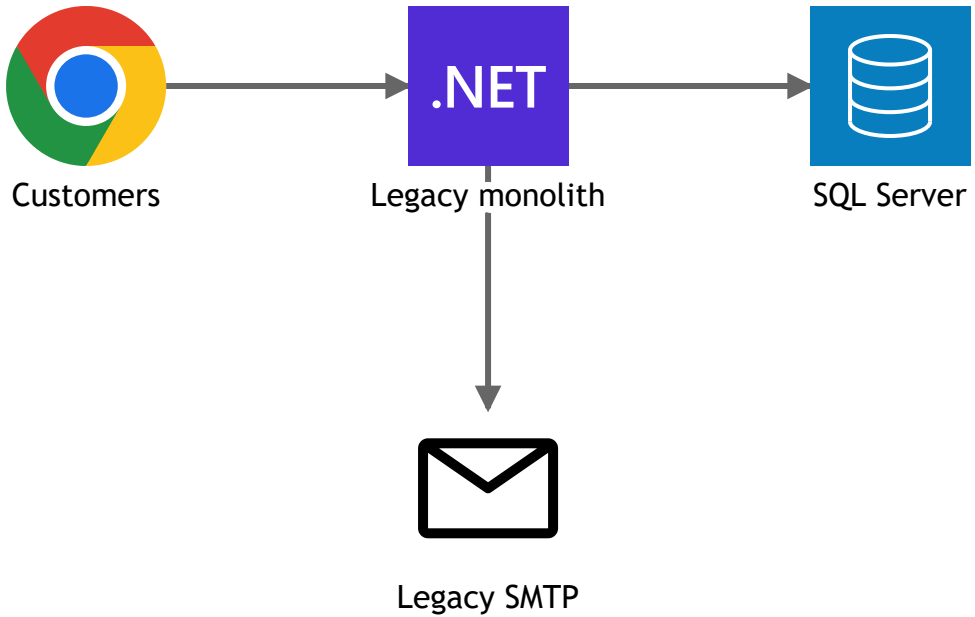
Extract notification worker

Mail outages delay order acknowledgement, so orders must be acknowledged with SMTP disabled and every confirmation sent after it returns.

Checkout sends the mail before it acknowledges the order

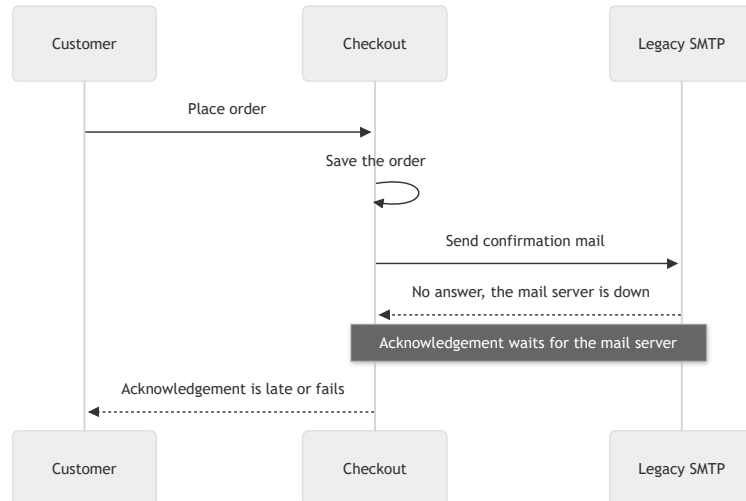
Component	What it is	What it does
Checkout	The order placement flow in the legacy monolith	Saves the order and sends the confirmation mail in the same request
Legacy SMTP	The existing mail server connection that the monolith uses	Delivers every confirmation; checkout waits for its answer
SQL Server	One shared relational database	Stores the order data
Order acknowledgement	The confirmation that tells the customer the order was accepted	Is returned only after the mail call finishes
Feature flags	Runtime switches for single behaviors	Turn one change off without undoing the others

Current architecture



A mail outage holds the order hostage

Checkout calls the mail server before it acknowledges the order, so a slow or failed mail server holds or fails the acknowledgement. The mail is not part of the order, yet its outage delays the order.



Outage delays orders

a mail outage delays or fails order acknowledgement

Every checkout

calls the mail server inside the request

SMTP disabled

orders still acknowledged, every confirmation sent after SMTP returns

Each option decides where the pending mail waits

Solution	What it is	What it does
Worker + Message Queue	A queue that buffers notification requests and a separate worker that sends the mail	Checkout publishes a message and acknowledges; the worker sends and retries until SMTP accepts
In-process async	The mail call moves to a background task inside the web application process	Checkout acknowledges at once while the same process sends the mail later
Transactional Outbox + Worker	The notification request is stored in the order's own database transaction and a worker sends it from that table	Commits order and request together, then the worker sends and retries until SMTP accepts

Durability costs a table or a broker; speed costs mail

Solution	Gains	Costs
Worker + Message Queue	Decoupled from the mail server Buffers an outage Retries without blocking checkout	Retries need idempotency and backoff New broker to operate Commit then publish can lose a message
In-process async	Quick to build No new component Acknowledgement stops waiting	Lost work on crash or recycle No retry after the process ends Nothing records what is unsent
Transactional Outbox + Worker	Order and request commit together No new broker Unsent mail stays visible in a table	Worker polls the database Checkout code changes once Duplicates possible without an idempotency key

Deciding signal: whether work can complete later and what happens to pending mail when the process dies. With SMTP disabled, measure acknowledgement time, the unsent backlog and its drain time, and kill the web process mid-backlog. Any lost mail rules out In-process async; a durable row in the order transaction that drains at the SMTP rate passes without a broker; a second consumer, fan-out or heavy polling picks Worker + Message Queue.

Smallest safe move

Transactional Outbox + Worker for the confirmation mail

- **Scope:** code and data; the confirmation mail only, same legacy SMTP server; order data and routes unchanged
- **Change:** insert one outbox row, with the order id as Idempotency Key, in the order transaction behind a Feature Flag; the Notification worker sends with Retry with Backoff
- **Safety net:** the row commits with the order, so a crash cannot lose it; In-process async is never used
- **Escape route:** flip the flag back to the synchronous SMTP call; unsent rows are sent first
- **Deferred:** Worker + Message Queue when a second consumer, fan-out or polling load appears

```
BEGIN TRAN;
INSERT INTO orders (id, total) VALUES (@id, @total);
INSERT INTO notification_outbox (order_id, template) VALUES (@id, 'confirmation');  -- order_id = idempotency key
COMMIT;  -- the worker sends the mail later
```

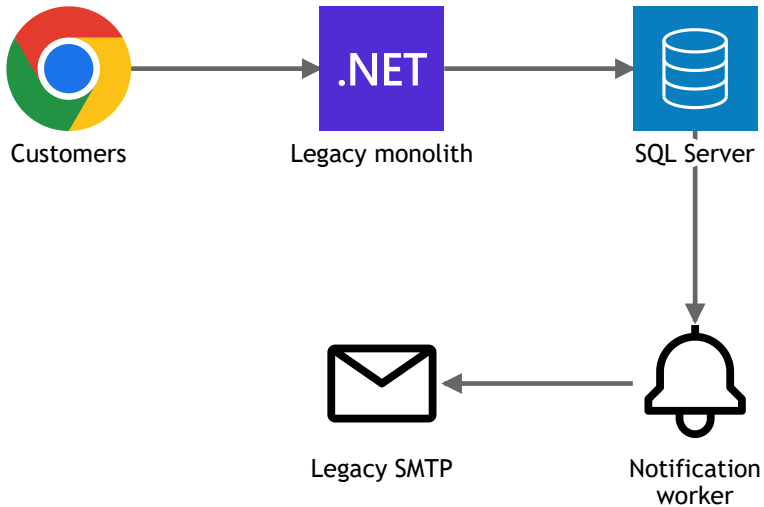
Verification: SMTP off, orders still accepted

Check	Pass condition
SMTP disabled and orders placed	Every order acknowledged at the normal speed, each with one unsent outbox row
SMTP restored	Every pending confirmation sent once; the unsent backlog drains to zero
Web process killed while mail is pending	No mail lost; the worker sends after the restart
Worker stopped and restarted mid-send while SMTP times out	No duplicate mail for the same order; retries back off
Fitness function	A pipeline test turns SMTP off, places an order, asserts acknowledgement plus one outbox row, then turns SMTP on and asserts exactly one mail; an alert fires when the oldest unsent row exceeds the agreed delay

Rollback: routing rollback: the flag returns checkout to the synchronous SMTP call at once, exposed to outages again. State rollback: unsent outbox rows are sent by the worker or a one-time script first; no order data changed, so no order is rolled back.

Stop: when orders are acknowledged during an SMTP outage and every confirmation is eventually sent once, no message queue or further notification platform is added.

Solution architecture



Checkout only writes the order and an outbox row. The notification worker reads unsent requests from SQL Server and sends them through the legacy SMTP server, so a mail outage no longer touches the order.

Glossary

Used in this step

Term	Definition
Worker + Message Queue	Notification requests buffered in a queue and sent by a separate worker
In-process async	Mail sent by a background task inside the web application process
Transactional Outbox + Worker	Notification request recorded in the order's database transaction and sent later by a worker
Notification worker	Separate process that sends mail outside the order request
Message Queue	Buffers messages for asynchronous processing by consumers.
Idempotency Key	Stable identifier recognizing multiple attempts at the same logical command
Retry with Backoff	Repeats failed operations with increasing delays.
Order acknowledgement	The confirmation that tells the customer the order was accepted
Legacy SMTP	The existing mail server connection that the monolith uses to send mail
Feature Flag	Runtime control enabling behavior without deploying new code
Routing rollback	Switching traffic or the writer flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Transactional Outbox	Atomically records business changes and outgoing events in the same database transaction.
Dead Letter Queue (DLQ)	Stores messages that could not be processed successfully.
Circuit Breaker	Stops requests to failing dependencies temporarily.
Dual Write	Attempt to update two independent stores from one business operation
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations

M35 · Business capability extraction

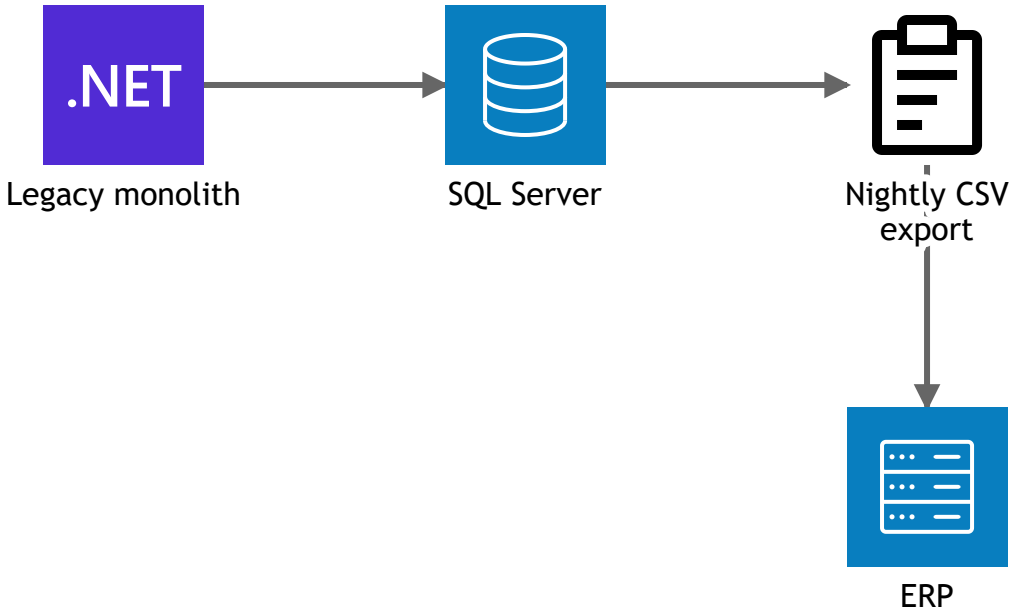
Modernize ERP incremental sync

Stock sync freshness must reach ≤5min, deletes included, where a nightly CSV export delivers it once a day.

Stock reaches the ERP once a night

Component	What it is	What it does
Legacy monolith	The .NET application that owns stock	Changes and deletes stock rows all day
SQL Server	One shared relational database	Holds the stock tables
Nightly CSV export	The existing full export of stock to the ERP, once a night	Writes one file that the ERP imports
ERP	The external enterprise resource planning system	Shows stock as of the last file, deleted rows included until then
ERP import	The ERP’s way of loading stock data	Takes the nightly CSV today

Current architecture



The ERP sees yesterday’s stock

The ERP gets stock levels once a night, so purchasing and fulfillment see old data, and a row deleted in legacy stays visible until the next file. The nightly job cannot deliver changes within minutes.

Old stock

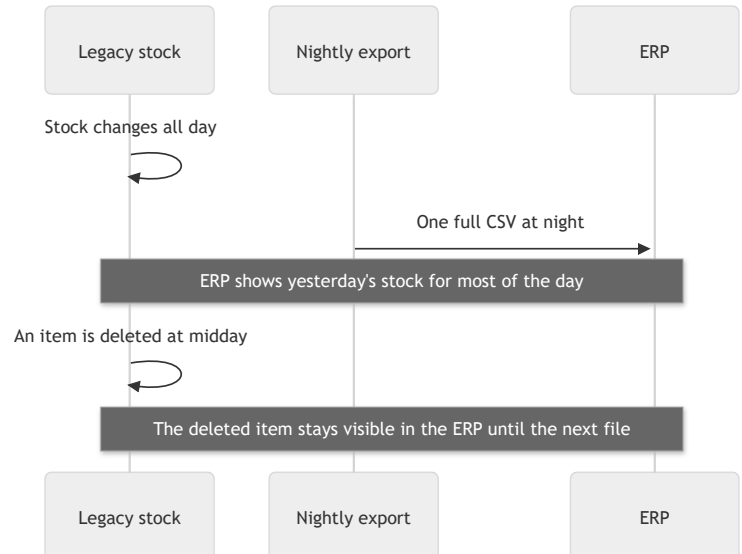
ERP stock is only as fresh as the last nightly CSV, and deleted rows linger

All day

stock changes and deletes arrive in legacy

≤5min

stock sync freshness, measured end to end including deletes



Each option trades lag, code change and meaning

Solution	What it is	What it does
CDC/ETL	Change data capture on the legacy stock tables feeding an extract-transform-load step into the ERP	Captures every insert, update and delete from the database log and loads it into the ERP in near real time
Outbox Events	Legacy records each stock change as a business event in the same transaction and publishes it	Delivers stock events with business meaning that the ERP applies
Incremental batch	A frequent job that exports only the rows changed since the last run, using a watermark	Repeats the existing export in small slices instead of one nightly file

Raw rows are cheap to get; meaning costs code

Solution	Gains	Costs
CDC/ETL	Few code changes Captures deletes too Near-real-time lag	Raw schema leaks to the ERP Log capture to operate Mapping step needed in the load
Outbox Events	Business meaning, not raw rows Consumers decoupled from the schema No event lost after commit	Every stock write path must change Event versioning and delivery to manage Implementation effort in undocumented code
Incremental batch	Simple: reuses the existing export No new infrastructure Few code changes	Lag up to the run interval A watermark cannot see deletes Needs a reliable change column

Deciding signal: freshness bound, change volume and rows versus business meaning. Measure max end-to-end lag including deletes at peak writes against ≤5min. Minutes are tolerable for a batch, so a run every 1 min (for example) that also finds deletes is cheapest; lag above ≤5min or deletes it cannot find picks CDC/ETL; a need for business meaning picks Outbox Events.

Smallest safe move

Incremental batch beside the nightly CSV

- **Scope:** data; the stock export only; legacy stock code and write ownership unchanged
- **Change:** export changed rows by Watermark every 1 min (for example) and find deletes by Key-set comparison; keep the Nightly CSV export as the correction
- **Safety net:** the nightly full CSV overwrites any drift; deletes are measured, because a watermark cannot see them
- **Escape route:** pause the incremental job; the nightly CSV remains the sync
- **Deferred:** CDC/ETL when max lag including deletes exceeds ≤5min or deletes are found too late; Outbox Events when the ERP needs business meaning and every stock write path can emit an event

```
SELECT sku, qty FROM stock WHERE row_version > @watermark;   -- changed rows
SELECT sku FROM erp_sent_keys EXCEPT SELECT sku FROM stock;  -- deleted rows
```

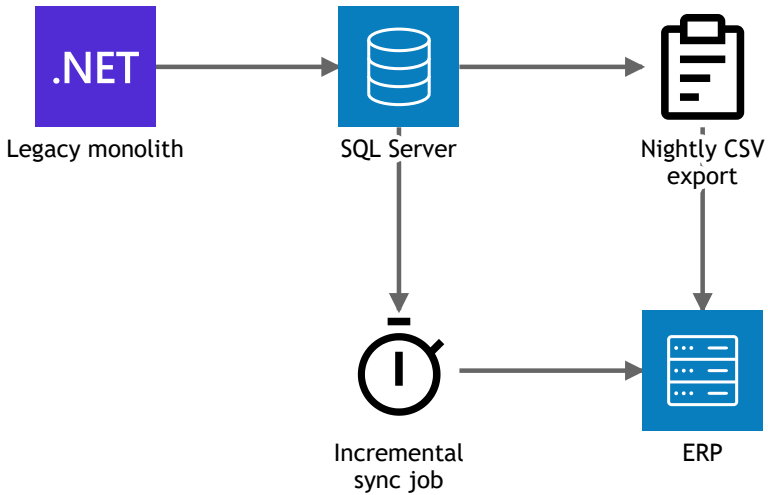
Verification: lag measured including deletes

Check	Pass condition
Stock changes and deletes at peak write rate over a working day	Max end-to-end lag from legacy commit to ERP visibility ≤5min, deletes included
Row deleted in legacy	Removed or zeroed in the ERP within ≤5min
Incremental state compared with the next nightly CSV	No unexplained difference in stock levels or keys
Incremental job stopped for several runs and restarted	Catches up from its watermark; no gap and no double application
Fitness function	A scheduled probe changes and deletes a canary stock row and asserts both reach the ERP within ≤5min; an alert fires on excess lag or nightly drift

Rollback: routing rollback: pausing the job returns the feed to the nightly CSV as before. State rollback: ERP stock changed by incremental slices is repaired by the next full CSV, which overwrites it with a known state; the job only reads, so legacy data is untouched.

Stop: when the measured max lag including deletes stays ≤5min, no CDC pipeline or event layer is added.

Solution architecture



The nightly export stays as the correcting path. A new incremental job reads changed rows and deleted keys from SQL Server and loads them into the ERP within the freshness bound.

Glossary

Used in this step

Term	Definition
CDC/ETL	Change data capture feeding an extract-transform-load step into the target
Outbox Events	Stock changes recorded as business events in the same transaction and published
Incremental batch	Frequent job that exports only the rows changed since the last run
Change Data Capture (CDC)	Detects and propagates changes made to database records.
ETL	Extract, transform and load data into a target system
Watermark	Marker of the last change already exported; the next run starts after it
Key-set comparison	Comparing the keys in legacy with the keys already sent to find deleted rows
Sync lag	Time from a change in legacy to its visibility in the ERP
Nightly CSV export	The existing full export of stock to the ERP, once a night
Freshness	Maximum age of data that the consumer accepts
Routing rollback	Switching traffic or the writer flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Transactional Outbox	Atomically records business changes and outgoing events in the same database transaction.
Domain Event	Statement that an important business fact occurred
Polling	Repeated requests to determine whether data changed
Batch Processing	Process groups of work together
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence

M36 · Business capability extraction

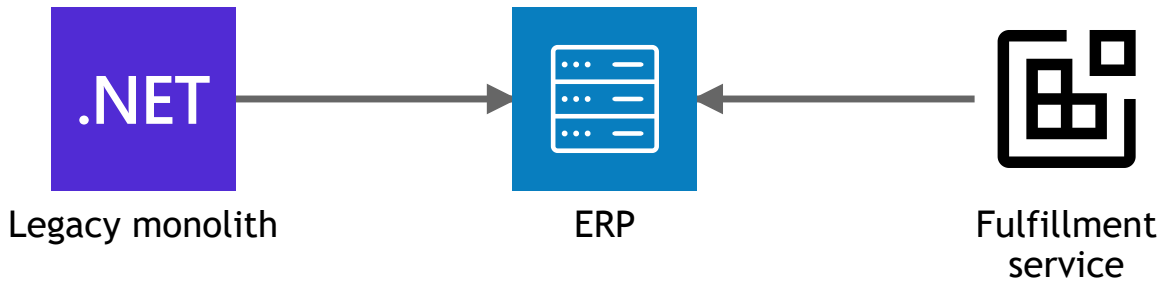
Reconcile ERP vs fulfillment model

New fulfillment recognizes a distinct lifecycle for orders and shipments, so scenario examples for partial shipments and returns must end in the expected state on both sides, with no shipment lost or counted twice.

The ERP merges shipments into one order status

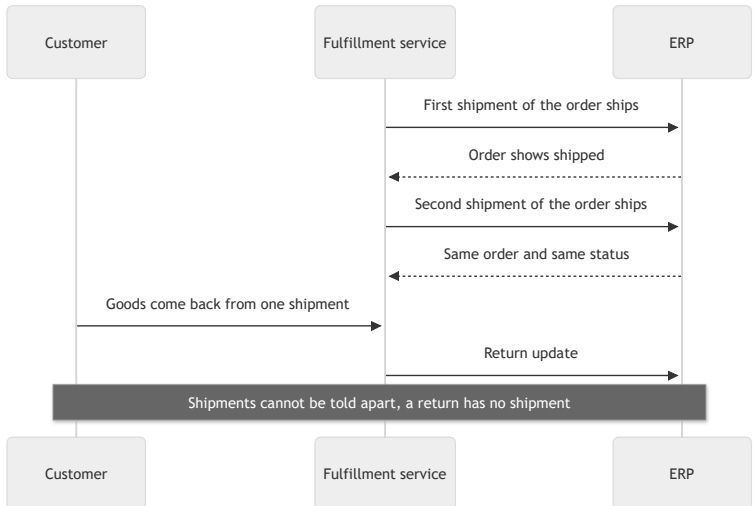
Component	What it is	What it does
Legacy monolith	The .NET app with the old Fulfillment module	Updates the ERP with one merged order status
Fulfillment service	The new service beside the legacy monolith	Models orders and shipments separately
ERP	The external ERP system	Records one status per order, shipments included
ERP port	The isolated port to the ERP	Keeps legacy order code free of ERP types
Anti-corruption layer	Mappings checked with domain examples	Translates between two models at a boundary
Stock sync	Incremental job that loads stock	Sends stock to the ERP in increments

Current architecture



One order, several shipments

The ERP records a shipment as a status on the order, so one order has one fulfillment state. The new fulfillment service models orders and shipments as separate lifecycles: one order can have several shipments and a return belongs to one shipment. Sharing the ERP record as it is erases these distinctions or sends them to the ERP as errors.



One merged state

per order in the ERP; fulfillment tracks orders and shipments separately

Several shipment events

for one order from partial shipments and returns

Expected state on both sides

in scenario examples for partial shipments and returns, with no shipment lost or counted twice

Each option decides where the two models meet

Solution	What it is	What it does
DDD Context Map + ACL	A context map of the fulfillment and ERP bounded contexts plus an anti-corruption layer (ACL) that translates at their boundary	Folds shipment events into the ERP's one order status and unfolds ERP updates into fulfillment's separate lifecycles
Canonical enterprise model	One shared model of order and shipment that every system adopts	Replaces the per-system models by a single uniform vocabulary
Adopt the ERP model as is	Fulfillment stores orders and shipments the way the ERP merges them	Passes ERP records through unchanged so that nothing needs translating

Translation costs effort, sharing costs agreement

Solution	Gains	Costs
DDD Context Map + ACL	Precise translation Each model keeps its own meaning Relationship between contexts explicit	Effort to build and test the translation Context map needs maintenance ERP status lags fulfillment
Canonical enterprise model	Uniform One vocabulary for every system No pairwise translation	Organization coupling ERP and fulfillment must change together Slow agreement across owners
Adopt the ERP model as is	Cheapest: nothing to build No translation to maintain	Partial shipments and returns cannot be shown ERP terms leak into fulfillment Every ERP change changes fulfillment

Deciding signal: degree of model mismatch and how many systems must agree. Replay the scenario examples, a partial shipment and a return after delivery, and count the ERP fields that cannot express them. A mismatch that shows in the examples, with two models to connect, picks DDD Context Map + ACL; examples that fit the ERP model without loss would make Adopt the ERP model as is enough, which the constraint rules out; many systems and one owning body needing the same vocabulary make a Canonical enterprise model worth its coordination cost.

Smallest safe move

DDD Context Map + ACL for one flow

- **Scope:** code and data; partial shipments first, returns second; ERP, data and interfaces unchanged
- **Change:** draw the context map; build the anti-corruption layer (ACL) below behind a feature flag beside the legacy Fulfillment module
- **Safety net:** neither model enters the other; mapping versioned and tested against the scenario examples; Adopt the ERP model as is is no shortcut
- **Escape route:** flag back to legacy; repair the ERP status forward by replaying logged events
- **Deferred:** Canonical enterprise model when many systems and one owning body need one vocabulary worth its coordination cost; Adopt the ERP model as is is rejected, not deferred

```
owns      fulfillment        →  the shipment lifecycle; the ERP keeps one status per order
fold      shipment events    →  the ERP's one order status
unfold    ERP order updates  →  fulfillment's separate lifecycles
unmapped  any other event    →  reject and log, never guess
never     ERP vocabulary into fulfillment, fulfillment's shipments to the ERP as orders
```

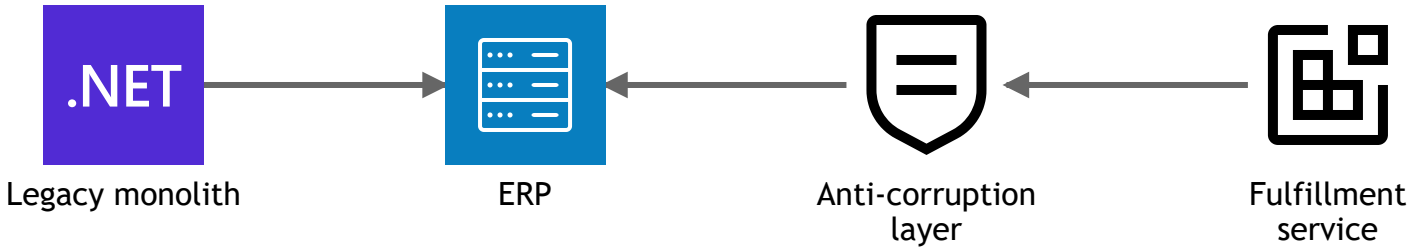
Verification: scenario examples replayed

Check	Pass condition
One order shipped in 2 shipments (for example)	ERP shows partially shipped, then shipped; fulfillment holds 2 shipments for 1 order
Return after delivery of one shipment	Return attached to that shipment; ERP order status and quantities equal the expected state
ERP update or event with no mapping injected	Rejected and logged; no guess, no shipment lost or counted twice
The same example events replayed	Same final state on both sides; nothing counted twice
Fitness function	The scenario examples run as contract tests in the pipeline and fail on any difference between the two models; a production counter of rejected events is alerted above zero until each one is explained

Rollback: routing rollback: the flag returns the flow to the legacy Fulfillment module and its merged ERP updates at once. State rollback: shipments created by the new service exist only there and are not copied into the legacy model; the ERP status is repaired forward by replaying the events through the corrected mapping.

Stop: when the translation passes the scenario examples for partial shipments and returns, no canonical enterprise model or further shared-model layer is added.

Solution architecture



The legacy monolith keeps its merged ERP updates. A translation layer now sits between the new fulfillment service and the ERP and replaces the direct connection.

Glossary

Used in this step

Term	Definition
DDD Context Map + ACL	Context map of the two contexts plus an anti-corruption layer translating at their boundary
Canonical enterprise model	One shared model of order and shipment that every system adopts
Adopt the ERP model as is	Fulfillment stores orders and shipments the way the ERP merges them
Context Map	Description of relationships between bounded contexts
Anti-Corruption Layer (ACL)	Protects a domain model from incompatible external system models.
Bounded Context	Boundary inside which a domain model and vocabulary have specific meaning
Lifecycle	The states an order or a shipment passes through over time
Partial shipment	One order delivered in more than one shipment
Return	Goods sent back after delivery, belonging to one shipment
Scenario example	A concrete case, such as a partial shipment or a return, with the state both systems must reach
Routing rollback	Switching traffic or the writer flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
DDD	Domain-driven design aligns software models with business concepts
Aggregate	Group of entities/values forming an invariant and transaction boundary
Domain Event	Statement that an important business fact occurred
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations

M37 · Business capability extraction

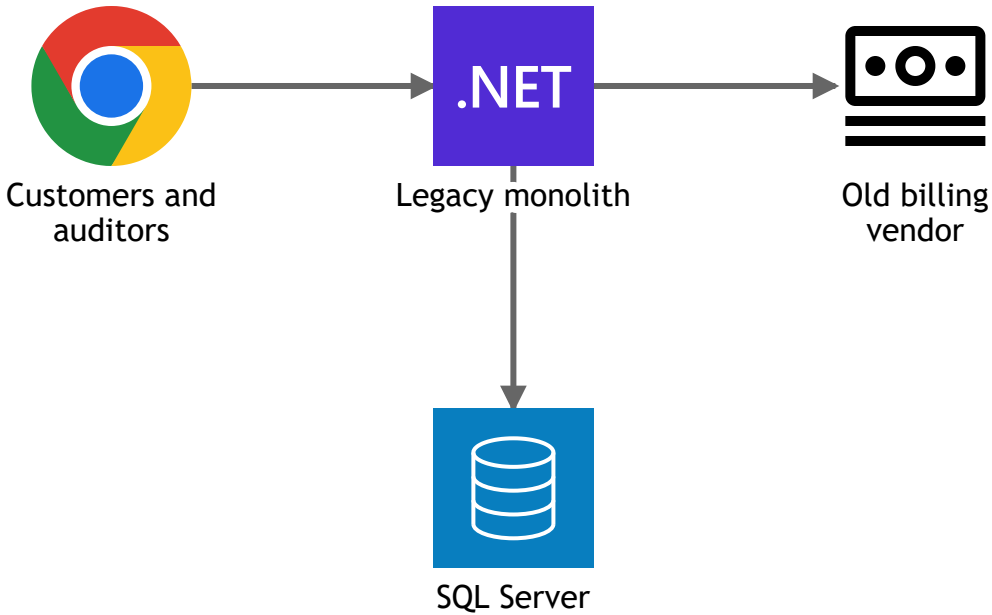
Switch billing provider

Old invoices must remain available during the provider transition: 100% of sampled historical invoices stay retrievable and identical.

Billing runs through a vendor whose contract is ending

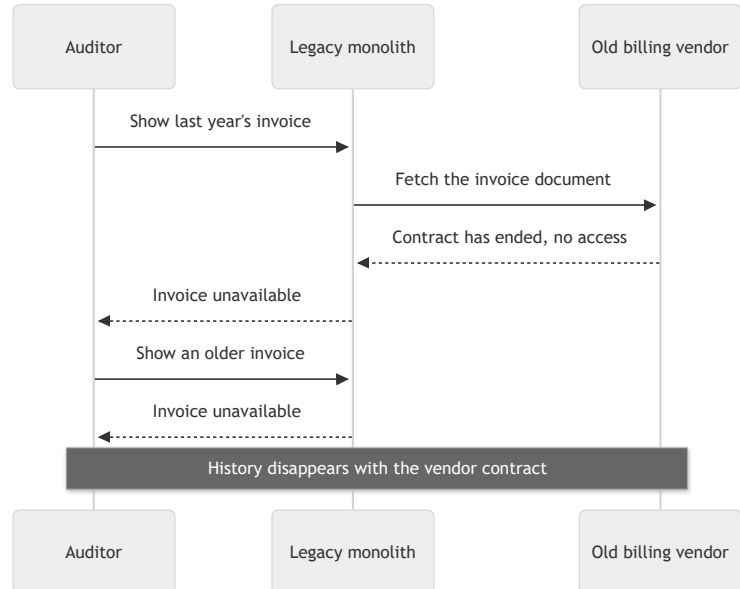
Component	What it is	What it does
Billing module	The invoicing code inside the legacy monolith	Issues invoices and refunds and retrieves invoices through the old vendor
Old billing vendor	The external provider whose contract terminates	Renders, stores and serves the invoice documents
SQL Server	One shared relational database	Stores the data of every module, including billing records
Compliance	The stakeholder for retention and audit	Requires every historical invoice to stay available unchanged
Feature flags	Runtime switches for single behaviors	Turn one change off without undoing the others

Current architecture



History disappears with the contract

The old billing vendor contract terminates, and every invoice is issued, refunded and retrieved through that vendor. Compliance needs the history to stay available unchanged for retention and audit, so the provider may change but the history may be neither lost nor rewritten.



Contract ends

and historical invoices are served only through the old vendor

New invoices and refunds

keep flowing while the provider changes

100%

of sampled historical invoices retrievable and identical to the originals

Each option decides where the provider boundary sits

Solution	What it is	What it does
Branch by Abstraction + Adapter	A billing interface inside the monolith with one adapter per provider	Sends issue, refund and retrieve calls through the interface to the old or the new adapter
Provider routing by tenant	A routing rule that sends each tenant's billing to the old or the new provider	Moves tenants group by group and keeps every other tenant on the old provider
Big-bang switch with re-import	Move all billing to the new provider on contract end and re-import old invoices into it	Re-creates every historical invoice as a new-provider document

Staging costs a second provider, rewriting costs the audit trail

Solution	Gains	Costs
Branch by Abstraction + Adapter	Staged switch behind one interface Old adapter stays as fallback Callers of Billing do not change	Two providers run side by side Temporary indirection to retire Adapters need contract tests
Provider routing by tenant	Controlled exposure per tenant A tenant can go back to the old provider	Reconciliation across two providers Routing table needs an owner Invoice number sequence spans providers
Big-bang switch with re-import	One provider after the switch No period of dual running	Historical invoices rewritten or lost Audit trail broken No way back after contract end

Deciding signal: invoice, refund and retrieval paths that call the vendor directly, how closely the new provider matches the old on replayed orders, retention requirements and time left on the contract. One seam with matching output picks the interface; differences to prove group by group add tenant routing; re-creating historical invoices fails the invariant.

Smallest safe move

Branch by Abstraction + Adapter around the Billing module

- **Scope:** code, data and billing calls; orders, payments and history stay untouched
- **Change:** Billing behind one billing interface with the old adapter; historical invoices into a read-only invoice archive with checksums; new adapter behind a feature flag, tried on replayed orders
- **Safety net:** historical invoices are never written or re-imported, which excludes Big-bang switch with re-import; interface and old adapter get an owner and a removal trigger
- **Escape route:** flip the flag back while the old contract is active; afterwards correct forward with a credit note
- **Deferred:** Provider routing by tenant when output differences must be proven group by group

```
public interface IBilling { Invoice Issue(Order o); Credit Refund(InvoiceId id); Invoice Retrieve(InvoiceId id); }
```

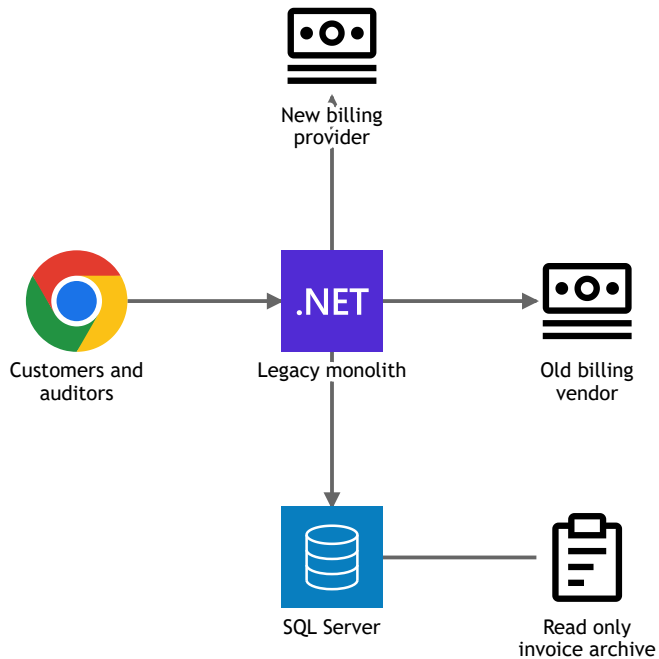
Verification: invoices, refunds and history compared

Check	Pass condition
Sampled historical invoices retrieved through the billing interface	Identical to the originals, checksums match the checksum manifest, none missing
New adapter on replayed orders, nothing sent to customers	Totals, tax lines and numbering match the old output with 0 unexplained differences
Refund of a new invoice and of a historical invoice	New invoice refunded by the new provider; historical refund is a credit note that references the unchanged original
Provider flag switched back during the transition	New invoices issued by the old adapter again, no failed issue, issued invoices unchanged
Fitness function	A scheduled job retrieves sampled historical invoices and alerts on any checksum mismatch or missing invoice; a pipeline contract test runs the same issue, refund and retrieve cases against both adapters

Rollback: routing rollback: the provider flag returns new invoices and refunds to the old adapter at once, possible only while the old contract is active. State rollback: an invoice issued by the new provider cannot be recalled or edited; correct it forward with a credit note and a new invoice and reconcile totals between the providers. Historical invoices never change, so they need no rollback.

Stop: when the new provider issues invoices and refunds and every historical invoice is retrievable unchanged from the archive, no tenant routing or further billing layer is added; the old adapter is removed after the retention check.

Solution architecture



The monolith keeps its callers and gains a second provider behind the same interface. Historical invoices are served from the read-only archive, so they outlive the old vendor contract.

Glossary

Used in this step

Term	Definition
Branch by Abstraction	Introduce stable interface then change the implementation behind it gradually
Adapter	Translates one system's calls and records into another's, one per provider
Provider routing by tenant	A rule that sends each tenant's billing to the old or the new provider
Big-bang switch with re-import	Switching every customer at once and re-creating old invoices in the new provider
Billing interface	The stable contract for issue, refund and retrieve that both providers implement
Invoice archive	Read-only store of issued invoices that outlives the old vendor contract
Checksum manifest	List of checksums used to prove an archived invoice is unchanged
Retention	The period for which records must stay available for audit
Credit note	A new document that corrects an issued invoice without editing it
Routing rollback	Switching traffic or the provider flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Anti-Corruption Layer (ACL)	Protects a domain model from incompatible external system models.
Feature Flag	Runtime control enabling behavior without deploying new code
Parallel Run	Execute old and new paths side by side and compare without duplicate business effects
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Forward Fix	Repair the new implementation without restoring the old system
Replace with SaaS	Substitute commercial cloud or packaged product for custom capability

M38 · Business capability extraction

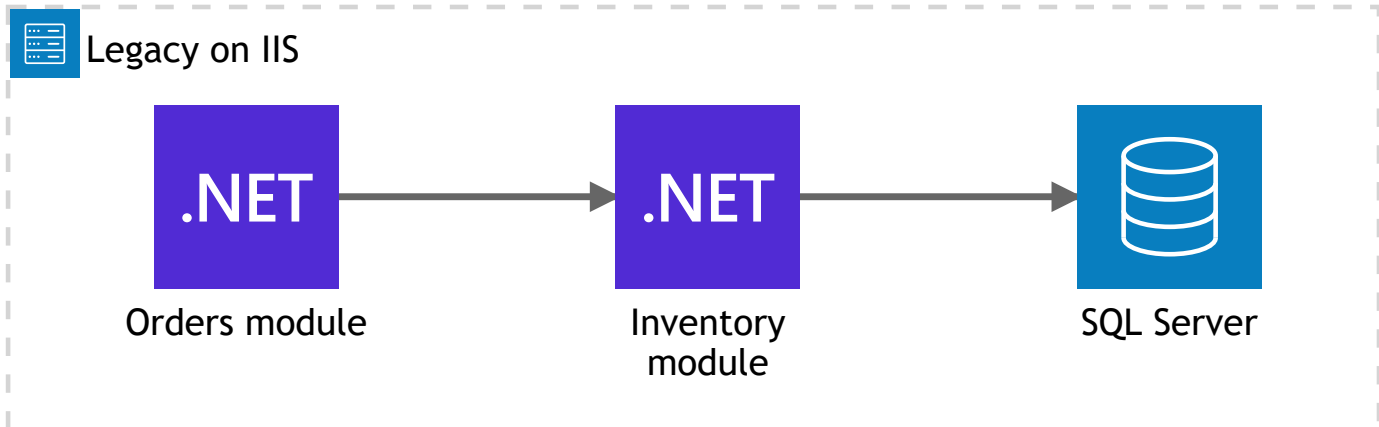
Locate inventory transaction boundary

Inventory deployments are needed independently, but orders and inventory still share one transaction, so every stock write, invariant and lock has to be mapped first.

Orders and inventory commit in one transaction

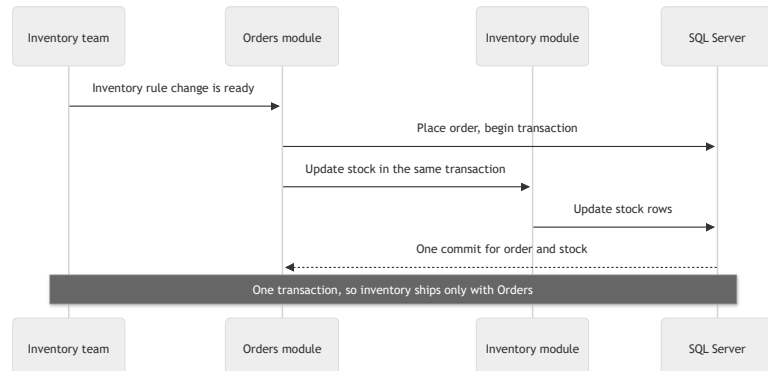
Component	What it is	What it does
Orders module	Order placement code inside the legacy monolith	Creates the order and updates stock in the same database transaction
Inventory module	Stock code inside the legacy monolith	Reserves, releases and adjusts the stock rows
SQL Server	One shared relational database	Commits order rows and stock rows together in one transaction
Characterization tests	Tests that record what the code does today	Show that order and stock behavior stays the same after a change
Feature flags	Runtime switches for single behaviors	Turn one change off without undoing the others

Current architecture



One transaction ties inventory to every release

Inventory rules change often, yet each change ships with the whole monolith because placing an order and updating stock run in one database transaction. Before inventory can deploy alone, the invariants and locks that tie the two together have to be known, or a split would silently break the stock guarantee.



Every inventory change
waits for a full monolith release

One database transaction
changes order rows and stock rows together

Every stock write mapped
to its invariant and lock, none outside one inventory command interface

Each option puts the boundary in a different place

Solution	What it is	What it does
Modular boundary first	Orders and Inventory as separate modules inside one deployable and one database	Keeps the single transaction and limits which code may touch stock
Inventory command interface	A small set of commands (reserve, release, adjust) that is the only way to change stock	Hides stock tables and locks behind a contract that can later cross a process boundary
Split DB early	Move the stock tables to a separate database before the boundary is known	Gives inventory its own database and deployment at once, with order and stock committed separately

A safe boundary costs coupling, an early split costs correctness

Solution	Gains	Costs
Modular boundary first	Safe: the transaction stays local No new network calls Cheap: code moves only	Shared database remains Inventory still ships with every release Boundary held only by convention
Inventory command interface	Internals hidden behind a contract Callers independent of stock tables Same contract in process or over the network	A new API to design and version Commands must keep the transaction's guarantees Every caller needs changing
Split DB early	Full data autonomy at once Inventory schema leaves the shared database	Order and stock commits can disagree Atomic invariants become distributed Hard to reverse after writes

Deciding signal: rate of coordinated releases and number of invariants spanning orders and inventory. Frequent coordinated releases with few spanning invariants pick the command interface; many invariants that must stay atomic, or rare coordinated releases, leave the modular boundary as enough; a split before they are covered makes local guarantees distributed.

Smallest safe move

Inventory command interface in front of the stock tables

- **Scope:** code and data access; one in-process interface, same transaction, same database, no new network call
- **Change:** map every statement that changes or locks stock; route callers through the interface behind a feature flag that keeps the direct update path
- **Safety net:** the single local transaction stays, so Split DB early is avoided and never-negative is unchanged; the interface is contract-tested
- **Escape route:** flag back to the direct path; no data moved, so no state rollback
- **Deferred:** Split DB early until a protocol covers every spanning invariant; Modular boundary first when many invariants must stay atomic or releases are rarely coordinated

```
public interface IInventoryCommands { Reservation Reserve(Sku s, int qty); void Release(ReservationId id); void Adjust(Sk
```

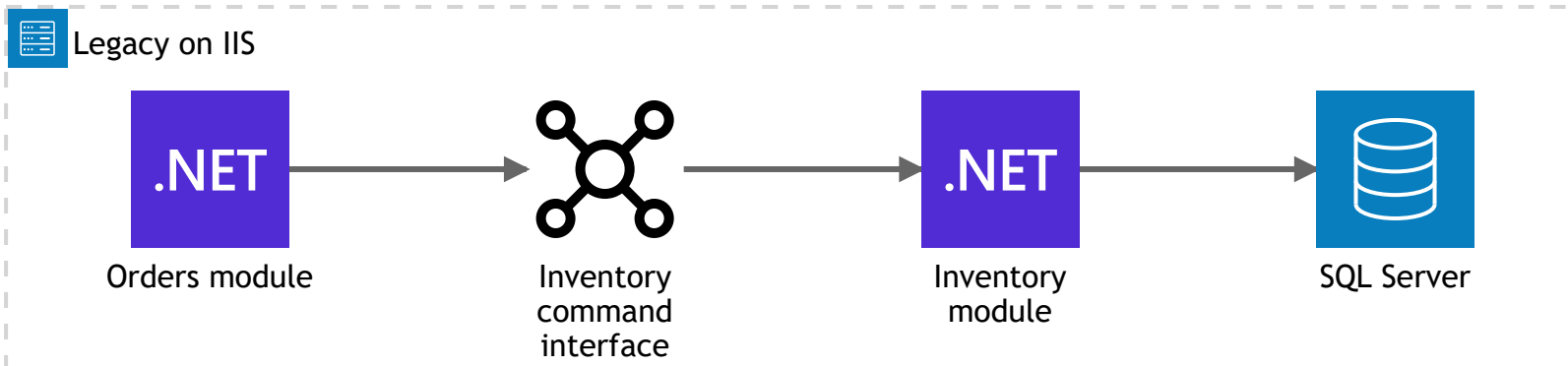
Verification: every stock write mapped and routed

Check	Pass condition
Inventory of stock-changing statements	Every statement and lock that changes stock listed with its caller, invariant and transaction; no unknown writer
Order placement through the command interface on characterization tests	Same order and stock results as the direct path; stock never negative
Code search and database trace for stock updates outside the interface	None found; direct updates exist only behind the old-path flag
Contract test of the interface with a breaking change made on purpose	Passes on the current interface; the breaking change fails the pipeline
Fitness function	A pipeline check fails when code outside the inventory module references the stock tables; the contract test fails on an incompatible change; an invariant query alerts on negative stock

Rollback: routing rollback: the feature flag returns every caller to the direct update path at once; the interface is in process, so no traffic moves. No state rollback is needed because no data and no transaction boundary moved; if a check finds a stock difference, reconcile stock counts against order lines and repair forward.

Stop: when the mapped invariants fit one local transaction behind the command interface, no database split is added yet; when the mapping shows many atomic invariants or coordinated releases are rare, the modular boundary stays as it is and nothing is extracted.

Solution architecture



Orders now reach stock only through the command interface, still inside one transaction and one deployable. The same contract can later cross a process boundary.

Glossary

Used in this step

Term	Definition
Modular boundary first	Separate Orders and Inventory modules inside one deployable and one database
Inventory command interface	The only way to change stock: reserve, release and adjust commands
Split DB early	Moving stock tables to another database before the boundary is known
Database transaction	Atomic execution boundary for related data modifications
Shared Database	Several modules or services use same persistence system or schema
Invariant	A business rule that must hold after every change, such as stock never negative
Lock	Stops other transactions changing a row until the holder commits
Contract test	Consumer expectations verified against a provider
Feature Flag	Runtime control enabling behavior without deploying new code
Routing rollback	Switching traffic or the flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Aggregate	Group of entities/values forming an invariant and transaction boundary
Database Per Service	Each service owns its private persistence and schema
Reservation Pattern	Temporarily claim business resource pending workflow completion
Saga Pattern	Coordinates distributed business transactions using local transactions and compensations.
Branch by Abstraction	Introduce stable interface then change the implementation behind it gradually
Write Ownership Handover	Transfer authoritative writes for bounded data from legacy to replacement

M39 · Business capability extraction

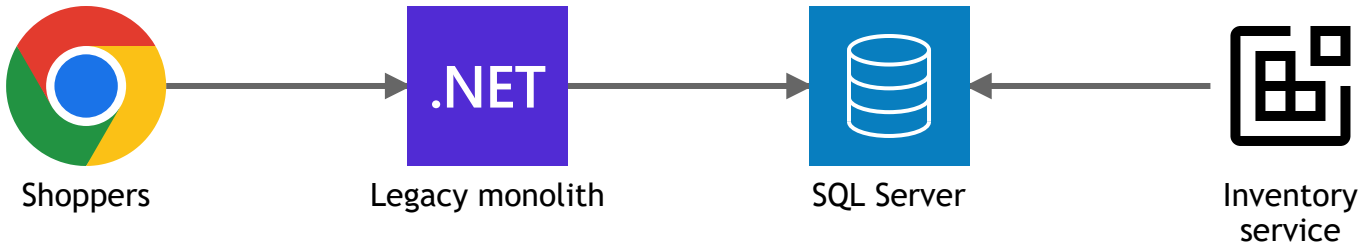
Move inventory ownership

Concurrent requests may oversell the last unit, so stock needs exactly one writer and zero oversells.

Inventory deploys alone, but stock still has several writers

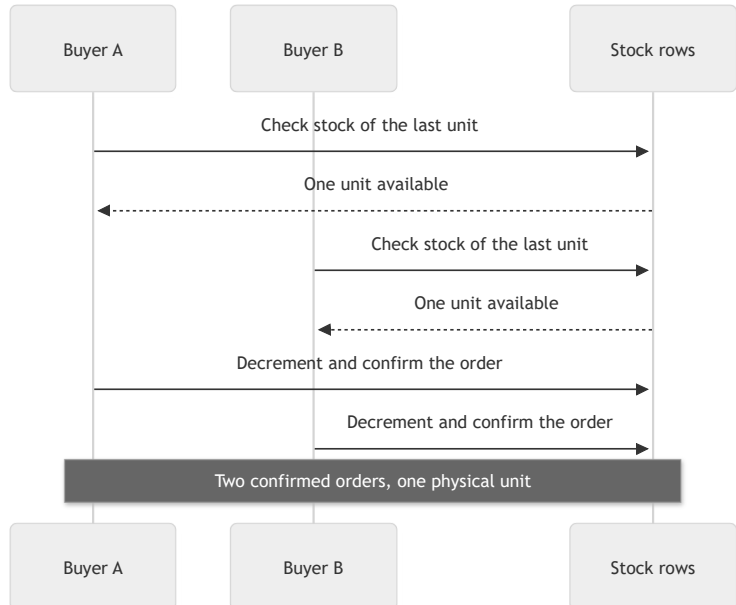
Component	What it is	What it does
Inventory service	The separate service behind the inventory command interface	Deploys on its own; reserves, releases and adjusts stock
Order flow	Order placement code in the legacy monolith	Checks availability, then decrements and confirms the order
SQL Server	One shared relational database	Still holds the stock rows that more than one code path can change
Feature flags	Runtime switches for single behaviors	Turn one change off without undoing the others

Current architecture



Two buyers can both get the last unit

Inventory now deploys on its own, yet stock can still be changed by the service and by older code paths, and an order checks availability before it decrements. When two buyers order the last unit at the same moment, both pass the check and both orders are confirmed.



Two orders

for the last unit can both be confirmed

Concurrent last-unit purchases

for example 50 buyers for 1 unit

Zero oversells

stock never negative

Each option decides who may change stock

Solution	What it is	What it does
Single Writer + Reservation	The inventory service is the only writer of stock, and orders claim units through reservations	Checks and decrements available stock in one atomic step per reservation, with expiry, before an order is confirmed
Partitioned ownership by SKU	Stock ownership split by SKU, with exactly one writer per partition	Routes every reserve call to the partition that owns the SKU
Saga alone	Order steps coordinated only by compensating actions, with no single stock writer	Decrements stock after the order is accepted and compensates if the decrement fails

One writer costs coordination, no writer costs correctness

Solution	Gains	Costs
Single Writer + Reservation	Stock never negative One authority for every stock row Unconfirmed reservations expire and release units	Orders wait for a reservation call Expiry and release logic to run Hot SKUs queue behind one writer
Partitioned ownership by SKU	Writers scale by partition Smaller blast radius per partition A hot partition can be isolated	Routing by SKU on every call Partition map needs an owner Rebalancing moves live ownership
Saga alone	No lock held across services Fits long-running flows	Both buyers pass the stock check first Compensation runs after the oversell Customers see cancelled orders

Deciding signal: code paths that can still change stock, peak reserve rate on the hottest SKUs and reserve latency against its target. One writer within its target at peak picks Single Writer + Reservation; a hot-SKU rate that saturates it picks partitioning; two paths decrementing without one atomic check (Saga alone) fails the invariant.

Smallest safe move

Single Writer + Reservation for stock

- **Scope:** write ownership and data; stock rows move to the service's own database, orders stay
- **Change:** reconcile, fence legacy stock writes (Write Fencing), drain, reconcile again, activate the single writer; each atomic reserve carries an idempotency key and an expiry
- **Safety net:** availability is never approved from a cache or copy; Saga alone is not the guard; legacy stock paths are fenced before activation
- **Escape route:** before the first reservation, unfence legacy and flip the writer flag back; after it, reverse replication or a forward fix
- **Deferred:** Partitioned ownership by SKU when hot SKUs saturate one writer

```
UPDATE stock SET available = available - @qty
WHERE sku = @sku AND available ⩾ @qty;    -- 0 rows changed means not enough stock
```

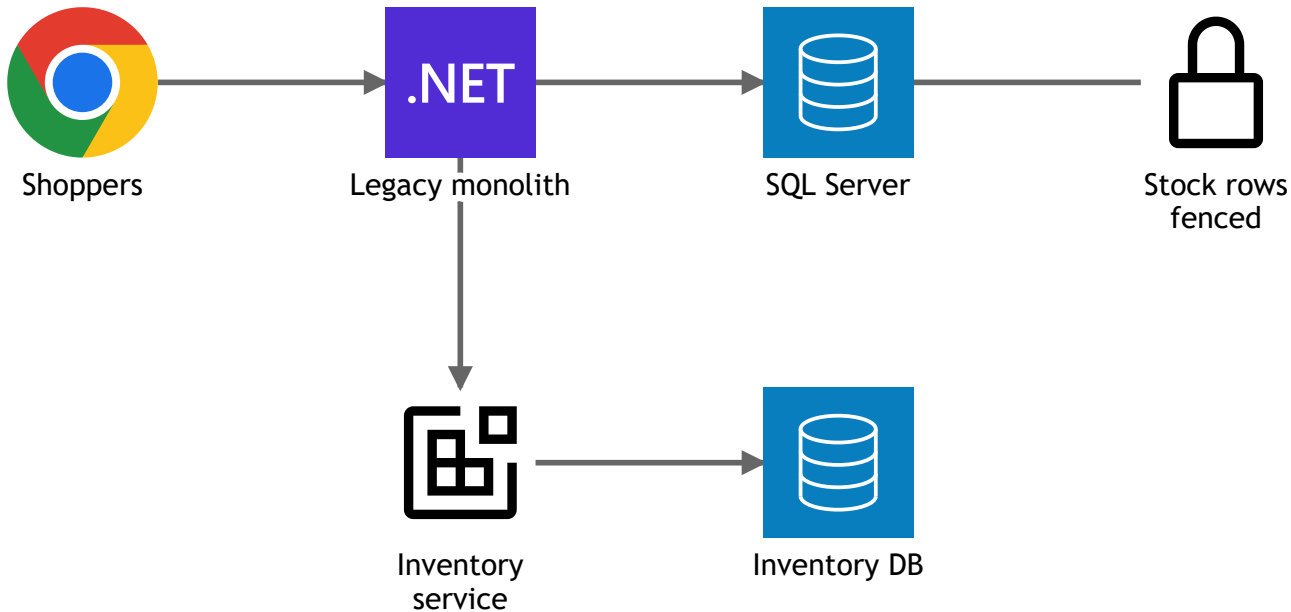
Verification: concurrent last-unit purchases

Check	Pass condition
Concurrent last-unit purchases (for example 50 buyers for 1 unit) through the order flow	Exactly one reservation succeeds, the others are rejected; stock never negative
Stock write attempted by a legacy path after the fence	Rejected; legacy stock rows unchanged
The same reserve request sent twice with one idempotency key	One reservation and one decrement
Reservation left unconfirmed past its expiry	Units released back to available stock and counts reconcile
Fitness function	A scheduled concurrency test of last-unit purchases fails on any oversell; an invariant monitor alerts on negative or unreconciled stock; the cutover script refuses to activate the writer without zero unexplained drift

Rollback: routing rollback is safe only before the service accepts its first reservation: unfence legacy and set the writer flag back. After that it is unsafe, because the legacy stock rows lack the new movements and switching back could sell units twice. State rollback then means reverse replication of the stock movements into SQL Server plus reconciliation, or a forward fix on the service; both are planned before the cutover.

Stop: when the single writer passes the concurrent last-unit test at peak with zero oversells and reserve latency within its target, no partitioning or saga-based stock guard is added.

Solution architecture



The monolith reaches stock only through the inventory service, the single writer on its own database. The stock rows in SQL Server stay readable behind the fence.

Glossary

Used in this step

Term	Definition
Single Writer + Reservation	Only the inventory service changes stock; orders claim units through reservations
Partitioned ownership by SKU	Stock split by SKU, each partition with exactly one writer
Saga alone	Compensating steps with no single atomic stock check
Reservation	Temporarily claim business resource pending workflow completion
Oversell	Confirming more orders than the physical stock can fulfil
Atomic reserve	One conditional update that claims units only when enough remain
Invariant	A business rule that must hold after every change, such as stock never negative
Idempotency Key	Stable identifier recognizing multiple attempts at the same logical command
Write Fencing	Prevent stale writers from modifying state after ownership changes
Reverse Replication	Propagate authoritative new-state changes back to old consumers temporarily
Routing rollback	Switching traffic or the writer flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Saga Pattern	Coordinates distributed business transactions using local transactions and compensations.
Write Ownership Handover	Transfer authoritative writes for bounded data from legacy to replacement
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Forward Fix	Repair the new implementation without restoring the old system
Database Per Service	Each service owns its private persistence and schema

M40 · Business capability extraction

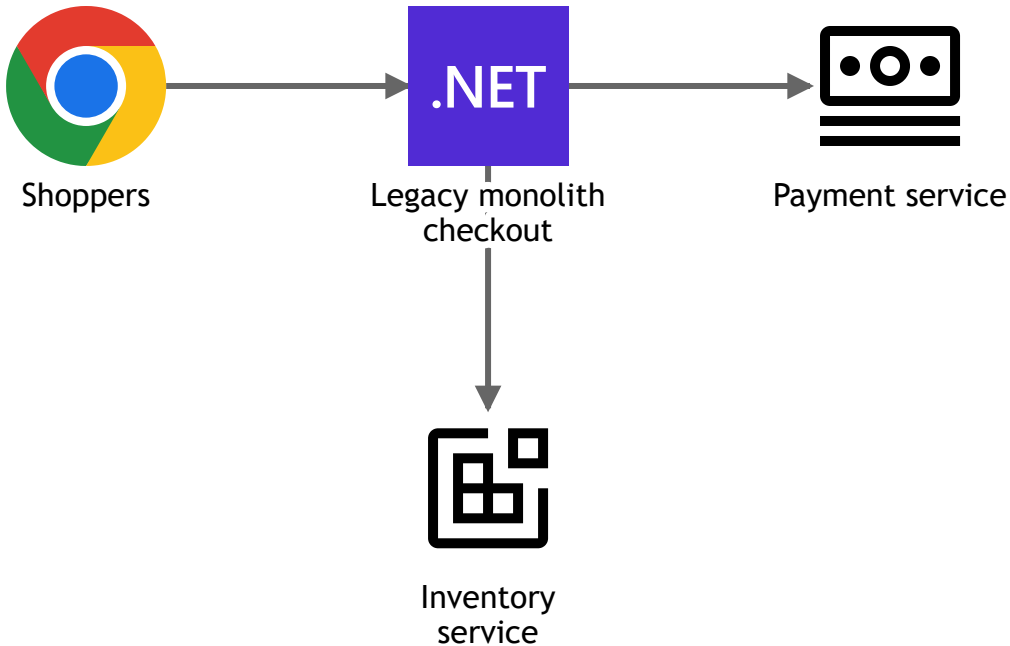
Coordinate checkout across services

Partial failures leave paid-but-unreserved orders, so checkout needs a recorded workflow that ends with 0 paid-but-unreserved orders.

Payment and inventory are independent, and no transaction covers both

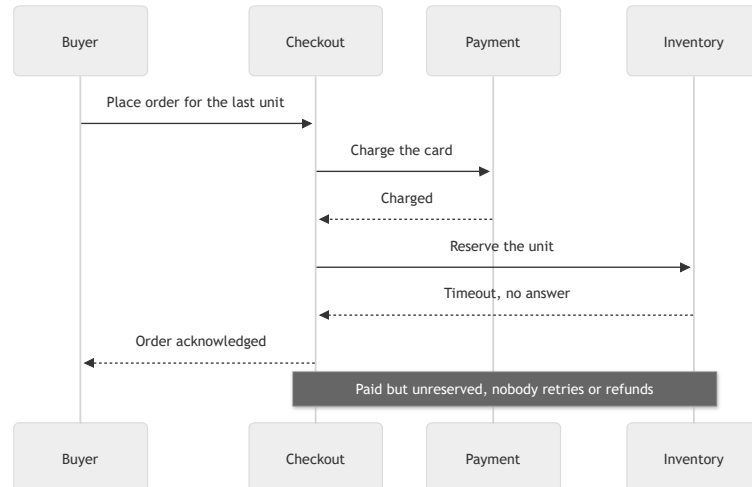
Component	What it is	What it does
Checkout flow	The order placement route in the legacy monolith	Reserves stock, charges payment and confirms the order, one call after another
Payment service	The separate service around the payment integration	Charges the card and refunds on request
Inventory service	The service that is the single writer of stock	Reserves and releases units
SQL Server	The shared database that still holds the order rows	Commits the order, but no longer covers payment or stock
Feature flags	Runtime switches for single behaviors	Turn one change off without undoing the others

Current architecture



A customer can pay for stock that is not reserved

Checkout now calls two independent services, and no transaction covers both. When the charge succeeds and the reservation then fails or times out, the customer has paid for an order with no stock behind it and nobody notices.



Partial failures

leave paid-but-unreserved orders

A failure at each step

including timeouts and duplicate requests

0 paid-but-unreserved orders

each order ends confirmed, or cancelled with release and refund

Each option decides who drives the checkout steps

Solution	What it is	What it does
Saga orchestration	One coordinator drives checkout as ordered local steps, each with a compensation	Calls reserve, charge and confirm in order, records every state and runs release or refund when a step fails
Choreography	Each service reacts to events from the others, with no central coordinator	Inventory, payment and orders publish and consume events, and failure events trigger compensations
Keep transaction in monolith	Checkout stays one local transaction and one code path in the legacy monolith	Commits the order and the payment record together in SQL Server and calls the services inside that unit of work

Visible compensation costs a coordinator, autonomy costs visibility

Solution	Gains	Costs
Saga orchestration	Compensation visible in one place Every order has a recorded state Failures retried or repaired centrally	A coordinator to build and run Compensation is not an atomic rollback Every step needs an idempotent handler
Choreography	Services stay autonomous No central coordinator to scale Another reacting service is easy to add	Order state spread across services Hard to see why an order is stuck Compensation logic scattered
Keep transaction in monolith	Safer until separation is needed No new coordinator or event flow Familiar failure handling	A remote call cannot be rolled back Checkout stays tied to monolith releases Services' own commits stay uncovered

Deciding signal: steps and participants in checkout, partial failures that leave a half state, and whether one order's state is visible in one place. A money path with refunds picks orchestration; loosely coupled reactions such as mail and ERP pick choreography; an invariant inside one local boundary keeps the monolith transaction, which no longer holds for two stores.

Smallest safe move

Saga orchestration on the checkout route

- **Scope:** code and order-state ownership on one route; payment, inventory and other routes unchanged
- **Change:** one saga record per order; reserve, charge, confirm in order; idempotency keys, retries and a sweeper for stuck sagas; old flow behind a flag
- **Safety net:** compensation is a recorded action, not an atomic rollback; reserve before charge, so a failed reservation never leaves a charge
- **Escape route:** flag returns new orders to the old flow; sagas in flight finish or compensate
- **Deferred:** Choreography for loosely coupled reactions such as mail and ERP; Keep transaction in monolith if payment and stock return to one boundary

```
reserve stock → charge payment → confirm order
on failure:  refund payment → release reservation → cancel order
```

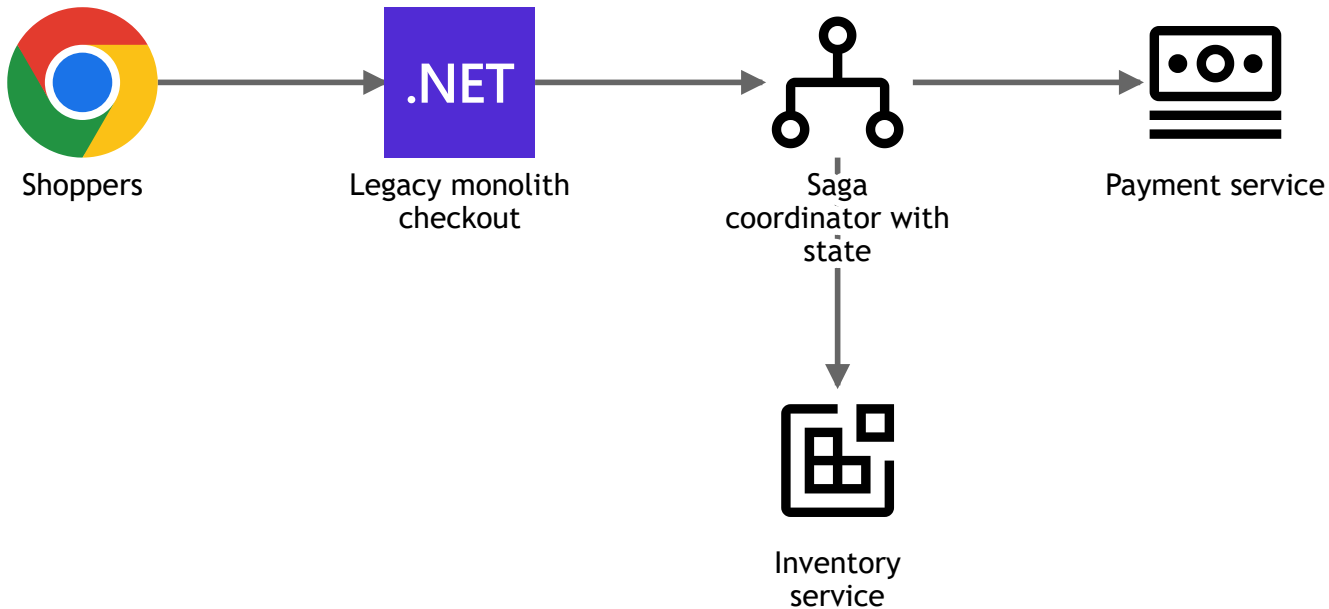
Verification: a failure injected at every step

Check	Pass condition
Failure injected after the stock reservation and before the charge	Reservation released, order cancelled, no charge
Failure injected after the charge and before the confirmation	Order completes on retry, or the charge is refunded and the reservation released; never paid and unreserved at rest
Duplicate checkout request and duplicate payment callback with one idempotency key	One order, one charge, one reservation
Coordinator process killed in the middle of a saga	The saga resumes from its stored state; no order stays in a half state
Fitness function	A failure-injection test fails each step in turn in the pipeline; a reconciliation query alerts on any order paid without a reservation or pending past its time limit

Rollback: routing rollback: the feature flag returns new orders to the old synchronous checkout at once; sagas already in flight finish or compensate. State rollback: a charge or reservation already made cannot be undone atomically; compensation is a new action, and any paid-but-unreserved order found afterwards is repaired forward by reserving again or refunding, found by reconciliation.

Stop: when failure injection leaves no paid-but-unreserved order and the coordinator inside the monolith is enough, no separate workflow engine, choreography or event bus is added.

Solution architecture



Checkout still starts in the monolith, but a coordinator with stored state now drives the payment and inventory steps and their compensations.

Glossary

Used in this step

Term	Definition
Saga orchestration	One coordinator runs the local steps and compensations of a business transaction
Choreography	Services react to each other's events with no central coordinator
Keep transaction in monolith	Checkout stays one local transaction inside the legacy monolith
Compensation	A new action that undoes the business effect of an earlier step, not atomically
Saga state	Stored record of which checkout steps ran for one order
Paid-but-unreserved order	An order whose payment succeeded but whose stock was never reserved
Idempotency Key	Stable identifier recognizing multiple attempts at the same logical command
Reservation	Temporarily claim business resource pending workflow completion
Local transaction	Atomic execution boundary for related data modifications in one database
Failure injection	Making a step fail on purpose to observe how the flow recovers
Routing rollback	Switching traffic or the flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Saga Pattern	Coordinates distributed business transactions using local transactions and compensations.
Monolith	Entire application deployed as a single unit.
Event Bus	Infrastructure for distributing events between components.
Transactional Outbox	Atomically records business changes and outgoing events in the same database transaction.
Forward Fix	Repair the new implementation without restoring the old system

M41 · Business capability extraction

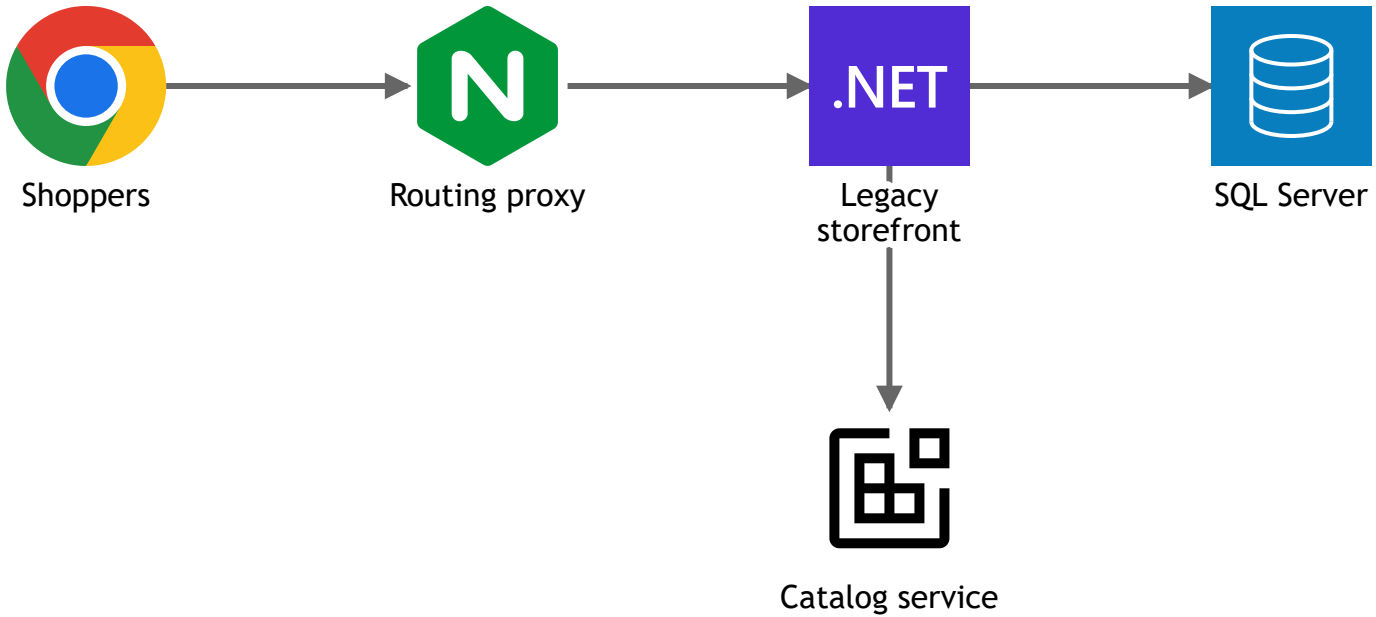
Migrate UI route by route

The new account/catalog UI has a different release schedule, so the storefront must move 1 UI route at a time, and any route must return to legacy with 0 other routes changed.

Every storefront page ships with the legacy monolith

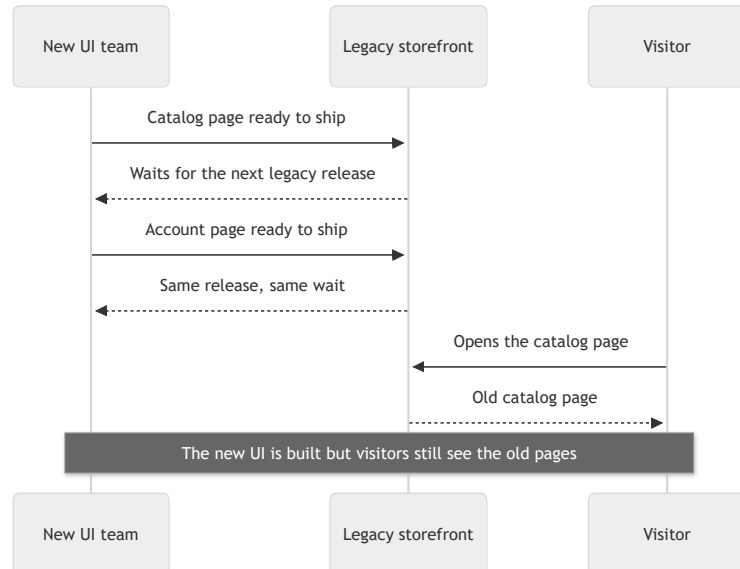
Component	What it is	What it does
Routing proxy	The public entry point in front of legacy	Forwards every path to the monolith; can send one path elsewhere by flag
Legacy storefront	The server-rendered pages of the .NET monolith	Renders catalog, account, cart and checkout pages and keeps the login session
SQL Server	The shared relational database	Still holds the order and account data
Catalog service	The replacement catalog with its own database, the only product writer	Serves product data to the legacy pages
Feature flags	Runtime switches for single behaviors	Turn one change off without undoing the others

Current architecture



A new UI that cannot ship before legacy does

A new UI team builds the account and catalog pages on its own schedule, yet the storefront is one server-rendered application, so every new page waits for the next legacy release. The whole storefront cannot be replaced at once, because its routes share the legacy login session and unmoved routes must keep working.



New UI pages

wait for the legacy release schedule

One server-rendered storefront

all routes ship together and share one login session

1 UI route

served by the new UI, then returned to legacy, with 0 other routes changed

Each option decides how the storefront is split

Solution	What it is	What it does
Strangler frontend route-by-route	A routing rule per URL path sends that path to the new UI and every other path to legacy	Replaces the storefront one route at a time, with unmoved routes still served by legacy
Microfrontend	A shell page that loads independently developed and deployed UI modules at runtime	Lets each team release its part of a page alone, without a release of the others
old UI retain	Keep the working server-rendered storefront unchanged	Serves every route from legacy, so new pages ship only with legacy releases

Incremental moves cost session work, autonomy costs a runtime

Solution	Gains	Costs
Strangler frontend route-by-route	Incremental: one route at a time One flag returns one route New UI releases on its own schedule	Sessions: routes share login state Two UIs must look alike meanwhile Routing rules need an owner
Microfrontend	Autonomous: each team releases alone Several modules can share one page	Runtime complexity: shell, loading, versions Shared styles and state need rules Server-rendered pages need a shell first
old UI retain	Safe: minimal risk to working pages No new routing or session work	Evolution restricted to the legacy schedule Debt and costs persist New UI cannot ship on its own

Deciding signal: how often a legacy release blocks the new UI, how many routes the proxy can separate by path, how many of them need the legacy login session, and how many UI teams release parts of one page. Routes separable by path with one UI team pick Strangler frontend route-by-route; more than one team on a page picks Microfrontend; a route that works and rarely changes picks old UI retain. The choice changes when routes cannot be separated by path.

Smallest safe move

Strangler frontend route-by-route for the catalog pages

- **Scope:** traffic and code on one read-only route; the rest, including the legacy session, unchanged
- **Change:** the new UI renders catalog pages from the catalog service; the proxy sends /catalog to it by route flag
- **Safety net:** a health check falls back to legacy; no signed-in state on the new route; every route rule has an owner and a removal trigger, so the compatibility layer does not become debt
- **Escape route:** switch the route flag back
- **Deferred:** Microfrontend when more than one team releases one page; old UI retain for rarely changing routes; the account route until single sign-on is needed

```
location /catalog { proxy_pass http://new_ui; }   # moved by route flag
location /account { proxy_pass http://legacy; }   # waits for the session problem
location /        { proxy_pass http://legacy; }   # everything else stays
```

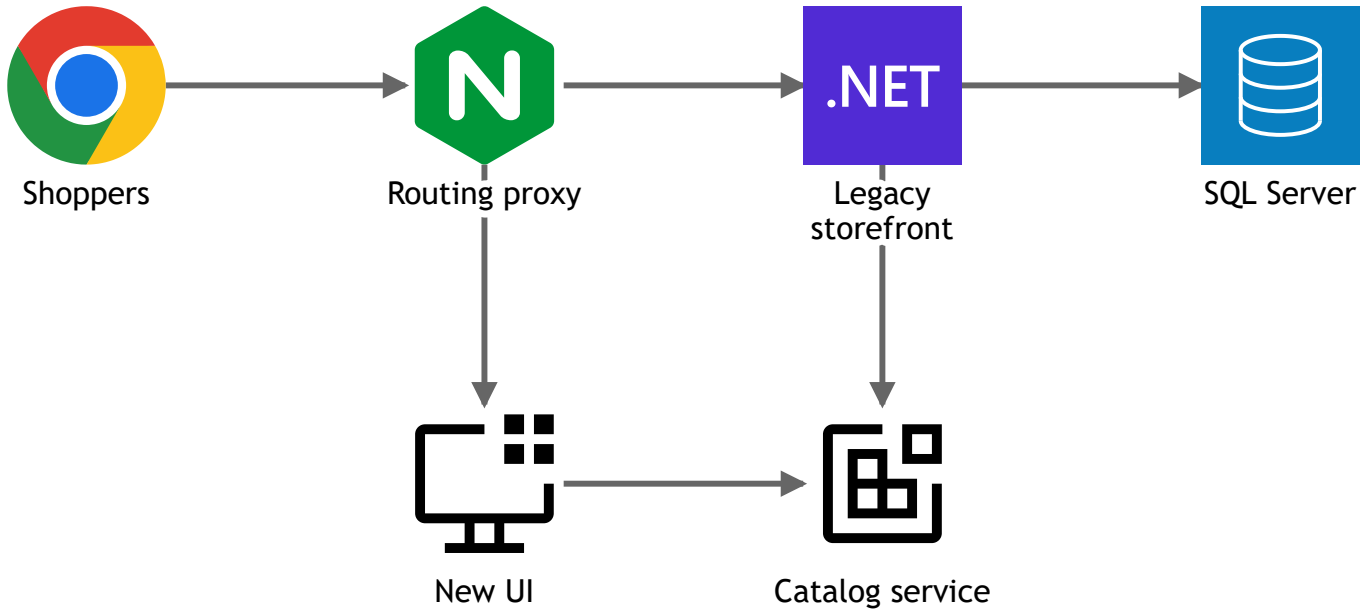
Verification: one route out and back

Check	Pass condition
Route /catalog moved to the new UI by route flag	Same URL; products come from the catalog service; error rate and response time recorded against the legacy baseline
New UI released in a window with no legacy release	The change goes live and the legacy storefront is not redeployed
New UI made unhealthy on purpose	Health check fails; the proxy serves the legacy catalog pages with no visitor-visible error
Route flag switched back	Only /catalog answers from legacy after one configuration change; every other route matches its baseline responses; no failed requests
Fitness function	A pipeline smoke test requests every storefront path through the proxy and asserts which system answers each and that one flag changes only its own route; error rate per route is alerted

Rollback: routing rollback: the route flag returns /catalog to legacy at once and no other route is touched. No state rollback is needed, because the catalog route only reads from the catalog service and writes nothing. A later route that writes data needs a state rollback plan: reconcile what the new UI wrote, or repair forward.

Stop: when routes move to the new UI without a legacy release and the remaining routes keep working from legacy, no microfrontend shell is added, and routes that rarely change stay on legacy.

Solution architecture



Shoppers keep every URL. The routing proxy sends the catalog path to the new UI, which reads the catalog service, while every other path and the login session stay in the legacy storefront.

Glossary

Used in this step

Term	Definition
Strangler frontend route-by-route	Send selected URL paths to the new UI while every other path stays on legacy
Microfrontend	Frontend divided into independently developed and deployed modules
old UI retain	Keep working legacy capability unchanged while other parts evolve
Route	One URL path of the storefront, served by legacy or by the new UI
Routing proxy	Stable public entry point that sends each route to legacy or to the new UI
Route flag	Feature flag that selects legacy or the new UI for one route
Shell	Host page that loads independently deployed UI modules at runtime
Legacy session	Server-side login state of the .NET application, tracked by a session cookie
Compatibility layer	Code or configuration that keeps old clients working while the system changes
Health check	Probe that tells the proxy whether a target can take traffic
Routing rollback	Switching traffic or the flag back to the previous path
State rollback	Restoring data to its earlier content, which traffic switching cannot do

Related concepts

Term	Definition
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations
Routing Façade	Stable public entrypoint routing selected capabilities to old or new system
Feature Flag	Runtime control enabling behavior without deploying new code
Canary Routing	Shift a small controlled segment of live traffic to a new implementation
SSO	Reuse sign-in for several applications

M42 · Business capability extraction

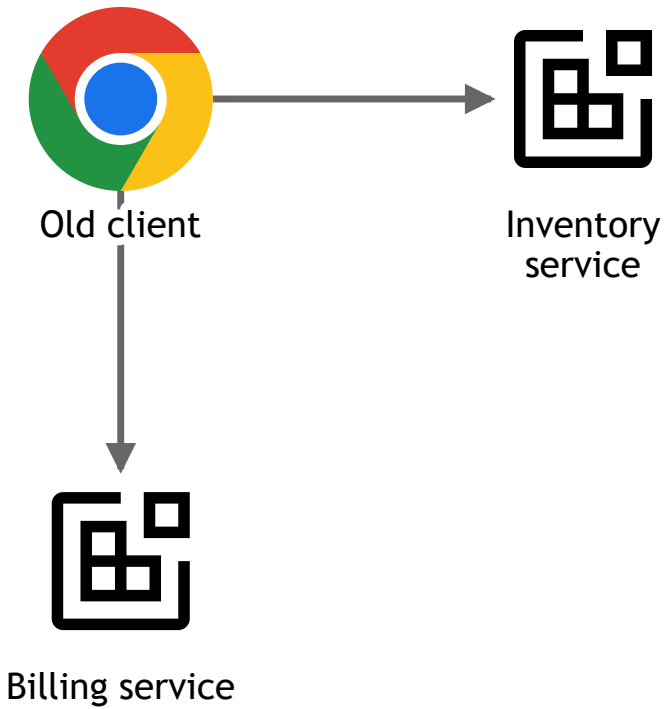
Guard independently deployed contracts

An API schema change breaks an old client, so every provider release has to be proven against the clients that have not migrated.

Several teams deploy services, and old clients still call them

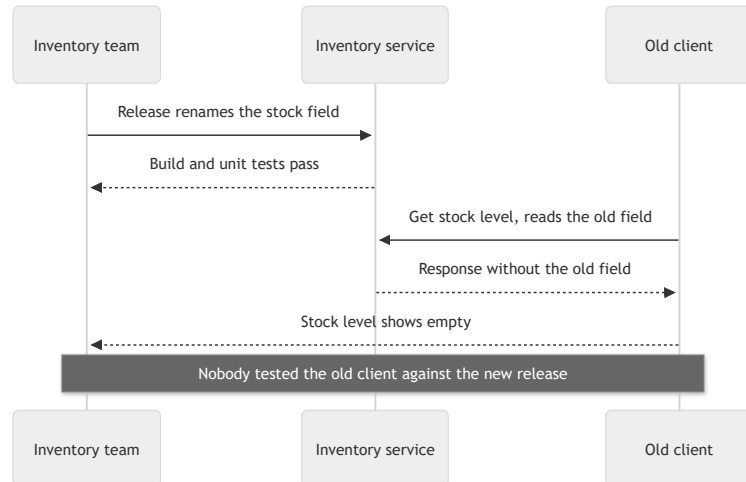
Component	What it is	What it does
Service teams	One team per extracted service	Release on their own schedule, with no shared release train
Provider services	Extracted services such as inventory and billing	Expose APIs and change their schemas
Old clients	Legacy screens and callers that have not migrated	Read fields by name and cannot release with a provider
Team pipelines	One automated build and deploy path per service	Test and ship a service, with no check against its consumers
Feature flags	Runtime switches for single behaviors	Turn one change off without undoing the others

Current architecture



A release that nobody tested against the old client

The inventory team renames a field in its stock response and ships on its own schedule. An old client that has not migrated still reads the old name and shows an empty stock level.



Old client breaks

in production after a provider release renames a field

Several teams

release on their own schedules, with no shared release train

0 old clients broken

by a provider release; the old client is tested against the new provider build before it ships

Each option guards the contract at a different point

Solution	What it is	What it does
Consumer Driven Contract Test	Each consumer publishes the requests and fields it relies on as a contract	Verifies every provider build against those contracts and blocks the release on a mismatch
Versioning + Expand-Contract	Old and new schema versions are served side by side until the last consumer has moved	Adds the new shape beside the old one, migrates consumers, then removes the old shape
Event schema registry	A central store of event schemas that checks compatibility when a producer publishes	Rejects a schema change that would break the registered consumers of an event stream
Coordinated release	The provider and every consumer are released together after a joint test	Avoids a mismatch by making independent deployment wait for the slowest team

Evidence is cheap; overlap and coordination are not

Solution	Gains	Costs
Consumer Driven Contract Test	Evidence from real consumer usage A failure names the broken consumer	Contracts need upkeep: test lifecycle and churn Consumers that publish nothing stay uncovered
Versioning + Expand-Contract	Safe: old clients keep the old shape Covers callers nobody can name	Overlap: duplicated formats until the sunset Old version needs an owner and a sunset date
Event schema registry	Governance: one place for schema rules Checked on every publish	Covers events, not synchronous API calls Registry and its rules need an owner
Coordinated release	No new tooling Simple to explain	Ends independent deployment Old clients cannot release with the provider

Deciding signal: consumers per provider, rate of interface change, and whether each caller can be named and can publish a contract. Known consumers that can publish: contract test. A caller that cannot be named or changed: versioning with overlap. Many event consumers: registry. Any option that releases every consumer together fails the constraint.

Smallest safe move

A Consumer Driven Contract Test on one provider

- **Scope:** code and evidence; the inventory API and one old client; schema, traffic, data and writes unchanged
- **Change:** record the old client's real calls as a contract, replay it against every provider build before release
- **Safety net:** a failing contract blocks the release and names the consumer; no Coordinated release; needed changes ship as an additive change first
- **Escape route:** redeploy the previous provider build; the contract stage is only a test
- **Deferred:** Versioning + Expand-Contract when a breaking change cannot be absorbed or a caller cannot publish; Event schema registry when events are the changing contract; Coordinated release never returns

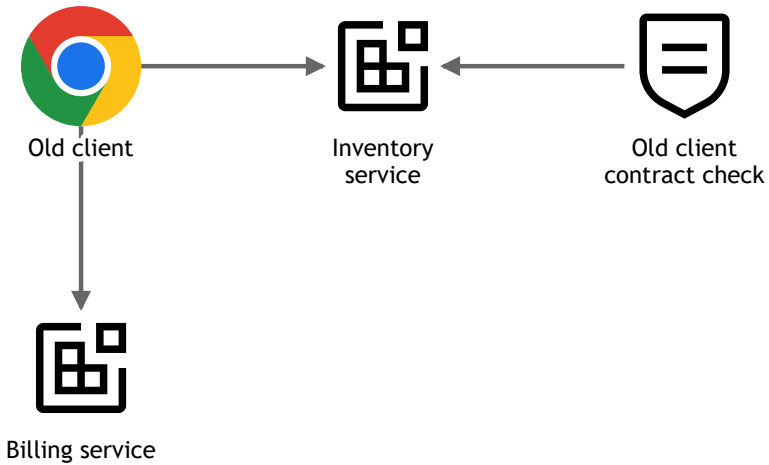
Verification: old client against the new provider build

Check	Pass condition
Old client contract replayed against the new provider build	Every recorded request returns the expected fields, types and status codes
Provider build with a field renamed on purpose	The pipeline fails before release and names the old client
Additive change: a new optional field added	The contract still passes and the old client is unchanged
Provider released in a window with no consumer release	It deploys alone and the old client keeps working in production
Fitness function	Contract verification is a required stage of the provider pipeline; a failing or missing contract blocks the release; an alert fires when a contract has no owner or was not verified against the latest build

Rollback: routing rollback: redeploy the previous provider build or switch the route back by flag; the old client never saw another shape. No state rollback is needed because the move adds a test stage and changes no data; a later expand step drops nothing, and the contract phase starts only after zero reads.

Stop: when every provider release passes the old clients' contracts and no schema change breaks a client in production, no versioning scheme, registry or release train is added.

Solution architecture



The old client and both services stay as they are. A contract check built from the old client's calls now runs against every inventory build before it ships.

Glossary

Used in this step

Term	Definition
Consumer Driven Contract Test	Consumer expectations verified against a provider
Versioning + Expand-Contract	Serve old and new versions side by side, then remove the old one
Event schema registry	Central store of event schemas that checks compatibility when a producer publishes
Coordinated release	Provider and every consumer released together after a joint test
Contract	A consumer's recorded expectations of a provider: paths, fields, types and status codes
Provider	The service whose API other clients call
Consumer	A client or service that calls a provider's API
Additive change	A new optional field or endpoint that leaves every existing one untouched
Sunset	Agreed date and owner for removing an old field, version or view
Fitness function	Measurable or executable architectural quality rule

Related concepts

Term	Definition
Expand-Contract	Add compatible representation, migrate consumers, then remove old representation
Database View	Named query presenting an abstraction over underlying tables
Routing Façade	Stable public entrypoint routing selected capabilities to old or new system
Feature Flag	Runtime control enabling behavior without deploying new code
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations

Phase 6

Infrastructure change and retirement

Steps in this phase

Step	Forcing constraint
M43 Rehost legacy runtime	Hosting contract expires before safe rewrite
M44 Move unsupported runtime incrementally	Security policy sets fixed end-of-support deadline
M45 Modernize legacy database hosting	Availability target exceeds old hardware
M46 Bridge legacy and modern identity	One SSO login across UI generations
M47 Unify audit across old and new	Financial edits require cross-system traceability
M48 Reduce cross-environment latency	Synchronous cross-environment calls exceed p95 and cost
M49 Decommission remaining legacy	License/hosting cost dominates value
M50 Stop unnecessary modernization	Further decomposition does not improve business outcomes

M43 · Infrastructure change and retirement

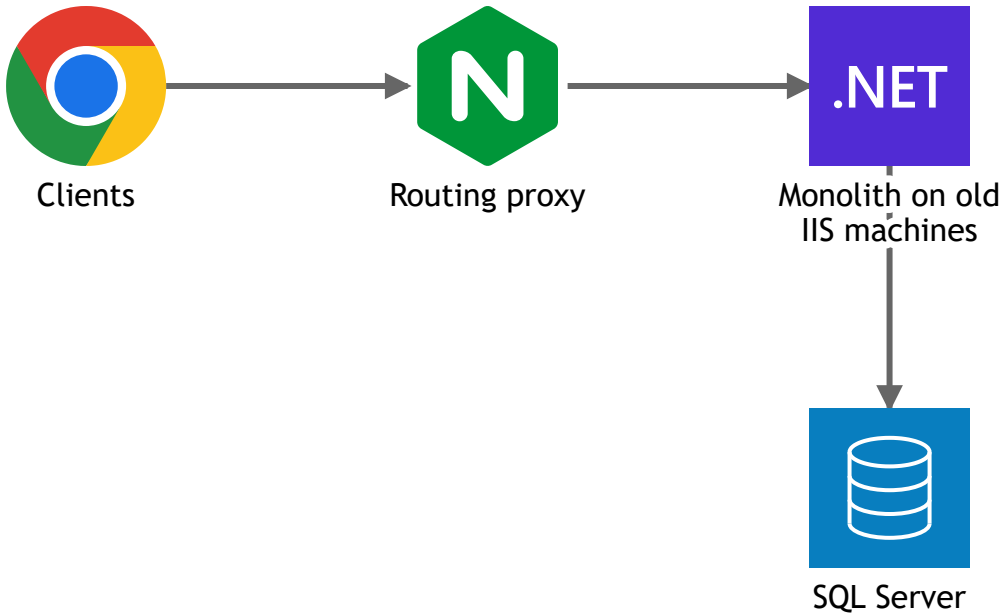
Rehost legacy runtime

The hosting contract for the IIS machines expires before any rewrite can be proven safe, so the same release has to run on supported hosting first.

The IIS machines are losing data-center support

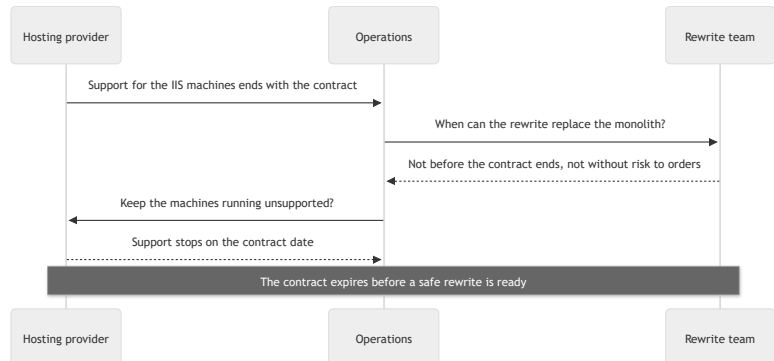
Component	What it is	What it does
Legacy monolith	12-year-old .NET Framework application	Serves the UI and APIs; runs every module
IIS machines	The Microsoft web servers that host the monolith	Run the application; lose data-center support when the contract ends
Hosting contract	Agreement that provides the machines and their support	Ends on a fixed date, before a safe rewrite
Routing proxy	Stable public entry point in front of legacy	Sends each route to a chosen target
SQL Server	One shared relational database	Stores the data of every module; stays on its current hardware

Current architecture



A contract date that the rewrite cannot beat

The data-center contract for the IIS machines ends before any rewrite of the monolith can be proven safe. The application has to run on supported hosting by that date without changing what it does.



Support ends

for the IIS machines when the hosting contract ends

Fixed end date

and no rewrite that can be proven safe before it

Same release

and artifacts run on supported hosting with SLOs at the baseline before the contract ends

Each option changes a different amount of the application

Solution	What it is	What it does
Rehost VM	Move the same IIS application to virtual machines on new hosting	Runs the same release and artifacts on new VMs and shifts traffic to them
Replatform managed hosting	Adapt the application to a managed hosting runtime	Hands runtime and operating-system upkeep to the provider and adjusts configuration to fit
Rebuild	Rewrite the application for the new platform	Replaces the monolith with new code before the contract ends

A deadline buys speed with debt, and anything else with time

Solution	Gains	Costs
Rehost VM	Rapid hosting migration No application code change Old machines stay as fallback	Technical debt stays Still VMs to patch and operate Two hosts paid during the overlap
Replatform managed hosting	Less operations work Provider patches the platform	Compatibility work before any traffic moves Configuration and dependency changes Test time the deadline may not allow
Rebuild	Technical debt removed Clean design for the new platform	High risk with a fixed date Hosting move and rewrite combined Undocumented rules must be rediscovered

Deciding signal: time left on the contract against a compatibility run of the same artifacts on managed hosting, plus managed-feature value and behavior risk. A hard deadline or any behavior difference picks Rehost VM; a clean run with time to spare picks Replatform managed hosting. Rebuild never meets a hosting date; the choice changes when the run passes with room before the contract ends.

Smallest safe move

Rehost VM: the same release on new VMs

- **Scope:** infrastructure and traffic; same artifacts on new VMs; code, database, clients and URLs unchanged
- **Change:** deploy the same release, route a small share of traffic by Canary Routing, compare error rate and p95 latency, then move route by route
- **Safety net:** hosting is the only variable; no replatform or Rebuild mixed in; old machines stay until the contract ends; debt noted
- **Escape route:** route traffic back to the old machines through the proxy
- **Deferred:** Replatform managed hosting when a compatibility run passes with time to spare; Rebuild only as business-driven step-by-step replacement

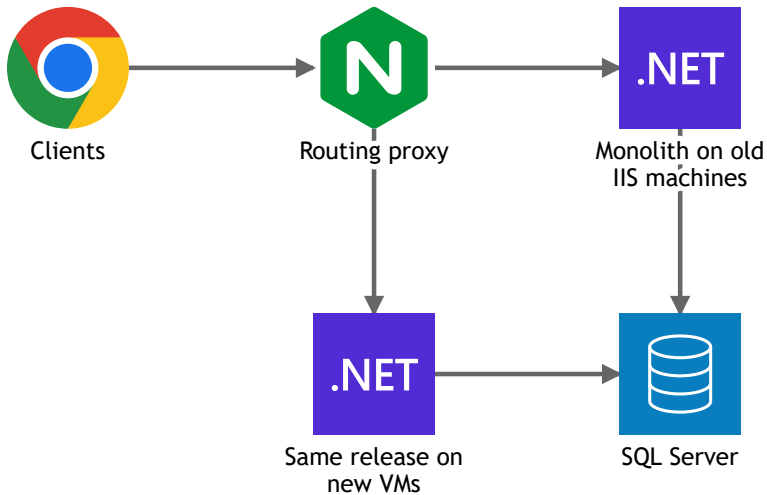
Verification: the same release on new hosting

Check	Pass condition
Artifact deployed to the new VMs	Checksum equals the artifact on the old machines; 0 application code changes
SLOs under a small share of production traffic	Error rate and p95 latency within the baseline recorded on the old machines
Characterization tests and business invariants on the new hosting	Tests pass unchanged and no acknowledged order is missing
Traffic switched back to the old machines	All traffic on the old machines after one proxy change, no failed requests
Fitness function	The pipeline deploys the same artifact to both hosts and runs smoke and characterization tests on each; an SLO alert compares new hosting with the baseline; the old machines' decommission date is tracked against the contract end

Rollback: routing rollback: the proxy returns routes to the old machines while they are still under contract. No state rollback is needed because the database did not move and both hosts write to the same SQL Server, so there is one data authority; in-memory sessions on the new VMs end at rollback and users sign in again.

Stop: when the same release runs on supported hosting with SLOs at the baseline before the contract ends, no replatform or rebuild is added.

Solution architecture



The code, the clients and the database stay as they are. The proxy can now route to the same release on new VMs, and the old machines remain the fallback.

Glossary

Used in this step

Term	Definition
Rehost VM	Move workload to new infrastructure with minimal application changes
Replatform managed hosting	Adapt runtime/hosting without redesigning domain functionality
Rebuild	Rewrite the application for the new platform
Hosting contract	Agreement that provides the machines and their support until a fixed end date
IIS	Microsoft web server that hosts the application
Artifact	The built release package that is deployed to a host
SLO	Target value for a service level, such as error rate or latency
Technical debt	Outdated or shortcut code that makes later change costlier
Canary Routing	Shift a small controlled segment of live traffic to a new implementation
Fitness function	Measurable or executable architectural quality rule

Related concepts

Term	Definition
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations
Retain	Keep working legacy capability unchanged while other parts evolve
Replace with SaaS	Substitute commercial cloud or packaged product for custom capability
Parallel Run	Execute old and new paths side by side and compare without duplicate business effects
Rollback	Return to a previously running version or state

M44 · Infrastructure change and retirement

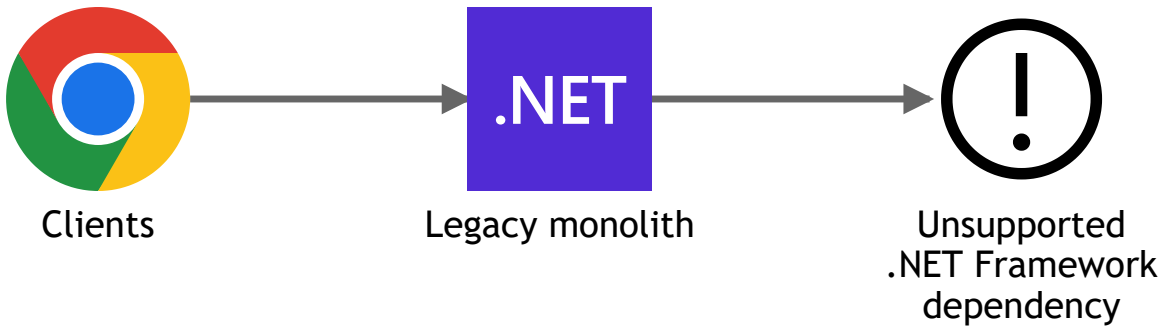
Move unsupported runtime incrementally

A security policy sets a fixed end-of-support deadline for an old .NET Framework dependency, so it has to leave production slice by slice before that date.

The monolith still depends on a runtime that is out of support

Component	What it is	What it does
Legacy monolith	.NET Framework application on supported hosting	Calls the old dependency from several modules
Old dependency	A .NET Framework component that is out of support	Does work the modules rely on, with no more security fixes
Security policy	The compliance rule for what may run in production	Sets one fixed end-of-support deadline
Characterization Testing	Tests that record actual legacy behavior	Show whether a change alters what the application does
Feature flags	Runtime switches for single behaviors	Turn one change off without undoing the others

Current architecture



A fixed deadline meets an all-at-once upgrade

A security policy fixes the date after which the old dependency may no longer run in production. Several modules call it, so one big upgrade changes everything at once while the deadline does not move.

Out of support

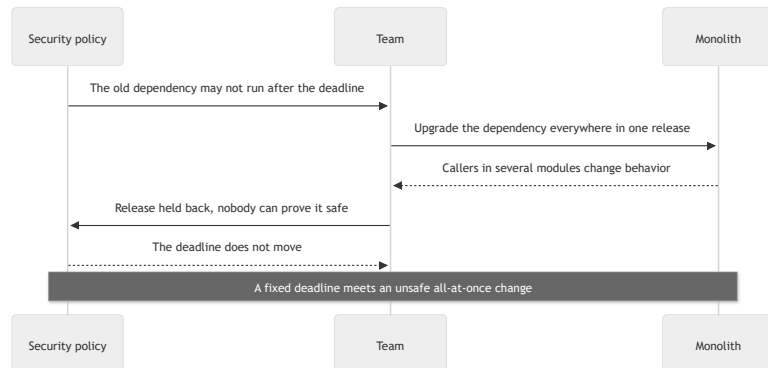
an old .NET Framework dependency that several modules call

One fixed deadline

set by security policy; the deadline does not move

0 unsupported dependencies

in production by the deadline, tracked slice by slice



Each option moves the old runtime out in a different way

Solution	What it is	What it does
Branch by Abstraction	Introduce a stable interface in front of the old dependency, then change the implementation behind it gradually	Moves the callers to the interface first, then swaps the implementation one slice at a time
Extract old component	Move the unsupported component into its own process or service	Runs the old runtime apart from the rest, which calls it over the network
Supported in-place upgrade	Upgrade the dependency to a supported version inside the monolith	Changes the version and fixes the breaking changes in the same process
Big-bang rewrite	Rewrite the affected modules on a supported runtime and release them together	Replaces the dependency and all its callers in one release

Isolation costs mixed runtimes; the shortcuts cost certainty

Solution	Gains	Costs
Branch by Abstraction	Isolates the old code from its callers One slice at a time Old implementation stays as fallback	Mixed runtimes until the last slice Temporary indirection
Extract old component	Runtime separation: the rest can move on Old component is fenced in	Network hop and new failure modes Component stays unsupported until replaced
Supported in-place upgrade	Easier if compatible No new boundary and no network hop	Breaking changes hit every caller at once Works only if a supported version exists
Big-bang rewrite	Clean supported code Nothing old left in those modules	Cutover risk lands on the deadline Undocumented behavior is lost

Deciding signal: can a seam be introduced, behavior preserved and a narrow slice moved, against the time left? Per dependency: a supported version that passes the tests picks the in-place upgrade; otherwise Branch by Abstraction, and Extract old component when it cannot share a process. The choice changes when the projected finish passes the deadline.

Smallest safe move

Branch by Abstraction on the first slice

- **Scope:** code and evidence; one dependency and its callers in one module; traffic, data and writes unchanged
- **Change:** pick one slice, put an interface over it, move the callers to it, put the supported implementation behind it, switch by feature flag
- **Safety net:** characterization tests pass on both implementations; the old one stays as fallback; no Big-bang rewrite; the count is published each release
- **Escape route:** flip the flag back to the old implementation
- **Deferred:** Supported in-place upgrade where a supported version passes the tests; Extract old component when it cannot share a process; Big-bang rewrite never returns

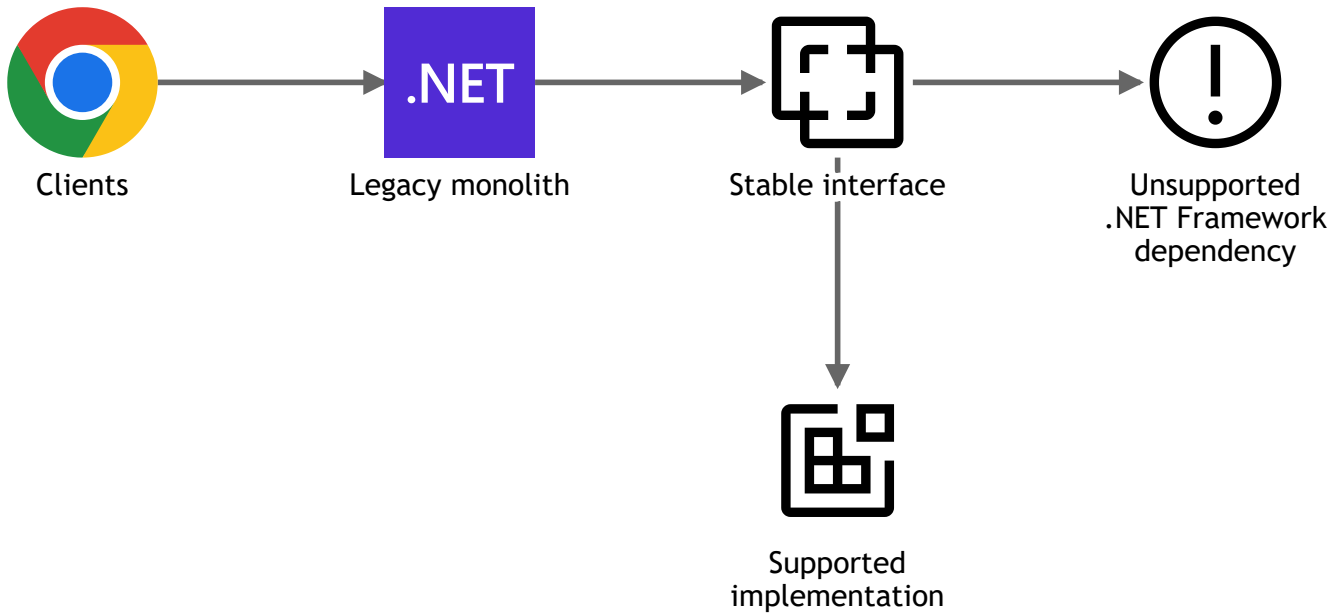
Verification: track dependencies slice by slice

Check	Pass condition
Inventory of unsupported dependencies	Every one is listed with an owner and a slice; the count is the baseline
Characterization tests on the old and the new implementation of the slice	Identical results on the same recorded inputs
Slice switched to the supported implementation by flag	Behavior and SLOs at the baseline; the unsupported count drops by 1
Flag switched back	The old implementation serves again after one change, with no failed requests
Fitness function	The pipeline fails when a new unsupported dependency appears; a scan publishes the burn-down each release; an alert fires when the projected finish passes the deadline

Rollback: routing rollback: the feature flag returns the slice to the old implementation at once. No state rollback is needed while the slice is stateless; where a slice stores data, the old implementation keeps writing the old format until the old code is removed, and a bad new write is repaired forward from the stored source.

Stop: when the count of unsupported dependencies reaches 0 before the deadline, no further extraction, rewrite or abstraction layer is added, and the interfaces that served their slice are removed.

Solution architecture



Callers now meet a stable interface. The flag decides per slice whether the old dependency or the supported implementation answers.

Glossary

Used in this step

Term	Definition
Branch by Abstraction	Introduce stable interface then change the implementation behind it gradually
Extract old component	Move the unsupported component into its own process and call it over the network
Supported in-place upgrade	Move the dependency to a supported version inside the same process
Big-bang rewrite	Replace a whole capability in one release instead of slice by slice
End-of-support deadline	Unsupported legacy runtime cannot be hosted after compliance date
Unsupported dependency	A library or runtime version that no longer receives security fixes
Mixed runtimes	Old and supported runtimes serving production at the same time
Slice	One dependency and its callers that move as a unit
Characterization Testing	Tests documenting actual legacy behavior before refactoring
Feature Flag	Runtime control enabling behavior without deploying new code
Fitness function	Measurable or executable architectural quality rule

Related concepts

Term	Definition
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations
Refactor in Place	Improve internal structure without changing external behavior
Rehost	Move workload to new infrastructure with minimal application changes
Replatform	Adapt runtime/hosting without redesigning domain functionality
Parallel Run	Execute old and new paths side by side and compare without duplicate business effects

M45 · Infrastructure change and retirement

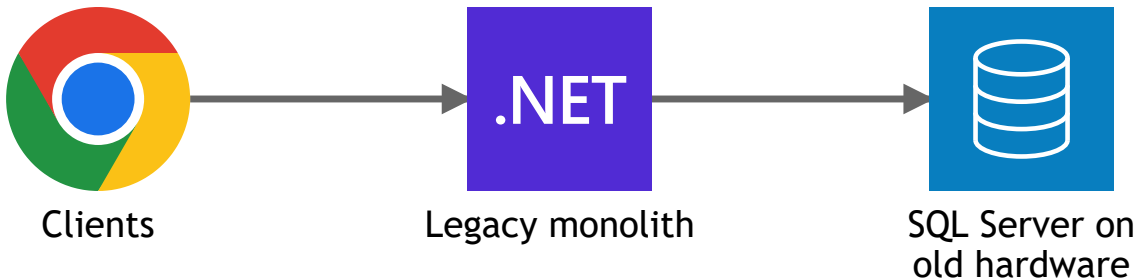
Modernize legacy database hosting

The availability target is higher than the unsupported on-prem SQL Server hardware can deliver, so failover has to be rehearsed and measured before authority moves.

The shared database runs on hardware that is out of support

Component	What it is	What it does
Legacy monolith	12-year-old .NET Framework application	Reads and writes one shared database for every module
SQL Server	One shared relational database on-prem	Stores the data of every module and is the one write authority
Old hardware	The on-prem servers that host SQL Server	Are unsupported and cap the availability that can be promised
Availability target	The agreed level of service	Is set above what the old hardware can deliver

Current architecture



A target the old hardware cannot reach

The shared SQL Server runs on unsupported hardware, and the business now expects more availability than that hardware can give. After a failure the restore from backup alone takes longer than the target allows.

Unsupported

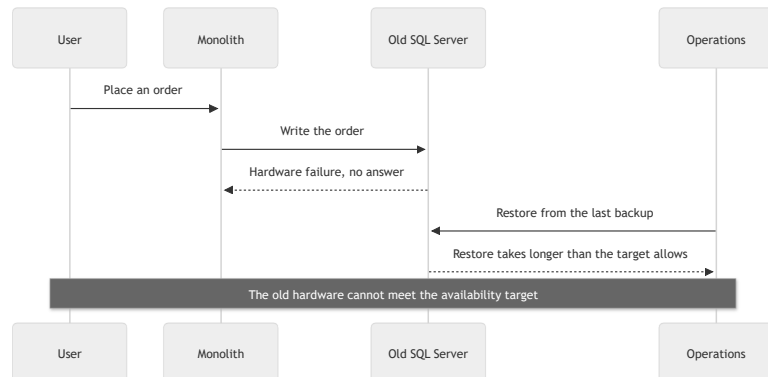
on-prem SQL Server hardware that cannot meet the availability target

One database

written by every module, so an outage stops the whole monolith

Measured RTO and RPO

meet the target, for example RTO ≤ 5 min and RPO ≤ 30 s (illustrative)



Each option changes how much of the database stack moves

Solution	What it is	What it does
Managed SQL Server + replication	Run the same SQL Server engine as a managed service and replicate the legacy database to it	Keeps a continuously updated managed copy, rehearses failover and moves the writer only after proof
Change engine	Move the data to a different database engine	Converts schema, queries and procedures to the new engine and migrates the data
Retain old hardware	Keep the database on the unsupported servers with spare parts and backups	Leaves the database where it is and answers a failure with a restore from backup

Familiar engines cost money, a new engine costs conversion

Solution	Gains	Costs
Managed SQL Server + replication	Familiar engine and SQL Failover can be rehearsed Less hardware to run	Managed service and replication costs Replication lag sets the RPO Cutover needs a write freeze
Change engine	Strategic platform choice Not tied to SQL Server	SQL conversion of schema and procedures Behavior differences to retest Failback is harder across engines
Retain old hardware	No data movement No cutover risk	Availability stays capped by old hardware Recovery means a restore from backup Unsupported hardware gets harder to repair

Deciding signal: measured RPO and RTO of a rehearsed failover against the target, SQL compatibility and cost. A managed replica that meets both in rehearsal picks Managed SQL Server + replication; a service that cannot, or costs too much, with affordable SQL conversion picks Change engine. Retain old hardware fails the constraint.

Smallest safe move

Managed SQL Server + replication, proven before cutover

- **Scope:** infrastructure and data; one managed replica; code, schema and clients unchanged; write authority moves once, after proof
- **Change:** replicate continuously, rehearse failover on a copy, then Write Freeze at a cutoff, reconcile and switch the connection setting
- **Safety net:** Write Fencing makes the old server read-only before the managed one accepts writes; zero drift required; Retain old hardware is only the fallback
- **Escape route:** before the first managed write, unfreeze and point back; after it, Reverse Replication, or a forward fix
- **Deferred:** Change engine when the managed service misses RPO and RTO or its cost dominates

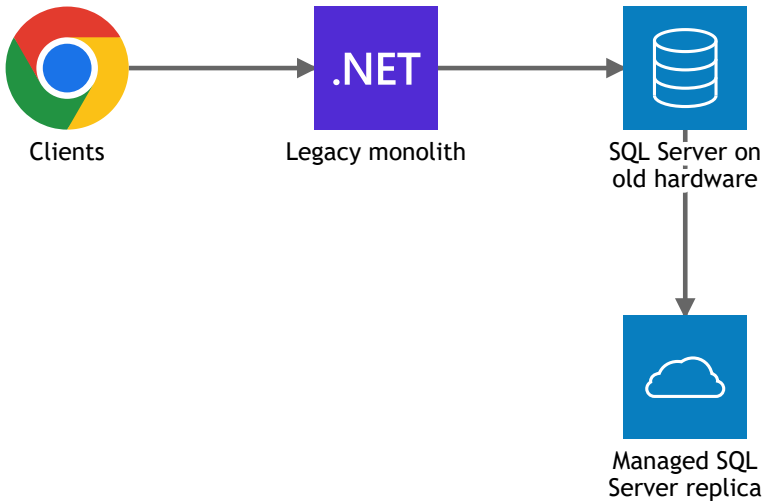
Verification: replicate, rehearse, then cut over

Check	Pass condition
Replication lag under production write load	Lag within the RPO target, for example RPO ≤ 30 s, over a full business cycle
Recovery Drill: failover on the managed copy	The application serves again within the RTO target, for example RTO ≤ 5 min, with no acknowledged order missing
Data Reconciliation at the cutoff	Row counts, checksums and order totals match; zero unexplained drift
Writes after the Data Cutover	Only the managed server accepts writes; the old server rejects them
Fitness function	A scheduled drill records RTO and RPO and fails when either target is exceeded; a lag alert fires before the RPO would be missed; nightly reconciliation runs until the old server is retired

Rollback: routing rollback is safe only before the first write on the managed server: unfreeze the old server and point the application back. After that write, switching back would drop it, so the problem becomes a state rollback: reverse replication into the old server plus reconciliation, or a forward fix, both planned before the cutover.

Stop: when the rehearsed failover on managed SQL Server with replication meets the availability target, no change of engine and no further data layer is added.

Solution architecture



The application and the on-prem server stay as they are. The managed replica receives every change, so failover can be rehearsed and measured before any write authority moves.

Glossary

Used in this step

Term	Definition
Managed SQL Server + replication	Same SQL Server engine run as a managed service and kept current by replication
Change engine	Move the data to a different database engine, which means converting the SQL
Retain old hardware	Keep the database on the unsupported servers with spare parts and backups
RPO	Maximum targeted period of data loss
RTO	Maximum targeted recovery duration after disruption
Failover	Switches operations to standby resources after failure.
Recovery Drill	Exercise failover/restoration to measure actual RTO/RPO
Write Freeze	Temporarily reject/queue writes to allow consistent transition
Data Cutover	Point at which authority for selected reads or writes changes to new system
Write Fencing	Prevent stale writers from modifying state after ownership changes
Reverse Replication	Propagate authoritative new-state changes back to old consumers temporarily
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence

Related concepts

Term	Definition
Active-Passive	One deployment processes traffic while another waits
Synchronous Replication	Acknowledge a commit only after required replicas persist it
Asynchronous Replication	Acknowledge commit before all replicas confirm persistence
Hot Standby	Fully ready fallback environment
Forward Fix	Repair the new implementation without restoring the old system

M46 · Infrastructure change and retirement

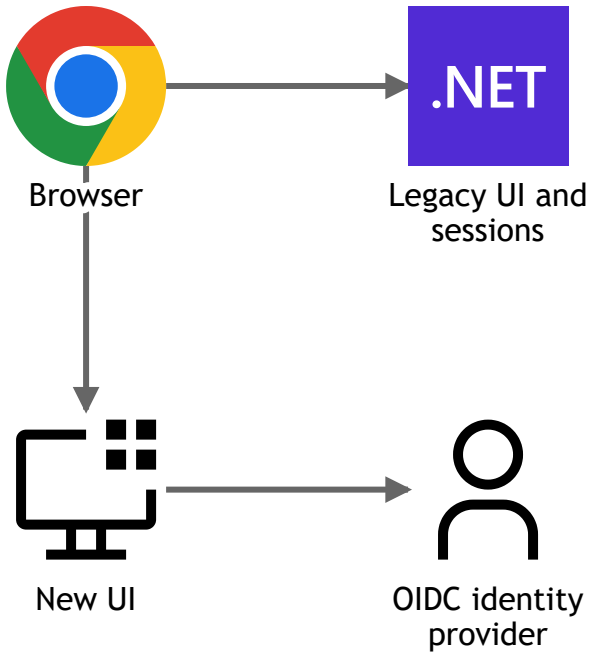
Bridge legacy and modern identity

One SSO login has to work across both UI generations, although legacy sessions and the new OIDC identity differ.

Two UI generations keep two different notions of a signed-in user

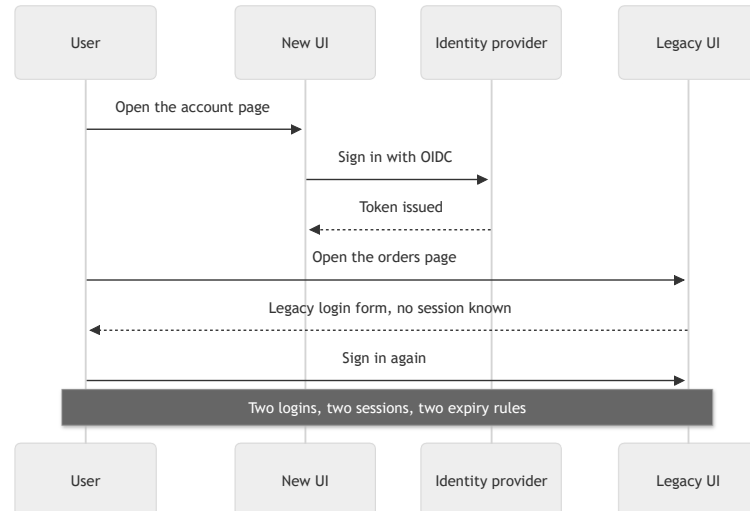
Component	What it is	What it does
Legacy UI	Server-rendered .NET pages	Signs users in with its own login form
Legacy session	Server-side login state tracked by a session cookie	Holds the user and roles until it expires
New UI	New account and catalog routes	Signs users in through the OIDC identity provider and holds its tokens
Identity provider	The OIDC service that authenticates users	Issues identity tokens to the new UI
Feature flags	Runtime switches for single behaviors	Turn one change off without undoing the others

Current architecture



A second login at every seam between the UIs

The new UI signs users in through an OIDC identity provider, while the legacy UI keeps its own login and session. A user who moves between them meets a second login, and nothing makes expiry, logout or a removed role reach both.



Second login

a user signed in through OIDC must sign in again on a legacy page

Two session models

legacy sessions and OIDC tokens, with different expiry and logout rules

1 SSO login

covers both UIs, and expiry, logout and privilege changes agree

Each option decides where the two identity models meet

Solution	What it is	What it does
Identity façade / token bridge	A translating layer in front of the legacy UI that accepts the OIDC identity and creates the legacy session	Exchanges an OIDC token for a legacy session and a legacy session for a token, so one login serves both UIs
Add OIDC to legacy	Change the legacy application so that it signs users in through the OIDC provider itself	Replaces the legacy login with OpenID Connect so both UIs trust the same tokens
Shared session cookie	One session cookie that both UI generations read	Lets the new UI trust the legacy session instead of an OIDC token

A bridge costs translation, a legacy change costs regressions

Solution	Gains	Costs
Identity façade / token bridge	Incremental: one route or cohort at a time Legacy login code stays untouched	Translation of claims, roles and expiry Another component in the login path Bridge needs an owner and a sunset
Add OIDC to legacy	Less bridge to run One identity model in both UIs	Old code changes in the login path Regression risk in sessions and roles
Shared session cookie	Cheap: no translation layer No change to the identity provider	Tied to the legacy session format New services still need OIDC tokens Expiry and logout follow legacy rules

Deciding signal: trust boundaries, the legacy roles and session rules to translate, and how safely the login code can change. A small translation and untested login code pick the bridge; a growing translation table or login code rebuilt anyway picks OIDC in legacy. A shared cookie fails at the first service that needs a token.

Smallest safe move

Identity façade / token bridge for one route and cohort

- **Scope:** traffic and code; one legacy route and one cohort (for example staff accounts); legacy login and user data unchanged
- **Change:** the bridge maps OIDC claims to a legacy session and, by token exchange, gives the new UI a token; feature flag per cohort
- **Safety net:** legacy login stays as fallback; fails closed; no Shared session cookie; owner and sunset keep the compatibility layer from becoming debt
- **Escape route:** flag the cohort back to the legacy login
- **Deferred:** Add OIDC to legacy when translation keeps growing or the login code is rebuilt anyway

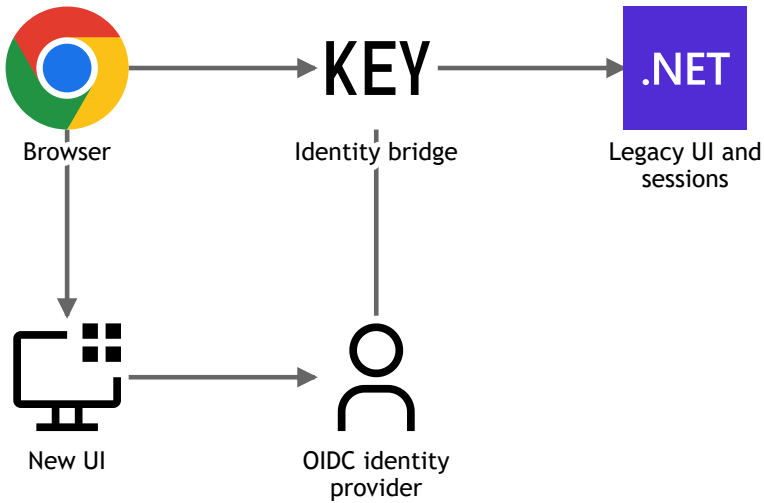
Verification: expiry, logout and privileges in both UIs

Check	Pass condition
One login, both UIs: sign in once, open a legacy route and a new UI route	No second login prompt on either UI
Session expiry	Both UIs reject the expired session at the same time
Logout in one UI	The session in the other UI ends too and the back button restores no access
Privilege removed in the identity provider	The legacy route denies the action no later than the next token refresh
Fitness function	A synthetic user runs sign-in, a legacy route, a new route, a role change, logout and expiry on every deploy and on a schedule; any second prompt, stale session or action allowed after logout fails it

Rollback: routing rollback: the cohort flag returns users to the legacy login at once. State rollback: no business data was written; the sessions the bridge created are invalidated and users sign in again; roles changed in the identity provider during the pilot are reconciled with the legacy roles before the cohort moves back, or repaired forward.

Stop: when one login covers both UIs and the expiry, logout and privilege checks pass, the legacy login code is not changed and no further identity layer is added; the bridge is removed with the last legacy route.

Solution architecture



The legacy UI, the new UI and the identity provider stay as they are. The identity bridge in front of the legacy UI turns an OIDC login into a legacy session, so one login serves both.

Glossary

Used in this step

Term	Definition
Identity façade / token bridge	Translating layer in front of legacy that exchanges legacy sessions and OIDC tokens
Add OIDC to legacy	Change the legacy login to sign users in through the OIDC provider itself
Shared session cookie	One session cookie that both UI generations read
SSO	Reuse sign-in for several applications
OIDC	OpenID Connect: identity layer providing authentication on top of OAuth 2.0
Identity provider	Authenticates users and issues identity credentials
Token exchange	Trade one credential, such as a legacy session, for another, such as an OIDC token
Legacy session	Server-side login state of the .NET application, tracked by a session cookie
Claim	A statement about the user, such as a role, carried in a token
Compatibility layer	Code or configuration that keeps old clients working while the system changes
Feature Flag	Runtime control enabling behavior without deploying new code
Fitness function	Measurable or executable architectural quality rule

Related concepts

Term	Definition
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations
Routing Façade	Stable public entrypoint routing selected capabilities to old or new system
Zero Trust	Require verification for access rather than assuming network trust
MFA	Require more than one authentication factor
RBAC	Grants permissions through predefined roles.

M47 · Infrastructure change and retirement

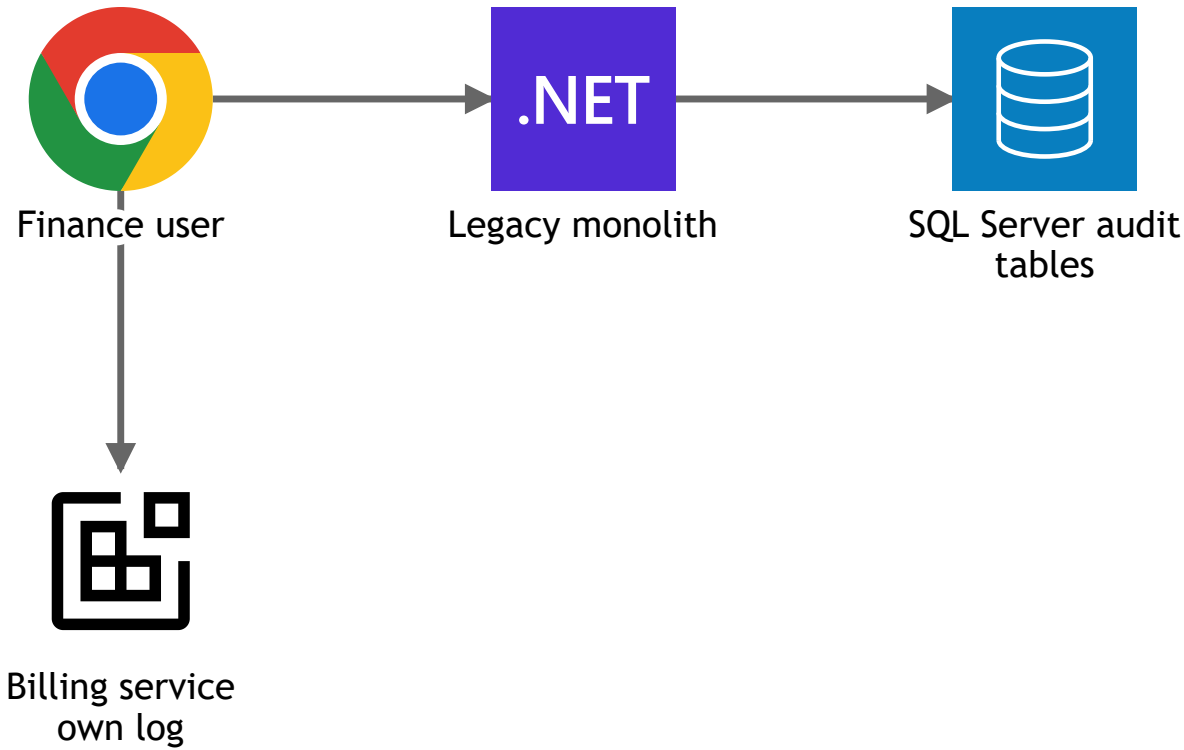
Unify audit across old and new

Financial edits require cross-system traceability: one correlation id must return actor, operation, outcome and time from both systems.

Every system keeps its own audit trail

Component	What it is	What it does
Finance user	Staff member who corrects invoices, prices and discounts	Makes financial edits through the old and the new screens
Legacy monolith	.NET application on IIS that owns the order record	Applies its part of the edit and writes audit rows
SQL Server audit tables	Audit rows inside the shared database	Hold legacy audit records with no link to the new service
Billing service	The new billing capability beside legacy	Applies invoice edits and keeps its own log lines
Correlation ID	Identifier that tags log lines of one request	Exists on the legacy path only, not across both

Current architecture



Nobody can follow one edit across both systems

An invoice correction touches the billing service and the legacy order record. Each side keeps its own record and nothing joins them, so compliance cannot reconstruct a financial edit.

Fragmented

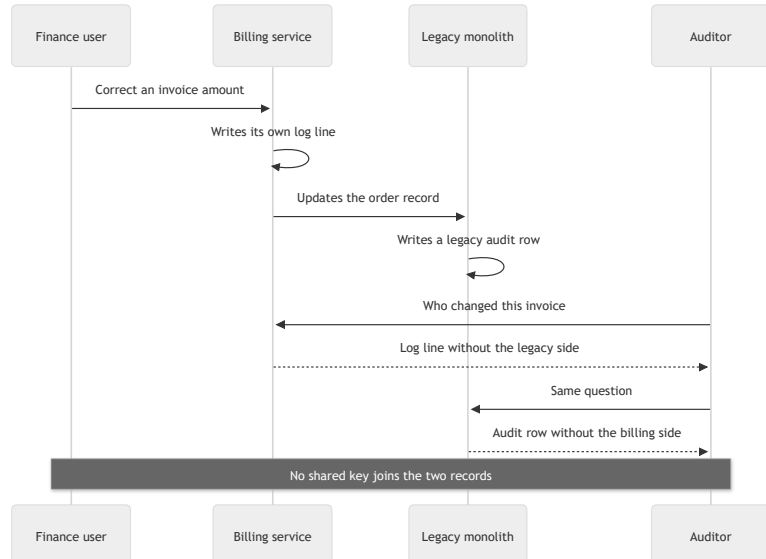
legacy audit rows and billing log lines share no key for one edit

Every edit

that touches both the legacy path and the new path

≥99.99% within 60 s

of completed edits answer one correlation id query from both systems (for example, an illustrative SLO)



Each option decides where the audit record is born

Solution	What it is	What it does
Correlation ID + Immutable Audit Log	One identifier carried through every hop of a financial edit, with each system appending its record to one write-once log	Lets one query by correlation id return actor, operation, outcome and time from both systems
Transactional Outbox	Atomically records business changes and outgoing events in the same database transaction	Writes each audit event with the edit in one transaction; a relay then publishes it to the audit log
Sampled tracing	Distributed tracing that keeps only a sample of the requests across both systems	Shows request paths and timings across legacy and the new service

Evidence costs retention, durability costs a relay

Solution	Gains	Costs
Correlation ID + Immutable Audit Log	One id joins both systems Tamper resistance from immutable storage Evidence for every edit, not a sample	Retention and deletion policy to define Personal data in records needs privacy rules Audit write sits in the edit path
Transactional Outbox	Durable event capture No edit without its event Edit does not wait for the log	Relay to build and operate Records arrive late by the relay lag Legacy schema and write path change
Sampled tracing	Request visibility across both systems Cheap to switch on	Unsampled edits leave no record No integrity or retention guarantee Fails the strict audit invariant

Deciding signal: can the record be appended in the edit path, and how often is the log unavailable? Measure completeness (edits with a queryable record) against ≥99.99% within 60 s (for example, an illustrative SLO), the share of edits rejected because the append failed, and the lag behind the edit. Rare failures pick Correlation ID + Immutable Audit Log; frequent rejections or one committed edit without a record pick Transactional Outbox; a sample fails the invariant at once.

Smallest safe move

Correlation ID + Immutable Audit Log for invoice corrections

- **Scope:** evidence and data for invoice corrections; clients, traffic and write ownership do not change
- **Change:** pass one correlation id to the billing service and the legacy call; each appends one record to an append-only audit log
- **Safety net:** old audit rows stay the record until the log is complete, then the edit must fail closed when its append fails; Sampled tracing is never audit evidence
- **Escape route:** a feature flag returns the append to observe-only; appended records stay
- **Deferred:** Transactional Outbox when appends reject too many edits or a committed edit lacks a record

```
{ "correlationId": "c-7f3a", "system": "billing", "actor": "user-17",
  "operation": "invoice.correct", "outcome": "committed", "at": "2026-11-02T09:14:03Z" }
```

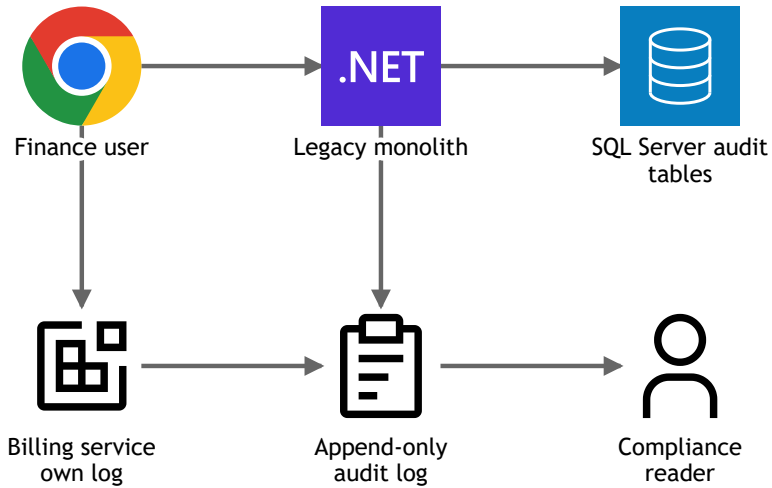
Verification: one edit traced through both systems

Check	Pass condition
One invoice correction through the billing service that updates the legacy order record	One query by its correlation id returns actor, operation, outcome and time from both systems, in order
Completeness over the completed financial edits of the test period	0 committed edits without a record; ≥99.99% of records queryable within 60 s (for example, the illustrative SLO)
Audit log made unavailable on purpose	The edit is rejected, no committed edit lacks a record and an alert fires
Update and delete attempted on a record with the application identity	Both rejected; the record is unchanged
Fitness function	A scheduled job joins committed edits in both systems with the audit log by correlation id and alerts on any edit without a record; a pipeline test fails when one cross-system edit does not return both records

Rollback: routing rollback is not needed, because no traffic moves; the flag returns the append to observe-only. State rollback is not needed either, because records are append-only: a wrong record is corrected by a compensating record, and a gap is repaired forward by backfilling it from the legacy audit rows and billing logs, marked as backfilled.

Stop: when one correlation id reconstructs every sampled financial edit across both systems from an append-only log and no committed edit lacks a record, no outbox, relay or further audit platform is added.

Solution architecture



The legacy monolith and the billing service both append to one audit log that compliance reads by correlation id; the old audit rows and billing logs stay beside it until the log is complete.

Glossary

Used in this step

Term	Definition
Correlation ID + Immutable Audit Log	A correlation id on every hop plus one write-once log of audit records
Transactional Outbox	Atomically records business changes and outgoing events in the same database transaction.
Sampled tracing	Distributed tracing that keeps only a sample of requests, so it cannot prove every edit
Correlation ID	Identifier carried across related operations
Audit Log	Recorded business/security action with actor, time and outcome
Immutable Storage	Data retention mechanisms resisting unauthorized modifications
Strict audit	Must reconstruct actor, operation, outcome and time with integrity
Financial edit	A change to an invoice, price or discount that affects money
Append-only	Records can be added but never changed or deleted
Fail closed	Reject the edit when its audit record cannot be written
Relay	Process that reads outbox rows and publishes them to the log

Related concepts

Term	Definition
Distributed Tracing	Tracks requests across multiple services and dependencies.
Event Sourcing	Persists state changes as an append-only sequence of events.
Change Data Capture (CDC)	Detects and propagates changes made to database records.
Dual Write	Attempt to update two independent stores from one business operation
Feature Flag	Runtime control enabling behavior without deploying new code

M48 · Infrastructure change and retirement

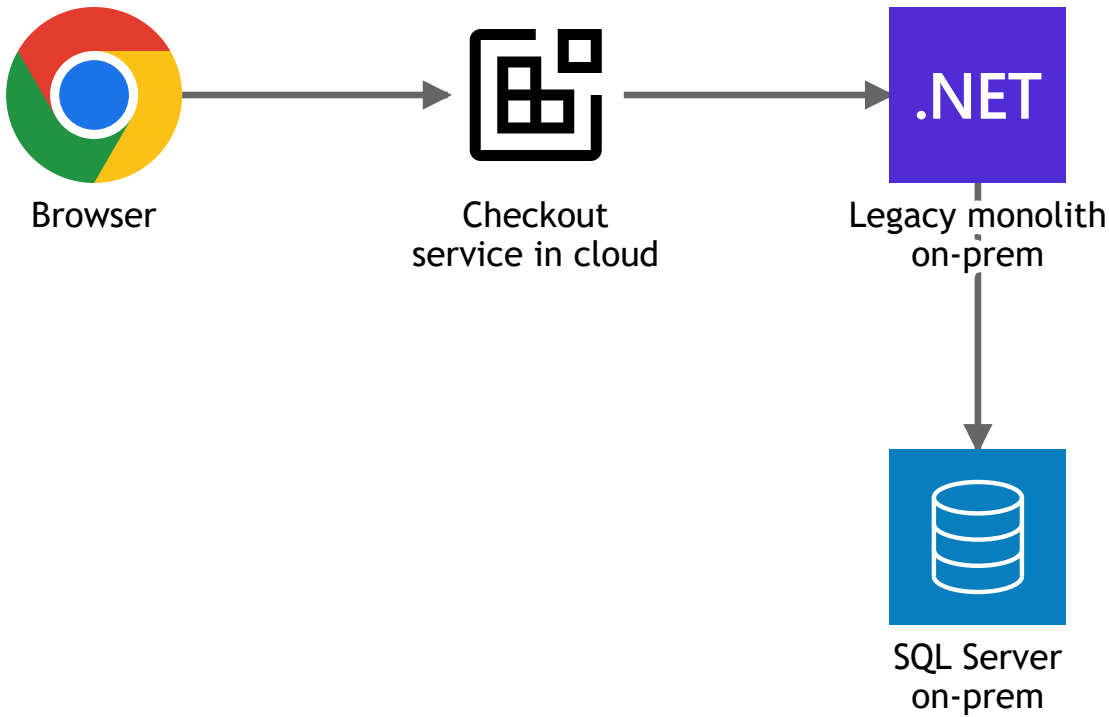
Reduce cross-environment latency

Synchronous cross-environment calls between the cloud services and the on-prem legacy system exceed p95 and cost.

The old system stays on-prem while new services run in the cloud

Component	What it is	What it does
Browser	The customer’s web client	Sends every request to the checkout service
Checkout service	New service running in the cloud	Handles checkout and calls legacy for data it does not own
Legacy monolith	.NET application on IIS in the on-prem site	Serves the modules that are not extracted yet
SQL Server	Shared database in the on-prem site	Holds the legacy data behind those modules
Cross-environment link	The network path between on-prem and cloud	Carries every synchronous call and bills it as egress

Current architecture



Every request pays for the trip across the boundary

The checkout service runs in the cloud while legacy and its database stay on-prem. Each customer request makes several synchronous calls across the boundary, so it waits for every round trip and the traffic is billed.

p95 exceeded

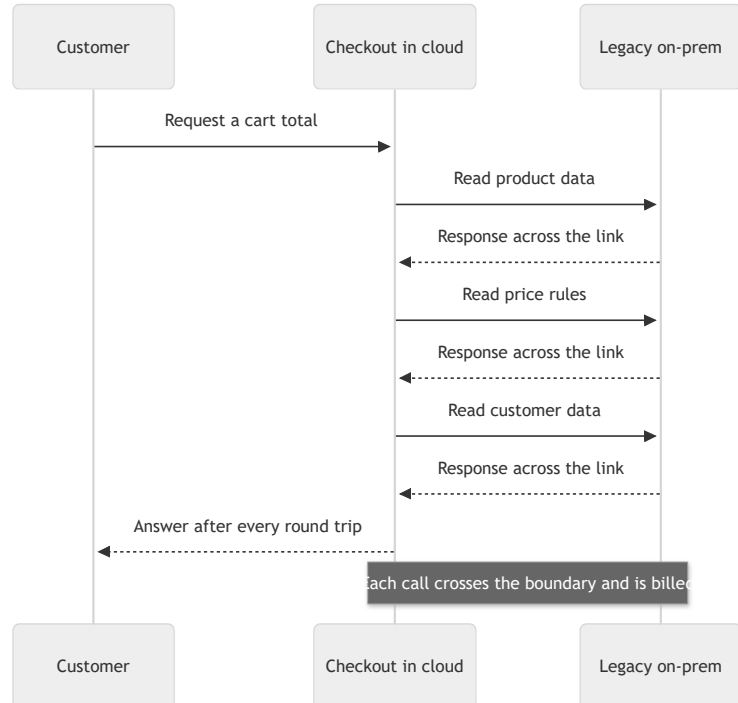
each request waits for several synchronous round trips between cloud and on-prem

Billed as egress

cross-environment calls on every request

p95 within target

and cross-environment cost down, measured per call pair against the baseline of each migration phase



Each option decides where the data and the caller live

Solution	What it is	What it does
Move coupled workload pair	Relocate the two components that call each other most so that both run in the same environment	Turns the chatty cross-environment calls into local calls
Private connectivity	A dedicated private network link between the on-prem site and the cloud	Gives the calls a steadier path without changing any code
Local projection	A read-optimized copy of the on-prem data kept next to the cloud service and built from its change events	Answers reads locally and receives changes asynchronously from legacy
Asynchronous messaging	Calls whose result is not needed at once are sent through a message queue	Lets the caller continue without waiting for the other environment

Locality is fast, a link is steady, and a copy is stale

Solution	Gains	Costs
Move coupled workload pair	Local calls, no round trip across the link Cuts egress for that pair	Migration effort and cutover risk Shared data has to move too
Private connectivity	Minimal code change More predictable latency	Recurring circuit cost Distance and round trips remain
Local projection	Fast independent reads No cross-environment call on reads	Data can be stale Lag and state synchronization complexity
Asynchronous messaging	Caller no longer waits on the link Absorbs on-prem outages	Result is not immediate Queue and idempotent consumers to run

Deciding signal: calls per request (fan-out), p95 per call, bytes billed per request, the maximum data age the business accepts and whether the caller needs the result at once. High fan-out of reads with tolerable data age picks Local projection; a chatty pair that cannot accept stale data picks Move coupled workload pair; p95 dominated by path jitter or bandwidth picks Private connectivity; only an acknowledgement needed picks Asynchronous messaging. The choice changes when replication lag exceeds the accepted data age.

Smallest safe move

Local projection for the one read that dominates p95

- **Scope:** data and traffic for one read-only call; legacy code, writes and other calls do not change
- **Change:** measure each call pair, build the projection from change events, then route that read by flag
- **Safety net:** only reads that tolerate the accepted data age use the copy; a lag alert guards it; synchronization stays limited to one projection
- **Escape route:** flip the flag; the synchronous call was never removed
- **Deferred:** Private connectivity when path jitter or bandwidth dominates p95; Move coupled workload pair when a pair stays chatty; Asynchronous messaging when only an acknowledgement is needed

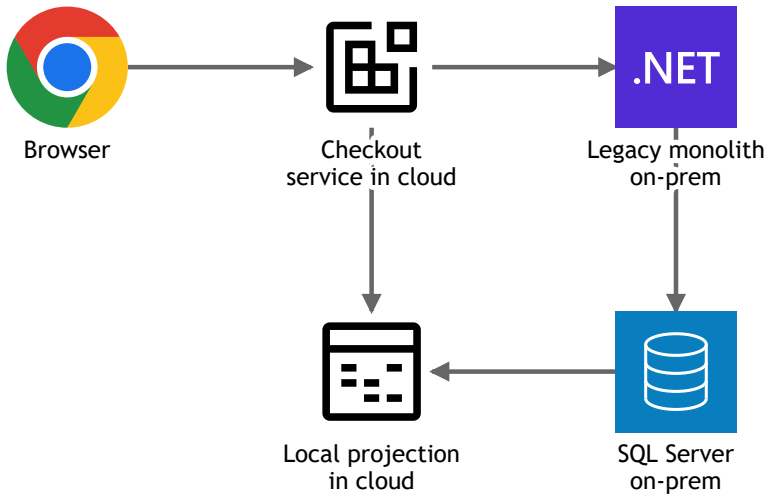
Verification: latency and egress per migration phase

Check	Pass condition
Baseline per cross-environment call pair: calls per request, p95 per call, bytes and egress cost	Recorded for every pair before any change
Projected read after the change	Request p95 back within target; 0 cross-environment calls for that read; its egress falls to the change stream
Replication lag of the projection against the accepted data age	Lag stays within the age the business accepts; an alert fires when it does not
Projection made stale or unavailable on purpose	Reads fall back to the synchronous call, no request fails and no acknowledged order is lost
Fitness function	A synthetic checkout request is timed in every migration phase and fails the pipeline when p95 or egress per request exceeds the recorded target; a lag alert guards data age

Rollback: routing rollback: the flag returns the read to the synchronous call at once, because that path was kept. State rollback is not needed, because the projection is a derived copy and nothing authoritative moved; a damaged or stale projection is rebuilt from legacy by replaying the change stream.

Stop: when request p95 and cross-environment cost are within target after the one projection, no private circuit, workload move or message queue is added.

Solution architecture



The checkout service now reads the one hot call from a projection in the cloud, fed by change events from SQL Server; the synchronous call to legacy stays for writes and fresh reads.

Glossary

Used in this step

Term	Definition
Move coupled workload pair	Relocate the two components that call each other most to the same environment
Private connectivity	A dedicated private network link between the on-prem site and the cloud
Local projection	Read-optimized representation built from updates or events, kept next to the caller
Asynchronous messaging	The caller sends a message and continues; a queue buffers it for the consumer
Cross-environment call	A call between the on-prem system and the cloud services
p95	The response time that 95 percent of requests stay under; shows the slow tail
Egress	Data that leaves an environment, usually billed by volume
Fan-out	Number of downstream calls that one request triggers
Data age	How old a copy may be before the business cannot accept it
Replication lag	Delay between a change in legacy and its arrival in the projection

Related concepts

Term	Definition
Materialized Query Projection	Read-optimized representation built from updates or events
Message Queue	Buffers messages for asynchronous processing by consumers.
Asynchronous Replication	Acknowledge commit before all replicas confirm persistence
Change Data Capture (CDC)	Detects and propagates changes made to database records.
Feature Flag	Runtime control enabling behavior without deploying new code

M49 · Infrastructure change and retirement

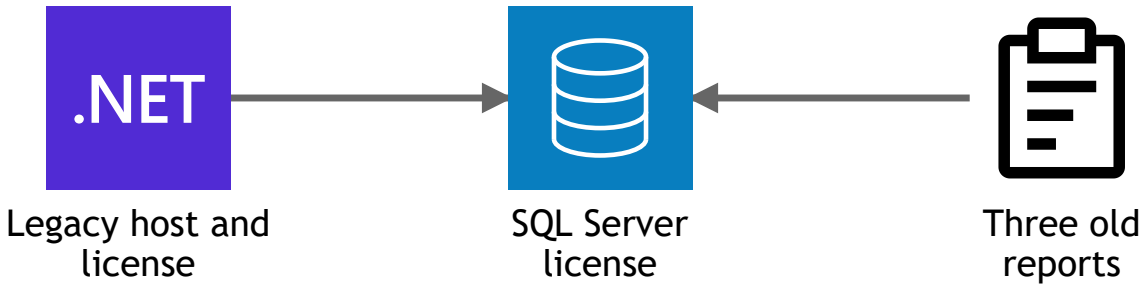
Decommission remaining legacy

Only three old reports still query legacy, and its license and hosting cost now dominates the value it gives.

A few old reports keep the whole legacy system alive

Component	What it is	What it does
Old reports	Three reports that business users still run	Query the legacy database directly
Legacy host	.NET application on IIS that serves almost nothing else	Is still licensed and hosted as a whole
SQL Server	The legacy database	Holds the data the reports read and the history that must be retained
License and hosting	Recurring cost of keeping legacy running	Dominates the value of the few reports
Retention requirements	Compliance rules for invoices and the audit trail	Keep historical records readable for a defined period

Current architecture



Cost says retire, but nobody can prove it is safe

Only three old reports still query legacy, yet the license and hosting of the whole system cost more than they are worth. Switching it off is tempting, but a reader nobody listed would break silently and retained records could be lost.

License and hosting dominate

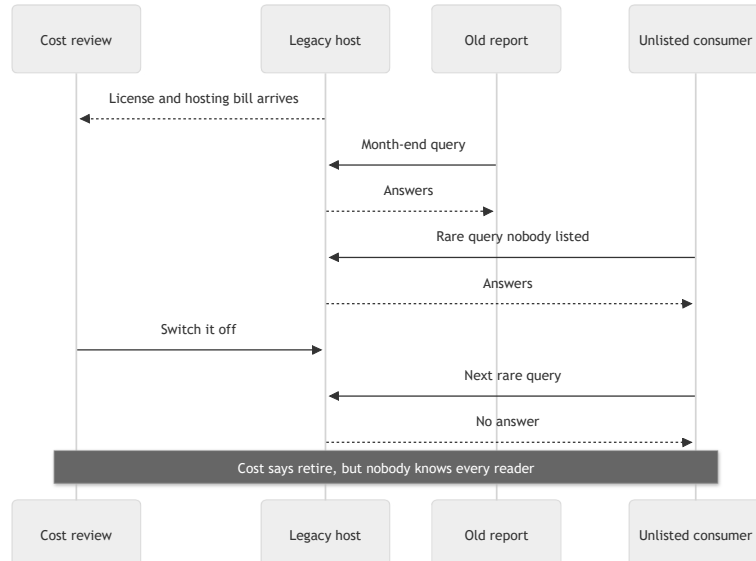
legacy runs for a few old reports and its cost exceeds their value

Three old reports

still query legacy, and nobody can list every reader with certainty

0 mandatory live paths

to legacy, and archived report access tested



Each option decides what happens to legacy and its data

Solution	What it is	What it does
Consumer inventory + read-only archive + retire	List every remaining consumer, move the data that must be kept into a read-only archive, then switch legacy off	Removes license and hosting cost once no live path and no retention need remain
Keep legacy	Leave legacy running unchanged for the few reports	Serves the reports from the live system and keeps paying for it
Replace with SaaS	Substitute a commercial reporting product for the custom reports	Rebuilds the three reports on hosted software fed with the archived data
Switch off legacy now	Turn legacy off on a cost-review date without listing its consumers	Stops license and hosting cost at once

Retiring cuts cost, but only after proof of zero readers

Solution	Gains	Costs
Consumer inventory + read-only archive + retire	Lowers license and hosting cost Archive keeps retained records readable	Restoring legacy later is complex Dependency discovery risk
Keep legacy	Safe: nothing changes Reports keep working	License and hosting cost continues Debt: old parts and risk stay
Replace with SaaS	Less custom maintenance Reports run on a hosted product	Vendor constraints and lock-in Reports and data still have to move
Switch off legacy now	Fastest cost cut No archive to build	An unlisted reader breaks silently Retained records may be lost

Deciding signal: consumers still calling legacy (traffic and query logs over a full reporting cycle, confirmed with owners), retention needs per dataset and license and hosting cost. 0 live callers with every kept report running from a tested archive picks Consumer inventory + read-only archive + retire; a consumer that cannot be re-pointed keeps Keep legacy; reports that outgrow an archive pick Replace with SaaS; a date without an inventory fails the invariant.

Smallest safe move

Consumer inventory + read-only archive + retire, in that order

- **Scope:** evidence, data and infrastructure for the three reports and legacy itself; nothing else changes
- **Change:** log every caller for a full reporting cycle, archive the data the reports and retention need, re-point the reports, then power legacy off
- **Safety net:** power-off waits for 0 live callers and a passed archive test; Switch off legacy now is never used
- **Escape route:** keep the final backup until retention ends; legacy can be restored
- **Deferred:** Replace with SaaS when reports outgrow the archive; Keep legacy while a consumer cannot be re-pointed

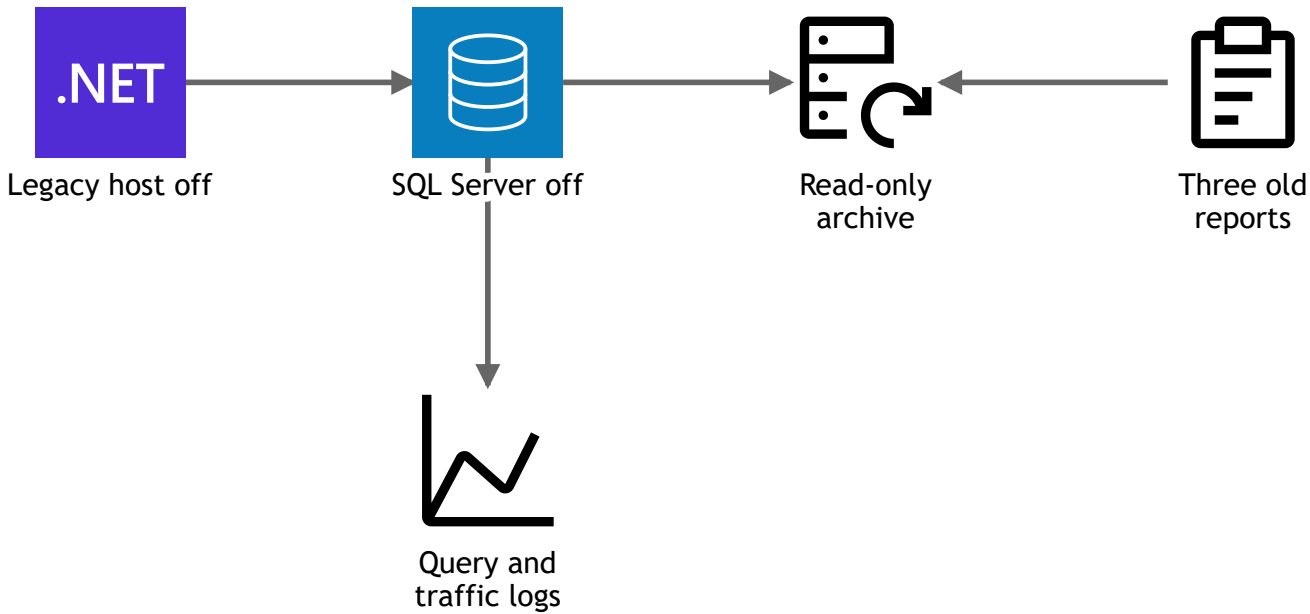
Verification: zero live paths and a working archive

Check	Pass condition
Traffic and query logs over a full reporting cycle, plus owner confirmation	0 mandatory live callers; every remaining caller identified and re-pointed or retired
Each kept report run against the read-only archive and compared with legacy	Same figures; every report that must stay available runs from the archive
Retention check of the archive against legacy	Every record class that must be kept is readable; counts and checksums equal legacy
Legacy powered off, consumers watched	No failed request or support ticket; a restore rehearsal from the final backup works
Fitness function	An alert on any connection to legacy after power-off; a scheduled smoke test runs each kept report from the archive; a deletion gate blocks removal until retention and backup checks pass

Rollback: routing rollback: power legacy on again from the final backup and re-point the consumer; possible because the shutdown is a power-off, but it is the complex restore. State rollback is not needed, because legacy was read-only and nothing was written after the archive; a reader found late is served forward from the archive or by the restore, then compared with the archive.

Stop: when 0 live callers remain, every kept report and record is served from the read-only archive and the cost is gone, legacy stays off and no SaaS product, replacement platform or further migration is added.

Solution architecture



The three reports now read a read-only archive, and the logs on SQL Server show 0 callers, so the legacy host and database are switched off while the archive keeps retained records readable.

Glossary

Used in this step

Term	Definition
Consumer inventory + read-only archive + retire	List every consumer, keep the data that must stay in a read-only archive, then switch legacy off
Keep legacy	Leave the old system running unchanged and accept its cost and debt
Replace with SaaS	Substitute commercial cloud or packaged product for custom capability
Switch off legacy now	Shut legacy down on a date without listing its consumers; the trap
Retire	Remove software capability once consumers and retention requirements are handled
Consumer inventory	List of every system, job and person that still reads legacy
Read-only archive	Copy of the data that stays readable but accepts no changes
Retention requirement	How long records such as invoices and the audit trail must stay readable
Query log	Record of every query and connection made to legacy
Mandatory live path	A consumer that needs legacy to answer in production

Related concepts

Term	Definition
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations
Expand-Contract	Add compatible representation, migrate consumers, then remove old representation
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Forward Fix	Repair the new implementation without restoring the old system
Feature Flag	Runtime control enabling behavior without deploying new code

M50 · Infrastructure change and retirement

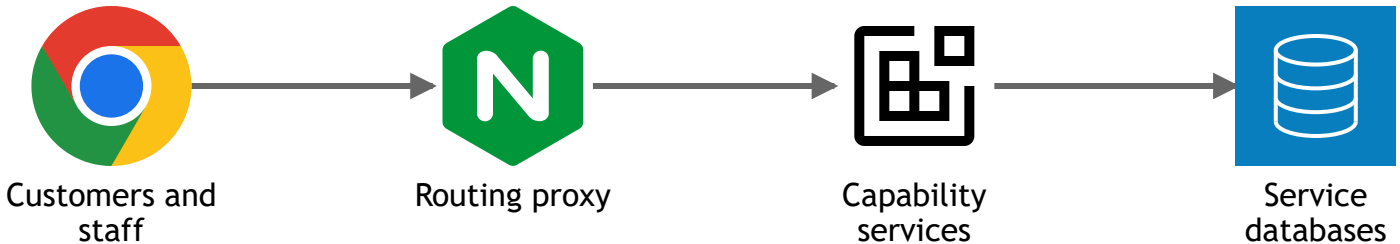
Stop unnecessary modernization

The architecture meets all target SLOs, so further decomposition does not improve business outcomes.

The current architecture already meets every target SLO

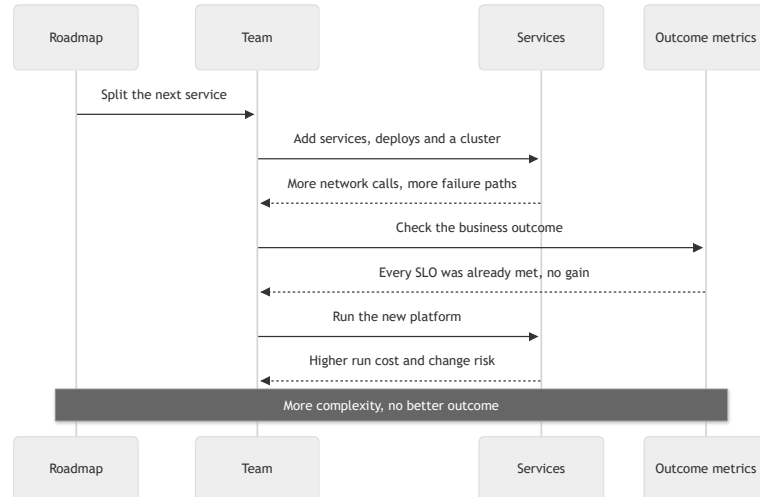
Component	What it is	What it does
Customers and staff	People who use the shop and the back office	Reach every capability through one entry point
Routing proxy	Stable public entry point	Sends each route to the service that owns it
Capability services	Catalog, notification, billing, inventory and checkout, each deployed on its own	Meet every target SLO today
Service databases	Each service owns its data	Hold the data behind each capability
Target SLOs	Measurable quality targets such as latency and availability	Are checked continuously by fitness functions

Current architecture



More decomposition would add cost without a better outcome

The legacy remnants are gone and every target SLO is met, yet the roadmap still lists further splits and a container platform. Each split adds deployments, network calls and failure paths without moving a business outcome.



No outcome gain

further decomposition improves no business outcome: every target SLO is already met

Roadmap still lists splits

further splits and a container platform are planned

Measured gap first

no extraction without one, and every restart trigger defined and monitored

Each option decides whether more structure has a measured reason

Solution	What it is	What it does
Stop migration	Declare the current architecture the target and add no further decomposition	Keeps the services as they are and watches outcome metrics against restart triggers
Microservices/Kubernetes	Independently deployable services organized around business capabilities, run on a container cluster	Gives one boundary its own service and platform when a constraint needs it
Continue decomposition by plan	Keep extracting services because the roadmap lists them	Delivers the planned splits whatever the outcome metrics say

Stopping prevents complexity, a plan alone adds risk

Solution	Gains	Costs
Stop migration	Prevents complexity No new change risk Effort goes to business features	Remaining coupling stays as it is Needs measured restart triggers Pressure to modernize returns without data
Microservices/Kubernetes	Independent deployment and scaling Team autonomy per capability	Only pays when actual constraints demand Cluster and platform to operate Distributed failure modes
Continue decomposition by plan	Roadmap and momentum continue Clear next tasks	Added change risk, no outcome gain Distributed overhead without a constraint Partial optimization can raise total cost

Deciding signal: coordinated change frequency, independent deployment or scaling need, operations skill and cross-domain invariants, with SLO status, cost per completed transaction and the added change risk of each extraction. All target SLOs met and no boundary that pays the distributed overhead picks Stop migration; a measured constraint the current structure cannot meet picks Microservices/Kubernetes for that boundary only; a roadmap item without a measured gap is Continue decomposition by plan and fails. The choice changes when a target SLO is missed over a review period.

Smallest safe move

Stop migration, with measured triggers that restart it

- **Scope:** evidence and governance only; no code, traffic, data or write ownership changes
- **Change:** record the decision, set outcome metrics and a baseline, and give each restart trigger a metric, a threshold and an owner
- **Safety net:** triggers are measured conditions: a missed target SLO, coordinated change as the main delay, an independent scale need; Continue decomposition by plan starts nothing without a measured gap
- **Escape route:** reverse the decision record; nothing in the system changed
- **Deferred:** Microservices/Kubernetes for one boundary when a measured constraint demands it

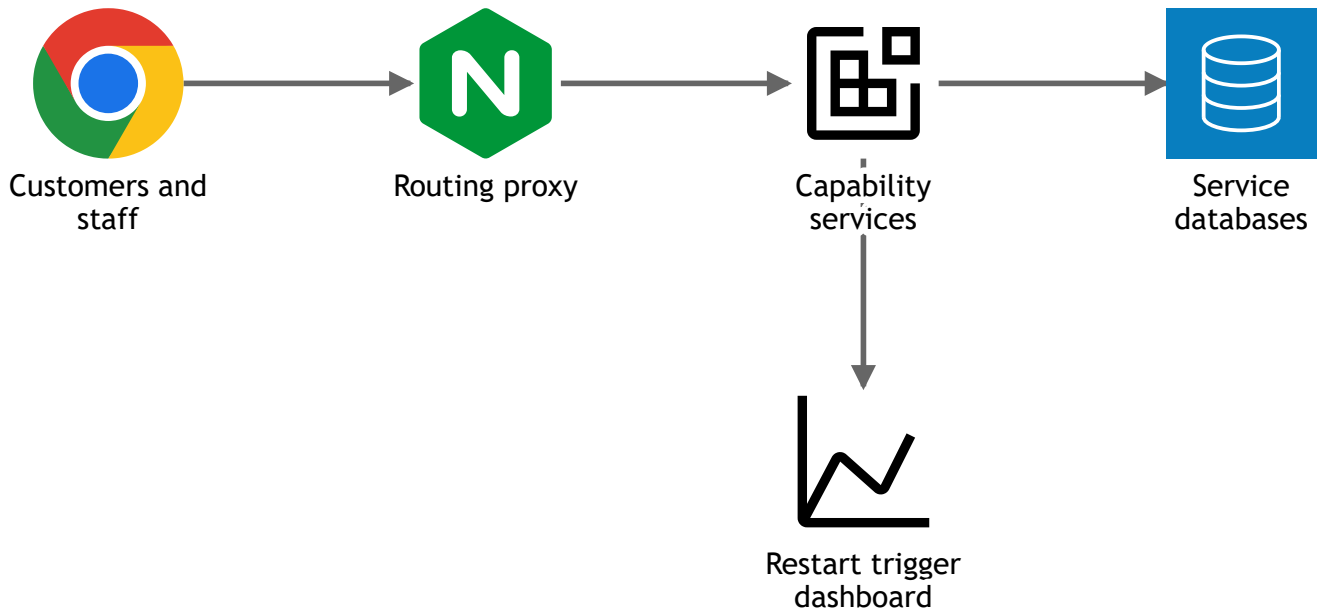
Verification: outcome metrics and change risk reassessed

Check	Pass condition
Outcome metrics reassessed against the baseline	All target SLOs met; the metrics show no gap that more decomposition would close
Added change risk of each planned extraction: blast radius, rollback, dependencies	Each names the SLO or outcome it would improve; none without a measured gap is started
Restart triggers defined	Each has a metric, a threshold set from the baseline and an owner, and is fed with live data
A trigger fired on purpose in a test	A modernization review opens with the evidence attached
Fitness function	A scheduled job compares the outcome metrics with the trigger thresholds and opens a review only when a trigger fires; the target SLOs stay automated checks in the pipeline

Rollback: routing rollback is not applicable, because nothing moves; state rollback is not needed, because no data or ownership changes. The stop decision is reversed by a decision record, not by traffic; when a trigger fires, restart with the smallest move for one boundary and repeat the decision loop.

Stop: the stop condition is the whole point: while every target SLO is met and no restart trigger has fired, no further decomposition, Kubernetes platform or new layer is added.

Solution architecture



The architecture is unchanged; the only addition is a trigger dashboard that watches the services and reopens modernization when a measured trigger fires.

Glossary

Used in this step

Term	Definition
Stop migration	Declare the current architecture the target and add no further decomposition
Microservices/Kubernetes	Independently deployable services organized around business capabilities, run on a container cluster
Continue decomposition by plan	Keep extracting services because the roadmap says so, not because a constraint demands it; the trap
Target SLO	Measurable quality target, such as latency or availability, that the architecture must meet
Outcome metric	Measure of a business result, such as cost per completed transaction or change lead time
Restart trigger	A measured condition with a threshold and an owner that reopens modernization
Trigger dashboard	View that compares the outcome metrics with the restart triggers
Decomposition	Splitting an application into smaller separately deployable parts
Change risk	Chance and blast radius of a failed change, including how it is reversed
Coordinated change	A change that needs several modules or teams to release together
Fitness Function	Measurable or executable architectural quality rule

Related concepts

Term	Definition
Modular Monolith	Single deployment with clearly separated internal modules.
Microservices	Independently deployable services organized around business capabilities.
Kubernetes Cluster	Group of machines orchestrating containerized applications.
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations
Feature Flag	Runtime control enabling behavior without deploying new code